



SAPIENZA
UNIVERSITÀ DI ROMA

Reti degli Elaboratori

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF.SSA GAIA MASELLI

25 settembre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Questo testo è stato integrato con i libri [1] e [2].

Indice

Prefazione	1
1 Introduzione al corso	7
1.1 Le reti	7
1.1.1 LAN (Local Area Network)	7
1.1.2 WAN (Wide Area Network)	8
1.2 Commutazione (Switching)	9
1.2.1 Reti a commutazione di circuito	10
1.2.2 Reti a commutazione di pacchetto	10
1.3 Internet	11
1.4 L'accesso a Internet	11
1.4.1 Accesso via rete telefonica	11
1.4.2 Accesso tramite Ethernet	12
1.4.3 Accesso wireless	12
1.5 Capacità e prestazioni delle reti	12
1.5.1 Ampiezza di banda e bit rate	12
1.5.2 Throughput	13
1.5.3 Latenza (ritardo)	14
1.5.4 Perdita di pacchetti	16
1.6 Hardware e software	16
2 Organizzazione dei protocolli in livelli	17
2.1 Due scenari	17
2.1.1 Principi della strutturazione in livelli	18
2.1.2 Collegamenti logici	19
2.2 Lo stack protocollare TCP/IP	19
2.2.1 Architettura a livelli	20
2.2.2 I livelli nello stack protocollare TCP/IP	20
2.2.3 Descrizione dei livelli dello stack protocollare TCP/IP	20
2.2.4 Incapsulamento e decapsulamento	22
2.2.5 Indirizzamento	23
2.2.6 Multiplexing e demultiplexing	23
2.3 Il modello OSI	23
2.3.1 L'insuccesso del modello OSI	24
3 La storia di Internet	25
3.1 La storia iniziale	25
3.2 ARPANET	25
3.3 Nascita della rete Internet	25
3.4 Internet oggi	26
3.5 Gli standard della rete Internet	26
3.6 Amministrazione della rete Internet	26

4	Livello applicazione	27
4.1	Introduzione	27
4.1.1	L'offerta dei servizi	27
4.1.2	Paradigmi del livello applicazione	28
4.2	WEB e HTTP	29
4.2.1	Panoramica Web	29
4.2.2	HTTP (HyperText Transfer Protocol)	31
4.2.3	HTTP con connessioni persistenti e non persistenti	31
4.2.4	Formato dei messaggi HTTP	33
4.2.5	Cookie	36
4.2.6	Caching	38
4.3	DNS (Domain Name System)	41
4.3.1	Servizi forniti da DNS	41
4.3.2	Panoramica del funzionamento DNS	42
4.3.3	Record e messaggi DNS	45
4.4	Protocollo FTP	47
4.4.1	Connessione di controllo	48
4.4.2	Connessione dati	48
4.5	Posta elettronica	49
4.5.1	Architettura della posta elettronica	49
4.5.2	User agent	50
4.5.3	Message Transfer Agent: SMTP	51
4.5.4	Message Access Agent	52
4.5.5	Multipurpose Internet Mail Extensions (MIME)	52
4.5.6	Webmail	53
5	Livello trasporto	55
5.1	Introduzione	55
5.1.1	I servizi del livello trasporto	55
5.1.2	API di comunicazione	59
5.1.3	Protocolli Internet di livello trasporto	60
5.2	Protocollo UDP (User Datagram Protocol)	61
5.2.1	Protocollo semplice	62
5.2.2	Struttura dei datagrammi utente	62
5.2.3	Servizi UDP	63
5.3	Protocollo TCP (Transmission Control Protocol)	63
5.3.1	Servizi TCP	64
5.3.2	Stop-and-wait	68
5.3.3	Go-Back-N (GBN)	71
5.3.4	Ripetizione selettiva	75
5.3.5	Protocolli bidirezionali: piggybacking	79
5.3.6	Segmenti TCP	79
5.3.7	Connessione TCP	81
5.3.8	Controllo degli errori	84
5.3.9	Controllo del flusso	90
5.3.10	Controllo della congestione	93
5.3.11	Timeout e stima del tempo di andata e ritorno	98
6	Livello di rete	101
6.1	Introduzione	101
6.1.1	Servizi a livello rete	101
6.1.2	Packet switching	102
6.1.3	Struttura di un router	105
6.1.4	Elaborazione alle porte di ingresso e inoltro	106
6.1.5	Struttura di commutazione	107
6.1.6	Elaborazione alle porte di uscita	109
6.1.7	Dove si verifica l'accodamento?	109
6.2	Protocolli di livello rete	111
6.2.1	Formato di datagrammi IPv4	112

6.2.2	Frammentazione	114
6.2.3	Indirizzi IPv4	117
6.2.4	Inoltro dei datagrammi IP	126
6.2.5	ICMP	129
6.3	Routing	131
6.3.1	Algoritmi d'instradamento	132
6.3.2	Protocollo di routing	141
6.3.3	Routing multicast	151
6.4	IPv6	156
6.4.1	Formato dei datagrammi IPv6	157
6.4.2	Passaggio da IPv4 a IPv6	158
7	Livello di collegamento	161
7.1	Introduzione	161
7.1.1	Servizi offerti dal livello di collegamento	162
7.1.2	Dov'è implementato il livello di collegamento?	163
7.1.3	Tecniche di rilevazione e correzione degli errori	164
7.2	Protocolli di accesso multiplo	166
7.2.1	Protocolli a suddivisione del canale	167
7.2.2	Protocolli ad accesso casuale	169
7.2.3	Protocolli a rotazione	175
7.3	Indirizzamento di collegamento	176
7.3.1	Indirizzi MAC	176
7.3.2	Protocollo per la risoluzione degli indirizzi (ARP)	177
7.4	LAN cablate: Ethernet	182
7.4.1	Ethernet standard	183
7.4.2	Fast Ethernet (100 Mbps)	186
7.4.3	Gigabit Ethernet	186
7.4.4	10-Gigabit Ethernet	186
7.4.5	LAN virtuali	187
7.5	Switch di livello collegamento	187
7.5.1	Apprendimento	188
8	Reti wireless	189
8.1	LAN wireless	189
8.1.1	Introduzione	189
8.1.2	IEEE 802.11	193
8.1.3	Bluetooth	200
8.2	Accesso al mezzo mediante suddivisione del canale	202
8.2.1	Frequency-Division Multiple Access (FDMA)	202
8.2.2	Time Division Multiple Access (TDMA)	202
8.2.3	Code-Divison Multiple Access (CDMA)	203

Capitolo 1

Introduzione al corso

In questo corso si studieranno principalmente servizi e protocolli di rete (TCP/IP, DNS, HTTP, NAT, SSL), in particolare, il focus principale è sul networking ovvero il trasferimento di dati dal dispositivo client al server e viceversa.

1.1 Le reti

Una **rete** è composta di dispositivi (nodi) in grado di scambiarsi informazioni, quali sistemi terminali (end system), e dispositivi di interconnessione. I sistemi terminali possono essere di due tipi: *host* o *server*. Un *host* è una macchina in genere di proprietà degli utenti e dedicata ad eseguire applicazioni, quale un computer desktop, un portatile, un cellulare o un tablet. Un *server* è tipicamente un computer con elevate prestazioni destinato a eseguire programmi che forniscono servizio a diverse applicazioni utente come, per esempio, la posta elettronica o il Web.

I server sono installati in uffici centrali e vengono gestiti dagli amministratori di sistema. Rientrano tra i server anche le periferiche di rete, come per esempio le stampanti, che vengono spesso condivise tra i vari utenti.

I dispositivi di interconnessione rigenerano/modificano il segnale che ricevono e si distinguono in: *router* che sono dispositivi che collegano una rete ad altre reti; *switch* (commutatori) che collegano più sistemi terminali a livello locale; *modem* che trasformano la codifica dei dati. Tutti questi dispositivi vengono collegati utilizzando mezzi trasmissivi cablati o wireless genericamente chiamati *link* (collegamenti).

I collegamenti cablati possono essere in rame o fibra ottica e un bit viaggia da un sistema terminale a un altro, passando per una serie di coppie trasmittente-ricevente. I mezzi di trasmissione cablati sono: il doppino intrecciato, formato da due fili di rame distinti (tradizionale cavo telefonico) o intrecciato da quattro coppie di fili (Ethernet); il cavo coassiale, formato da due conduttori in rame concentrici, usato per cablatura di reti locali ad alta velocità ed è molto resistente alle interferenze; la fibra ottica, è un mezzo sottile e flessibile che conduce impulsi di luce (ciascun impulso rappresenta un bit), ha alta frequenza trasmissiva, elevata velocità di trasmissione punto-punto, basso tasso di errore ed è immune all'interferenza elettromagnetica. Per questi motivi è il mezzo prevalente delle dorsali internet.

Con i mezzi trasmissivi wireless, i segnali si propagano nell'atmosfera e nello spazio esterno. Il vantaggio di questo tipo di sistemi è che trasportano segnali nello spettro elettromagnetico e non richiedono l'installazione fisica di cavi, ma allo stesso tempo possono essere soggetti alla rifrazione, ostruzione da parte di ostacoli o interferenza. I principali tipi di canali radio sono: microonde terrestri, LAN, wide-area e satellitari.

1.1.1 LAN (Local Area Network)

Una LAN (Local Area Network), o rete locale, è solitamente una rete privata che collega i sistemi terminali in un singolo ufficio (azienda, università) e non specifica un numero minimo o massimo di dispositivi. Ogni sistema terminale nella LAN ha un indirizzo che lo identifica univocamente nella rete.

In passato tutti i dispositivi di una rete locale erano collegati mediante un cavo condiviso, o mezzo broadcast, per cui il pacchetto inviato da un dispositivo veniva ricevuto da tutti. Il destinatario elaborava il pacchetto, mentre gli altri lo dovevano ignorare. Oggi la maggior parte delle LAN utilizza uno switch di interconnessione, al quale ogni dispositivo in rete è direttamente connesso. Lo switch è in grado di riconoscere l'indirizzo di destinazione e di inviare il pacchetto al solo destinatario senza inviarlo agli altri dispositivi. Lo switch riduce il traffico nella LAN e consente a più coppie di dispositivi di comunicare contemporaneamente fra di loro, posto che non vi siano dispositivi sorgente o destinazione in comune.

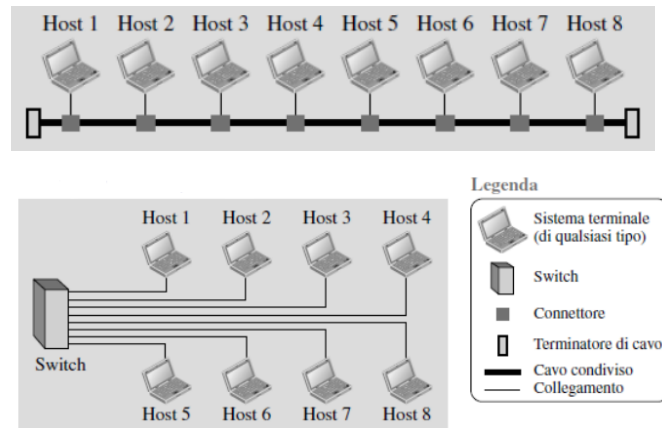


Figura 1.1: LAN con cavo condiviso e con switch

1.1.2 WAN (Wide Area Network)

Anche una WAN (Wide Area Network), o rete geografica, è un'interconnessione di dispositivi in grado di comunicare, ma esistono notevoli differenze rispetto alle LAN. Queste ultime sono solitamente di estensione limitata, mentre una WAN ha un'estensione decisamente superiore, potendo servire una città, una regione, una nazione o addirittura l'intero globo. Una LAN interconnette prevalentemente sistemi terminali, mentre una WAN interconnette anche switch, router o modem. Una LAN è solitamente di proprietà dell'organizzazione che la utilizza, mentre una WAN è generalmente creata e gestita da un operatore di telecomunicazioni, Internet Service Provider (ISP), che fornisce i suoi servizi alle organizzazioni che ne fanno uso. Oggi esistono due tipi di WAN: punto-punto e a commutazione (switched).

- Una WAN punto-punto è una rete che collega due dispositivi di comunicazione tramite un mezzo trasmissivo (cavo o wireless);
- Una WAN a commutazione (switched) è una rete con più di due punti di terminazione e viene utilizzata nelle dorsali dell'odierna rete globale.

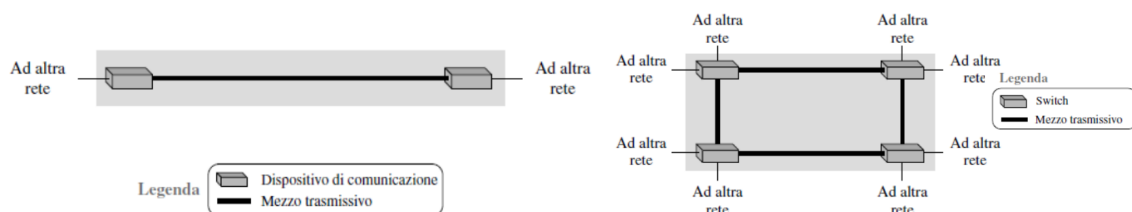
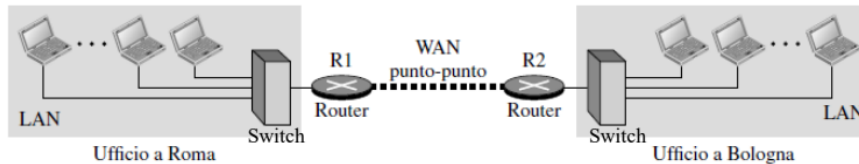


Figura 1.2: WAN punto-punto e WAN a commutazione

Al giorno d'oggi è raro vedere LAN o WAN isolate: esse sono in genere connesse fra di loro, formando una internetwork, o internet.

Esempio 1.1.1 – Internetwork

Si ipotizzi che un'azienda abbia due uffici in città di regioni differenti, Roma e Bologna. In ciascun ufficio esiste una LAN che consente agli impiegati di comunicare l'uno con l'altro. Per rendere possibile la comunicazione fra gli impiegati di uffici differenti, l'azienda affitta un'apposita WAN punto-punto da un ISP, per esempio una compagnia telefonica, per collegare le due LAN. A questo punto l'azienda ha realizzato una internetwork, o una rete internet privata, che consente la comunicazione fra le due reti.



Quando un host in un ufficio invia un messaggio a un altro host nello stesso ufficio, lo switch trasmette il messaggio all'host indirizzato. Quando invece un host nell'ufficio di Roma invia un messaggio a un host nell'ufficio di Bologna, è il router R1 che instrada il pacchetto al router R2 per raggiungere il destinatario.

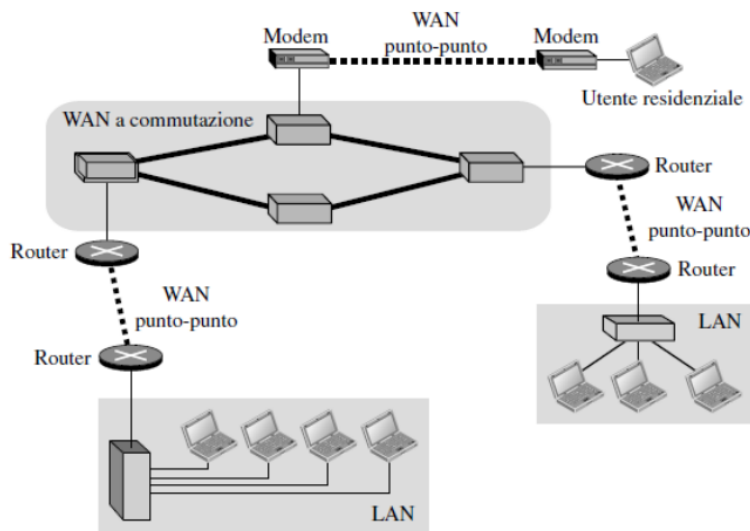


Figura 1.3: Rete eterogenea composta da quattro WAN e tre LAN

La rete GARR (Gruppo per l'Armonizzazione delle Reti della Ricerca) interconnette ad altissima capacità università, centri di ricerca, biblioteche, musei, scuole e altri luoghi in cui si fa istruzione, scienza, cultura e innovazione su tutto il territorio nazionale. E' un'infrastruttura in fibra ottica che utilizza le più avanzate tecnologie di comunicazione e si sviluppa su circa 15.000 km tra collegamenti di dorsale e di accesso. Oggi la capacità delle singole tratte della dorsale arriva a 100 Gbps, mentre quella dei collegamenti di accesso può raggiungere i 40 Gbps in base alle necessità di banda della sede. Grazie alla grande scalabilità delle tecnologie utilizzate, queste capacità possono evolvere facilmente insieme alle necessità degli utenti.

1.2 Commutazione (Switching)

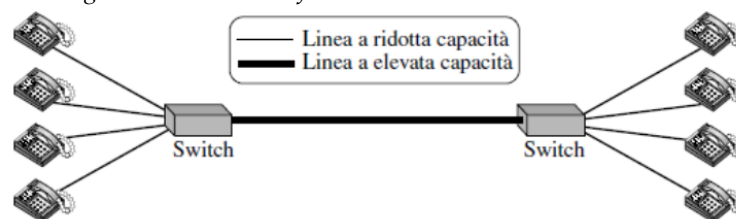
Si è affermato che una internet è una combinazione di link e dispositivi capaci di scambiarsi informazioni. In particolare i sistemi terminali appartenenti alla rete comunicano tra di loro per mezzo di dispositivi come switch e router che si trovano nel percorso (o rotta) tra il sistema sorgente e destinazione. In base al metodo adottato per determinare il percorso tra due sistemi terminali in comunicazione si possono distinguere due tipi di reti: a commutazione di circuito e a commutazione di pacchetto.

1.2.1 Reti a commutazione di circuito

In una rete a commutazione di circuito (circuit-switched network) tra due dispositivi è sempre disponibile un collegamento dedicato, chiamato circuito; lo switch che collega i due dispositivi può solo attivarlo o disattivarlo. Le risorse necessarie al circuito sono riservate per l'intera durata della comunicazione e le informazioni riguardanti il circuito vengono mantenute dalla rete. Anche se esistono più percorsi tra due dispositivi in comunicazione, solo uno di questi verrà usato per l'intera comunicazione e comunicazioni diverse tra gli stessi dispositivi possono usare circuiti stabiliti su percorsi diversi.

Esempio 1.2.1 – Reti a commutazione di circuito

Quattro apparecchi telefonici su ciascun lato sono collegati a uno switch che mette in comunicazione un apparecchio telefonico da un lato con un apparecchio telefonico dall'altro lato. La linea tra i due switch è una linea di comunicazione a elevata capacità che può gestire contemporaneamente quattro canali voce; tale capacità può essere condivisa fra tutte le coppie di apparecchi telefonici. Gli switch impiegati in questo esempio sono in grado di inoltrare informazioni ma non di memorizzarle.



Si prendano ora in considerazione le due situazioni seguenti. Nella prima tutti gli apparecchi telefonici sono occupati: quattro persone da un lato sono in comunicazione con le quattro persone all'altro lato e la capacità della linea è completamente utilizzata. Nella seconda situazione solo un apparecchio telefonico da un lato è collegato a un apparecchio telefonico dell'altro lato: si impiega dunque solamente un quarto della capacità della linea. Questo comporta che una rete a commutazione di circuito risulta efficiente solamente quando è utilizzata alla sua massima capacità, mentre nella maggior parte del tempo è inefficiente perché sottoutilizzata.

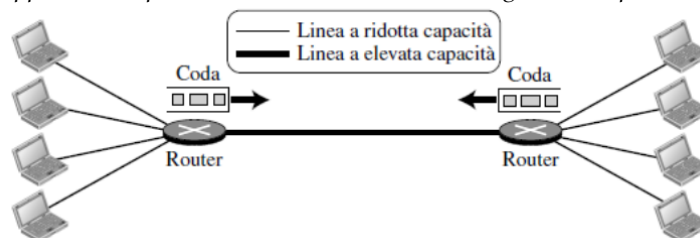
In questo tipo di rete le risorse (ad es. ampiezza di banda - bandwidth) vengono suddivise in "pezzi" per frequenza o per tempo, e ciascun "pezzo" viene allocato ai vari collegamenti.

1.2.2 Reti a commutazione di pacchetto

In questa rete di computer la comunicazione fra i due lati viene effettuata trasmettendo blocchi di dati chiamati pacchetti. Invece di avere una comunicazione continua, i due dispositivi comunicano scambiandosi pacchetti di dati. Gli switch sono in grado di memorizzare i pacchetti (store), oltre che di inoltrarli (forward), essendo i pacchetti entità indipendenti che possono essere memorizzate e inviate successivamente. Non c'è un percorso specifico che viene usato per il trasferimento di dati tra due dispositivi: blocchi di dati, anche dello stesso file o comunicazione, possono prendere percorsi diversi nel viaggio dalla sorgente alla destinazione e arrivare a destinazione in un ordine diverso.

Esempio 1.2.2 – Reti a commutazione di pacchetto

Un router in una rete a commutazione di pacchetto possiede una coda dove possono essere memorizzati i pacchetti prima di essere inoltrati. Si ipotizzi che la capacità della linea di comunicazione fra i due router sia solo il doppio della capacità delle linee dei dati che collegano i computer ai router.



Nel caso in cui due soli computer, uno per ciascun lato, dovessero comunicare fra di loro, non vi sarebbe alcun tempo di attesa per i pacchetti, che verrebbero inoltrati appena disponibili. Se invece i pacchetti arrivassero a un router quando la linea di comunicazione fra i due router fosse già utilizzata alla sua massima capacità, i pacchetti dovrebbero essere memorizzati per essere successivamente inoltrati nell'ordine di arrivo.

Questo esempio dimostra che una rete a commutazione di pacchetto è più efficiente di una rete a commutazione di circuito, ma che i pacchetti potrebbero incorrere in qualche ritardo.

1.3 Internet

Notazione 1.3.1 – Internet

internet (con la i minuscola) rappresenta due o più reti interconnesse. L'internet più famosa è chiamata Internet (I maiuscola) ed è composta da migliaia di reti interconnesse.

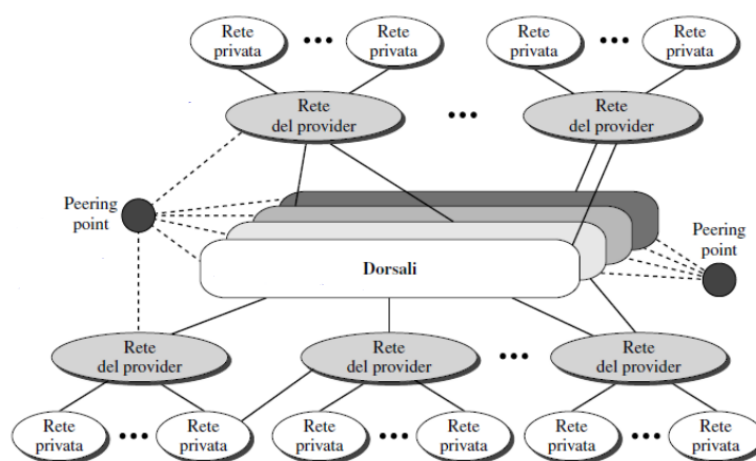


Figura 1.4: Rappresentazione concettuale di Internet

La figura 1.4 rappresenta Internet come un insieme di dorsali (backbone), reti dei provider e reti private. Al livello più elevato le dorsali sono reti particolarmente estese di proprietà di qualche compagnia telefonica. Queste reti dorsali sono interconnesse tramite sistemi di commutazione notevolmente complessi, chiamati peering point. Al secondo livello vi sono reti più piccole, chiamate reti dei provider, che utilizzano a pagamento i servizi delle dorsali. Le reti dei provider sono connesse alle dorsali e a volte ad altre reti di provider. Le reti private sono reti ai confini di Internet (last hop network), più vicine ai sistemi terminali, che utilizzano i servizi a pagamento forniti dalle reti dei provider.

Le dorsali e le reti dei provider sono anche chiamate collettivamente ISP (Internet Service Provider). Le dorsali sono spesso chiamate ISP internazionali, le reti dei provider ISP nazionali o regionali.

1.4 L'accesso a Internet

Internet è una internetwork che consente a qualsiasi utente di farne parte. L'utente, tuttavia, deve essere fisicamente collegato a un ISP, solitamente mediante una WAN punto-punto. Il collegamento che connette l'utente al primo router di Internet è detto rete di accesso.

1.4.1 Accesso via rete telefonica

Oggi la maggior parte delle abitazioni e delle piccole aziende è dotata di servizi telefonici, quindi è collegata a una rete telefonica. Poiché molte reti telefoniche sono collegate a Internet, un'opzione

è quella di collegarsi a Internet modificando la linea telefonica fra la sede del dispositivo che vuole connettersi e la centrale telefonica con una WAN punto-punto. Questo può avvenire in due modi.

1. *Servizio dial-up*. Questa soluzione consiste nell'inserire sulla linea telefonica un modem che converte i dati digitali (del computer) in analogici (per trasmetterli sulla linea telefonica) e viceversa. Sfortunatamente il servizio dial-up è molto lento; inoltre, quando la linea è utilizzata per il collegamento a Internet non può essere utilizzata per le normali conversazioni telefoniche (vocali). Si tratta quindi di una soluzione adatta a clienti residenziali o a piccole aziende che si connettono a Internet solo sporadicamente.
2. *Servizio DSL (Digital Subscriber Line)*. Negli ultimi anni numerose compagnie telefoniche hanno modificato le proprie linee telefoniche per poter fornire ai clienti residenziali e alle piccole imprese servizi di accesso a Internet ad alta velocità. La tecnologia DSL supporta la comunicazione digitale ad alta velocità sulla linea telefonica. Questo servizio consente anche di utilizzare simultaneamente la linea telefonica per il traffico dati e per le comunicazioni vocali.

1.4.2 Accesso tramite Ethernet

Nelle reti aziendali e universitarie la tecnologia usata per l'accesso a Internet è l'Ethernet dove lo switch (Ethernet) della LAN è generalmente collegato a un router istituzionale che è connesso ai router della dorsale.

1.4.3 Accesso wireless

La connettività wireless si è ampiamente diffusa. Con il continuo sviluppo di tecnologie di accesso wireless, i clienti residenziali e le piccole aziende possono convenientemente utilizzare un'opportuna combinazione di collegamenti wireless e via cavo per accedere a Internet. Esempi di accessi wireless sono: Wi-Fi con access point (locale) connesso alla Ethernet cablata e raggio di azione di qualche decina di metri oppure cellulare, dove si usa la rete cellulare con access point (base station) della compagnia telefonica cellulare, con raggio di azioni di decine di chilometri.

1.5 Capacità e prestazioni delle reti

Un argomento di cui si parla spesso nell'ambito delle reti è la velocità della rete o di un collegamento, ovvero quanto velocemente si riesce a trasmettere e ricevere i dati. In realtà, il concetto di velocità è molto ampio e coinvolge vari fattori che influiscono sulle prestazioni (performance) di una rete. Nel caso di una rete a commutazione di pacchetto, le metriche che ne determinano le prestazioni si misurano in termini di: ampiezza di banda, bit rate, throughput, ritardi e perdita di pacchetti.

1.5.1 Ampiezza di banda e bit rate

Con il termine ampiezza di banda si indicano due concetti leggermente diversi ma strettamente legati.

1. Al livello di caratterizzazione di un canale o di un sistema trasmissivo, l'ampiezza di banda (bandwidth), o banda, è una quantità che si misura in hertz e rappresenta la larghezza dell'intervallo di frequenze utilizzato dal sistema trasmissivo, ovvero l'intervallo di frequenze che un mezzo fisico consente di trasmettere senza danneggiare il segnale in maniera irrecuperabile. In generale, maggiore è l'ampiezza di banda, maggiore è la quantità di informazione che può essere veicolata attraverso il mezzo trasmissivo.
2. Quando viene espressa in bit al secondo (bps), la banda rappresenta invece il bit o transmission rate (velocità di trasmissione), per brevità rate, ovvero la quantità di bit al secondo che un certo link garantisce di trasmettere.

I due concetti sono legati ma non coincidenti, in quanto il bit rate di un link dipende sia dalla banda (in hertz) che dalla specifica tecnica di trasmissione, o formato di modulazione digitale utilizzato. In generale, il bit rate è proporzionale alla banda in hertz. Infine, per banda di un tipo di rete, si intende il bit rate garantito (nominalmente) dai suoi link. Per esempio si può dire che il rate di un link Fast Ethernet è di 100 Mbps. Ciò significa che tale rete può inviare al massimo 100 Mbps. Il rate fornisce quindi un'indicazione della capacità della rete di trasferire dati.

1.5.2 Throughput

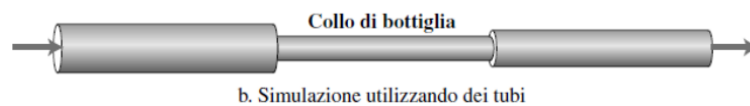
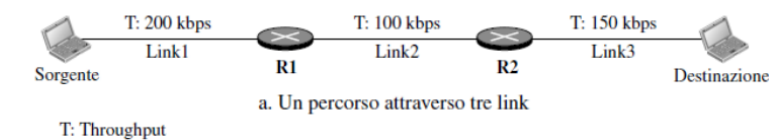
Il throughput indica quanto velocemente riusciamo effettivamente a inviare i dati tramite una rete. Sebbene a prima vista il rate e il throughput sembrino la stessa cosa, in realtà non lo sono. Un link può avere un rate di B bps, ma possiamo inviare solo T bps tramite quel link, con T sempre inferiore a B . In altre parole, il rate è una misura della potenziale velocità di un link; il throughput è una misura dell'effettiva velocità di un link (quanto velocemente riusciamo a inviare i dati in realtà). Per esempio, possiamo avere un link con un rate di 1 Mbps, ma i dispositivi connessi alle estremità del link possono gestire solamente 200 kbps. Ciò significa che non possiamo inviare più di 200 kbps attraverso quel link.

Immaginiamo un'autostrada progettata per far transitare 1000 auto al minuto da un punto a un altro. Tuttavia, se c'è traffico sulla strada, tale cifra può essere ridotta a 100 auto al minuto. Il rate è 1000 auto al minuto, il throughput 100 auto al minuto.

E' importante notare che il throughput è definito come il numero dei bit che passano attraverso un punto della rete in un secondo. In un percorso da una sorgente a una destinazione un pacchetto può passare attraverso numerosi link, ognuno con un throughput diverso. Come possiamo allora determinare il throughput dell'intero percorso?

Esempio 1.5.1 – Throughput

Supponiamo di avere tre link, come mostrato in figura:



I dati possono scorrere alla velocità di 200 kbps nel link 1. Tuttavia, quando i dati arrivano al router R1, non possono passare alla stessa velocità. I dati devono essere accodati al router e inviati a 100 kbps. Quando i dati arrivano al router R2, potrebbero essere inviati alla velocità di 150 kbps ma non ci sono sufficienti dati da inviare. In altre parole, la velocità media del flusso di dati nel link 3 è anch'essa di 100 kbps. Possiamo concludere che il throughput dei dati per questo percorso è di 100 kbps, la minore delle tre diverse velocità dei dati. La figura mostra anche che possiamo simulare il comportamento in ogni link con tubi di diverse dimensioni; il throughput medio è determinato dal collo di bottiglia, il tubo con il diametro minore.

In generale in un percorso con n link in serie abbiamo:

$$\text{Throughput} = \text{minimo}(T_1, T_2, \dots, T_n)$$

Anche se l'esempio illustrato mostra come calcolare il throughput quando i dati passano attraverso numerosi link, la situazione reale in Internet è che i dati normalmente passano attraverso due reti di accesso e la dorsale di Internet, come mostrato nella Figura 1.5.

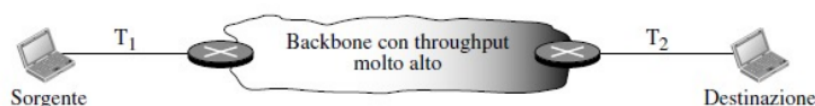


Figura 1.5: Throughput attraverso il backbone di Internet

La dorsale di Internet ha un throughput molto alto, nell'ordine di gigabit al secondo. Ciò significa che il throughput normalmente viene definito come il minimo tra i due link d'accesso che collegano la sorgente e la destinazione alla dorsale. La Figura 1.5 illustra tale situazione, in cui il throughput è il minimo di T_1 e T_2 . Ad esempio, se un server connesso ad Internet tramite una Fast Ethernet LAN con la velocità dei dati di 100 Mbps, ma un utente che vuole scaricare un file collegato ad Internet

tramite una linea telefonica commutata con velocità di dati di 40 kbps, il throughput è di 40 kbps. Il collo di bottiglia è certamente la linea commutata.

Dobbiamo parlare di un'altra situazione nella quale pensiamo al throughput. Il link tra due router non è sempre dedicato ad un flusso. Un router può raccogliere il flusso da varie sorgenti o distribuirlo tra diverse sorgenti. In questo caso il rate del link tra i due router è realmente condiviso tra i flussi e ciò va considerato quando si calcola il throughput. Per esempio, nella Figura 1.6 la velocità del link principale nel calcolo del throughput è solo 200 kbps in quanto il link viene condiviso tra tre percorsi.

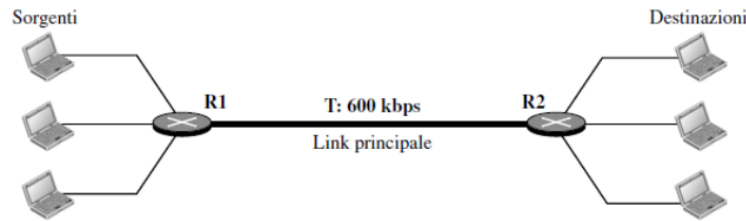


Figura 1.6: Effetto del throughput nei link condivisi

1.5.3 Latenza (ritardo)

Tutti noi ci aspettiamo risposte istantanee da una rete, ma un pacchetto che viaggia dalla sorgente alla destinazione accumula dei ritardi. La latenza, o ritardo (delay), definisce quanto tempo serve affinché un intero messaggio arrivi completamente a destinazione dal momento in cui il primo bit parte dalla sorgente. I ritardi si possono dividere in quattro tipi: ritardo di trasmissione, ritardo di propagazione, ritardo di elaborazione e ritardo di accodamento. La latenza, o ritardo totale, deriva dalla seguente formula:

$$\text{Latenza} = \text{ritardo di propagazione} + \text{ritardo di trasmissione} \\ + \text{ritardo di accodamento} + \text{ritardo di elaborazione}$$

Analizziamo ora i vari tipi di ritardo.

Ritardo di trasmissione

Quando un dispositivo sorgente invia un pacchetto, esso deve immettere sulla linea di trasmissione (ovvero trasmettere) i bit del pacchetto uno per uno. I pacchetti vengono trasmessi in first-come-first-served e se il primo bit del pacchetto viene trasmesso al tempo t_1 , e l'ultimo bit viene messo in linea al tempo t_2 il ritardo di trasmissione del pacchetto è $(t_2 - t_1)$. Chiaramente il ritardo di trasmissione è maggiore per un pacchetto più lungo e minore se il mittente riesce a trasmettere più velocemente. In altre parole il ritardo di trasmissione è:

$$\text{Ritardo}_{tr} = (\text{Lunghezza del pacchetto}) / (\text{Rate})$$

Per esempio in una Fast Ethernet LAN con un rate di 100 milioni di bit al secondo e un pacchetto di 10.000 bit, servono $(10.000) / (100.000.000)$ o 100 microsecondi affinché tutti i bit del pacchetto vengano trasmessi.

Ritardo di propagazione

Il ritardo di propagazione è il tempo che serve ad un bit per viaggiare dal punto A al punto B nel mezzo di trasmissione. Il ritardo di propagazione dipende dalla velocità di propagazione del mezzo, tipicamente data dalla velocità della luce ($3 * 10^8 m/s$ nell'etere, $2 * 10^8 m/s$ nel rame). Il ritardo di propagazione dipende anche dalla distanza tra sorgente e destinazione, ovvero dalla lunghezza del link. In altre parole il ritardo di propagazione è:

$$\text{Ritardo}_{pr} = (\text{Distanza}) / (\text{Velocità di propagazione})$$

Per esempio se la lunghezza di un collegamento in una WAN punto-punto è 2.000 metri e la velocità di propagazione dei bit nel cavo è $2 * 10^8 m/s$, allora il ritardo di propagazione è 10 microsecondi.

Ritardo di elaborazione

Il ritardo di elaborazione è il tempo che serve a un router o a un sistema terminale per ricevere un pacchetto dalla sua porta di input, rimuovere l'intestazione del pacchetto, eseguire una procedura di rilevamento errori e consegnare il pacchetto alla porta di output (nel caso di un router) o al protocollo del livello superiore (nel caso di un sistema terminale).

Ritardo di accodamento

Il ritardo di accodamento è tipico dei router. Come analizzeremo meglio in seguito, un router ha una coda di input collegata ad ognuna delle sue porte di ingresso per memorizzare i pacchetti in attesa di elaborazione; il router ha anche una coda di output collegata ad ognuna delle sue porte di uscita per memorizzare i pacchetti in attesa di trasmissione. Il ritardo di accodamento per un pacchetto si misura come il tempo in cui il pacchetto attende nella coda di input e in quella di output di un router. Possiamo paragonare la situazione ad un aeroporto trafficato. Alcuni aerei dovranno attendere l'autorizzazione d'atterraggio (ritardo di input); altri aspetteranno l'autorizzazione al decollo (ritardo di output).

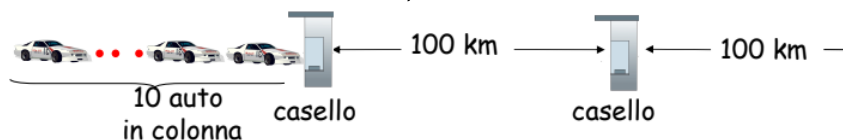
Questo tipo di ritardo può variare da pacchetto a pacchetto, fa uso di misure statistiche e dipende dal tasso di arrivo, dal rate, e dalla lunghezza dei pacchetti. Si definisce intensità di traffico come:

$$\text{Intensità di traffico} = \frac{\text{Lunghezza del pacchetto} \times \text{Tasso medio di arrivo dei pacchetti}}{\text{Rate di trasmissione}}$$

Se l'intensità di traffico è ~ 0 allora ci sarà poco ritardo; se $\rightarrow 1$ il ritardo si fa consistente; se > 1 si ha più lavoro in arrivo di quanto può essere svolto, con conseguente ritardo medio infinito.

Esempio 1.5.2 – Casello autostradale

a) Le automobili viaggiano (ossia "si propagano") alla velocità di 100 km/h. Il casello serve (ossia "trasmette") un'auto ogni 12 secondi (rate). Quanto tempo occorre perché le 10 auto in carovana abbiano superato il secondo casello? b) Le auto ora "si propagano" alla velocità di 1000 km/h. Al casello adesso occorre 1 min per servire ciascuna auto. Le prime auto arriveranno al secondo casello prima che le ultime auto della colonna lascino il primo?



- (a) Il tempo richiesto al casello per trasmettere l'intera colonna sull'autostrada è $10 / (5 \text{ auto/min}) = 2 \text{ min}$ (equivale al tempo di trasmissione). Il tempo richiesto a un'auto per viaggiare dall'uscita di un casello fino al casello successivo è $100 \text{ km} / (100 \text{ km/h}) = 1 \text{ hr}$ (corrisponde al tempo di propagazione). Il tempo che intercorre da quando l'intera colonna di vetture si trova di fronte al casello di partenza fino al momento in cui raggiunge quello successivo è la somma del ritardo di trasmissione e del ritardo di propagazione, ovvero 62 minuti.
- (b) Sì! Dopo 7 minuti, la prima auto sarà al secondo casello, e tre auto saranno ancora in coda davanti al primo casello.

Prodotto rate-ritardo

Il rate e il ritardo sono due metriche delle prestazioni di un link. Tuttavia, ciò che è realmente importante nella comunicazione dei dati è il loro prodotto.

Supponiamo di avere un link con rate di 1 bps e un ritardo di 5 secondi. Cosa rappresenta il prodotto rate-ritardo? Osservando la Figura 1.7, possiamo vedere che il prodotto rate-ritardo (1×5) è il numero massimo di bit che il link può contenere. Non possono esserci più di 5 bit per volta sul link.

Possiamo pensare al link tra due punti come a un tubo. La sezione trasversale del tubo rappresenta il rate e la lunghezza rappresenta il ritardo. Possiamo dire che il volume del tubo definisce il prodotto rate-ritardo.

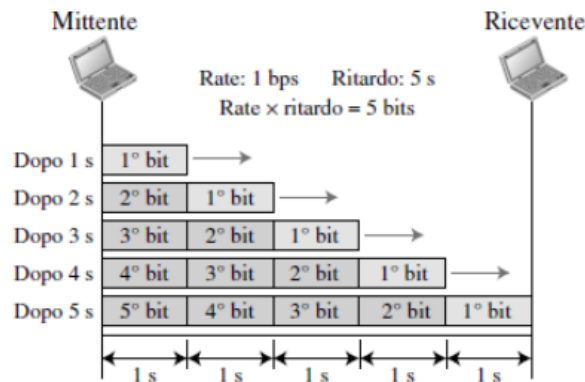


Figura 1.7: Prodotto rate ritardo

1.5.4 Perdita di pacchetti

Un'altra questione che influisce gravemente sulle prestazioni della comunicazione è il numero di pacchetti persi durante la trasmissione. Quando un router riceve un pacchetto mentre ne sta elaborando un altro, il pacchetto ricevuto deve essere memorizzato nel buffer di input in attesa del suo turno. Un router tuttavia ha un buffer di input di dimensioni limitate. Può arrivare il momento in cui il buffer è pieno e il pacchetto successivo deve essere saltato. L'effetto della perdita del pacchetto è che deve essere inviato nuovamente dal nodo precedente o dal sistema terminale che lo ha generato, il che a sua volta può creare un'eccedenza di dati e causare la perdita di altri pacchetti, oppure non essere ritrasmesso affatto.

Ma cosa significano effettivamente *packet delay* e *packet loss* nella "vera" Internet? Una traceroute (tracert) è un programma diagnostico che fornisce una misura del ritardo dalla sorgente a tutti i router lungo il percorso Internet punto-punto verso la destinazione. Per fare questo invia gruppi di tre pacchetti, ogni gruppo con tempo di vita incrementale (da 1 a n , massimo valore = 30) che raggiungeranno il router i ($i = 1, \dots, n$) sul percorso verso la destinazione. Il router i restituirà i pacchetti al mittente e il mittente calcola l'intervallo tra trasmissione e risposta.

L'output del comando traceroute presenta 6 colonne: il numero di router sulla rotta; il nome del router; l'indirizzo del router; il tempo di andata e ritorno 1° pacchetto; il tempo di andata e ritorno 2° pacchetto; il tempo di andata e ritorno 3° pacchetto. Il tempo di andata e ritorno (round trip time - RTT) include i 4 ritardi visti precedentemente. Può accadere (se c'è congestione o se si segue un percorso diverso) che il RTT del router n sia maggiore del RTT del router $n + 1$ a causa dei ritardi di accodamento. Se la sorgente non riceve risposta da un router intermedio (o ne riceve meno di 3) allora pone un asterisco al posto del tempo RTT.

1.6 Hardware e software

Si è fornita una panoramica della struttura e delle prestazioni di Internet, che è costituita da numerose reti di varie dimensioni interconnesse tramite opportuni dispositivi di comunicazione. Tuttavia per poter comunicare non è sufficiente assicurare questi collegamenti, ma è necessario utilizzare sia dell'hardware che del software. Nel capitolo successivo si descriverà come l'hardware e il software vengano coordinati tramite l'impiego di protocolli organizzati in livelli (layer).

Capitolo 2

Organizzazione dei protocolli in livelli

Un termine che si utilizza frequentemente quando si parla di Internet è protocollo.

Un **protocollo** definisce le regole che il mittente e il destinatario, così come tutti i sistemi intermedi coinvolti, devono rispettare per essere in grado di comunicare.

In situazioni particolarmente semplici potrebbe essere sufficiente un solo protocollo, in situazioni più complesse potrebbe essere opportuno suddividere i compiti fra più livelli (layer), nel qual caso è richiesto un protocollo per ciascun livello: si parla dunque di layering di protocolli, o di organizzazione (o strutturazione) dei protocolli in livelli.

2.1 Due scenari

Esempio 2.1.1 – Esempio di comunicazione

In questo esempio si sviluppano due scenari per comprendere meglio la necessità di organizzare i protocolli in livelli:

1. Si ipotizzi che Maria e Anna siano vicine di casa con molti interessi in comune. La comunicazione fra di loro avviene direttamente con un solo livello, faccia a faccia e nella stessa lingua.

Anche in questo semplice scenario è comunque necessario rispettare un certo insieme di regole. Per prima cosa Anna e Maria sanno che è buona norma salutarsi appena si incontrano. Secondo, sono coscienti di poter utilizzare uno stile di linguaggio adatto al loro livello di confidenza. Terzo, riconoscono di doversi astenere dal parlare mentre la controparte sta parlando. Quarto, sono coscienti che la conversazione dovrebbe essere un dialogo piuttosto che un monologo: entrambe le parti devono avere la possibilità di esprimersi. Quinto, sanno che è opportuno scambiare qualche convenevole prima di lasciarsi.

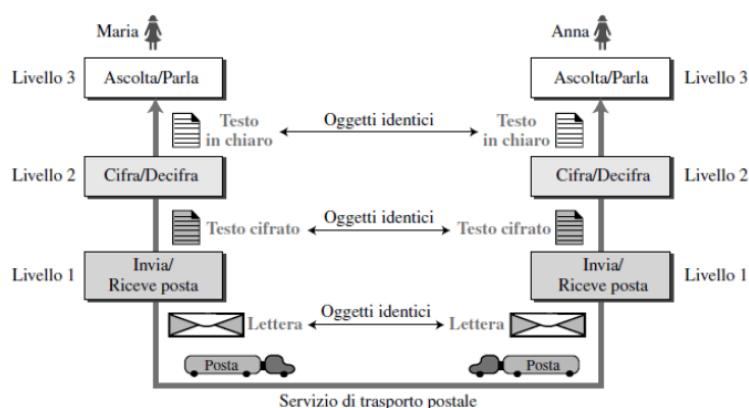


2. Si ipotizzi che ad Anna venga offerta una posizione migliore nella azienda in cui lavora, e che debba traslocare in altra sede in una città molto lontana da Maria. Le due amiche decidono dunque di continuare le loro conversazioni utilizzando il servizio postale, ma senza rischiare che altre persone possano venire a conoscenza delle loro idee se le loro lettere venissero intercettate. Di conseguenza si accordano per crittografare i messaggi: il mittente di una lettera la critta per renderla incomprensibile a un eventuale intruso, il destinatario della lettera la decritta per recuperare il messaggio originale. In questo

caso la comunicazione fra Anna e Maria avviene su tre livelli. Si ipotizza che le due amiche abbiano tre macchine ciascuna, per portare a termine i compiti a ciascun livello.

Si ipotizzi ora che Maria invii la prima lettera ad Anna: comunica quindi con la macchina al terzo livello. La macchina di terzo livello ascolta quanto dice Maria e crea il testo in chiaro, che viene passato alla macchina del secondo livello. Questa riceve il testo in chiaro, lo cripta e ottiene il testo cifrato che viene passato alla macchina del primo livello. Quest'ultima, prende il testo cifrato, lo mette in una busta, aggiunge gli indirizzi del mittente e del destinatario e la invia.

Dal lato di Anna la macchina al primo livello recupera la busta dalla cassetta delle lettere, toglie quindi il testo cifrato dalla busta e lo consegna alla macchina di secondo livello. Questa decrypta il messaggio ottenendo il testo in chiaro che passa alla macchina di terzo livello. Quest'ultima prende il testo in chiaro e lo legge.



La strutturazione in livelli dei protocolli consente di suddividere un compito complesso in compiti più semplici. Nel esempio si sarebbe potuto utilizzare una sola macchina per portare a termine il compito delle tre macchine. Tuttavia se Maria e Anna decidessero che la tecnica crittografica utilizzata dalla seconda macchina non fosse più adeguata per proteggere i loro segreti, dovrebbero sostituire l'intera macchina. Nella situazione descritta, invece, è sufficiente sostituire solamente la macchina di secondo livello, lasciando inalterate le altre due. Questa tecnica viene chiamata modularizzazione, che in questo caso significa sostanzialmente indipendenza dei livelli. Un modulo, o meglio un livello in questo contesto, può essere descritto come una black box con opportuni ingressi e uscite, senza doversi preoccupare della modalità con la quale i dati in ingresso vengano trasformati nei dati di uscita. Se due macchine forniscono lo stesso output dato il medesimo input, possono essere considerate equivalenti. Per esempio Anna e Maria potrebbero acquistare la macchina di secondo livello da due produttori differenti, purché entrambe le macchine producano il medesimo testo cifrato dal medesimo testo in chiaro e viceversa.

Uno dei vantaggi della strutturazione in livelli è la possibilità di separare i servizi offerti e la loro implementazione. Un livello deve essere in grado di utilizzare un insieme di servizi dal livello inferiore per offrire servizi al livello superiore, indipendentemente da come sia implementato. Per esempio Maria potrebbe decidere di non comprare la macchina del primo livello ed effettuare i compiti richiesti personalmente.

Un altro vantaggio della strutturazione in livelli, che non si evidenzia in questi semplici esempi ma che diverrà evidente quando si discuterà l'organizzazione in livelli di Internet, è che la comunicazione non coinvolge necessariamente sempre solo due sistemi terminali: spesso vengono infatti coinvolti dei sistemi intermedi (per esempio, router) che richiedono solo alcuni livelli. Se non si utilizzasse il layering i sistemi intermedi dovrebbero avere la medesima complessità dei sistemi terminali, cosa che renderebbe il sistema più costoso.

2.1.1 Principi della strutturazione in livelli

In questo paragrafo si presentano alcuni principi fondamentali relativi alla strutturazione in livelli. Innanzitutto, quando è richiesta una comunicazione bidirezionale, è necessario rendere ciascun livello capace di effettuare i due compiti opposti, uno per ciascuna direzione. Per esempio il compito

del terzo livello è ascoltare (in una direzione) e parlare (nell'altra direzione), il secondo livello deve essere in grado di crittografare e decrittografare, il primo livello deve poter inviare e ricevere la posta.

Il secondo principio da seguire riguarda gli oggetti in input/output sotto ciascun livello di entrambi i lati: devono essere identici. Per esempio l'oggetto sotto al livello 3 di entrambi i lati deve essere una lettera in chiaro, l'oggetto sotto al livello 2 deve essere una lettera cifrata. L'oggetto sotto al livello 1 di entrambi i lati deve essere una busta.

2.1.2 Collegamenti logici

Se si rispettano i due principi precedenti è possibile immaginare un collegamento logico fra i livelli, come mostrato nella Figura 2.1. Questo collegamento logico (virtuale) è come se consentisse una comunicazione diretta fra i pari livelli delle due parti.

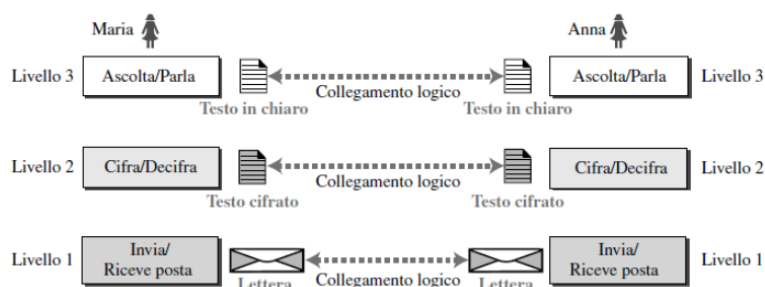


Figura 2.1: Collegamento logico tra i livelli

2.2 Lo stack protocollare TCP/IP

Chiariti i concetti di layering e di collegamento logico fra i livelli, è possibile introdurre l'architettura TCP/IP, ovvero la strutturazione in livelli dei protocolli utilizzati in Internet. Si tratta di una gerarchia di protocolli costituita da moduli interagenti, ciascuno dei quali fornisce funzionalità specifiche. Il termine gerarchia significa che ciascun protocollo di livello superiore è supportato dai servizi forniti dai protocolli di livello inferiore. Tale organizzazione dei protocolli viene chiamata pila o stack protocollare TCP/IP. Definita in origine in termini di quattro livelli software soprastanti a un livello hardware, la pila TCP/IP è oggi intesa come composta di cinque livelli.

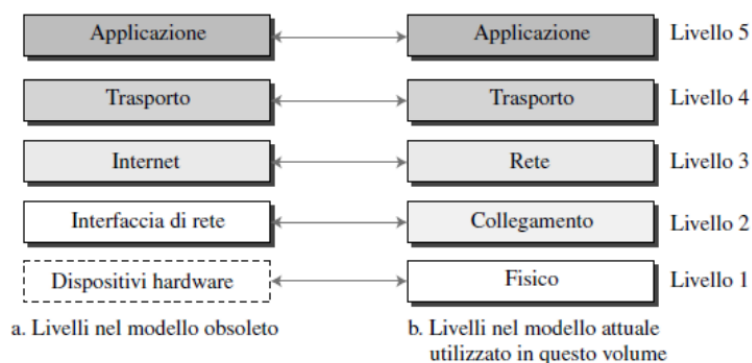
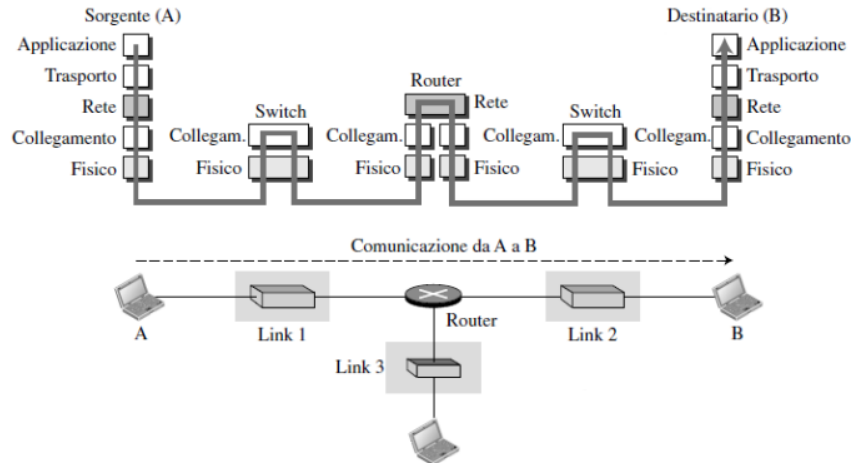


Figura 2.2: Livelli nello stack protocollare TCP/IP

2.2.1 Architettura a livelli

Esempio 2.2.1 – Comunicazione in una internet

Con l'obiettivo di illustrare come i vari livelli dello stack protocollare TCP/IP siano coinvolti nella comunicazione fra due host, si ipotizza che si vogliano utilizzare i protocolli TCP/IP in una piccola internet costituita da tre (link) LAN, ciascuna con uno switch di collegamento (data-link), interconnesse tramite un router.



Si supponga che l'host A comunichi con l'host B. L'host sorgente crea un messaggio a livello applicazione e lo trasmette attraversando verso il basso tutti e cinque i livelli in modo tale che possa essere fisicamente inviato all'host destinatario. Questo riceve il messaggio al livello fisico e lo consegna attraverso gli altri livelli all'applicazione destinataria.

Il router è coinvolto solamente per tre livelli: i livelli trasporto e applicazione non hanno ragione di esistere in un router che venga utilizzato esclusivamente per scopi di instradamento. Sebbene un router abbia sempre un livello rete, può utilizzare n combinazioni di livelli fisico e collegamento dove n è il numero di link ai quali è collegato. Infatti ciascun collegamento potrebbe utilizzare un proprio specifico protocollo di livello fisico o collegamento. Per esempio in figura il router ha tre collegamenti fisici, ma il messaggio inviato da A a B coinvolge due soli link. Ciascun link potrebbe utilizzare differenti protocolli a livello fisico e collegamento: il router deve poter ricevere un pacchetto dal Link 1 utilizzando una coppia di protocolli e instradarlo sul Link 2 utilizzando una seconda coppia di protocolli.

Uno switch di collegamento, invece, utilizza solo due livelli: collegamento e fisico. Sebbene ciascuno switch nell'illustrazione precedente abbia due collegamenti differenti, entrambi utilizzano lo stesso insieme di protocolli: uno switch di livello 2 quindi, a differenza di un router, utilizza un solo livello fisico e un solo livello di collegamento.

2.2.2 I livelli nello stack protocollare TCP/IP

L'impiego del concetto di collegamento logico semplifica la comprensione dei compiti di ciascun livello. Il compito dei livelli applicazione, trasporto e rete è di tipo end-to-end, ovvero fra i due host terminali coinvolti nella comunicazione. Invece il compito dei livelli di collegamento e fisico è hop-to-hop, dove un hop è un host o un router. In altre parole il dominio dei compiti dei tre livelli superiori è la internet, mentre il dominio dei compiti dei due livelli inferiori è il link.

Si ricordi che sotto ciascun livello di ogni dispositivo si indicano gli oggetti identici e nonostante il collegamento logico al livello rete sia fra i due host, gli oggetti sono identici unicamente fra due hop contigui, poiché il router potrebbe frammentare il pacchetto a livello rete e inviare più pacchetti di quanti ricevuti.

2.2.3 Descrizione dei livelli dello stack protocollare TCP/IP

Compreso il concetto di comunicazione logica, si descrivono brevemente i compiti di ciascun livello.

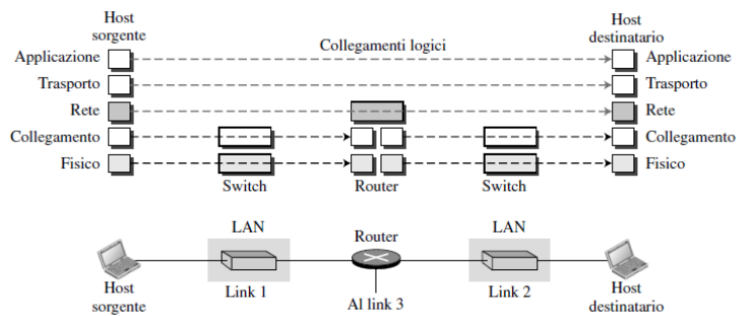


Figura 2.3: Collegamenti logici fra i livelli dello stack protocollare TCP/IP

Livello applicazione

Il collegamento logico fra i due livelli applicazione è end-to-end. I due livelli applicazione si scambiano dei messaggi come se fossero direttamente collegati. Naturalmente è necessario sapere che la comunicazione reale avviene attraversando tutti i livelli.

La comunicazione al livello applicazione avviene tra due processi (due programmi eseguiti a questo livello). Per comunicare, un processo invia una richiesta all'altro processo e ne riceve una risposta: la comunicazione processo-a-processo è il compito del livello applicazione.

I protocolli utilizzati in questo livello sono: HTTP (Hypertext Transfer Protocol), SMTP (Simple Mail Transfer Protocol), FTP (File Transfer Protocol), TELNET (Terminal Network), SSH (Secure Shell) e DNS (Domain Name System).

Livello trasporto

Anche la connessione logica a livello di trasporto è end-to-end. Il livello di trasporto nell'host sorgente riceve il messaggio dal livello applicazione, lo incapsula in un pacchetto di livello di trasporto (chiamato segmento) e lo invia, tramite la connessione logica (virtuale), al livello di trasporto dell'host destinatario.

I protocolli principali in questo livello sono: TCP (Transmission Control Protocol) e UDP (User Datagram Protocol).

Livello rete

Il compito del livello di rete (network) è quello di far arrivare i pacchetti dall'host sorgente all'host destinazione. La comunicazione a questo livello è host-to-host, ma dato che vi possono essere dei router nel tragitto fra la sorgente e la destinazione, questi hanno il compito di scegliere il miglior percorso (o rotta) per ciascun pacchetto. Si può affermare che il livello rete è responsabile della comunicazione host-to-host e dell'instradamento e inoltro dei pacchetti attraverso i possibili percorsi.

Il livello rete in Internet comprende il protocollo principale, l'IP (Internet Protocol), che definisce il formato del pacchetto chiamato datagramma.

Livello di collegamento

Si è visto come una internet sia costituita da più LAN e WAN interconnesse. Vi possono essere diversi percorsi (sequenze di link) anche parzialmente sovrapposti che un datagramma può utilizzare per viaggiare dalla sorgente alla destinazione. Quando un router ha determinato il link successivo da utilizzare, il livello di collegamento (data-link) si occupa di trasferire il pacchetto attraverso tale collegamento. E' anche possibile che vengano utilizzati protocolli differenti per ciascun tipo di link, in ogni caso il livello di collegamento si occupa del trasferimento del pacchetto attraverso il collegamento.

L'architettura TCP/IP non definisce alcun protocollo specifico per il livello di collegamento, ma supporta tutti i protocolli standard e proprietari. Qualsiasi protocollo che possa prendere in consegna il datagramma e trasferirlo attraverso il link è sufficiente per il livello rete. I pacchetti al livello di collegamento sono chiamati frame.

Livello fisico

Il livello fisico si occupa di trasferire i singoli bit di un frame attraverso il link. Sebbene il livello fisico sia il livello più basso nella famiglia di protocolli TCP/IP, la comunicazione fra due dispositivi al livello fisico è ancora una comunicazione logica, poiché vi è un ulteriore livello nascosto: il mezzo trasmissivo al di sotto del livello fisico. Di conseguenza i bit ricevuti in un frame dal livello di collegamento vengono in realtà trasformati e inviati attraverso il mezzo trasmissivo, ma è comunque possibile pensare che l'unità logica trasmessa fra due livelli fisici in due dispositivi sia il bit.

2.2.4 Incapsulamento e decapsulamento

L'incapsulamento/decapsulamento è un concetto molto importante relativo alla strutturazione in livelli di una rete.

La Figura 2.4 illustra l'incapsulamento nell'host sorgente, il decapsulamento nell'host destinataria ed entrambe le operazioni nel router.

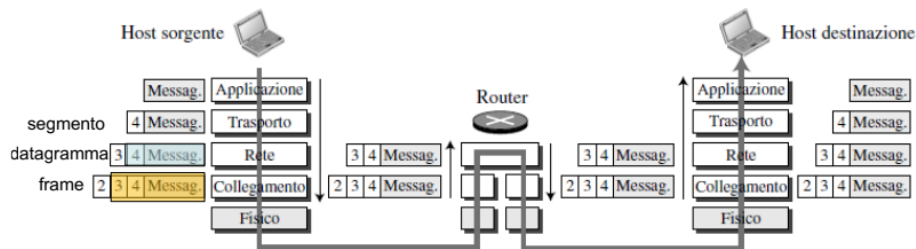


Figura 2.4: Incapsulamento e decapsulamento

La sorgente effettua esclusivamente l'incapsulamento.

1. A livello applicazione i dati da scambiare vengono chiamati messaggi. Un messaggio solitamente non contiene alcuna intestazione (header) o trailer (informazioni di controllo in coda). Il messaggio viene passato al livello trasporto.
2. Il livello trasporto considera il messaggio ricevuto come payload, ovvero come carico dati, che deve trasportare e gli aggiunge l'intestazione di livello trasporto che contiene varie informazioni per la gestione del pacchetto a quel livello (per esempio, gli identificatori dei programmi applicativi sorgente e destinatario, informazioni necessarie per la consegna end-to-end del messaggio ecc.). Questo procedimento di considerare il pacchetto ricevuto dal livello sovrastante come carico dati e aggiungergli un'intestazione è detto incapsulamento. Al livello di trasporto il risultato dell'incapsulamento è un segmento o un datagramma utente, a seconda del protocollo usato. Il livello trasporto passa quindi questo pacchetto al livello rete.
3. Il livello rete considera il pacchetto di livello trasporto passatogli come proprio carico dati e gli aggiunge un'intestazione. Questo contiene gli indirizzi degli host sorgente e destinatario e altre informazioni utili. Il risultato è un pacchetto di livello rete, chiamato datagramma, che viene dunque passato al livello di collegamento.
4. Il livello di collegamento considera il pacchetto di livello rete come payload e gli aggiunge la propria intestazione, che contiene l'indirizzo del livello di collegamento dell'host o del next-hop (il router). Il risultato è un pacchetto di livello di collegamento, chiamato frame, che viene quindi passato al livello fisico per la trasmissione.

Nel router si effettua sia l'incapsulamento che il decapsulamento poiché questi è collegato a due o più link.

1. L'insieme di bit ricevuti dal livello fisico viene passato al livello di collegamento, che decapsula il frame (ovvero rimuove l'header) e passa il datagramma al livello di rete.
2. Il livello di rete consulta solamente gli indirizzi di sorgente e destinatario nell'header del datagramma e consulta la propria tabella di instradamento per decidere il next-hop al quale consegnare il datagramma. Il datagramma viene quindi passato al livello di collegamento del link interessato.

3. Il livello di collegamento incapsula quindi il datagramma in un frame e lo passa al livello fisico per la trasmissione.

Ciascun livello nell'host destinatario rimuove la propria intestazione dal pacchetto ricevuto estraendone il payload; questo viene passato al protocollo del livello sovrastante, fino a quando il messaggio raggiunge il livello applicazione.

2.2.5 Indirizzamento

E' necessario presentare un altro concetto relativo alla gerarchia di livelli in Internet: l'indirizzamento. Come discusso in precedenza, questo modello prevede una comunicazione logica fra coppie di livelli. Qualsiasi comunicazione che coinvolge due parti richiede due indirizzi: l'indirizzo sorgente e l'indirizzo destinazione. Sebbene possa sembrare che siano necessarie cinque paia di indirizzi, un paio per livello, normalmente è sufficiente utilizzarne solo quattro poiché il livello fisico non richiede indirizzi: l'unità dati scambiata a livello fisico è infatti il bit, che decisamente non può avere un indirizzo.

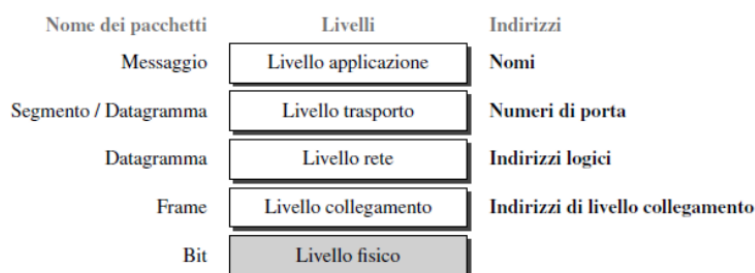


Figura 2.5: Indirizzamento nel modello TCP/IP

Come evidenziato in Figura 2.5 esiste una relazione fra livello, indirizzo utilizzato in quel livello e nome del relativo pacchetto.

2.2.6 Multiplexing e demultiplexing

Dato che lo stack protocollare TCP/IP prevede più protocolli nello stesso livello, è necessario eseguire il multiplexing alla sorgente e il demultiplexing alla destinazione. Multiplexing in questo caso significa che un protocollo di un livello può incapsulare (uno alla volta) i pacchetti ottenuti da più protocolli del livello immediatamente superiore; demultiplexing significa che un protocollo può decapsulare e consegnare i pacchetti a più protocolli del livello immediatamente superiore.

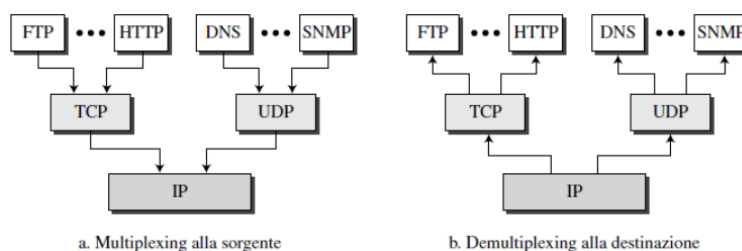


Figura 2.6: Multiplexing e demultiplexing

Per poter effettuare le operazioni di multiplexing e demultiplexing un protocollo deve avere un campo nel proprio header per identificare a quale protocollo appartengano i pacchetti incapsulati.

2.3 Il modello OSI

Sebbene nel contesto di Internet il modello TCP/IP sia quello più diffuso, esso non è l'unico modello esistente. L'ISO (International Organization for Standardization), fondata nel 1947, è un'organizzazione internazionale dedicata alla definizione di standard universalmente accettati, organizzazione

di cui fanno parte oltre tre quarti delle nazioni. Il modello OSI (Open Systems Interconnection) è uno standard ISO, introdotto alla fine degli anni '70, che concerne tutti gli aspetti della reti.

Un sistema aperto è un insieme di protocolli che consente a due sistemi qualsiasi di comunicare indipendentemente dalla loro (differente) architettura sottostante. Lo scopo del modello OSI è facilitare la comunicazione fra sistemi di comunicazione differenti senza dovere imporre vincoli alla logica dell'hardware e del software utilizzati. Il modello OSI non è un protocollo, ma un modello per comprendere e progettare architetture di rete flessibili, robuste e interoperabili. E' stato dunque ideato come base per lo sviluppo di protocolli dello stack OSI.

Il modello OSI è un framework stratificato per il progetto di sistemi di rete che consentono la comunicazione fra qualsiasi tipo di dispositivo. E' suddiviso in sette livelli ognuno dei quali descrive una parte del meccanismo di trasferimento dell'informazione attraverso una rete.

Confrontando il modello OSI con il modello TCP/IP si scopre che nell'architettura TCP/IP mancano i livelli sessione e presentazione. Questi due livelli vengono solitamente considerati parte del livello applicazione di TCP/IP.

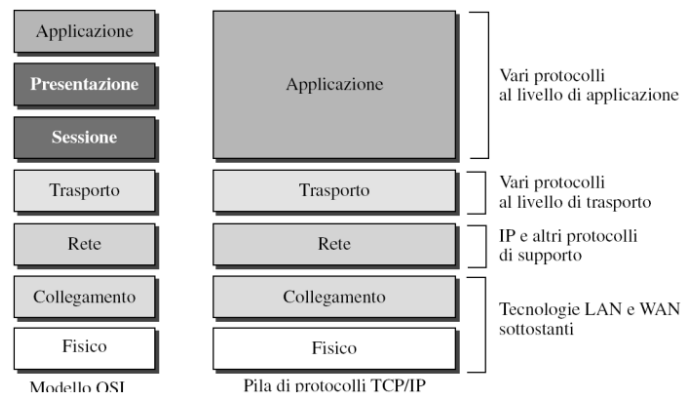


Figura 2.7: TCP/IP e il modello OSI

2.3.1 L'insuccesso del modello OSI

Il modello OSI venne pubblicato dopo la definizione dell'architettura TCP/IP e la maggior parte degli esperti arrivò a ipotizzare che avrebbe completamente rimpiazzato TCP/IP. In realtà questo non avvenne per varie ragioni. Per prima cosa l'OSI venne pubblicato quando il TCP/IP era già ampiamente diffuso e gli si erano già dedicate notevoli risorse: un'eventuale sostituzione avrebbe rappresentato un costo notevole. La seconda ragione è che alcuni livelli nel modello OSI, come presentazione e sessione, non sono mai stati completamente specificati: il software corrispondente non è dunque mai stato completamente sviluppato. Infine anche quando alcune organizzazioni implementarono l'OSI in qualche applicazione, non riuscirono a dimostrare delle prestazioni tali da convincere le autorità di Internet a sostituire il TCP/IP con il modello OSI.

Capitolo 3

La storia di Internet

Questo capitolo vuole mostrare come Internet sia evoluta da una rete privata a una rete globale in meno di quarant'anni.

3.1 La storia iniziale

Prima del 1960 esistevano già alcune reti di telecomunicazione, come le reti telegrafiche e le reti telefoniche. Queste reti erano adatte a un tipo di comunicazione a velocità costante: era necessario stabilire una connessione fra due utenti prima di poter trasmettere il messaggio codificato (telegrafo) o la voce (telefonia). Una rete di computer, invece, deve essere in grado di gestire traffico bursty (a raffica), ovvero dati trasmessi a varie velocità in tempi differenti. Il mondo attendeva l'invenzione delle reti a commutazione di pacchetto.

La teoria della commutazione di pacchetto per il traffico bursty venne introdotta per la prima volta da Leonard Kleinrock nel 1961 al MIT. Contemporaneamente altri due ricercatori, Paul Baran al Rand Institute e Donald Davies al National Physical Laboratory in Inghilterra pubblicarono alcuni articoli sulle reti a commutazione di pacchetto.

3.2 ARPANET

A metà degli anni '60 i calcolatori centrali dei centri di ricerca erano dei dispositivi a sé stanti. Calcolatori prodotti da industrie diverse non erano in grado di comunicare tra loro. L'Advanced Research Projects Agency (ARPA) del Department of Defense (DOD) era alla ricerca di un metodo per connettere diversi calcolatori, in modo che i ricercatori finanziati con i suoi fondi potessero facilmente condividere i risultati del loro lavoro, riducendo così i costi ed evitando di duplicare gli sforzi.

Nel 1967, a un incontro dell'Association for Computing Machinery (ACM), l'ARPA presentò le sue idee per ARPANET, una piccola rete di calcolatori connessi tra loro. L'idea di base era la seguente: ogni calcolatore sarebbe stato collegato a una macchina dedicata, chiamata Interface Message Processor (IMP). Gli IMP, a loro volta, sarebbero stati collegati tra loro. Ogni IMP doveva essere capace di comunicare con gli altri IMP e con il calcolatore host a esso collegato.

Nel 1969 ARPANET divenne una realtà. I quattro nodi costituiti dalla University of California a Los Angeles (UCLA), dalla University of California a Santa Barbara (UCSB), dallo Stanford Research Institute (SRI) e dalla University of Utah, furono collegati attraverso gli IMP e divennero una rete. Il software che realizzava la comunicazione tra i vari calcolatori host fu chiamato Network Control Protocol (NCP).

3.3 Nascita della rete Internet

Nel 1972 Vint Cerf e Bob Kahn, entrambi parte del nucleo del gruppo ARPANET, collaborarono a ciò che chiamarono l'Internetting Project. Il loro scopo era quello di collegare diverse reti in modo che un host di una delle reti potesse comunicare con un altro host di un'altra rete. I problemi da risolvere furono numerosi: dimensione dei pacchetti, interfacce e frequenze di trasmissione diverse, ma anche differenti requisiti di attendibilità.

3.4 Internet oggi

Siamo tutti testimoni della rapidissima evoluzione sia dell'infrastruttura che delle applicazioni di Internet, divenuta oggi una rete che fornisce servizi a livello mondiale. L'incessante sviluppo di nuove applicazioni è il motivo principale della sua diffusione.

Gli anni '90 hanno conosciuto un'esplosione delle applicazioni basate sul World Wide Web (WWW), inventato al CERN da Tim Berners-Lee. Questa invenzione ha aperto Internet alle applicazioni commerciali.

I recenti sviluppi nelle applicazioni multimediali come il Voice Over IP (telefonia su Internet), il video over IP (Skype), la condivisione di video (YouTube) e la televisione su Internet (PPLive) hanno moltiplicato il numero di utenti e il tempo che questi spendono sulla rete.

Le applicazioni di rete peer-to-peer costituiscono un nuovo settore con notevoli potenzialità di sviluppo.

3.5 Gli standard della rete Internet

Uno standard Internet è una specifica che è stata rigorosamente esaminata e controllata, utile e accettata da chi utilizza la rete Internet. È un insieme di regole formalizzate che devono necessariamente essere seguite. Esiste una procedura rigorosa attraverso la quale una specifica diviene uno standard Internet. Il primo stadio di una specifica è quello di bozza Internet (Internet draft). Una bozza Internet è un documento preliminare senza alcuno status ufficiale, la cui vita media è dell'ordine di sei mesi. Su suggerimento delle autorità che regolano la rete Internet una bozza può essere pubblicata come Request for Comments (RFC, richiesta di commenti). Una RFC attraversa, durante la sua vita, sei diversi stadi o livelli di maturità: proposta di standard, bozza di standard, standard Internet, livello storico, sperimentale e informativo.

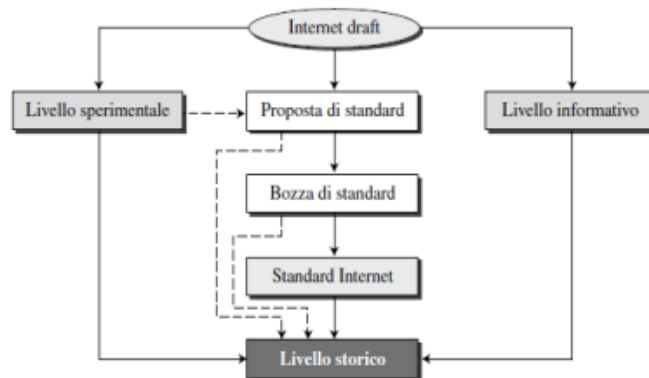


Figura 3.1: Studi di maturazione di una RFC

3.6 Amministrazione della rete Internet

La rete Internet, nata sostanzialmente nel mondo della ricerca, si è ampiamente sviluppata e ha raggiunto una base di utenti molto vasta con significative attività commerciali. Numerosi gruppi che coordinano aspetti diversi della rete Internet hanno guidato la sua crescita e il suo sviluppo. Tra questi troviamo: Internet Society (ISOC), Internet Architecture Board (IAB), Internet Engineering Task Force (IETF), Internet Assigned Numbers Authority (IANA) e Internet Corporation for Assigned Names and Numbers (ICANN), Network Information Center (NIC).

Capitolo 4

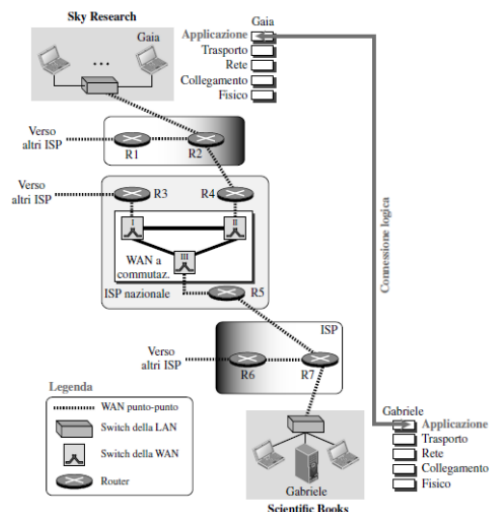
Livello applicazione

4.1 Introduzione

Il livello applicazione fornisce servizi all'utente. La comunicazione è fornita per mezzo di una connessione logica: questo significa che i livelli applicazione nei due lati della comunicazione agiscono come se esistesse un collegamento diretto (immaginario) attraverso il quale poter inviare e ricevere messaggi.

Esempio 4.1.1 – Connessione logica a livello applicazione

La figura illustra uno scenario nel quale un ricercatore che lavora nell'azienda Sky Research deve ordinare un libro relativo alle proprie ricerche dalla libreria online Scientific Books. La connessione logica avviene tra il livello applicazione di un client in Sky Research e il livello applicazione di un server nella libreria. I due utenti (Gaia e Gabriele) possono immaginare che tra di essi esista un canale logico bidirezionale, attraverso il quale si possono inviare dei messaggi. La comunicazione reale, tuttavia, avviene attraverso più dispositivi (Gaia, R2, R4, R5, R7 e Gabriele) e vari canali fisici.



4.1.1 L'offerta dei servizi

Tutte le reti di comunicazione sviluppate prima di Internet avevano l'obiettivo di fornire servizi ai propri utenti; la maggior parte di queste reti, tuttavia, era inizialmente progettata per offrire un servizio specifico.

Anche Internet venne originalmente progettata con lo stesso scopo: fornire servizi agli utenti in tutto il mondo. Un nuovo protocollo che viene aggiunto ad un dato livello deve essere progettato in modo tale da utilizzare i servizi offerti da uno dei protocolli di livello inferiore. Se un protocollo viene eliminato da un livello, è necessario apportare le modifiche necessarie ai protocolli del livello immediatamente superiore, che utilizzavano i servizi del protocollo rimosso.

Il livello applicazione, tuttavia, ha caratteristiche peculiari rispetto agli altri protocolli della pila, visto che si trova nel livello più alto. I protocolli di questo livello non forniscono servizi ad altri protocolli di livello superiore, ma utilizzano solamente i servizi offerti dai protocolli del livello inferiore (quello di trasporto). Questo significa che eliminare protocolli da questo livello è un'operazione facile, così come aggiungere nuovi protocolli purché questi siano in grado di utilizzare i servizi di uno dei protocolli di livello trasporto.

Dato che il livello applicazione è l'unico che fornisce servizi agli utenti di Internet, la sua flessibilità consente di aggiungere nuovi protocolli con estrema facilità.

Protocolli standard e non standard

Per agevolare le operazioni di Internet, i protocolli utilizzati nei primi quattro livelli dello stack TCP/IP devono essere standardizzati e opportunamente documentati. Per una maggiore flessibilità, tuttavia, i protocolli di livello applicazione possono essere sia standard che non standard.

- Esistono diversi protocolli di livello applicazione che sono stati standardizzati e documentati dagli enti responsabili della gestione di Internet. Ogni protocollo standard è costituito da una coppia di programmi che interagiscono con l'utente e con il livello trasporto per fornire un servizio specifico.
- E' possibile creare un'applicazione non standard scrivendo due programmi che forniscono servizi agli utenti, facendo uso dei servizi di trasporto. La possibilità di creare protocolli non standard che non richiedono alcuna approvazione: un'azienda privata può sviluppare il proprio protocollo specifico di livello applicazione per comunicare con i propri uffici sparsi nel mondo, utilizzando i servizi offerti dai primi quattro livelli dello stack TCP/IP, senza essere costretta ad utilizzare alcun programma applicativo standard.

4.1.2 Paradigmi del livello applicazione

Dovrebbe ormai essere chiaro che per utilizzare Internet sono necessari due programmi che interagiscono l'uno con l'altro: uno eseguito su un certo computer in qualche parte del mondo, l'altro eseguito su un secondo computer in qualsiasi altro luogo. I due programmi hanno la necessità di scambiarsi messaggi sfruttando l'infrastruttura di Internet. I due programmi applicativi devono essere entrambi in grado di richiedere e offrire servizi, oppure ciascuno deve occuparsi di uno dei due compiti? Con il passare del tempo, sono stati sviluppati due approcci (paradigmi) diversi: il paradigma client/server e quello peer-to-peer.

Paradigma client/server

L'approccio tradizionale è chiamato client/server, ed è stato il paradigma più utilizzato fino a pochi anni fa. Con questo paradigma il fornitore di servizi è un programma applicativo chiamato processo server, sempre in esecuzione in attesa che un altro programma applicativo, chiamato processo client, apra una connessione via Internet e richieda un servizio. Vi sono solitamente un numero limitato di processi server pronti ad offrire uno specifico servizio, ma numerosi client che richiedono servizi da questi server. Il processo server deve essere in esecuzione permanente, il processo client viene mandato in esecuzione solo quando ha bisogno di ottenere un servizio.

Sebbene la comunicazione nel paradigma client/server avvenga fra due programmi applicativi, il ruolo di ciascun programma è totalmente differente. Infatti non è possibile eseguire un client come programma server e viceversa.

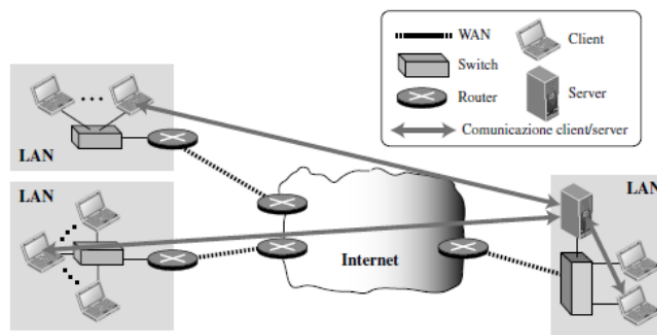


Figura 4.1: Esempio di architettura client/server

Un problema di questo paradigma è che il carico di comunicazione risulta concentrato sul server, che deve quindi utilizzare un computer con molte risorse di comunicazione e computazione.

Un secondo problema è che deve esservi un fornitore di servizi disponibile a sobbarcarsi il costo della complessa infrastruttura necessaria per offrire un particolare servizio. Affinché questo possa avvenire, il servizio dovrà quindi necessariamente fornire un qualche tipo di ritorno economico.

Numerosi servizi tradizionali utilizzano ancora questo paradigma, tra cui il World Wide Web (WWW) con il protocollo associato HTTP (HyperText Transfer Protocol), il File Transfer Protocol (FTP), il protocollo Secure Shell (SSH), la posta elettronica e così via.

4.2 WEB e HTTP

Fino agli anni '90 Internet veniva principalmente utilizzata da ricercatori, docenti e studenti universitari per raggiungere host remoti, trasferire file, ricevere e inviare notizie e per la posta elettronica. Poi, nei primi anni '90, comparve sulla scena una nuova importante applicazione: il World Wide Web. Il Web è stata la prima applicazione Internet che ha catturato l'attenzione del pubblico, cambiando profondamente il modo di interagire all'interno e all'esterno degli ambienti di lavoro. Forse ciò che attira di più gli utenti è che il Web opera su richiesta (on demand): si può avere ciò che si vuole, quando lo si vuole. Inoltre, è facilissimo per tutti rendere disponibili informazioni sul Web: chiunque può diventare editore a costi estremamente bassi e i collegamenti ipertestuali e i motori di ricerca ci aiutano a navigare in un oceano di siti.

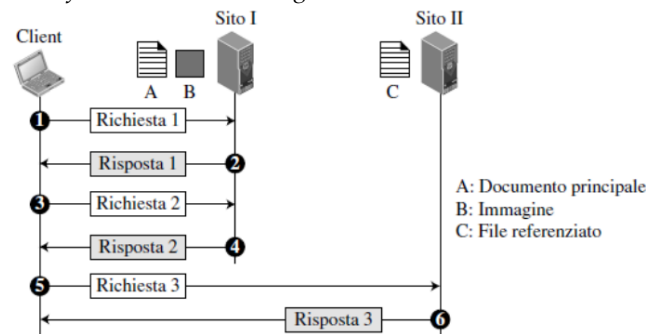
4.2.1 Panoramica Web

Architettura

Il Web è oggi un servizio client/server distribuito, nel quale un client che utilizza un browser può accedere a un servizio offerto da un server. Il servizio fornito è tuttavia distribuito su numerose locazioni fisiche chiamate siti. Ogni sito contiene uno o più documenti chiamati pagine web, ciascuna delle quali può contenere collegamenti (o riferimenti) ad altre pagine web nello stesso sito o in altri siti.

Esempio 4.2.1 – Architettura Web

Si supponga di voler ottenere un documento scientifico che contiene un riferimento a un altro documento testuale e un riferimento a un'immagine.



Il documento principale e l'immagine sono memorizzati in due file separati nello stesso sito (file A e file B), mentre il documento testuale referenziato è memorizzato in un altro file in un altro sito (file C). Poiché si hanno tre file differenti, sono necessarie tre transazioni per poter vedere il documento completo. La prima transazione (richiesta/risposta) consente di ottenere una copia del documento principale (file A), che contiene dei riferimenti (puntatori) agli altri due file. Quando viene ottenuta una copia del documento principale, l'utente può attivare il collegamento all'immagine per innescare la seconda transazione e ottenere una copia dell'immagine (file B). Se l'utente desidera vedere il contenuto del file di testo collegato, può attivarne il collegamento facendo click con il mouse sul relativo link, innescando la terza transazione per ottenere una copia del file C. Si noti che nonostante i file A e B siano memorizzati nello stesso sito, si tratta comunque di file indipendenti con nomi e indirizzi differenti. Sono necessarie due transazioni per poterli ottenere.

Esistono numerosi browser, capaci di interpretare e visualizzare i documenti Web, che utilizzano sostanzialmente la medesima architettura. Qualsiasi browser è composto solitamente da tre compo-

nenti: il controllore, il modulo che implementa il protocollo client e gli interpreti. Il controllore riceve l'input dalla tastiera o dal mouse e sfrutta i protocolli client per accedere al documento. Ottenuto il documento, il controllore utilizza uno degli interpreti per visualizzarlo sullo schermo. Il protocollo client può essere uno dei protocolli che verranno descritti in seguito, come HTTP o FTP. Gli interpreti possono gestire HTML, Java, JavaScript o altro, a seconda del tipo di documento. Internet Explorer, Google Chrome e Mozilla Firefox sono esempi di browser.

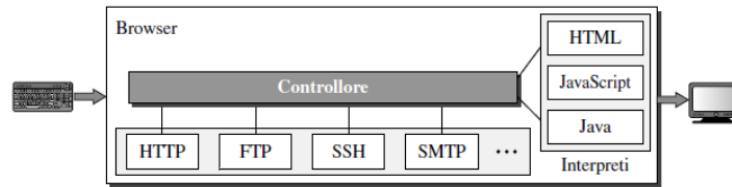


Figura 4.2: Architettura generale di un browser

URL (Uniform Resource Locator)

Una pagina Web, essendo un file, deve avere un identificatore univoco per distinguerla dalle altre pagine. Per identificare una pagina Web sono necessari tre identificatori: host, porta e percorso. Tuttavia, prima di definire la pagina Web è necessario indicare al browser quale applicazione client/server si desidera utilizzare, specificando il cosiddetto protocollo. Per identificare una pagina Web sono quindi necessari quattro parametri: il primo indica lo strumento che deve essere utilizzato per recuperare la pagina Web, la combinazione degli altri tre identifica l'oggetto ricercato (pagina Web).

- Il primo identificatore è un'abbreviazione che indica il protocollo client/server necessario per il trasferimento. Nella maggior parte dei casi si tratta di HTTP, ma è possibile utilizzare anche altri protocolli come FTP (File Transfer Protocol).
- L'identificatore dell'host può essere l'indirizzo IP o il nome univoco assegnato al server. Gli indirizzi IP possono essere espressi in notazione decimale puntata (per esempio 64.23.56.17). Il nome è solitamente il nome di dominio che identifica univocamente l'host.
- La porta, un intero di 16 bit, è predefinito per molte applicazioni client/server standard. Per esempio se si utilizza il protocollo HTTP per accedere alle pagine Web, il numero di porta è 80. E' possibile utilizzare una porta differente, ma in questo caso è necessario indicarne esplicitamente il numero.
- Il percorso identifica la posizione e il nome del file nel filesystem. Il formato di questo identificatore dipende normalmente dal sistema operativo utilizzato. In Unix un percorso è una sequenza di nomi di directory seguita dal nome del file, tutti separati da slash (il carattere "/").

Questi quattro elementi sono raggruppati assieme sotto forma di URL (Uniform Resource Locator), come ad esempio: `protocol://host/path` è utilizzato nella maggior parte dei casi quando la porta utilizzata è quella standard e `protocol://host:porta/path` è utilizzato quando è necessario specificare il numero di porta.

Documenti Web

I documenti Web possono essere raggruppati in tre categorie: statici, dinamici e attivi.

- I documenti statici sono documenti dal contenuto predeterminato che vengono memorizzati in un server. Il client può solamente ottenere una copia di tali documenti. In altri termini, i contenuti sono determinati al momento della loro creazione, non al momento del loro utilizzo.
- Un documento dinamico viene creato dal server Web alla ricezione di una richiesta di un browser. All'arrivo della richiesta, il server esegue un programma applicativo o uno script che crea il documento dinamico: il server restituisce al browser l'output del programma o dello script eseguito. Dato che viene creato un nuovo documento per ogni richiesta, il suo contenuto può variare da una richiesta all'altra (ad esempio la richiesta di un documento che contiene data e ora).

- In numerose applicazioni è utile poter eseguire programmi o script sul computer client: si tratta dei cosiddetti documenti attivi. Si supponga per esempio che si voglia eseguire un programma che interagisce frequentemente con l'utente. Tale programma deve necessariamente essere eseguito sul sito client dove ha luogo l'interazione. Quando un browser richiede un documento attivo, il server invia una copia del documento che incorpora script o programmi, che verranno quindi eseguiti nel browser (ovvero "lato client").

4.2.2 HTTP (HyperText Transfer Protocol)

HTTP (HyperText Transfer Protocol) definisce in che modo i client web richiedono le pagine ai web server e come questi ultimi le trasferiscono ai client. Quando l'utente richiede una pagina web (per esempio, cliccando su un collegamento ipertestuale), il browser invia al server messaggi di richiesta HTTP per gli oggetti nella pagina. Il server riceve le richieste e risponde con messaggi di risposta HTTP contenenti gli oggetti.

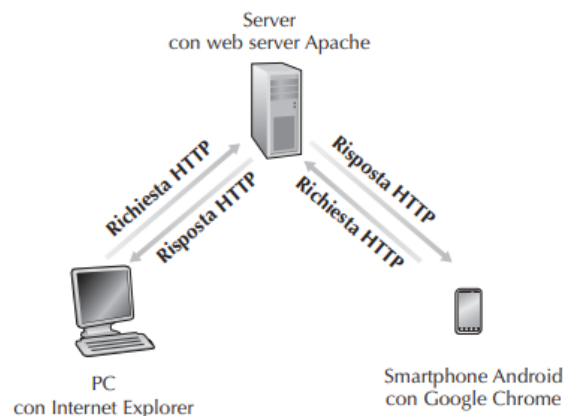


Figura 4.3: Comportamento richiesta-risposta di HTTP

HTTP utilizza TCP (anziché UDP) come protocollo di trasporto. Il client HTTP per prima cosa determina l'URL ed estrae host e filename in modo da iniziare una connessione TCP con il server. Una volta stabilita, il client invia richieste e riceve risposte HTTP, analogamente il server riceve richieste e invia messaggi di risposta.

4.2.3 HTTP con connessioni persistenti e non persistenti

In molte applicazioni per Internet, client e server comunicano per un lungo periodo di tempo, con il client che inoltra una serie di richieste e il server che risponde a ciascuna di esse. A seconda dell'applicazione e del suo impiego la serie di richieste potrebbe essere effettuata in sequenza, periodicamente a intervalli regolari o in maniera intermittente. Quando tale interazione client-server ha luogo su TCP, gli sviluppatori dell'applicazione devono prendere una decisione importante: ciascuna coppia richiesta/risposta deve essere inviata su una connessione TCP separata o devono essere inviate tutte sulla stessa connessione TCP? Nel primo approccio si dice che l'applicazione usa connessioni non persistenti, mentre nel secondo usa connessioni persistenti.

Connessioni non persistenti

Esempio 4.2.2 – Connessioni non persistenti

Seguiamo passo dopo passo il trasferimento di una pagina web dal server al client nel caso di connessioni non persistenti. Supponiamo che la pagina consista di un file HTML principale e di 10 immagini JPEG, e che tutti gli undici oggetti risiedano sullo stesso server. Ipotizziamo che l'URL del file HTML principale sia `http://www.someSchool.edu/someDepartment/home.index`.

1. Il processo client HTTP inizializza una connessione TCP con il server `www.someSchool.edu` sulla porta 80, che è la porta di default per HTTP. Il server

HTTP all'host `www.someSchool.edu` in attesa di una connessione TCP alla porta 80 "accetta" la connessione e avvisa il client.

2. Il client HTTP trasmette un messaggio di richiesta (con l'URL) nella socket della connessione TCP. Il messaggio indica che il client vuole l'oggetto `someDepartment/home.index`.
3. Il processo server HTTP riceve il messaggio di richiesta, recupera l'oggetto `/someDepartment/home.index` dalla memoria (centrale o di massa), lo incapsula in un messaggio di risposta HTTP che viene inviato al client attraverso la socket.
4. Il processo server HTTP comunica a TCP di chiudere la connessione.
5. Il client HTTP riceve il messaggio di risposta. La connessione TCP termina. Il messaggio indica che l'oggetto incapsulato è un file HTML. Il client estrae il file dal messaggio di risposta, esamina il file HTML e trova i riferimenti ai 10 oggetti JPEG.
6. Vengono quindi ripetuti i primi cinque passi per ciascuno degli oggetti JPEG referenziati.

I passi appena riportati illustrano l'utilizzo di connessioni non persistenti, in cui ogni connessione TCP viene chiusa dopo l'invio dell'oggetto da parte del server: vale a dire che ciascuna trasporta soltanto un messaggio di richiesta e un messaggio di risposta. Pertanto, in questo esempio, quando l'utente richiede una pagina web, vengono generate 11 connessioni TCP.

Prima di procedere facciamo un calcolo approssimativo per stimare l'intervallo di tempo che intercorre tra la richiesta di un file HTML da parte del client e il momento in cui l'intero file viene ricevuto dal client. A questo scopo, definiamo il round-trip time (RTT), che rappresenta il tempo impiegato da un piccolo pacchetto per viaggiare dal client al server e poi tornare al client. RTT include i ritardi di propagazione, di accodamento nei router e nei commutatori intermedi nonché di elaborazione del pacchetto.

Consideriamo ora che cosa succede quando un utente fa click con il mouse su un collegamento ipertestuale. Questa azione fa sì che il browser inizializzi una connessione TCP con il web server. Ciò comporta un handshake (letteralmente, "stretta di mano") a tre vie (three-way handshake): il client invia un piccolo segmento TCP al server, quest'ultimo manda una conferma per mezzo di un piccolo segmento TCP. Infine, il client dà anch'esso una conferma di ritorno al server. Le prime due parti dell'handshake a tre vie richiedono un RTT. Dopo il loro completamento, il client invia un messaggio di richiesta HTTP combinato con la terza parte dell'handshake. Quando il messaggio di richiesta arriva al server, quest'ultimo inoltra il file HTML sulla connessione TCP. La richiesta-risposta HTTP consuma un altro RTT. Pertanto, il tempo di risposta totale è, approssimativamente, di due RTT, più il tempo di trasmissione da parte del server del file HTML.

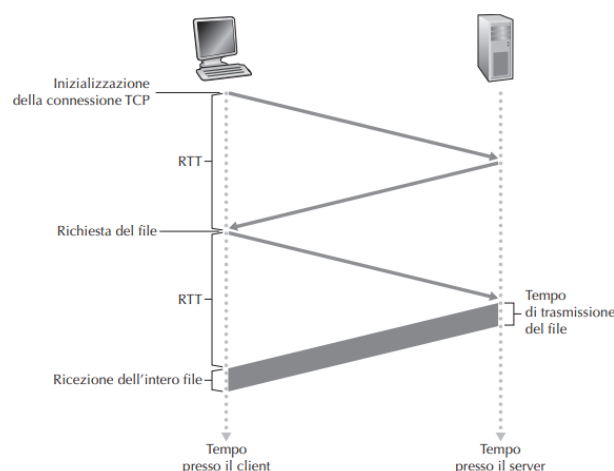


Figura 4.4: Calcolo approssimato del tempo necessario per richiedere e ricevere un file HTML

Connessioni persistenti

Le connessioni non persistenti presentano alcuni limiti: il primo è che per ogni oggetto richiesto occorre stabilire e mantenere una nuova connessione. Per ciascuna di queste connessioni si devono allocare buffer e mantenere variabili TCP sia nel client sia nel server. Ciò pone un grave onere sul web server, che può dover servire contemporaneamente richieste provenienti da centinaia di diversi client. In secondo luogo, come abbiamo appena descritto, ciascun oggetto subisce un ritardo di consegna di due RTT, uno per stabilire la connessione TCP e uno per richiedere e ricevere un oggetto.

Con HTTP nelle connessioni persistenti il server lascia la connessione TCP aperta dopo l'invio di una risposta, per cui le richieste e le risposte successive tra gli stessi client e server possono essere trasmesse sulla stessa connessione. In particolare, non solo il server può inviare un'intera pagina web (nell'esempio, il file HTML principale e le 10 immagini) su una sola connessione TCP permanente, ma può anche spedire allo stesso client più pagine web, in questo modo è necessario solo un solo RTT di connessione per tutti gli oggetti referenziati (+ un RTT per ogni oggetto ricevuto dal server). In generale, il server HTTP chiude la connessione quando essa rimane inattiva per un dato lasso di tempo.

4.2.4 Formato dei messaggi HTTP

Le specifiche HTTP includono la definizione dei due formati dei messaggi HTTP, di richiesta e di risposta.

Messaggio di richiesta

Esempio 4.2.3 – Messaggio di richiesta HTTP

Ecco un tipico messaggio di richiesta HTTP:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Notiamo innanzitutto che il messaggio è scritto in testo ASCII, in modo che l'utente sia in grado di leggerlo. Inoltre, notiamo che consiste di cinque righe, ciascuna seguita da un carattere di ritorno a capo (carriage return) e un carattere di nuova linea (line feed). L'ultima riga è seguita da una coppia di caratteri di ritorno a capo e nuova linea aggiuntivi.

La prima riga è detta riga di richiesta (request line) e quelle successive righe di intestazione (header lines). La riga di richiesta presenta tre campi: il campo metodo, il campo URL e il campo versione di HTTP. Il campo metodo può assumere diversi valori, tra cui GET, POST, HEAD, PUT e DELETE. Il browser, che implementa la versione HTTP/1.1, sta richiedendo l'oggetto /somedir/page.html.

La riga Host: www.someschool.edu specifica l'host su cui risiede l'oggetto. Includendo la linea di intestazione Connection: close, il browser sta comunicando al server che non si deve occupare di connessioni persistenti, ma vuole che questi chiuda la connessione dopo aver inviato l'oggetto richiesto. La riga di intestazione User-agent: specifica il tipo di browser che sta effettuando la richiesta al server, in questo caso Mozilla/5.0, un browser Firefox. Infine, Acceptlanguage: indica che l'utente preferisce ricevere una versione in francese dell'oggetto se disponibile; altrimenti, il server dovrebbe inviare la versione di default.

A questo punto concentriamoci sul formato generale di un messaggio di richiesta.

Abbiamo visto che i tipi di metodi della riga di richiesta sono: GET, POST, HEAD, PUT e DELETE.

- Il metodo GET è usato quando il client vuole scaricare un documento dal server. Il documento richiesto è specificato nell'URL. Il server normalmente risponde con il documento richiesto nel corpo del messaggio di risposta.
- Il metodo POST è usato per fornire input al server (contenuto dei campi di un form) e, l'input arriva al server nel corpo dell'entità. In realtà le richieste generate con il form non usano necessariamente il metodo POST. Anzi, i form HTML usano spesso il metodo GET e includono i

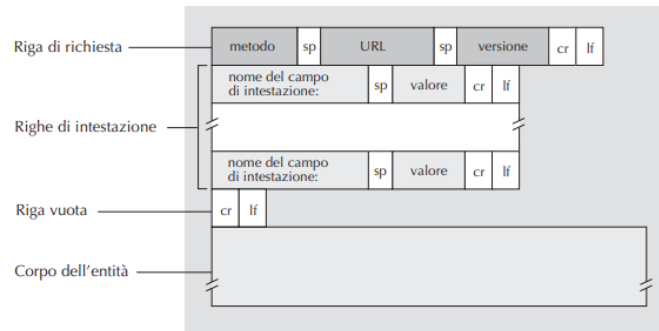


Figura 4.5: Formato generale dei messaggi di richiesta di HTTP

dati immessi (nei campi del form) nell'URL richiesto. Per esempio, se un form impiega il metodo GET e presenta due campi, e se i dati immessi sono scimmie e banane, allora l'URL avrà la struttura: `www.somesite.com/animalsearch?scimmie&banane`.

- Il metodo HEAD è simile a GET, è usato quando il client non vuole scaricare il documento ma solo alcune informazioni sul documento (come ad esempio la data dell'ultima modifica). Nella risposta il server non inserisce il documento ma solo degli header informativi.
- Il metodo PUT è utilizzato per memorizzare un documento nel server. Il documento viene fornito nel corpo del messaggio e la posizione di memorizzazione nell'URL. Il metodo PUT viene anche utilizzato dalle applicazioni che richiedono di inviare oggetti ai web server.
- Il metodo DELETE consente la cancellazione di un oggetto su un server.

Messaggio di risposta

Esempio 4.2.4 – Messaggio di risposta HTTP

Presentiamo ora un tipico messaggio di risposta HTTP che potrebbe rappresentare la risposta al messaggio di richiesta dell'esempio precedente.

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

Osserviamo tre sezioni: una riga di stato iniziale, sei righe di intestazione e il corpo. Quest'ultimo è il fulcro del messaggio: contiene l'oggetto richiesto (rappresentato da: `data data data data data ...`).

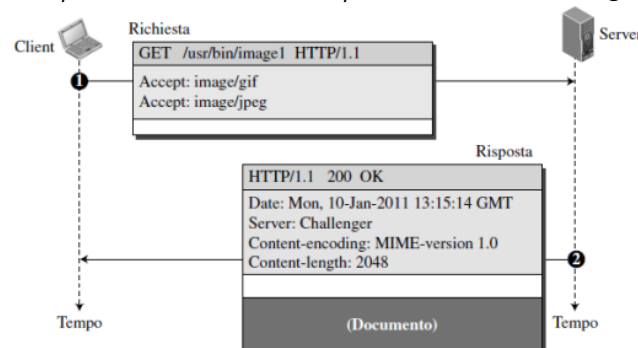
La riga di stato presenta tre campi: la versione del protocollo, un codice di stato e un corrispettivo messaggio di stato. In questo esempio, la riga di stato indica che il server sta usando HTTP/1.1 e che tutto va bene (ossia che il server ha trovato e sta inviando l'oggetto richiesto). Il server utilizza la riga di intestazione `Connection: close` per comunicare al client che ha intenzione di chiudere la connessione TCP dopo l'invio del messaggio. La riga `Date:` indica l'ora e la data di creazione e invio, da parte del server, della risposta HTTP. Si noti che non si tratta dell'istante in cui l'oggetto è stato creato o modificato per l'ultima volta, ma del momento in cui il server recupera l'oggetto dal proprio file system, lo inserisce nel messaggio di risposta e invia il messaggio. La riga `Server:` indica che il messaggio è stato generato da un web server Apache. La riga `Last-Modified:` indica l'istante e la data il cui l'oggetto è stato creato o modificato per l'ultima volta. La riga di intestazione `ContentLength:` contiene il numero di byte dell'oggetto inviato. La riga `Content-Type:` indica che l'oggetto nel corpo è testo HTML.

Il codice di stato e l'espressione associata indicano il risultato della richiesta. Tra i più comuni codici di stato e relative espressioni troviamo:

- 200 OK: la richiesta ha avuto successo e in risposta si invia l'informazione.
- 301 Moved Permanently: l'oggetto richiesto è stato trasferito in modo permanente; il nuovo URL è specificato nell'intestazione Location: del messaggio di risposta.
- 400 Bad Request: si tratta di un codice di errore generico che indica che la richiesta non è stata compresa dal server.
- 404 Not Found: il documento richiesto non esiste sul server.
- 505 HTTP Version Not Supported: il server non dispone della versione di protocollo HTTP richiesta.

Esempio 4.2.5 – Esempio GET

In questo esempio il client preleva un documento dal percorso /usr/bin/image1.

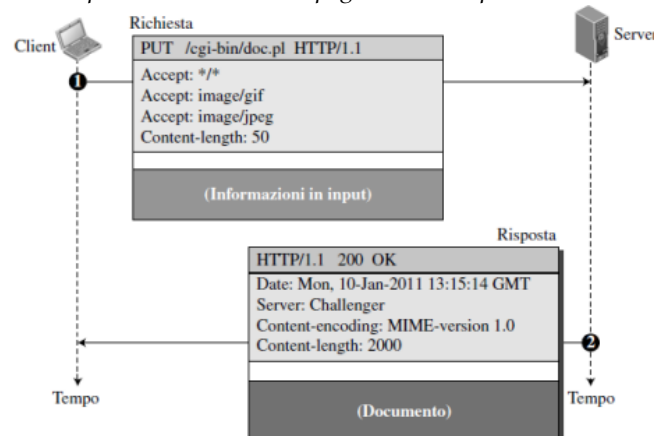


La riga di richiesta contiene il metodo GET, l'URL e la versione (1.1) del protocollo HTTP. L'intestazione è costituita da due righe in cui si specifica che il client accetta immagini nei formati GIF e JPEG. Il messaggio di richiesta non ha corpo.

Il messaggio di risposta contiene la riga di stato e quattro righe di intestazione che contengono la data, il server, il metodo di codifica del contenuto e la lunghezza del documento. Il corpo del messaggio segue l'intestazione.

Esempio 4.2.6 – Esempio PUT

In questo esempio il client spedisce al server una pagina Web da pubblicare.



La riga di richiesta contiene il metodo PUT, l'URL e la versione (1.1) del protocollo HTTP. L'intestazione è costituita da quattro righe d'intestazione. Il corpo del messaggio di richiesta contiene la pagina Web inviata.

Il messaggio di risposta contiene la riga di stato e quattro righe di intestazione. Il documento creato è incluso nel corpo del messaggio di risposta.

4.2.5 Cookie

Il Web è stato inizialmente progettato come ambiente "senza stato" (stateless): il server risponde alle richieste inviate dal client senza mantenere alcuna traccia del contesto, senza memorizzare alcuna informazione relativa alla sequenza di richieste inviate dal client, che sono quindi totalmente isolate. Questa modalità è perfettamente in linea con gli obiettivi iniziali del Web: poter ottenere facilmente documenti pubblicamente disponibili. Questo approccio oggi è decisamente inadeguato, visto che il Web ha anche altre funzioni: alcuni siti Web vogliono offrire contenuti personalizzati in base alle preferenze dell'utente; alcuni siti Web vogliono mantenere il carrello nei siti di commercio elettronico; alcuni siti Web devono limitare l'accesso ai soli utenti registrati.

Tutte queste funzioni richiedono un meccanismo di memorizzazione dello stato o conservazione del contesto: a questo scopo sono stati sviluppati i cookie.

Interazione utente-server

La tecnologia dei cookie presenta quattro componenti:

1. una riga di intestazione nel messaggio di risposta HTTP;
2. una riga di intestazione nel messaggio di richiesta HTTP;
3. un file cookie mantenuto sul sistema terminale dell'utente e gestito dal browser dell'utente;
4. un database sul server.

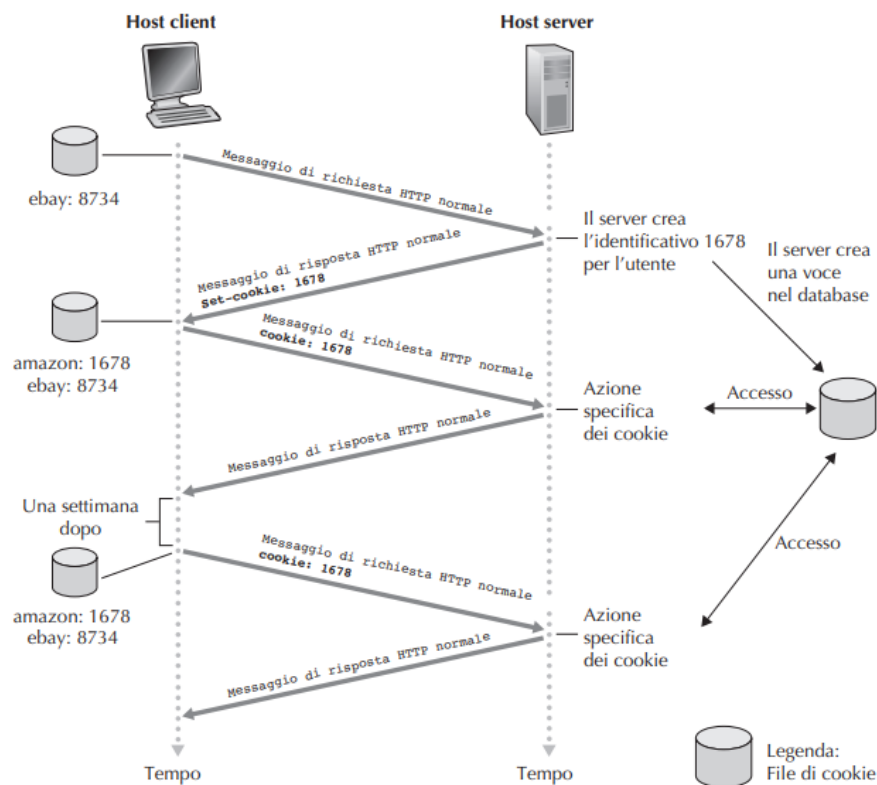
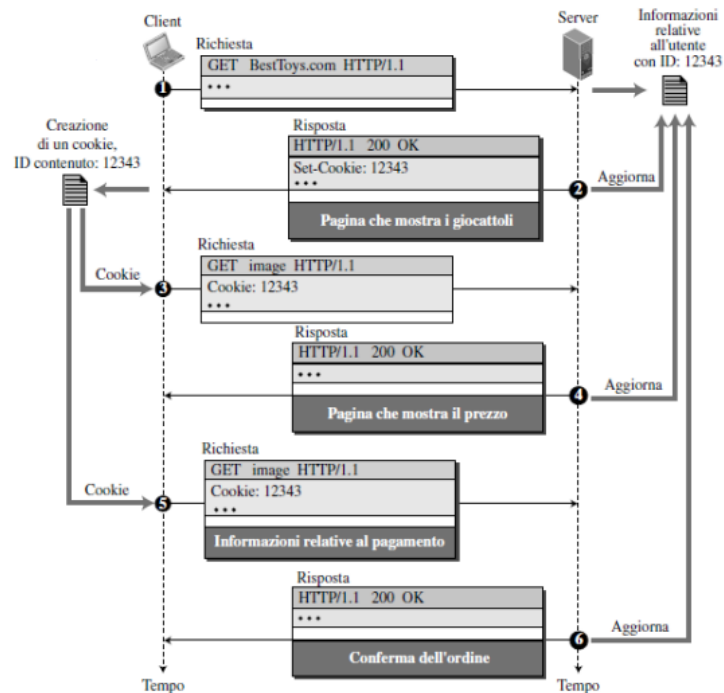


Figura 4.6: Memorizzazione dello stato dell'utente con i cookie

Esempio 4.2.7 – Cookie

Viene illustrato uno scenario nel quale un negozio elettronico sfrutta opportunamente i cookie.



Si supponga che un cliente desideri comprare un giocattolo da un negozio elettronico chiamato BestToys. ① Il browser del cliente invia una richiesta al server BestToys. Questo crea per il cliente un carrello della spesa vuoto e gli assegna un identificatore (per esempio 12343). ② Il server invia quindi un messaggio di risposta che contiene le immagini di tutti i giocattoli disponibili, con un link associato a ciascun giocattolo che consente all'utente di selezionarlo. Questo messaggio di risposta contiene anche la riga di intestazione Set-Cookie con valore 12343. Il browser sul computer client visualizza le immagini e memorizza il valore del cookie in un file chiamato BestToys. Il cookie non viene visualizzato al cliente ma solamente memorizzato. ③ Il cliente seleziona ora uno dei giocattoli: il browser invia una richiesta e include l'identificativo 12343 nella riga di intestazione Cookie. Sebbene il server stia servendo più clienti contemporaneamente, quando riceve la richiesta e controlla l'intestazione trova il valore 12343 come cookie. Il server è quindi in grado di capire che il cliente non è nuovo e cerca quindi il carrello della spesa con identificativo 12343. Il carrello viene aperto per inserirvi il giocattolo scelto. ④ Il server invia ora un'altra risposta al cliente per comunicare il prezzo totale e richiederne il pagamento. ⑤ Il cliente fornisce le informazioni relative alla propria carta di credito e invia una nuova richiesta, sempre con identificativo 12343 come valore del cookie. ⑥ Il server che riceve la richiesta riconosce l'identificativo 12343: accetta quindi l'ordine (e il pagamento) e invia una conferma come risposta. Il server archivia le informazioni aggiuntive sul cliente che ha ottenuto durante la transazione. Se il cliente accederà nuovamente al negozio, il browser invierà automaticamente il cookie e il negozio accederà immediatamente a tutte le informazioni archiviate.

Il cookie inviato al client è un identificatore di sessione (SID). Per evitare che il cookie sia utilizzato da utenti "maligni" l'identificatore è composto da una stringa di numeri. Ad esempio:

```
== Server -> User Agent ==
Set-Cookie: SID=31d4d96e407aad42
```

Il client ogni volta che manda una richiesta al server fornisce il suo identificatore (cookie): il browser consulta il file cookie, estrae il numero di cookie per il sito che si vuole visitare e lo inserisce nella richiesta http. Ad esempio:

```
== User Agent -> Server ==
Cookie: SID=31d4d96e407aad42
```

Il server mediante il cookie fornito dal client accede al relativo file e fornisce risposte personalizzate.

Il server chiude una sessione inviando al client una intestazione Set-Cookie nel messaggio con Max-Age=0. L'attributo Max-Age definisce il tempo di vita in secondi di un cookie. Dopo tot secondi il client dovrebbe rimuovere il cookie. Il valore zero indica che il cookie deve essere rimosso subito.

4.2.6 Caching

Il caching ha come obiettivo migliorare le prestazioni dell'applicazione web. Un modo semplice consiste nel salvare le pagine richieste per riutilizzarle in seguito senza doverle richiedere al server. E' una tecnica efficiente con pagine che vengono visitate molto spesso. Il caching può essere eseguito da Browser o Proxy.

Browser caching

Il browser può mantenere una cache delle pagine visitate personalizzabile dall'utente. Esistono vari meccanismi per la gestione della cache locale: l'utente può impostare il numero di giorni dopo i quali i contenuti della cache vengono cancellati; la pagina può essere mantenuta in cache in base alla sua ultima modifica (es. modificata un'ora prima); si possono utilizzare informazioni nei campi intestazione dei messaggi per gestire la cache.

Web caching: server proxy

Una web cache, nota anche come proxy server, è un'entità di rete che soddisfa richieste HTTP al posto del web server effettivo. Il proxy ha una propria memoria su disco (una cache) in cui conserva copie di oggetti recentemente richiesti. Il browser di un utente può essere configurato in modo che tutte le richieste HTTP dell'utente vengano innanzitutto dirette al proxy server. Una volta configurato il browser, ogni richiesta di oggetto da parte del browser viene inizialmente diretta al proxy.

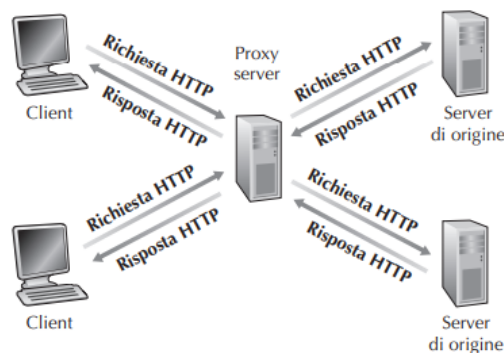
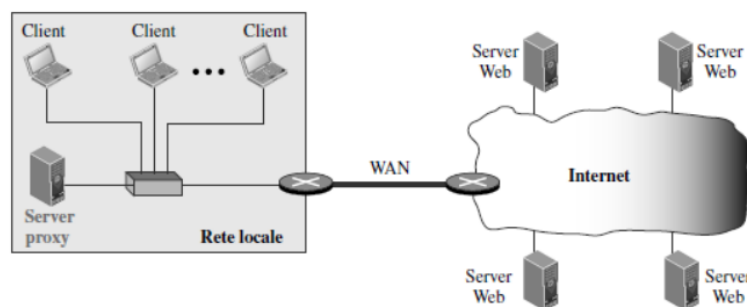


Figura 4.7: Client che richiedono oggetti attraverso un proxy server

Esempio 4.2.8 – Server proxy in una rete locale

Si mostra un esempio di impiego di un server proxy in una rete locale, per esempio una rete aziendale o un campus universitario.



Il proxy è installato all'interno della rete locale. Quando il browser di uno dei client genera una richiesta HTTP, questa viene diretta per prima cosa al proxy. Se quest'ultimo ha già la pagina

corrispondente invia la risposta al client. In caso contrario il proxy agisce da client e invia la richiesta al server Web in Internet. Quando la risposta viene restituita, il proxy ne memorizza una copia nella propria memoria cache prima di inviarla al client che l'aveva richiesta.

Si noti che il proxy è contemporaneamente server e client: quando riceve richieste da un browser e gli invia risposte agisce da server, quando invia richieste e riceve risposte da un server di origine funziona da client.

Generalmente un proxy server è acquistato e installato da un ISP. Per esempio un'università può installare un proxy sulla propria rete e configurare tutti i browser della propria sede per puntare a quel proxy. Oppure un grande ISP (quale ComCast) potrebbe installare uno o più proxy nella propria rete e preconfigurare i browser in modo che vi puntino.

Il web caching si è sviluppato in Internet per due ragioni. Innanzitutto, un proxy può ridurre in modo sostanziale i tempi di risposta alle richieste dei client, in particolare se l'ampiezza di banda che costituisce il collo di bottiglia tra il client e il server di origine è molto inferiore rispetto all'ampiezza di banda minima tra client e proxy. Se esiste una connessione ad alta velocità tra il client e il proxy, come spesso avviene, e se l'oggetto è nella cache, questa sarà in grado di consegnare rapidamente l'oggetto al client. In secondo luogo i proxy possono ridurre sostanzialmente il traffico sul collegamento di accesso a Internet, con il vantaggio di non dover aumentare l'ampiezza di banda frequentemente e ottenere quindi una riduzione dei costi. Inoltre i proxy possono ridurre in modo sostanziale il traffico globale del Web in Internet, migliorando di conseguenza le prestazioni di tutte le applicazioni.

Esempio 4.2.9 – Server proxy

La figura mostra due reti: la rete di un ente e la parte pubblica di Internet.

La rete dell'ente è una LAN ad alta velocità. Un collegamento a 15 Mbps connette un router della prima rete a uno della seconda. Supponiamo che la dimensione media di un oggetto sia 1 Mbit e che i browser dell'ente abbiano una frequenza media di 15 richieste ai server di origine al secondo. Ipotizziamo che i messaggi di richiesta HTTP siano trascurabilmente piccoli e non creino pertanto traffico nelle reti o nel collegamento di accesso (tra i due router). Supponiamo, inoltre, che la quantità di tempo che intercorre da quando il router sul lato Internet del collegamento di accesso inoltra una richiesta HTTP a quando riceve la risposta sia mediamente di due secondi. In modo informale ci riferiamo a questo ultimo ritardo come "ritardo Internet". Il tempo totale di risposta, ossia il tempo che intercorre tra la richiesta da parte del browser di un oggetto fino alla corrispondente ricezione dell'oggetto, è la somma del ritardo sulla rete locale, del ritardo di accesso (ritardo tra i due router) e del ritardo Internet.

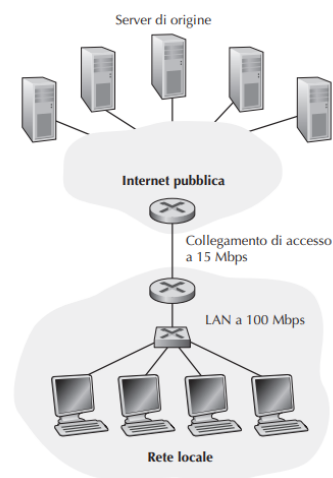
L'intensità di traffico sulla rete locale è pari a:

$$(15 \text{ richieste/secondo}) * (1 \text{ Mbit/richiesta}) / (100 \text{ Mbps}) = 0,15$$

mentre l'intensità di traffico sul collegamento di accesso (dal router Internet al router dell'ente) vale:

$$(15 \text{ richieste/secondo}) * (1 \text{ Mbit/richiesta}) / (15 \text{ Mbps}) = 1$$

Un'intensità di traffico di 0,15 su una rete locale provoca generalmente alcune decine di millisecondi di ritardo che può, quindi, essere trascurato. Tuttavia quando l'intensità di traffico si avvicina a 1 il ritardo su un collegamento diventa notevole e cresce senza limiti. Pertanto il tempo di risposta medio per soddisfare le richieste diventa dell'ordine dei minuti, se non superiore, il che è inaccettabile per gli utenti. Chiaramente occorre fare qualcosa. Una possibilità consiste nell'incremento della banda sul collegamento di accesso a Internet, per esempio da 15 Mbps a 100 Mbps. Ciò abbasserà l'intensità di traffico sul collegamento di accesso fino



a 0,15, il che si traduce in ritardi trascurabili tra i due router. In questo caso, il tempo totale di risposta sarà di circa due secondi, ossia il ritardo Internet. Ma questa soluzione implica che l'ente debba aggiornare il proprio collegamento a Internet, il che può risultare costoso. Una soluzione alternativa consiste nell'adozione di un proxy nella rete dell'istituzione. Le percentuali di successo (o hit rate), cioè la frazione di richieste soddisfatte dalla cache, variano in pratica tra 0,2 e 0,7. A scopo esemplificativo, supponiamo che per la nostra istituzione l'hit rate sia 0,4. Dato che i client e il proxy sono collegati alla stessa rete locale ad alta velocità, il 40% delle richieste verrà soddisfatto dalla cache quasi immediatamente, ossia entro 10 millisecondi. Ciò nondimeno, il restante 60% delle richieste deve ancora essere soddisfatto dai server di origine. Ma ora solo il 60% degli oggetti richiesti passa attraverso il collegamento di accesso, e l'intensità di traffico sul collegamento di accesso si riduce da 1,0 a 0,6. In generale, un'intensità di traffico inferiore a 0,8 su un collegamento a 15 Mbps corrisponde a un piccolo ritardo, dell'ordine delle decine di millisecondi. Questo ritardo è trascurabile rispetto ai due secondi di ritardo Internet. Sulla base di queste considerazioni, il ritardo medio è pertanto:

$$0,4 * (0,01\text{secondi}) + 0,6 * (2,01\text{secondi})$$

che è solo leggermente superiore a 1,2 secondi. Questa seconda soluzione fornisce un tempo di risposta perfino inferiore rispetto alla prima e non richiede l'aggiornamento del collegamento dell'istituzione a Internet. Questa deve, ovviamente, acquistare e installare un proxy, una spesa comunque contenuta: molti proxy sono software di pubblico dominio che possono essere eseguiti su PC economici.

GET condizionale

Sebbene il web caching riduca i tempi di risposta percepiti dall'utente, introduce un nuovo problema: la copia di un oggetto che risiede in cache potrebbe essere scaduta. In altre parole, l'oggetto ospitato nel web server potrebbe esser stato modificato rispetto alla copia nel client (sia esso un proxy o un browser). Fortunatamente, HTTP presenta un meccanismo che permette alla cache di verificare se i suoi oggetti sono aggiornati. Questo meccanismo è chiamato GET condizionale (conditional GET). Un messaggio di richiesta HTTP viene detto messaggio di GET condizionale se: (1) usa il metodo GET e (2) include una riga di intestazione If-modified-since.

Esempio 4.2.10 – GET condizionale

Per prima cosa un proxy invia un messaggio di richiesta a un web server per conto del browser richiedente:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
```

Poi, il web server invia al proxy un messaggio di risposta con l'oggetto richiesto:

```
HTTP/1.1 200 OK
Date: Sat, 3 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)
Last-Modified: Wed, 9 Sep 2015 09:23:24
Content-Type: image/gif
(data data data data data ...)
```

Il proxy inoltra l'oggetto al browser richiedente e pone anche l'oggetto nella cache locale. Va sottolineato che la cache memorizza con l'oggetto anche la data di ultima modifica. Poi, una settimana più tardi, un altro browser richiede lo stesso oggetto attraverso il proxy, e l'oggetto si trova ancora nella cache. Dato che tale oggetto può essere stato modificato nel web server durante la settimana trascorsa, il proxy effettua un controllo di aggiornamento inviando un GET condizionale. Più nello specifico invia:

```
GET /fruit/kiwi.gif HTTP/1.1
Host: www.exotiquecuisine.com
If-modified-since: Wed, 9 Sep 2015 09:23:24
```


Si osservi che il valore della riga di intestazione `If-modified-since:` equivale esattamente al valore della riga di intestazione `Last-Modified:` inviata dal server una settimana prima. Questo GET condizionale sta comunicando al server di inviare l'oggetto solo se è stato modificato rispetto alla data specificata. Supponiamo che l'oggetto non sia stato modificato dalle 9:23:24 del 9 settembre 2015. Allora il web server invia un messaggio di risposta al proxy:

```
HTTP/1.1 304 Not Modified
Date: Sat, 10 Oct 2015 15:39:29
Server: Apache/1.3.0 (Unix)
(corpo vuoto)
```

Notiamo che in risposta a un GET condizionale, il web server invia ancora un messaggio di risposta, ma non include l'oggetto richiesto, in quanto ciò implicherebbe solo spreco di banda e incrementerebbe il tempo di risposta percepito dall'utente, in particolare se l'oggetto è grande. La riga di stato `304 Not Modified` comunica al proxy che può procedere e inoltrare al browser richiedente la copia dell'oggetto presente in cache.

4.3 DNS (Domain Name System)

Gli host Internet possono essere identificati in vari modi. I nomi degli host (hostname), ad esempio `www.google.com`, risultano abbastanza appropriati per l'uomo, ma forniscono ben poca informazione sulla loro collocazione all'interno di Internet. Inoltre, dato che questi nomi sono costituiti da un numero variabile di caratteri alfanumerici, sarebbero difficilmente elaborabili dai server. Per tali motivi, gli host vengono identificati anche dai cosiddetti indirizzi IP.

Un indirizzo IP consiste di quattro byte e presenta una rigida struttura gerarchica. Ha una forma del tipo `121.7.106.83`, in cui ogni punto separa uno dei byte espressi con un numero decimale compreso tra 0 e 255. Un indirizzo IP è gerarchico perché, leggendolo da sinistra a destra, otteniamo informazioni sempre più specifiche sulla collocazione dell'host in Internet (ossia sulla sua appartenenza a quale rete all'interno della rete di reti).

4.3.1 Servizi forniti da DNS

Abbiamo appena visto che esistono due modi per identificare gli host: il nome e l'indirizzo IP. Le persone preferiscono il primo, mentre i router prediligono gli indirizzi IP a lunghezza fissa e strutturati in modo gerarchico. Al fine di conciliare i due approcci è necessario un servizio in grado di tradurre i nomi degli host nei loro indirizzi IP. Si tratta del principale compito del domain name system (DNS) di Internet. DNS è un database distribuito implementato in una gerarchia di DNS server e un protocollo a livello di applicazione che consente agli host di interrogare il database. Il protocollo DNS utilizza UDP e la porta 53. DNS viene comunemente utilizzato da altri protocolli a livello di applicazione, tra cui HTTP e SMTP, per tradurre i nomi di host forniti dall'utente in indirizzi IP.

Esempio 4.3.1 – Interazione con HTTP

Consideriamo che cosa succede quando un browser (ossia un client HTTP) in esecuzione sull'host di un utente richiede l'URL `www.someschool.edu/index.html`.

L'host dell'utente, per essere in grado di inviare un messaggio di richiesta HTTP al web server `www.someschool.edu`, deve come prima cosa ottenere il suo indirizzo IP. Ciò avviene come segue:

1. La stessa macchina utente esegue il lato client dell'applicazione DNS.
2. Il browser estrae il nome dell'host, `www.someschool.edu`, dall'URL e lo passa al lato client dell'applicazione DNS.
3. Il client DNS invia una interrogazione (query) contenente l'hostname a un DNS server.
4. Il client DNS prima o poi riceve una risposta, che include l'indirizzo IP corrispondente all'hostname.

5. Una volta ricevuto l'indirizzo IP dal DNS, il browser può dare inizio a una connessione TCP verso il processo server HTTP collegato alla porta 80 di quell'indirizzo IP.

Oltre alla traduzione degli hostname in indirizzi IP, DNS mette a disposizione altri importanti servizi:

- *Host aliasing*. Un host dal nome complicato può avere uno o più sinonimi (alias). Per esempio, `relay1.west-coast.enterprise.com` potrebbe avere due sinonimi quali `enterprise.com` e `www.enterprise.com`. In questo caso, si dice che il nome `relay1.west-coast.enterprise.com` è un hostname canonico. I sinonimi, se presenti, sono generalmente più facili da ricordare rispetto ai nomi canonici. Il DNS può essere invocato da un'applicazione per ottenere l'hostname canonico di un sinonimo, così come l'indirizzo IP dell'host.
- *Mail server aliasing*. Un'applicazione di posta può invocare il DNS per ottenere il nome canonico di un sinonimo fornito, così come l'indirizzo IP dell'host.
- *Distribuzione del carico di rete (load distribution)*. Il DNS viene anche utilizzato per distribuire il carico tra server replicati, per esempio dei web server. I siti con molto traffico vengono replicati su più server, ognuno eseguito su un host diverso con un indirizzo IP differente. Nel caso di web server replicati, va dunque associato a ogni hostname canonico un insieme di indirizzi IP. Il database DNS contiene questo insieme di indirizzi. Quando i client effettuano una richiesta DNS per un nome associato a un insieme di indirizzi, il server risponde con l'intero insieme di indirizzi, ma ne varia l'ordinamento a ogni risposta. Dato che generalmente un client invia il suo messaggio di richiesta HTTP al primo indirizzo IP elencato nell'insieme, la rotazione DNS distribuisce il traffico sui server replicati.

4.3.2 Panoramica del funzionamento DNS

Dal punto di vista dell'applicazione nell'host utente, il DNS è una scatola nera che fornisce un servizio di traduzione semplice e diretto. Tuttavia nei fatti la scatola nera è costituita da un gran numero di DNS server distribuiti per il mondo e da un protocollo a livello di applicazione che specifica la comunicazione tra DNS server e host richiedenti. Un primo approccio potrebbe essere quello di utilizzare un DNS server contenente tutte le corrispondenze. In questo schema centralizzato, i client dirigerebbero semplicemente tutte le richieste al singolo server e quest'ultimo risponderebbe loro direttamente. Sebbene tale semplicità progettuale sia attraente, sarebbe inappropriata per l'attuale Internet, dotata di un vasto e sempre crescente numero di host. Tra i problemi legati a uno schema centralizzato ricordiamo i seguenti:

- *Un solo punto di fallimento*. Se il DNS server si guasta, ne soffre l'intera Internet.
- *Volume di traffico*. Un singolo DNS server dovrebbe gestire tutte le richieste (per tutte le richieste HTTP e i messaggi di posta elettronica generati da centinaia di milioni di host).
- *Database centralizzato distante*. Un singolo DNS server non può essere vicino a tutti i client. Se il server si trovasse a New York, le query provenienti dall'Australia dovrebbero viaggiare fino all'altro capo del mondo, magari su collegamenti lenti e congestionati, causando ritardi significativi.
- *Manutenzione*. Il singolo DNS server dovrebbe contenere record relativi a tutti gli host di Internet. Non solo tale database centralizzato sarebbe vasto, ma dovrebbe essere aggiornato frequentemente per tener conto di ogni nuovo host.

In conclusione, un database centralizzato su un singolo DNS server non è in grado di adattarsi alla crescita esponenziale della rete (ovvero, non è scalabile). Di conseguenza, il DNS è stato progettato in maniera distribuita e costituisce un magnifico esempio di come si possa implementare un database distribuito su Internet.

Gerarchia server DNS

Per trattare il problema della scalabilità, il DNS utilizza un grande numero di server, organizzati in maniera gerarchica e distribuiti nel mondo. Nessun DNS server ha le corrispondenze per tutti gli host in Internet, che sono invece distribuite tra tutti i DNS server. In prima approssimazione, esistono tre classi di DNS server: i root server, i top-level domain (TLD) server e i server autoritativi, organizzati in una gerarchia.

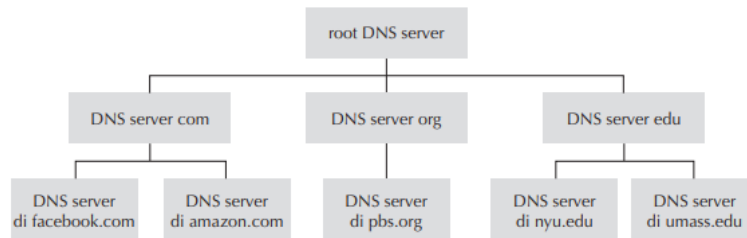


Figura 4.8: Gerarchia di server DNS

Esempio 4.3.2 – Iterazione tra classi DNS

Per comprendere l'interazione tra le classi, supponiamo che un client DNS voglia determinare l'indirizzo IP relativo all'hostname `www.amazon.com`.

Per fare ciò, il client dapprima contatta uno dei root server, che gli restituisce uno o più indirizzi IP relativi al server TLD per il dominio `com` (detto “dominio di primo livello” o anche “top-level”, in quanto è in cima alla gerarchia). Quindi contatta uno di questi server TLD, che gli restituisce uno o più indirizzi IP del server autoritativo per `amazon.com`. Infine, contatta uno dei server autoritativi per `amazon.com`, che gli restituisce l'indirizzo IP dell'hostname `www.amazon.com`.

Esaminiamo le tre classi di DNS server:

- *Root server.* In Internet esistono 400 root server, dislocati in tutto il mondo. Questi root server sono gestiti da 13 diverse organizzazioni. I root server forniscono gli indirizzi IP dei server TLD.
- *Top-level domain (TLD) server.* Questi server si occupano dei domini di primo livello quali `com`, `org`, `net`, `edu` e `gov`, e di tutti i domini di primo livello relativi ai vari paesi, come `uk`, `fr`, `ca` e `jp`. L'azienda Verisign Global Registry Services gestisce i TLD server per il dominio `com` e il Registro.it che ha sede a Pisa all'Istituto di Informatica e Telematica (CNR) gestisce il dominio `it`. I server TLD forniscono gli indirizzi IP dei server autoritativi.
- *DNS server autoritativi.* Ogni organizzazione dotata di host pubblicamente accessibili tramite Internet (quali web server e mail server) deve fornire record DNS pubblicamente accessibili che associno i nomi di tali host a indirizzi IP. Il DNS server autoritativo dell'organizzazione ospita questi record. Un'organizzazione può scegliere di implementare il proprio server autoritativo o di pagare un fornitore di servizi per ospitare questi record su un suo server. La maggior parte delle università e delle grandi società implementa e gestisce dei propri server autoritativi primario e secondario (di backup).

Esiste un altro importante tipo di DNS, detto DNS server locale, che non appartiene strettamente alla gerarchia di server, ma che è comunque centrale nell'architettura DNS. Ciascun ISP, come un'università, un dipartimento universitario, un'azienda o un ISP residenziale, ha un DNS server locale (detto anche default name server). Quando un host si connette a un ISP, quest'ultimo gli fornisce un indirizzo IP tratto da uno o più dei suoi DNS server locali. Quando un host effettua una richiesta DNS, la query viene inviata al DNS server locale, che opera da proxy e inoltra la query alla gerarchia dei DNS server.



Figura 4.9: Root DNS server nel mondo

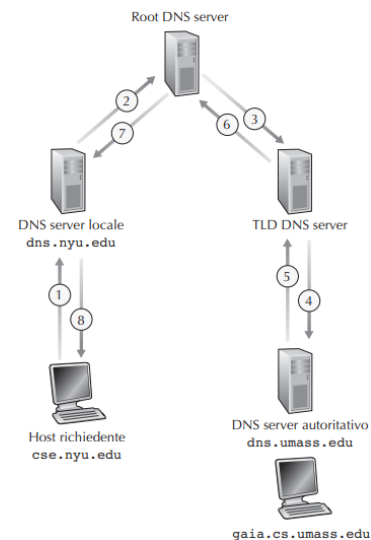
Esempio 4.3.3 – Query DNS iterativa

Supponiamo che l'host `cse.nyu.edu` voglia l'indirizzo IP di `gaia.cs.umass.edu`. Supponiamo, inoltre, che il DNS server locale per `cse.nyu.edu` sia `dns.nyu.edu`, mentre un server autoritativo per `gaia.cs.umass.edu` sia `dns.umass.edu`.

L'host `cse.nyu.edu` dapprima invia un messaggio di richiesta DNS al proprio server locale `dns.nyu.edu`. Il messaggio contiene il nome da tradurre, ossia `gaia.cs.umass.edu`. Il server locale inoltra il messaggio di richiesta a un root server. Quest'ultimo prende nota del suffisso `edu` e restituisce al server locale un elenco di indirizzi IP per i TLD server responsabili di `edu`. Il server locale rinvia quindi il messaggio di richiesta a uno di questi ultimi. Il TLD server prende nota del suffisso `umass.edu` e risponde con l'indirizzo IP del server autoritativo per l'Università del Massachusetts, ossia `dns.umass.edu`. Infine, il DNS server locale rimanda il messaggio di richiesta direttamente a `dns.umass.edu`, che risponde con l'indirizzo IP di `gaia.cs.umass.edu`.

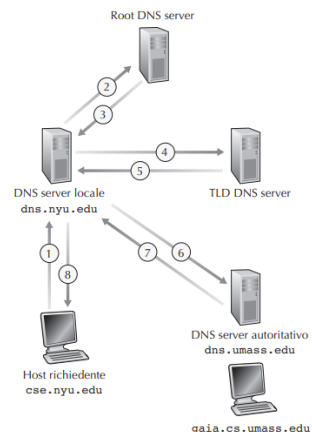
Si noti che in questo esempio, per ottenere la mappatura di un hostname, sono stati inviati ben otto messaggi DNS: quattro messaggi di richiesta e quattro messaggi di risposta.

In una query iterativa il server contattato risponde con il nome del server da contattare.

**Esempio 4.3.4 – Query DNS ricorsiva**

Supponiamo ancora che Università del Massachusetts abbia un proprio DNS server, chiamato `dns.umass.edu`. Ipotizziamo, inoltre, che ciascun dipartimento dell'università abbia un proprio server, competente per tutti gli host presenti in dipartimento.

In questo caso, quando il DNS server intermedio, `dns.umass.edu`, riceve una richiesta da `dns.nyu.edu` per un host il cui nome termina con `cs.umass.edu`, gli restituisce l'indirizzo IP di `dns.cs.umass.edu`, che rappresenta il server autoritativo per tutti i nomi di host terminanti con `cs.umass.edu`. Il server locale `dns.nyu.edu` invia quindi la richiesta al server autoritativo, che restituisce la corrispondenza desiderata al DNS server locale, il quale a sua volta la restituisce all'host richiedente.



Una query ricorsiva affida il compito di tradurre il nome al server DNS contattato.

DNS caching

Fino a questo momento abbiamo ignorato il DNS caching, una caratteristica di fondamentale importanza. In verità, il DNS sfrutta in modo estensivo il caching per migliorare le prestazioni di ritardo e per ridurre il numero di messaggi DNS che “rimbalzano” su Internet. L’idea alla base del DNS caching è molto semplice. In una concatenazione di richieste, il DNS server che riceve una risposta DNS (contenente, per esempio, la traduzione da hostname a indirizzo IP), può mettere in cache le informazioni contenute. Se una coppia hostname/indirizzo IP è nella cache di un DNS server e giunge al server un’altra richiesta con lo stesso hostname, il DNS server può fornire l’indirizzo IP desiderato, anche se non è autoritativo per tale indirizzo. Dato che gli host e le associazioni tra nome e indirizzo IP non sono in alcun modo permanenti, i DNS server invalidano le informazioni in cache dopo un periodo di tempo fissato (in genere di 2 giorni). Un DNS server locale può, inoltre, memorizzare in cache gli indirizzi IP dei TLD server, consentendogli di aggirare i root server nella catena di richieste (e ciò avviene di frequente).

4.3.3 Record e messaggi DNS

I server che implementano il database distribuito di DNS memorizzano i cosiddetti record di risorsa (RR, resource record), tra cui quelli che forniscono le corrispondenze tra nomi e indirizzi. Ogni messaggio di risposta DNS trasporta uno o più record di risorse.

Un record di risorsa contiene i seguenti campi: (Name, Value, Type, TTL). TTL è il time to live, ossia il tempo residuo di vita di un record e determina quando una risorsa vada rimossa dalla cache. Il significato di Name e Value dipende da Type:

- Se Type=A, allora Name è il nome dell’host e Value è il suo indirizzo IP. Pertanto un record di tipo A fornisce la corrispondenza tra hostname standard e indirizzo IP. Per esempio, (relay1.bar.foo.com, 145.37.93.126, A) è un record di tipo A.
- Se Type=NS, allora Name è un dominio (quale foo.com) e Value è l’hostname del DNS server autoritativo che sa come ottenere gli indirizzi IP degli host nel dominio. Questo record viene usato per instradare le richieste DNS successive alla prima nella concatenazione delle query. Per esempio, (foo.com, dns.foo.com, NS) è un record di tipo NS.
- Se Type=CNAME, allora Value rappresenta il nome canonico dell’host per il sinonimo Name. Questo record può fornire agli host richiedenti il nome canonico relativo a un hostname. Per esempio, (foo.com, relay1.bar.foo.com, CNAME) è un record CNAME.
- Se Type=MX, allora Value è il nome canonico di un mail server che ha il sinonimo Name. Per esempio, (foo.com, mail.bar.foo.com, MX) è un record MX. Questo tipo di record consente agli hostname dei mail server di avere sinonimi semplici.

Un DNS server autoritativo per un certo hostname contiene un record di tipo A per l’hostname. Anche se il server non fosse autoritativo, potrebbe tuttavia contenere un record di tipo A nella propria cache. Se un server non è quello autoritativo per un certo hostname, allora conterrà un record di tipo NS per il dominio che include l’hostname, e conterrà anche un record di tipo A che fornisce l’indirizzo IP del DNS server nel campo Value del record NS.

Esempio 4.3.5 – Record DNS

Supponiamo che un TLD server non sia competente per l’host corsi.di.uniroma1.it. Allora conterrà un record per un dominio che include l’host corsi.di.uniroma1.it, per esempio (umass.edu, dns.umass.edu, NS). Il TLD server conterrebbe inoltre un record di tipo A, che indica il DNS server dns.uniroma1.it in un indirizzo IP, per esempio (dns.umass.edu, 128.119.40.111, A).

Messaggi DNS

Abbiamo precedentemente fatto riferimento agli unici due tipi di messaggio DNS: le query e i messaggi di risposta che presentano, entrambi, lo stesso formato.

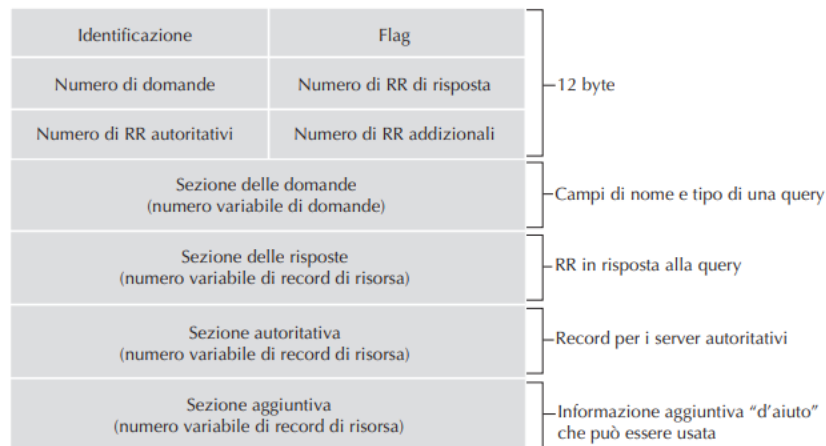


Figura 4.10: Formato dei messaggi DNS

La semantica dei campi dei messaggi DNS è la seguente:

- I primi 12 byte rappresentano la *sezione di intestazione*, che a sua volta contiene un certo numero di campi. Il primo è un numero di 16 bit che identifica la richiesta. Tale identificatore viene copiato nei messaggi di risposta a una richiesta, consentendo al client di far corrispondere le risposte ricevute con le query inviate. Troviamo poi il campo flag. Il primo di questi, il bit di richiesta/risposta (query/reply), indica se il messaggio è una richiesta (0) o una risposta (1). Un bit viene impostato nei messaggi di risposta quando il DNS server è autoritativo per il nome richiesto. Un ulteriore bit, chiamato di richiesta di ricorsione (recursion-desired flag), viene impostato quando un client (sia esso un host o un DNS server) desidera che il DNS server effettui ricorsione quando non dispone del record. Si imposta un campo da 1 bit di ricorsione disponibile (recursion-available) all'interno di una risposta se il DNS server supporta la ricorsione. Nell'intestazione troviamo anche quattro campi il cui nome comincia con "numero di", che indicano il numero di occorrenze delle quattro sezioni di tipo di dati che seguono l'intestazione.
- La *sezione delle domande* contiene informazioni sulle richieste che stanno per essere effettuate. Inoltre, include un campo nome con il nome che sta per essere richiesto, e un campo tipo che indica il tipo della domanda sul nome: per esempio, un indirizzo di host associato a un nome (tipo A) o il server di posta per un nome (tipo MX).
- In una risposta proveniente da DNS server, la *sezione delle risposte* contiene i record di risorsa relativi al nome originariamente richiesto. Ricordiamo che in ogni record di risorsa troviamo Type (A, NS, CNAME o MX), Value e TTL. Una risposta può restituire più RR, dato che un hostname può avere più indirizzi IP.
- La *sezione autoritativa* contiene i record di altri server autoritativi.
- La *sezione aggiuntiva* racchiude altri record utili: per esempio, se il campo di risposta relativo a una richiesta MX contiene un record di risorsa che fornisce l'hostname canonico del server di posta, la sezione aggiuntiva contiene un record di tipo A che fornisce l'indirizzo IP relativo all'hostname canonico del server di posta.

Inserimento di record nel database DNS

La precedente trattazione si è concentrata su come vengono recuperati i record dal database DNS. A questo punto potreste chiedervi come siano inseriti i record nel database. Vediamolo nel contesto di uno specifico esempio.

Esempio 4.3.6 – Inserimento di record nel database DNS

Supponiamo che abbiate appena fondato e avviato una nuova, entusiasmante società chiamata Network Utopia.

La prima cosa che sicuramente desiderate fare è registrare il nome di dominio `networkutopia.com` presso un ente di registrazione (registrar). Un registrar è un'azienda che verifica l'unicità del nome di dominio, lo inserisce nel database DNS e vi richiede una piccola somma di denaro per i propri servizi. Quando registrate il nome di dominio `networkutopia.com` presso un registrar, dovete fornirgli anche i nomi e gli indirizzi IP dei vostri DNS server autoritativi primario e secondario. Supponiamo che i nomi e gli indirizzi IP siano `dns1.networkutopia.com`, `dns2.networkutopia.com`, `212.2.212.1` e `212.212.212.2`. Per ciascuno di questi due server autoritativi l'ente si accerterà dell'inserimento di un record di tipo NS e di tipo A nei TLD server relativi al suffisso `com`. Più nello specifico, per il server autoritativo primario di `networkutopia.com` il registrar inserirebbe nel sistema DNS i due seguenti record di risorsa: (`networkutopia.com`, `dns1.networkutopia.com`, NS) e (`dns1.networkutopia.com`, `212.212.212.1`, A).

Dovrete inoltre accertarvi che il record di risorsa di tipo A per il vostro web server `www.networkutopia.com` e che il record di risorsa di tipo MX per il vostro server di posta `mail.networkutopia.com` vengano immessi nei vostri DNS server autoritativi. Una volta completati questi passi sarà possibile visitare il vostro sito web e spedire posta elettronica ai dipendenti della vostra società. Concludiamo la nostra trattazione su DNS verificando che questa affermazione sia vera.

Supponiamo che Alice, in Australia, voglia visitare la pagina web `www.networkutopia.com`. Come detto precedentemente, il suo host invierà una richiesta DNS al suo DNS server locale. Quest'ultimo contatterà poi un TLD server per il suffisso `com`. Il DNS server locale dovrà inoltre contattare un root server nel caso in cui l'indirizzo del TLD server responsabile per il suffisso `com` non si trovi in cache. Questo TLD server contiene i record di risorsa di tipo NS e A elencati precedentemente, dato che il registrar li ha inseriti in tutti i TLD server relativi al suffisso `com`. Il TLD server di `com` invia una risposta al server locale di Alice, e la risposta contiene i due record di risorsa. Il server locale invia poi una richiesta DNS a `212.212.212.1`, cercando il record di tipo A che corrisponde a `www.networkutopia.com`. Tale record fornisce l'indirizzo IP del web server desiderato, per esempio `212.212.71.4`, restituito poi dal server locale all'host di Alice. Il browser di Alice può ora inizializzare una connessione TCP verso l'host `212.212.71.4` e inviare una richiesta HTTP sulla connessione.

4.4 Protocollo FTP

FTP (File Transfer Protocol) è il protocollo standard offerto da TCP/IP per la copia di file da un host a un altro. Sebbene il trasferimento di un file possa sembrare un'operazione semplice, vi sono, in realtà, numerosi problemi ai quali bisogna prestare attenzione: i due sistemi coinvolti, per esempio, possono adottare convenzioni differenti per la denominazione dei file, oppure gestire in modo differente la struttura delle directory. Problemi di questo tipo vengono risolti in modo semplice ed elegante dal protocollo FTP.

Quando l'utente fornisce il nome dell'host remoto (`ftp NomeHost`), il processo client FTP stabilisce una connessione TCP sulla porta 21 con il processo server FTP. Stabilita la connessione, il client fornisce nome utente e password che vengono inviate sulla connessione TCP come parte dei comandi. Ottenuta l'autorizzazione del server il client può inviare uno o più file memorizzati nel file system locale verso quello remoto (o viceversa).

Il client ha tre componenti: interfaccia utente, processo di controllo e processo di trasferimento dati. Il server ha, invece, due sole componenti: processo di controllo e processo di trasferimento dati. La connessione di controllo viene realizzata tra i due processi di controllo e quella per lo scambio dei dati tra i due processi relativi.

La separazione del trasferimento dei dati e dei comandi rende il protocollo FTP piuttosto efficiente: la connessione che si occupa delle informazioni di controllo può usare regole molto semplici, così che lo scambio di informazioni si riduce allo scambio di una riga di comando (o di risposta) per ogni interazione. La connessione che si occupa dello scambio di dati, invece, deve fare uso di regole più complesse a causa della varietà delle informazioni scambiate.

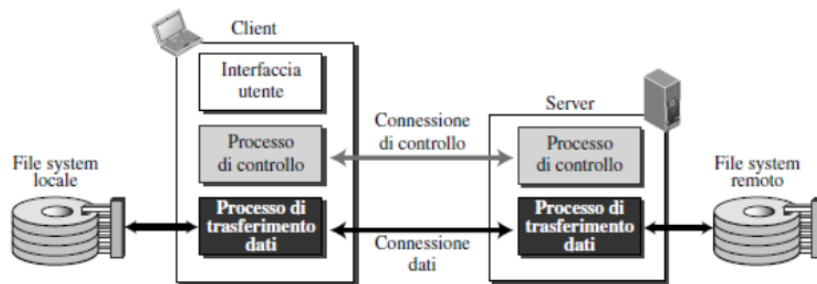


Figura 4.11: Protocollo FTP tra client e server

4.4.1 Connessione di controllo

La connessione di controllo (porta 21) viene usata per inviare informazioni di controllo. L'apertura della connessione di controllo viene richiesta dal client al comando `ftp NomeHost` e tutti i comandi eseguiti dall'utente sono trasferiti sulla connessione di controllo.

La comunicazione attraverso la connessione di controllo avviene per mezzo di caratteri con una codifica standard chiamata NVT ASCII, sia per i comandi sia per le risposte. Pur nella sua semplicità questo metodo è perfettamente adeguato perché l'interazione avviene mediante l'invio di un comando o di una risposta alla volta.

Durante questa connessione di controllo i comandi vengono inviati dal client al server e le risposte vengono inviate dal server al client. I comandi sono inviati dal processo di controllo del client FTP sotto forma di caratteri ASCII maiuscoli e possono opzionalmente essere seguiti da un argomento.

Ogni comando FTP genera almeno una risposta; le risposte sono composte da due parti: un numero di tre cifre e un testo. La parte numerica costituisce il codice della risposta, quella di testo contiene i parametri necessari o informazioni supplementari. La prima cifra fornisce lo stato del comando. La seconda cifra definisce l'area alla quale si applica lo stato. La terza cifra fornisce informazioni supplementari. Un esempio di codice di risposta è: 331 Username OK, password required.

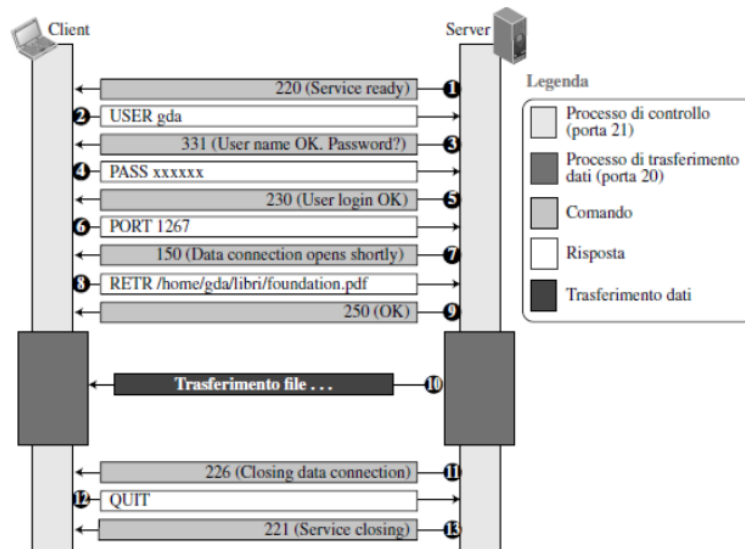
4.4.2 Connessione dati

Nella connessione per lo scambio di dati il server usa la porta 20; l'apertura di tale connessione avviene secondo uno schema completamente diverso da quello utilizzato per la connessione di controllo.

1. Il client, non il server, effettua un'apertura passiva usando una porta effimera (rimarrà in attesa di connessione); ciò viene fatto dal client perché è tale processo che invierà i comandi per il trasferimento dei file.
2. Il client invia questo numero di porta al server per mezzo del comando `PORT`.
3. Il server, ricevuto il numero di porta, effettua un'apertura attiva (aprirà la connessione) usando la porta nota 20 e quella effimera segnalata dal client.

Esempio 4.4.1 – Trasferimento di file

Viene illustrato un esempio d'uso di FTP per trasferire un file dal server al client.



La connessione di controllo rimane sempre aperta, la connessione dati viene invece aperta per il trasferimento del file. Appena completato il trasferimento, il processo di controllo server chiude la connessione dati. Dato che il processo di controllo client non ha altri file da trasferire, invia il comando QUIT che fa chiudere la connessione di controllo.

4.5 Posta elettronica

La posta elettronica (o e-mail) consente agli utenti di scambiarsi messaggi. La natura di questa applicazione è tuttavia differente da altre applicazioni trattate fino a questo momento. Per prima cosa la posta elettronica è considerata una transazione unidirezionale. Quando Gaia invia una e-mail a Gabriele, la risposta non è obbligatoria: Gabriele può decidere di rispondere oppure di non rispondere. Se risponde, si tratta comunque di un'altra transazione unidirezionale. Secondariamente non è ragionevole per Gabriele eseguire un programma server e attendere fino a quando qualcuno gli invia una e-mail. Gabriele potrebbe spegnere il computer quando non lo utilizza. Questo implica che l'idea di programmazione client/server deve essere realizzata in qualche altro modo, ad esempio utilizzando dei computer intermedi (styver).

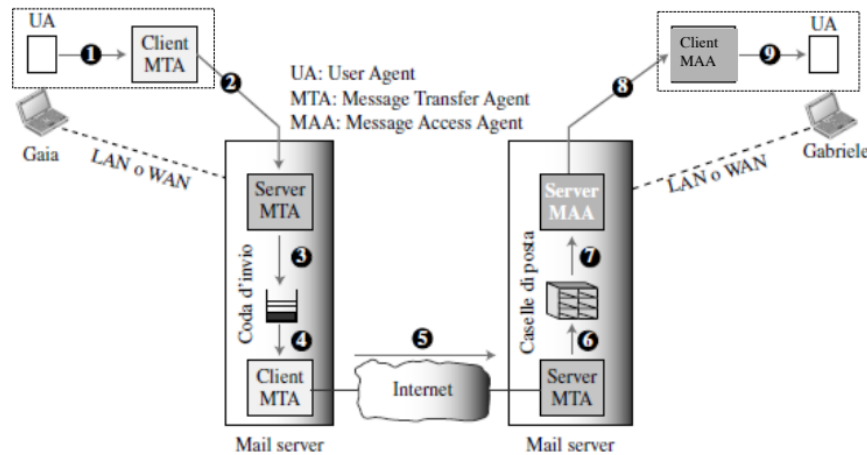
4.5.1 Architettura della posta elettronica

Abitualmente il mittente e il destinatario della posta elettronica sono collegati via LAN o WAN a due server di posta. L'amministratore ha creato una mailbox (casella di posta) per ogni utente dove memorizzare i messaggi ricevuti. Una mailbox non è altro che una porzione di spazio sulla memoria di massa del server, sotto forma di un file (o di una directory) con opportune restrizioni di accesso per far sì che solamente il proprietario possa accedere. Inoltre, l'amministratore ha creato anche una coda (spesso chiamata spool) dove memorizzare i messaggi in attesa di essere inviati.

Esempio 4.5.1 – Invio di una e-mail

Il semplice invio di una e-mail da Gaia a Gabriele richiede nove passi.

Gaia e Gabriele utilizzano tre agenti differenti: un User Agent (UA) usato per scrivere e inviare un messaggio o leggerlo, un Mail Transfer Agent (MTA) usato per trasferire il messaggio attraverso Internet e un Message Access Agent (MAA) usato per leggere la mail in arrivo.



Gaia, per inviare un messaggio a Gabriele, utilizza un programma UA per preparare il messaggio e inviarlo al proprio server di posta. Quest'ultimo memorizza il messaggio in attesa di essere inviato all'interno di un'apposita coda. Il messaggio, tuttavia, deve essere inviato tramite Internet dal mail server di Gaia al mail server di Gabriele usando un MTA. Sono quindi necessari due MTA in ciascun mail server: uno client e uno server. Analogamente alla maggior parte dei programmi client/server presenti in rete, il server deve essere in esecuzione permanente perché non può sapere quando il client richiederà una connessione. Il client, invece, può essere eseguito dal sistema quando in coda c'è un messaggio in attesa di essere spedito. L'user agent presso il sito di Gabriele gli consente di ricevere e leggere messaggi. Infine Gabriele utilizza un client MAA per ottenere i messaggi da un server MAA in esecuzione sul secondo server.

Ci sono due aspetti importanti da considerare. Primo, Gabriele è obbligato a passare per il server di posta e non può utilizzare il server MTA direttamente. Per utilizzare il server MTA direttamente, Gabriele dovrebbe mantenerlo sempre in esecuzione perché non può sapere quando arriverà un messaggio.

Secondo, si noti che Gabriele richiede un'ulteriore coppia di programmi client/server: i programmi per accedere ai messaggi di posta. Il programma client/server MTA, infatti, è un'applicazione di tipo push ("spinta"): il client "spinge" il messaggio al server. Gabriele invece richiede un programma di tipo pull ("tira"): il client deve "tirare" il messaggio dal server.

Il sistema di posta elettronica richiede due UA, due coppie client/server di MTA e una coppia client/server di MAA.

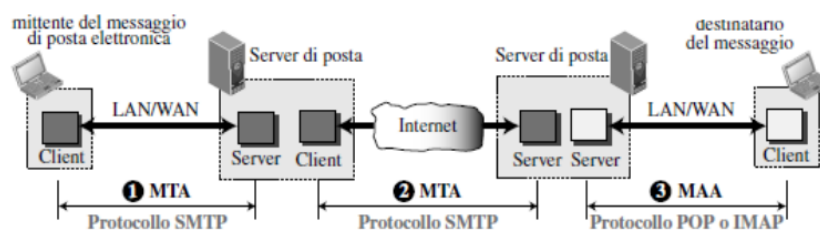


Figura 4.12: Protocolli utilizzati nella posta elettronica

4.5.2 User agent

Il primo componente di un sistema di posta elettronica è l'User Agent (UA). Il suo scopo è quello di semplificare all'utente il processo di invio e ricezione dei messaggi. Un user agent è un programma che consente di scrivere, leggere, rispondere e inoltrare i messaggi, ed è in grado inoltre di gestire le mailbox nei computer degli utenti.

Vi sono due tipi di user agent: con interfaccia a riga di comando e con interfaccia grafica (Graphical User Interface, GUI).

L'utente, per inviare una e-mail (tramite l'UA), crea messaggi che sono molto simili alla posta cartacea tradizionale; ogni messaggio di posta elettronica consiste di una sorta di busta (envelope) e del messaggio vero e proprio. La busta contiene solitamente l'indirizzo del destinatario, quello del mittente e altre informazioni. Il messaggio è suddiviso in intestazioni (header) e corpo (body); le intestazioni contengono l'indirizzo del mittente, quello del destinatario, l'argomento del messaggio e alcune altre informazioni, mentre il corpo contiene il messaggio vero e proprio.

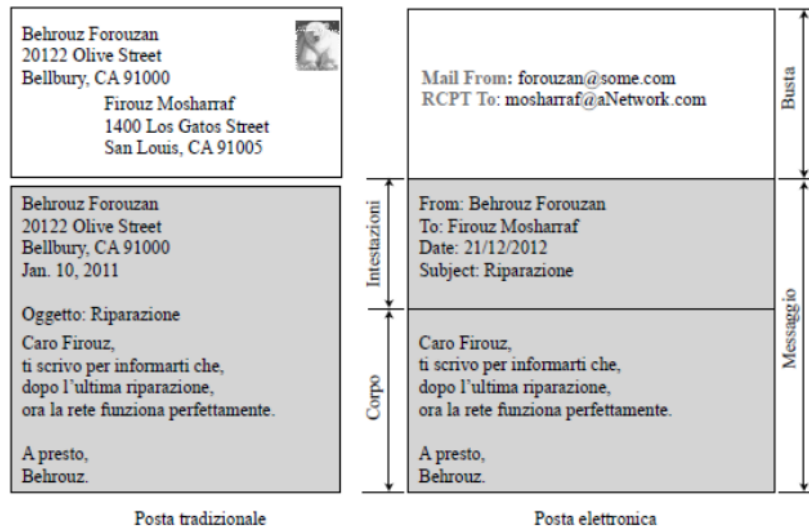


Figura 4.13: Formato di un messaggio di posta elettronica

4.5.3 Message Transfer Agent: SMTP

Il protocollo che definisce in modo formale l'interazione tra il client e il server MTA è chiamato SMTP (Simple Mail Transfer Protocol). SMTP è utilizzato in due diverse occasioni: fra il mittente ed il suo server di posta e tra i due server di posta. Quello che fa SMTP è semplicemente definire come deve avvenire l'interazione per mezzo di comandi e risposte.

Il trasferimento di un messaggio di posta elettronica viene realizzato dal protocollo SMTP mediante lo scambio di comandi e risposte tra client e server MTA. I comandi sono inviati da un client MTA a un server MTA, viceversa le risposte. I comandi e le risposte terminano tutti con la medesima coppia di caratteri (ritorno carrello e fine linea).

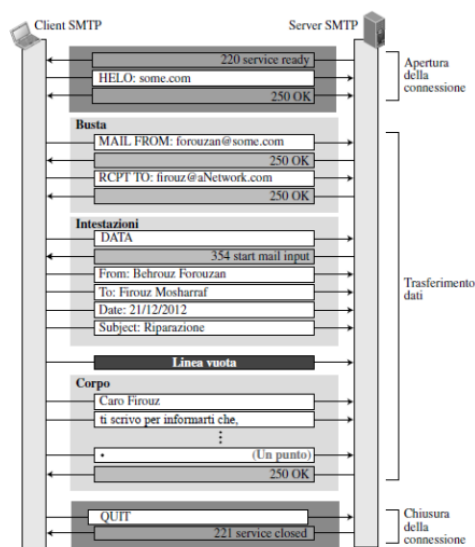
I comandi vengono inviati dal client al server; il loro formato è composto da una keyword (parola chiave) e uno o più argomenti. Le risposte sono formate da un codice a tre cifre seguito eventualmente da testo.

La consegna di un messaggio avviene in tre fasi distinte: apertura della connessione, trasferimento del messaggio di posta e chiusura della connessione.

- *Apertura della connessione.* Il server SMTP dà il via alla fase di apertura della connessione non appena il client effettua la connessione TCP con la porta nota 25.
- *Trasferimento del messaggio.* Dopo l'apertura della connessione tra il client e il server SMTP, può essere inviato un singolo messaggio dal mittente a uno o più destinatari.
- *Chiusura della connessione.* Il client, una volta trasferito il messaggio, chiude la connessione.

Esempio 4.5.2 – Trasferimento della posta

Nella parte della figura relativa al trasferimento del messaggio si sono separati i messaggi relativi alla preparazione della busta, dell'intestazione e del corpo. Si noti che la sequenza di passi successivi in questa figura viene ripetuta due volte in ciascun trasferimento di e-mail: la prima dal mittente del messaggio al server di posta locale, la seconda dal server di posta locale al server di posta remoto. Ovviamente, il server di posta locale, una volta ricevuto l'intero messaggio di posta, potrebbe accodarlo per inviarlo al server di posta remoto in un secondo tempo.

**4.5.4 Message Access Agent**

Il primo e il secondo stadio della trasmissione della posta elettronica utilizzano SMTP, che non è invece impiegato nel terzo stadio essendo un protocollo di tipo push: esso "spinge" il messaggio dal client al server. In altri termini, la direzione della maggior parte dei dati (messaggi) è dal client al server. Il terzo stadio richiede invece un protocollo di tipo pull: il client deve "tirare" i messaggi dal server, in questo caso la direzione della maggior parte dei dati è dal server al client. Il terzo stadio utilizza dunque un Message Access Agent.

Attualmente si utilizzano due protocolli per l'accesso ai messaggi: POP3 (Post Office Protocol, versione 3) e IMAP4 (Internet Mail Access Protocol, versione 4).

- Il protocollo Post Office Protocol versione 3 (POP3) è molto semplice ma ha funzionalità limitate. Il software client POP3 è installato sul computer del destinatario, mentre il software POP3 server viene installato sul server di posta. L'accesso alla posta avviene quando il client scarica i messaggi (le e-mail) dalla propria casella di posta che si trova sul server. Il client apre una connessione TCP verso il server sulla porta nota 110, quindi invia il proprio nome utente e relativa password per accedere alla casella di posta. A questo punto l'utente può richiedere la lista dei messaggi presenti e prelevarli, uno alla volta.

Il protocollo POP3 ha due modalità: delete (elimina) e keep (conserva). Nella modalità delete i messaggi sono automaticamente eliminati dalla mailbox dopo il prelievo. Nella modalità keep, invece, i messaggi vengono conservati. La modalità delete viene solitamente utilizzata quando l'utente lavora al proprio computer abituale e può salvare e organizzare la posta ricevuta dopo averla prelevata. La modalità keep è solitamente utilizzata quando l'utente non accede alla posta dalla propria postazione abituale (per esempio da un laptop): in questo caso legge la poste che viene però conservata per l'uso futuro.

- Un altro protocollo di accesso alla posta è l'Internet Mail Access Protocol versione 4 (IMAP4). Questo protocollo è simile al POP3 ma ha più funzionalità: è più potente e complesso. IMAP4, invece, fornisce varie funzionalità aggiuntive quali: controlla l'intestazione dei messaggi prima del prelievo; ricercare una stringa specifica nei messaggi prima del prelievo; prelevare i messaggi in modo parziale. Una funzionalità particolarmente utile quando vi sono limitazioni di banda; creare, cancellare o rinominare le mailbox sul server di posta; creare una gerarchia di cartelle all'interno della mailbox sul server a scopo di archiviazione.

4.5.5 Multipurpose Internet Mail Extensions (MIME)

La posta elettronica ha una struttura molto elementare, ma la sua semplicità comporta che il protocollo possa gestire soltanto messaggi nel formato standard NVT ASCII a 7 bit. Vi sono quindi delle

limitazioni; per esempio non possono essere inviati messaggi in lingue che non siano supportate dai caratteri ASCII a 7 bit e non possono essere inviate file binari contenenti audio o video.

Il protocollo MIME (Multipurpose Internet Mail Extensions) è un protocollo supplementare che permette l'invio di messaggi in formati diversi dall'ASCII. Esso agisce lato mittente convertendo tutti i dati in formato non ASCII al formato NVT ASCII e consegnando il risultato al client MTA per l'invio. Il messaggio viene ritrasformato al formato originale presso il destinatario.

In conclusione si può pensare al protocollo MIME come a un insieme di funzioni che si occupano di tradurre dati non ASCII in formato NVT ASCII e viceversa.

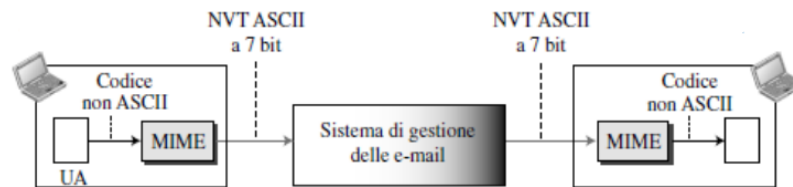


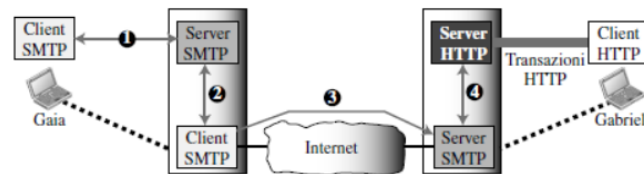
Figura 4.14: Il protocollo MIME

4.5.6 Webmail

La posta elettronica è un'applicazione talmente comune che vari siti Web offrono questo servizio a chiunque si registri, per esempio Hotmail, Yahoo e Google Mail.

Esempio 4.5.3 – Solo il destinatario usa HTTP

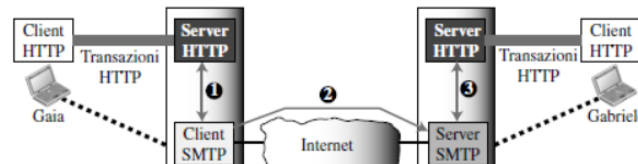
Gaia, il mittente, utilizza un server di posta tradizionale. Gabriele, il destinatario, ha un account su un server di Webmail.



Il trasferimento della posta dall'user agent di Gaia al suo server di posta viene effettuato tramite SMTP. Il trasferimento del messaggio dal server di posta mittente al server di posta destinatario viene nuovamente effettuato via SMTP. Tuttavia il trasferimento del messaggio dal server di destinazione (il server Web) al browser di Gabriele viene effettuato via HTTP. In altri termini, invece di utilizzare POP3 o IMAP si impiega HTTP (o preferibilmente HTTPS). Quando Gabriele vuole prelevare la sua posta, invia un messaggio HTTP di richiesta al sito Web (Hotmail per esempio). Il sito Web invia una form che Gabriele deve completare, che comprende nome utente e password. Se i parametri forniti sono corretti, l'elenco di e-mail viene trasferito dal server Web al browser di Gabriele in formato HTML. Ora Gabriele può scorrere l'elenco e visualizzare le proprie e-mail una alla volta.

Esempio 4.5.4 – Il mittente e il destinatario usano HTTP

Sia Gabriele che Gaia usano un server Web, non necessariamente lo stesso.



Gaia invia la mail e tutti i dati necessari (per esempio, l'indirizzo di Gabriele) al server Web attraverso HTTP. Il server Web usato da Gaia trasferisce il messaggio al client SMTP e lo invia al server di posta di Gabriele usando il protocollo SMTP. Gabriele riceve il messaggio tramite HTTP.

Capitolo 5

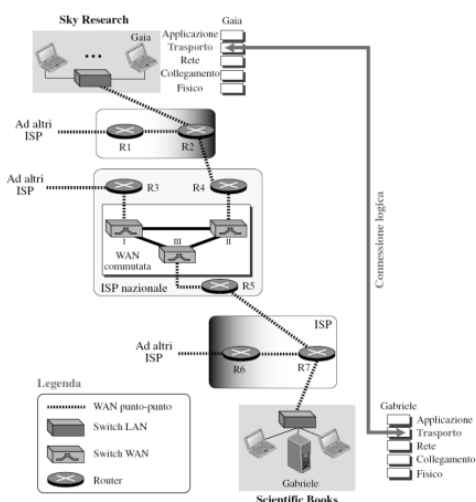
Livello trasporto

5.1 Introduzione

Il livello trasporto si colloca fra il livello rete e il livello applicazione, e fornisce un servizio di comunicazione tra processi fra i due livelli applicazione, uno sull'host locale e l'altro sull'host remoto. Il servizio di comunicazione viene fornito tramite una connessione logica, che consente ai due livelli applicazione (che possono essere situati in qualsiasi parte del mondo) di lavorare come se ci fosse una connessione diretta attraverso la quale inviare e ricevere i messaggi.

Esempio 5.1.1 – Livello trasporto

L'host di Gaia nell'azienda Sky Research crea una connessione logica a livello trasporto con l'host di Gabriele nell'azienda Scientific Books. Le due aziende comunicano a livello trasporto come se fra di loro esistesse un canale di comunicazione reale. Da notare che solamente i due sistemi finali (i computer di Gabriele e di Gaia) usano i servizi del livello trasporto, mentre i router intermedi utilizzano solo i primi tre livelli dell'architettura di rete.



5.1.1 I servizi del livello trasporto

Il livello trasporto si colloca fra il livello rete e il livello applicazione: fornisce quindi servizi al livello applicazione e riceve servizi dal livello rete.

Comunicazione tra processi

Il primo compito di un protocollo di livello trasporto è supportare la comunicazione tra processi. Un processo è un'entità (programma in esecuzione) di livello applicazione che usa i servizi del livello trasporto. Prima di descrivere la comunicazione tra processi, vediamo la comunicazione tra host.

Il livello rete si occupa della comunicazione tra dispositivi, ovvero della comunicazione tra host. Un protocollo di rete consegna i messaggi al computer destinatario, ma questo tipo di consegna è incompleta, perché i messaggi devono giungere ai processi cui erano indirizzati (sullo stesso host possono essere attivi svariati processi applicativi). E' a questo punto che intervengono i protocolli del livello trasporto, che completano la consegna passando i messaggi ai processi applicativi destinatari.

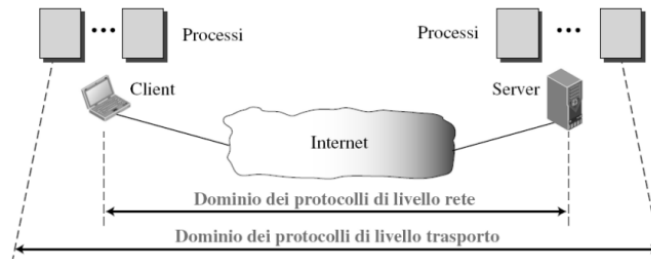


Figura 5.1: Livello rete e livello trasporto a confronto

Indirizzamento: i numeri di porta

Sebbene la comunicazione tra processi possa essere realizzata in diversi modi, la più diffusa è sicuramente quella basata sul paradigma client/server. Un processo sulla macchina locale, il client, necessita di servizi svolti da un processo solitamente su una macchina remota, il server. Va osservato, però, che la maggior parte dei sistemi operativi attuali è multiutente e multiprocesso, quindi l'host locale ha spesso diversi processi client attivi, così come l'host remoto può avere diversi processi server attivi. Affinché si possa stabilire una comunicazione tra i due dispositivi, è necessario un metodo per individuare: l'host locale, l'host remoto, il processo locale e il processo remoto. Gli host vengono individuati per mezzo del loro indirizzo IP.

Al client viene assegnato un numero di porta che viene detto effimero, ovvero "di breve durata", dato il breve tempo di vita di un processo client.

Anche al server deve essere associato un numero di porta, che, però, non può essere scelto a caso, perché tale numero deve essere in qualche modo noto al client. Una soluzione potrebbe essere l'invio di un pacchetto al fine di scoprire questo numero di porta, ma ciò creerebbe un carico inutile sulla rete. Nei protocolli TCP/IP si usano numeri universali per i server, che sono detti numeri di porta noti (well-known). Ogni client conosce il numero di porta noto del server corrispondente. Per esempio, mentre il client Daytime può usare il numero di porta effimero 52000, il server Daytime deve usare necessariamente il numero di porta noto 13.

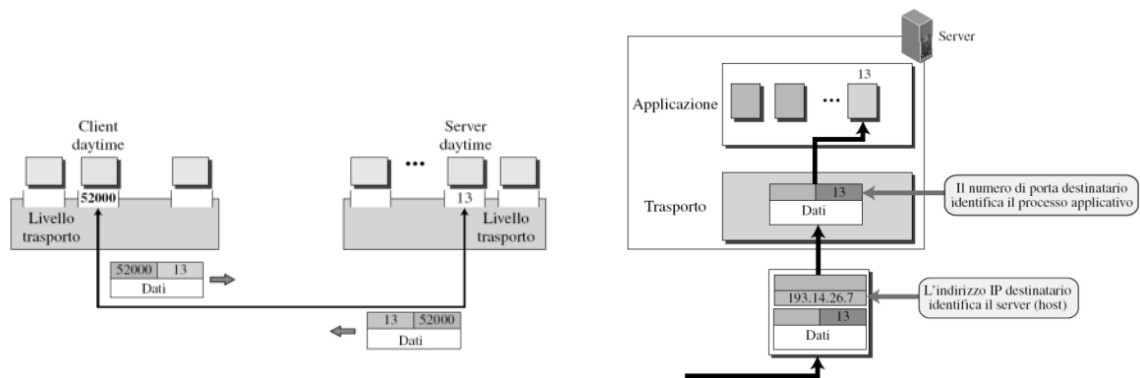


Figura 5.2: Numeri di porta

Deve essere chiaro che gli indirizzi IP e i numeri di porta giocano ruoli ben distinti nel selezionare la destinazione finale. L'indirizzo IP individua un host tra gli innumerevoli host di tutta la rete mondiale; il numero di porta individua uno dei processi applicativi del particolare host individuato dall'indirizzo IP.

Incapsulamento/decapsulamento

Per inviare un messaggio da un processo a un altro, il protocollo di livello trasporto incapsula e decapsula i messaggi. L'incapsulamento avviene dal lato del mittente. Quando un processo deve inviare un messaggio, lo passa al livello trasporto insieme ad altre informazioni, che dipendono dal protocollo del livello trasporto. Il livello trasporto riceve i dati e vi aggiunge la propria intestazione

(incapsulando il messaggio applicativo in un pacchetto del livello trasporto). I pacchetti a livello trasporto sono chiamati segmenti (nel protocollo TCP) o datagrammi utente (nel protocollo UDP).

Il decapsulamento avviene dal lato del destinatario. Quando il messaggio arriva al livello trasporto del destinatario, l'intestazione viene rimossa e il messaggio (decapsulato) viene consegnato al processo a livello applicazione.

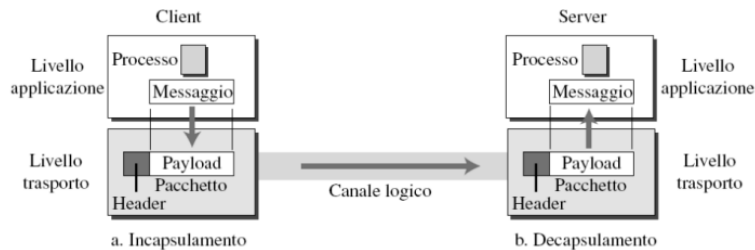


Figura 5.3: Incapsulamento e decapsulamento

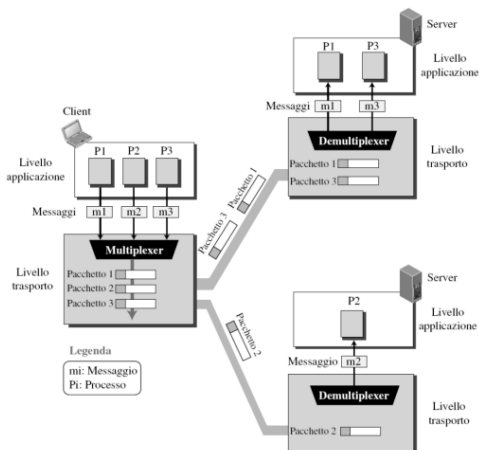
Multiplexing e demultiplexing

Il termine multiplexing (molti a uno) fa riferimento al caso in cui un'entità riceve informazioni da più di una sorgente, il termine demultiplexing fa riferimento al caso in cui un'entità trasmette informazioni a più di un destinatario. Il livello trasporto effettua il multiplexing nel sito mittente e il demultiplexing nel sito destinatario.

Esempio 5.1.2 – Multiplexing e demultiplexing

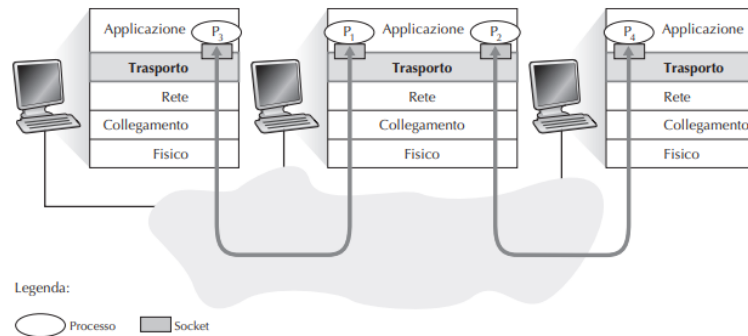
Tre processi client, P1, P2 e P3, sono in esecuzione su un host. I processi P1 e P3 devono inviare delle richieste al processo server corrispondente, in esecuzione su un server. Il processo client P2 deve inviare una richiesta al processo server corrispondente in esecuzione su un altro server.

Il livello trasporto nel client accetta i tre messaggi dai tre processi e crea tre pacchetti: agisce da multiplexer. I pacchetti 1 e 3 usano lo stesso canale logico per raggiungere il livello trasporto del primo server. Quando arrivano al server, il livello trasporto agisce da demultiplexer distribuendo i messaggi a due processi differenti. Il livello trasporto nel secondo server riceve il pacchetto 2 e lo consegna al processo corrispondente. Si noti che anche in questo caso si ha comunque un'operazione di demultiplexing sebbene vi sia un solo messaggio.



Esempio 5.1.3 – Multiplexing e demultiplexing

Supponiamo che vi troviate di fronte al vostro computer e che stiate scaricando pagine web mentre sono in esecuzione una sessione FTP e due sessioni Telnet. Avrete pertanto quattro processi applicativi in esecuzione: due Telnet, uno FTP e uno HTTP.

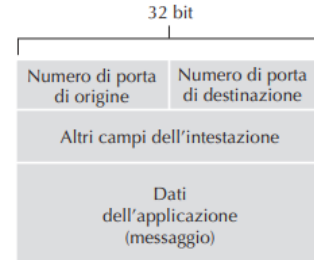


Il livello di trasporto nel vostro calcolatore, quando riceve dati dal livello di rete sottostante, li deve indirizzare a uno di questi quattro processi. Di conseguenza il livello di trasporto nell'host di ricezione in realtà non trasferisce i dati direttamente a un processo, ma piuttosto a una socket che fa da intermediario. Siccome, a ogni dato istante, può esserci più di una socket nell'host di ricezione, ciascuna avrà un identificatore univoco il cui formato dipende dal fatto che si tratti di socket UDP o TCP.

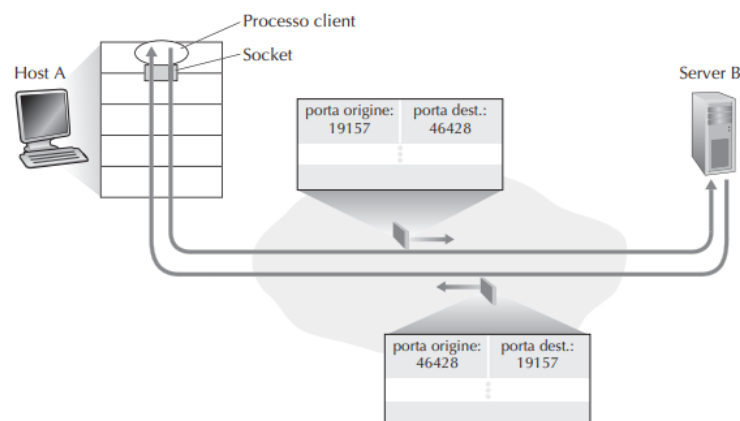
Il compito di trasportare i dati dei segmenti a livello di trasporto verso la giusta socket viene detto demultiplexing. Il compito di radunare frammenti di dati da diverse socket sull'host di origine e incapsulare ognuno con intestazioni a livello di trasporto (che verranno poi utilizzate per il demultiplexing) per creare dei segmenti e passarli al livello di rete, viene detto multiplexing.

Si noti che il livello di trasporto nell'host centrale deve effettuare il demultiplexing dal livello di rete di segmenti che possono arrivare sia per il processo P1 sia per P2; ciò avviene indirizzando i dati del segmento in ingresso alla giusta socket. Il livello di trasporto nell'host centrale deve, inoltre, raccogliere i dati in uscita dalle socket dei due processi, creare i segmenti a livello di trasporto e passarli al livello di rete.

Ora che abbiamo compreso i ruoli del multiplexing e del demultiplexing a livello di trasporto esaminiamo come vengono realizzati negli host. Da quanto visto in precedenza, sappiamo che il multiplexing a livello di trasporto richiede (1) che le socket abbiano identificatori unici e (2) che ciascun segmento presenti campi che indichino la socket cui va consegnato il segmento. Questi sono il campo del numero di porta di origine e il campo del numero di porta di destinazione. I numeri di porta sono di 16 bit e vanno da 0 a 65535, quelli che vanno da 0 a 1023 sono chiamati numeri di porta noti (well-known port number) e sono riservati per essere usati da protocolli applicativi ben noti quali HTTP (porta 80) e FTP (porta 21).



Esempio 5.1.4 – Numero di porta



Per quanto riguarda il numero di porta nel segmento che va da A verso B, il numero di porta

di origine serve come parte di un “indirizzo di ritorno”: quando B vuole restituire il segmento ad A, la porta di destinazione del segmento da B verso A assumerà il valore della porta di origine del segmento da A verso B. L’indirizzo di ritorno completo è costituito dall’indirizzo IP di A più il numero di porta di origine.

5.1.2 API di comunicazione

Come può un processo client comunicare con un processo server? Se si vuole sviluppare un programma capace di comunicare con un altro programma, è necessario un nuovo insieme di istruzioni per chiedere ai primi quattro livelli dello stack TCP/IP di aprire la connessione, inviare/ricevere dati e chiudere la connessione. Un insieme di istruzioni di questo tipo viene chiamato API (Application Programming Interface). Nella programmazione, un’interfaccia non è altro che un insieme di istruzioni usato per l’interazione tra due entità. In questo caso una delle entità è il processo a livello applicazione e l’altra è il sistema operativo che implementa i primi quattro livelli dello stack TCP/IP. Sono state sviluppate diverse API di comunicazione, tra le quali tre sono le più note: l’interfaccia socket (socket interface), TLI (Transport Layer Interface) e STREAMS.

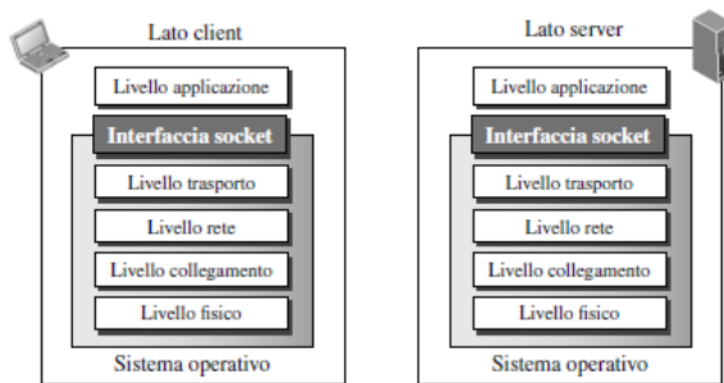


Figura 5.4: L’interfaccia socket nella pila dei protocolli

Socket

L’interfaccia socket è un insieme di istruzioni che consentono la comunicazione fra il livello applicazione e il sistema operativo; è un insieme di istruzioni che possono essere utilizzate da un processo per comunicare con un altro processo.

Sebbene una socket possa apparire come un terminale o un file, non è un’entità fisica di questo tipo: è un’astrazione. Si tratta in realtà di una struttura dati creata e utilizzata dal programma applicativo. Si può dire che, per quanto riguarda il livello applicazione, comunicare fra un processo client e un processo server significa comunicare tra due socket create nei due lati della comunicazione.

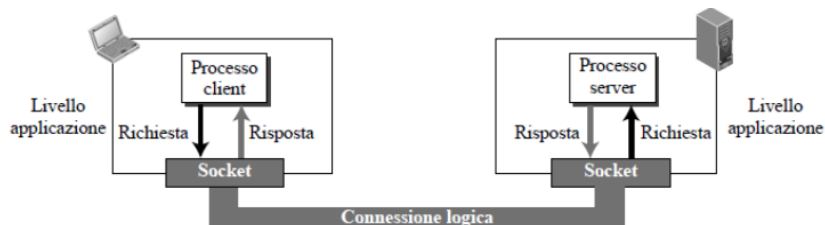


Figura 5.5: Impiego delle socket nella comunicazione tra processi

Il client considera la socket come l’entità che riceve le richieste e fornisce le risposte; il server vede la socket come l’entità che gli sottopone le richieste e alla quale inviare le risposte. Se si creano due socket, una a ciascun lato della comunicazione, definendo correttamente gli indirizzi mittente e destinatario, si possono utilizzare tutte le istruzioni disponibili per inviare e ricevere dati.

Socket address

L'interazione fra client e server è una comunicazione bidirezionale. Nella comunicazione bidirezionale si rende necessaria una coppia di indirizzi: locale (mittente) e remoto (destinatario). L'indirizzo locale in una direzione è l'indirizzo remoto nell'altra direzione e viceversa. Poiché la comunicazione nel paradigma client/server avviene fra due socket, è necessaria una coppia di socket address (indirizzi delle socket) per la comunicazione: un socket address locale ed uno remoto.

Un socket address deve innanzitutto identificare l'host sul quale il processo client o server è in esecuzione, definito univocamente dal proprio indirizzo IP, che solitamente è un intero a 32 bit. Ma sullo stesso computer potrebbero essere eseguiti contemporaneamente diversi processi client o server, di conseguenza è necessario utilizzare un altro identificatore per specificare il particolare client o server coinvolto nella comunicazione: il numero di porta, un intero a 16 bit. Un socket address sarà quindi composto dalla combinazione di un indirizzo IP e un numero di porta.



Dato che una socket specifica gli end-point (estremi) della comunicazione, è possibile affermare che una socket è identificata da una coppia di socket address, uno locale e uno remoto.

Individuare i socket address

Come può un client o un server individuare la coppia di socket address per effettuare la comunicazione? La situazione è differente a seconda del lato della comunicazione.

- *Lato server.* Il server ha bisogno di un socket address locale (server) e un socket address remoto (client) per comunicare.

Il socket address locale (server) viene fornito dal sistema operativo. Il sistema operativo conosce l'indirizzo IP del computer sul quale il processo server è in esecuzione. Il numero di porta di un processo server, invece, deve essere assegnato.

Il socket address remoto di un server è il socket address del client che si collega. Poiché il server può servire numerosi client, non può conoscere in anticipo il loro indirizzo, ma lo può identificare quando un client cerca di connettersi. Il socket address del client, contenuto nel pacchetto di richiesta inviato al server, diviene il socket address remoto utilizzato per rispondere al client.

Il socket address locale di un server è fissato e rimane invariato, mentre il socket address remoto varia a ogni interazione con un client differente.

- *Lato client.* Per comunicare, anche il client ha bisogno di due socket address, uno locale e uno remoto.

Anche il socket address locale (client) viene fornito dal sistema operativo, che conosce l'indirizzo IP del computer sul quale è in esecuzione il client. Il numero di porta invece è un intero di 16 bit che viene assegnato temporaneamente al client ogni volta che questo desidera iniziare una comunicazione. Il numero di porta deve essere scelto fra un insieme di interi definiti dagli enti di gestione di Internet chiamati numeri di porta effimeri (o temporanei). Il sistema operativo deve garantire che il numero di porta così assegnato non sia utilizzato da alcun altro processo client contemporaneamente in esecuzione.

Individuare il socket address remoto (server) comporta operazioni un po' più complesse. Un processo client, quando viene avviato, deve conoscere il socket address del server al quale vuole connettersi. Si hanno due possibili situazioni.

1. In alcuni casi l'utente che avvia il processo client conosce sia il numero di porta che l'indirizzo IP del computer sul quale è in esecuzione il server.
2. Sebbene tutte le applicazioni standard abbiano un numero di porta noto, nella maggior parte dei casi non si conosce l'indirizzo IP: il processo client solitamente conosce il numero di porta, trattandosi di un numero di porta noto, mentre l'indirizzo IP può essere ottenuto utilizzando un'altra applicazione client/server chiamata DNS (Domain Name System).

5.1.3 Protocolli Internet di livello trasporto

Avendo discusso i concetti generali relativi al livello trasporto, si approfondiscono ora due dei protocolli di livello trasporto impiegati in Internet, UDP e TCP. Questi protocolli sono situati fra il livello

applicazione e il livello rete e agiscono da intermediari fra i programmi applicativi e le operazioni della rete.

UDP è un protocollo di trasporto inaffidabile e privo di connessione, utilizzato per la sua semplicità ed efficienza nei casi in cui il controllo degli errori può essere gestito dai processi di livello applicazione. TCP è un protocollo affidabile, orientato alla connessione, che può essere utilizzato nelle applicazioni dove l'affidabilità è particolarmente importante.

5.2 Protocollo UDP (User Datagram Protocol)

UDP (User Datagram Protocol) è un protocollo del livello trasporto inaffidabile e privo di connessione. Non aggiunge nulla ai servizi di IP eccetto la comunicazione tra processi. Per quale motivo un processo dovrebbe utilizzare UDP, se offre così pochi servizi? Gli svantaggi hanno come contropartita alcuni vantaggi. UDP è un protocollo molto semplice con un overhead (carico aggiuntivo) minimo. Se un processo vuole inviare un piccolo messaggio senza doversi preoccupare troppo dell'affidabilità, si può utilizzare UDP. Inviare un messaggio con UDP richiede meno interazioni fra il mittente e il destinatario rispetto all'impiego del TCP.

Il mittente invia pacchetti uno dopo l'altro senza pensare al destinatario (può inviare dati a raffica perché non c'è controllo di flusso né di congestione).

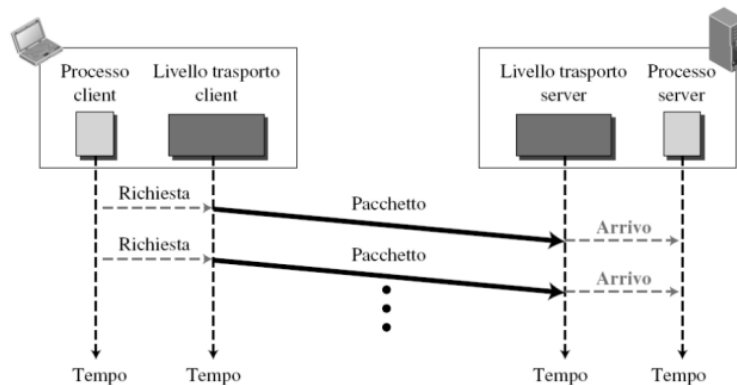


Figura 5.6: Diagramma di comunicazione

Il mittente deve dividere i suoi messaggi in porzioni di dimensioni accettabili dal livello di trasporto, a cui consegnarli uno per uno. Ogni pacchetto è indipendente dagli altri (la sequenza di arrivo può essere diversa da quella di spedizione). Non c'è coordinazione tra livello trasporto mittente e destinatario. Non è possibile implementare efficacemente controllo di flusso, controllo degli errori, controllo della congestione.

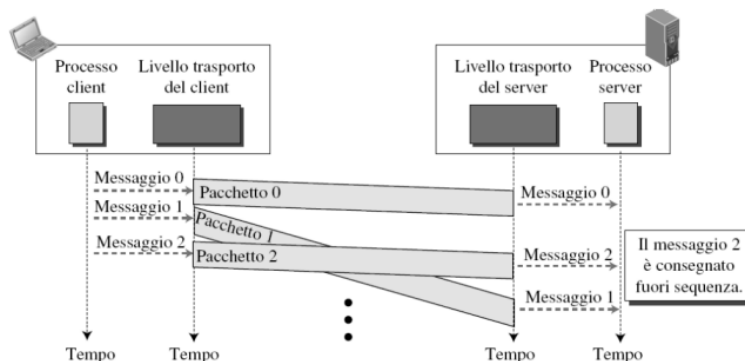


Figura 5.7: Servizi privi di connessione

Il comportamento di un protocollo di trasporto può essere rappresentato da un automa a stati finiti. L'automa rimane in uno stato fin quando non avviene un evento che può modificare lo stato dell'automa (transizione di stato) e fargli compiere un'azione.

5.2.1 Protocollo semplice

Un primo protocollo semplice, privo di connessione, senza controllo degli errori e del flusso suppone che il destinatario sa in grado di gestire immediatamente tutti i pacchetti ricevuti; in altri termini, il destinatario non viene mai sovraccaricato dai pacchetti in arrivo.



Figura 5.8: Protocollo semplice

Il livello trasporto mittente riceve un messaggio dal proprio livello applicazione, ne crea un pacchetto e lo invia. Il livello trasporto destinatario riceve il datagramma dal proprio livello rete, ne estrae il messaggio e lo consegna al proprio livello applicazione. I livelli trasporto del mittente e del destinatario forniscono quindi servizi di comunicazione ai rispettivi livelli applicazione.

Il mittente non può inviare pacchetti fino a quando il suo livello applicazione non ha un messaggio da spedire. Analogamente il destinatario non può consegnare un messaggio al proprio livello applicazione fino a quando non riceve un pacchetto. E' possibile rappresentare questi due vincoli usando due FSM, ognuna costituita dal solo stato ready (pronto). La macchina mittente rimane nello stato ready fino a quando arriva una richiesta dal processo nel livello applicazione. Quando avviene questo evento, la macchina mittente incapsula il messaggio in un pacchetto e lo invia alla macchina destinataria. La macchina destinataria rimane nello stato ready fino a quando non arriva un pacchetto dal mittente. Quando avviene questo evento, la macchina destinataria estrae il messaggio dal pacchetto e lo consegna al processo nel livello applicazione.

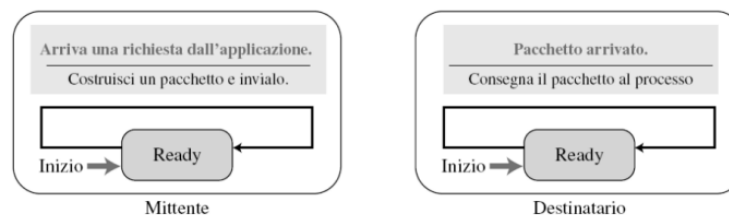


Figura 5.9: FSM per il protocollo semplice

5.2.2 Struttura dei datagrammi utente

I pacchetti UDP sono chiamati datagrammi utente; la loro intestazione è costituita da 4 campi di 2 byte (16 bit) per un totale di 8 byte fissi. I primi due campi definiscono i numeri di porta di mittente e destinatario. Il terzo campo definisce la lunghezza totale, intestazione più dati, del datagramma utente. I 16 bit possono definire una lunghezza totale da 0 a 65535 byte, ma in realtà la dimensione è inferiore perché il datagramma utente UDP viene inserito in un datagramma IP con lunghezza totale di 65535 byte. L'ultimo campo può contenere il checksum opzionale.

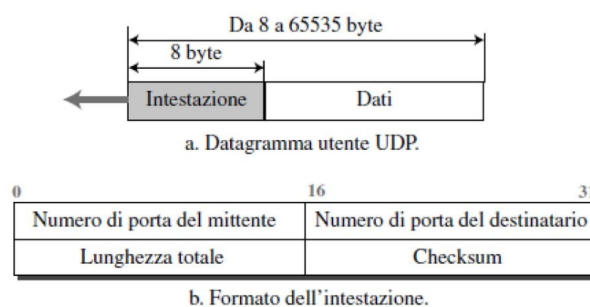


Figura 5.10: Formato dei datagramma utente

5.2.3 Servizi UDP

Checksum

Il checksum UDP serve per il rilevamento degli errori. In altre parole, viene utilizzato per determinare se i bit del segmento UDP sono stati alterati durante il loro trasferimento (per esempio, a causa di disturbi nei collegamenti o quando sono stati memorizzati in un router) da sorgente a destinazione. Lato mittente UDP effettua il complemento a 1 della somma di tutte le parole da 16 bit nel segmento, e l'eventuale riporto finale viene sommato al primo bit. Tale risultato viene posto nel campo checksum del segmento UDP.

Esempio 5.2.1 – Calcolo del checksum

Calcolare il checksum della seguente stringa di 32 bit: 11100110011001101101010101010101. Viene effettuata la somma tra le due metà e il riporto viene sommato al primo bit:

$$\begin{array}{r}
 1110011001100110 \quad + \\
 1101010101010101 \quad = \\
 \hline
 1101110111011101 \\
 \hline
 1011101110111100
 \end{array}$$

Il complemento a 1 si ottiene convertendo i bit 0 in 1 e viceversa. Di conseguenza, il checksum sarà:

0100010001000011

DNS usa UDP

DNS è un tipico esempio di protocollo a livello applicativo che utilizza UDP. Quando l'applicazione DNS in un host vuole effettuare una query, costruisce un messaggio di query DNS e lo passa a UDP. L'entità UDP in esecuzione sul sistema di destinazione aggiunge i campi d'intestazione al messaggio e trasferisce il segmento risultante al livello di rete. Quest'ultimo incapsula il segmento UDP in un datagramma e lo invia a un server DNS. L'applicazione DNS sull'host che ha effettuato la richiesta aspetta quindi una risposta. Se non ne riceve (magari perché la rete sottostante ha smarrito la richiesta o la risposta), l'applicazione tenta di inviare la richiesta a un altro DNS server oppure informa l'applicazione dell'impossibilità di ottenere una risposta. La semplicità della richiesta/risposta (molto breve) motiva l'utilizzo di UDP, che risulta più veloce. UDP è utilizzato anche perché consente un controllo più sottile a livello di applicazione su quali dati sono inviati e quando.

5.3 Protocollo TCP (Transmission Control Protocol)

Il protocollo TCP (Transmission Control Protocol) è un protocollo orientato alla connessione e affidabile. Per fornire un servizio orientato alla connessione TCP prevede esplicitamente i meccanismi di apertura della connessione, di trasferimento dei dati e di chiusura della connessione. Per fornire un servizio affidabile utilizza una combinazione di meccanismi GBN e SR, con il checksum per la rilevazione degli errori, la ritrasmissione dei segmenti smarriti o corrotti, la conferma cumulativa e selettiva e opportuni timer. TCP è il protocollo di livello trasporto più utilizzato in Internet.

Il protocollo TCP fornisce un servizio di trasporto affidabile tra processi basato, come per UDP, sui numeri di porta. Analogamente a UDP, TCP effettua il multiplexing in trasmissione e il demultiplexing in ricezione. A differenza di UDP, TCP è un protocollo orientato alla connessione, pertanto è necessario stabilire una connessione per ogni coppia di processi prima di avviare la comunicazione tra processi. Oltre ad essere orientato alla connessione, il protocollo TCP è anche orientato al flusso di dati (stream-oriented), ovvero permette al livello applicazione di trasmettere un flusso di dati.



Figura 5.11: Trasmissione di un flusso

5.3.1 Servizi TCP

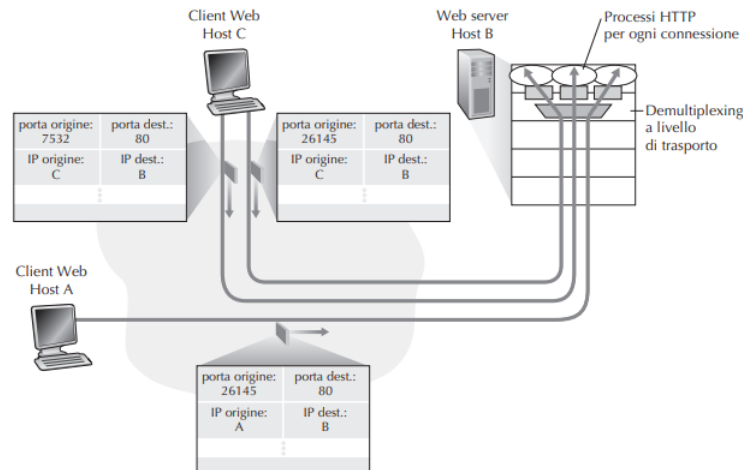
Demultiplexing orientato alla connessione

Una socket TCP è identificata da quattro parametri: indirizzo IP di origine, numero di porta di origine, indirizzo IP di destinazione e numero di porta di destinazione. Pertanto, quando un segmento TCP giunge dalla rete in un host, quest'ultimo utilizza i quattro valori per dirigere (fare demultiplexing) il segmento verso la socket appropriata. In particolare, due segmenti TCP in arrivo, aventi indirizzi IP di origine o numeri di porta di origine diversi, vengono diretti a due socket differenti, anche a fronte di indirizzo IP e porta di destinazione uguali, con l'eccezione dei segmenti TCP che trasportano la richiesta per stabilire la connessione.

L'host server può ospitare più socket TCP contemporanee collegate a processi diversi, ognuna identificata da una specifica quaterna di valori.

Esempio 5.3.1 – Demultiplexing orientato alla connessione

L'Host C dà inizio a due sessioni HTTP verso il server B, mentre l'Host A apre una sessione verso B. Gli Host A e C e il server B hanno ciascuno il proprio indirizzo IP, indicati rispettivamente con A, C e B. L'Host C assegna due diversi numeri di porta di origine (26145 e 7532) alle sue due connessioni HTTP.



Dato che l'Host A sta scegliendo i numeri di porta di origine indipendentemente da C, potrebbe anch'esso attribuire una porta di origine 26145 alla sua connessione HTTP. Ma questo non è un problema: il server B sarà ancora in grado di fare correttamente demultiplexing delle due connessioni che, pur avendo lo stesso numero di porta di origine, hanno indirizzi IP differenti.

Servizio orientato alla connessione

In un servizio orientato alla connessione, il client e il server devono per prima cosa stabilire una connessione logica. Solo dopo questa fase è possibile scambiare dati utente. Una volta terminato lo scambio dei dati, è necessario chiudere la connessione.

A livello trasporto il servizio orientato alla connessione coinvolge solamente i due host finali: si tratta di un servizio end-to-end (tra le due estremità). Questo significa che dovrebbe essere possibile utilizzare un protocollo orientato alla connessione a livello trasporto indipendentemente dal proto-

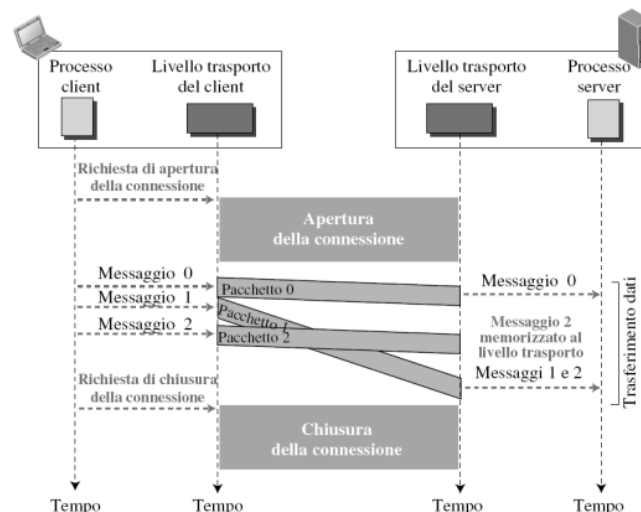


Figura 5.12: Servizi orientati alla connessione

collo di livello rete (con o senza connessione). In un protocollo orientato alla connessione è possibile implementare il controllo di flusso, il controllo degli errori e il controllo della congestione.

Rappresentazione tramite FSM

Il comportamento di un protocollo del livello trasporto può essere ben rappresentato con una macchina o automa a stati finiti (FSM, Finite State Machine). La FSM di un livello trasporto orientato alla connessione deve passare attraverso tre stati prima di raggiungere lo stato established. La macchina è nello stato closed (chiuso) quando non vi è connessione e rimane in questo stato fino a quando arriva una richiesta di apertura della connessione dal processo locale: la macchina invia un pacchetto di richiesta di apertura al livello trasporto remoto e si sposta allo stato open-wait-I (attesa di apertura I). La FSM locale, quando riceve il riscontro dall'altro lato, si sposta allo stato open-wait-II (attesa di apertura II). Quando la macchina è in questo stato, è stata completata l'apertura di una connessione unidirezionale; se è richiesta una connessione bidirezionale, la macchina deve rimanere in attesa in questo stato fino a quando anche l'altro lato richiede l'apertura della connessione. Quando riceve la richiesta, la macchina invia un riscontro e passa allo stato established. I due lati possono scambiarsi dati e relativi riscontri mentre sono entrambi nello stato established.

Per chiudere una connessione, il livello applicazione invia un messaggio di richiesta di chiusura al proprio livello trasporto. Questo invia un pacchetto di richiesta di chiusura all'altro lato e si sposta nello stato close-wait-I. Quando riceve un riscontro dall'altro lato, la macchina si sposta nello stato close-wait-II dove attende il pacchetto con la richiesta di chiusura dall'altro lato. All'arrivo del pacchetto la macchina invia un riscontro e si sposta nello stato closed.

Controllo di flusso

Quando un'entità produce dati che un'altra entità deve consumare, deve esistere un equilibrio fra la velocità di produzione e la velocità di consumo dei dati. Se i dati vengono prodotti a una velocità superiore rispetto a quella con cui possono essere consumati, il consumatore potrebbe trovarsi sovraccaricato ed essere costretto a eliminarne alcuni. Se i dati vengono prodotti a una velocità inferiore rispetto a quella con cui possono essere consumati, il consumatore deve rimanere in attesa, riducendo l'efficienza del sistema. Il controllo di flusso è legato alla prima problematica, dove si rende necessario evitare la perdita di dati al destinatario.

Nella comunicazione al livello trasporto si ha a che fare con quattro entità: il processo mittente, il livello trasporto del mittente, il livello trasporto del destinatario e il processo destinatario. Il processo mittente a livello applicazione è solo un produttore: produce dei dati e li passa al livello trasporto. Il livello trasporto del mittente ha un doppio ruolo: di consumatore e di produttore. Consuma i messaggi in arrivo dal produttore, li incapsula in pacchetti e li trasmette al livello trasporto del destinatario. Anche il livello trasporto del destinatario ha un doppio ruolo: è consumatore dei pacchetti in arrivo dal mittente ed è il produttore che decapsula i messaggi e li consegna al livello applicazione.

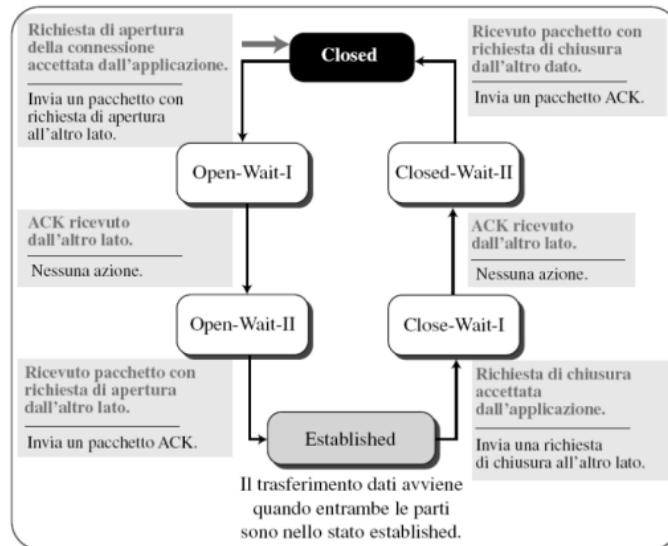


Figura 5.13: FSM per il livello trasporto orientato alla connessione

L'ultima consegna, tuttavia, agisce solitamente in pulling: il livello trasporto attende fino a quando il processo di livello applicazione richiede dati. Sono necessari almeno due casi di controllo di flusso: dal livello trasporto al livello applicazione dal lato mittente, e dal livello trasporto del destinatario al livello trasporto del mittente.



Figura 5.14: Controllo del flusso al livello trasporto

Sebbene il controllo di flusso possa essere realizzato in modi differenti, una delle soluzioni classiche consiste nell'utilizzare due buffer: uno al livello trasporto del mittente e l'altro al livello trasporto del destinatario. Un buffer è un insieme di locazioni di memoria che possono contenere dei pacchetti. La comunicazione delle informazioni di controllo di flusso può avvenire inviando segnali dal consumatore al produttore.

Il livello trasporto del mittente segnala al livello applicazione, quando ha il buffer saturo, di sospendere l'invio di messaggi; quando si libera dello spazio nel buffer segnala al livello applicazione che può riprendere l'invio di messaggi.

Controllo degli errori

In Internet, dato che il livello rete (IP) è inaffidabile, è necessario implementare l'affidabilità al livello trasporto se richiesta dall'applicazione. L'affidabilità può essere ottenuta aggiungendo i servizi di controllo degli errori al livello trasporto, che ha la responsabilità di:

- rilevare e scartare i pacchetti corrotti;
- tenere traccia dei pacchetti persi e scartati e gestirne la rispeditizione;
- riconoscere i pacchetti duplicati ed eliminarli;
- bufferizzare i pacchetti fuori sequenza fino a quando arrivano i pacchetti mancanti.

Il controllo degli errori, a differenza del controllo di flusso, coinvolge solamente i livelli trasporto del mittente e del destinatario. Si suppone infatti che i messaggi scambiati fra i livelli applicazione siano esenti da errori. Il livello trasporto del destinatario gestisce il controllo degli errori, nella maggior parte dei casi, segnalando il problema al livello trasporto del mittente.



Figura 5.15: Controllo degli errori al livello trasporto

Il controllo degli errori comporta che il livello trasporto del mittente sappia quale pacchetto debba rispedire e che il livello trasporto del destinatario sappia riconoscere i pacchetti duplicati o fuori sequenza. Questo può essere ottenuto numerando i pacchetti, ovvero aggiungendo ai pacchetti del livello trasporto un campo che indica il loro numero di sequenza. Quando un pacchetto viene corrotto o smarrito, il livello trasporto del destinatario può chiedere al livello trasporto del mittente di rispedirlo, identificandolo tramite il numero di sequenza. Il livello trasporto del ricevente può identificare anche i pacchetti duplicati, quando riceve due pacchetti con il medesimo numero di sequenza. I pacchetti fuori sequenza possono infine essere identificati verificando gli eventuali numeri di sequenza ricevuti disordinatamente.

I pacchetti vengono numerati sequenzialmente. Tuttavia, poiché è necessario inserire il numero di sequenza nell'intestazione di ogni pacchetto, occorre specificarne il valore massimo. Se l'intestazione del pacchetto prevede di allocare m bit per i numeri di sequenza, questi possono assumere i valori da 0 a $2^m - 1$. Se m vale 4, per esempio, si possono utilizzare i numeri di sequenza da 0 a 15. Naturalmente, raggiunto l'ultimo numero si può ricominciare dal primo. I numeri di sequenza utilizzati per il controllo degli errori sono considerati in modulo 2^m dove m è la dimensione del campo numero di sequenza espressa in bit.

Per notificare al mittente la corretta ricezione di uno o più pacchetti viene utilizzato il numero di riscontro (detto anche acknowledgment, ACK, o conferma). Il destinatario può semplicemente scartare (ignorare) i pacchetti corrotti. Il mittente può identificare i pacchetti persi utilizzando un timer. Quando invia un pacchetto, il mittente attiva un timer: se non riceve un ACK prima della sua scadenza rispedisce il pacchetto. Anche i pacchetti duplicati possono essere semplicemente scartati dal destinatario. I pacchetti fuori sequenza possono invece essere scartati (saranno trattati dal mittente come pacchetti persi) oppure temporaneamente memorizzati in attesa dell'arrivo dei pacchetti mancanti.

Integrazione del controllo degli errori e controllo di flusso

Si è visto che il controllo di flusso richiede l'impiego di due buffer, uno nel mittente e l'altro nel destinatario. Si è visto anche che il controllo degli errori richiede l'uso dei numeri di sequenza e di riscontri in entrambi i lati. Questi due requisiti possono essere combinati usando due buffer numerati, uno presso il mittente e uno presso il destinatario.

Nel mittente, quando si prepara un pacchetto si usa il numero della posizione libera successiva nel buffer, x , come numero di sequenza del pacchetto. Quando il pacchetto viene inviato, ne viene memorizzata una copia nella locazione x , in attesa della sua conferma da parte del destinatario. Quando si riceve una conferma relativa a un pacchetto inviato, il pacchetto memorizzato viene eliminato liberando la posizione di memoria che occupava. Un pacchetto con numero di sequenza y che arriva al destinatario, viene memorizzato nella locazione y fino a quando il livello applicazione è pronto a riceverlo. A quel punto può essere inviata la conferma di ricezione del pacchetto y .

Poiché il numero di sequenza è calcolato in modulo 2^m , i numeri di sequenza da 0 a $2^m - 1$ possono essere rappresentati con un cerchio. Il buffer viene rappresentato come un insieme di settori, chiamati finestra scorrevole (sliding window), che in ogni istante occupano una parte del cerchio. Nel mittente, quando viene inviato un pacchetto se ne contrassegna il settore corrispondente. Quando tutti i settori risultano contrassegnati, il buffer è completo e non possono essere accettati altri messaggi dal livello applicazione. Quando arriva un riscontro, viene eliminato il contrassegno dal settore corrispondente. Quando un insieme consecutivo di settori a partire dall'inizio della finestra risulta non contrassegnato, la finestra scorre sopra l'intervallo dei numeri di sequenza corrispondenti rendendo disponibile un numero di settori equivalente alla fine della finestra.

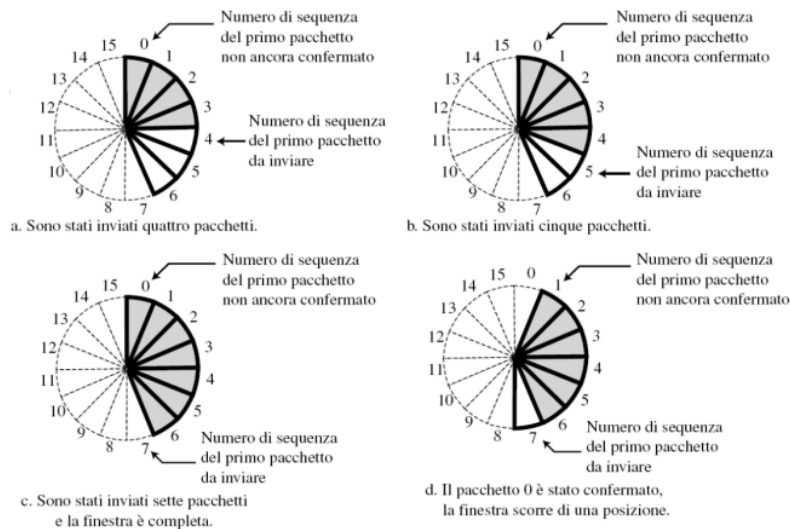


Figura 5.16: Finestra scorrevole con rappresentazione circolare

Si noti che la finestra scorrevole è solo un'astrazione: nella realtà si utilizzano delle variabili per contenere i numeri di sequenza del pacchetto successivo da inviare e dell'ultimo pacchetto inviato.

La maggior parte delle descrizioni dei protocolli rappresentano la finestra scorrevole con una rappresentazione lineare. L'idea è la stessa, ma la rappresentazione risultante è solitamente più sintetica.

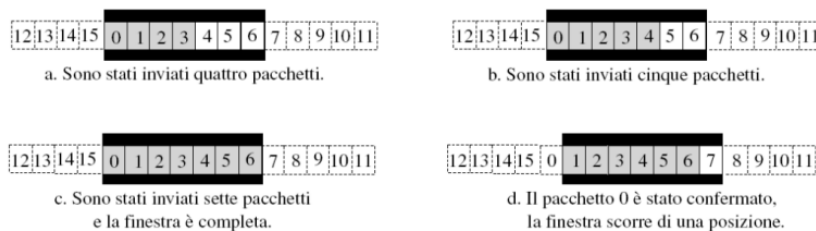


Figura 5.17: Finestra scorrevole con rappresentazione lineare

Controllo della congestione

La congestione in una rete può avvenire se il carico della rete - il numero di pacchetti inviati alla rete - è superiore alla capacità della rete - il numero di pacchetti che una rete può gestire. L'espressione controllo della congestione fa riferimento ai meccanismi e alle tecniche per controllare la congestione mantenendo il carico della rete al di sotto della sua capacità.

Ci si potrebbe chiedere per quale motivo esista la congestione in una rete: la congestione può avvenire in qualsiasi sistema che comporti un'attesa. La congestione in rete può avvenire poiché i router e gli switch hanno delle code - buffer dove vengono memorizzati i pacchetti prima e dopo la loro elaborazione. Un router, per esempio, ha una coda di input e una coda di output per ogni interfaccia. Se il router non riesce a elaborare i pacchetti alla stessa velocità con cui arrivano, le code si sovraccaricano e avviene la congestione. La congestione a livello trasporto è in realtà il risultato della congestione a livello rete che si manifesta a livello trasporto.

5.3.2 Stop-and-wait

Il secondo meccanismo preso in considerazione è orientato alla connessione ed è chiamato Stop and Wait; utilizza sia il controllo di flusso sia il controllo degli errori. Il mittente e il destinatario usano entrambi una finestra scorrevole di dimensione 1. Il mittente invia un pacchetto alla volta e ne attende l'ack (riscontro) prima di poter spedire il successivo. Per rilevare i pacchetti corrotti è necessario aggiungere un valore checksum (valore di controllo) a ogni pacchetto dati. Quando un pacchetto

arriva al destinatario, viene calcolato il checksum: se il valore corrisponde al checksum ricevuto, si invia un ack al mittente. Se il checksum non corrisponde, il pacchetto risulta corrotto e viene scartato senza informare il client. L'assenza di un ack da parte del destinatario costituisce un'indicazione per il mittente che il pacchetto è stato perso o scartato. Infatti, ogni volta che il mittente invia un pacchetto, inizializza un timer. Se arriva un ack prima che il timer sia scaduto, il timer viene arrestato e viene inviato il pacchetto successivo (se esiste un pacchetto pronto). Se invece il timer scade, il mittente rispedisce il pacchetto precedente, ipotizzando che sia stato corrotto o si sia smarrito. Questo comporta che il mittente deve tenere una copia del pacchetto fino a quando non ne riceve il riscontro. Si noti che in un dato istante, nel canale possono esserci un solo pacchetto e un solo riscontro.

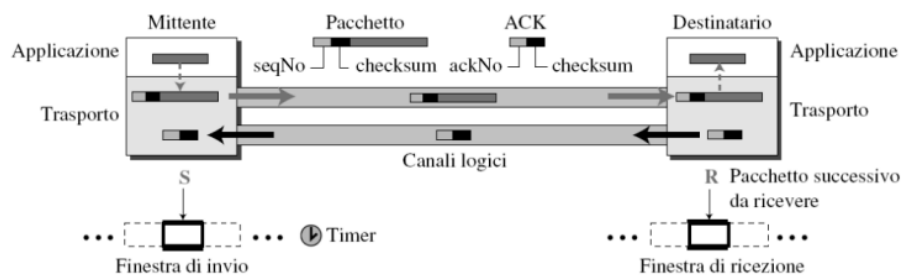


Figura 5.18: Stop and Wait

Il meccanismo Stop and Wait è orientato alla connessione e fornisce i servizi di controllo degli errori e del flusso (il controllo di flusso è garantito dal fatto che non si spedisce più di un pacchetto alla volta).

Numeri di sequenza e di riscontro

Per poter gestire i pacchetti duplicati, lo Stop and Wait utilizza i numeri di sequenza e di riscontro. Viene quindi aggiunto un campo all'intestazione del pacchetto, per contenere il valore del numero di sequenza del pacchetto. Una considerazione importante riguarda l'intervallo dei possibili valori dei numeri di sequenza: l'obiettivo è sempre minimizzare la dimensione dei pacchetti, o meglio delle informazioni di controllo nei pacchetti. Si vuole quindi identificare l'intervallo più piccolo possibile che possa consentire la comunicazione senza ambiguità. Ebbene: se si è utilizzato x come numero di sequenza, dopo di questo è sufficiente utilizzare il numero $x + 1$, mentre il numero $x + 2$ non è necessario. Per dimostrare questa affermazione, si supponga che il mittente abbia inviato il pacchetto con numero di sequenza x . Si possono verificare tre casi.

1. Il pacchetto arriva correttamente al destinatario, che invia un riscontro. Il riscontro arriva al mittente, che invia il pacchetto successivo numerato $x + 1$.
2. Il pacchetto risulta corrotto o non arriva al destinatario. In questo caso il mittente, scaduto il timeout, rispedisce il pacchetto con numero di sequenza x , che viene normalmente riscontrato dal destinatario.
3. Il pacchetto arriva correttamente al destinatario, ma il riscontro del destinatario viene perso o risulta corrotto. Il mittente rispedisce quindi il pacchetto (con numero di sequenza x) dopo il timeout. In questo caso il pacchetto ricevuto è un duplicato, ma il destinatario se ne accorge perché aspettandosi il pacchetto $x + 1$ riceve il pacchetto x .

Si può quindi osservare che i numeri di sequenza x e $x + 1$ sono necessari per disambiguare i casi 1 e 3, ma non vi è alcuna necessità del numero di sequenza $x + 2$. Nel caso 1 il pacchetto può essere numerato nuovamente con x , poiché i pacchetti x e $x + 1$ sono già stati riscontrati e non vi possono essere ambiguità nei due siti. Nei casi 2 e 3 il nuovo pacchetto è $x + 1$ non $x + 2$. Se sono necessari solo x e $x + 1$, è possibile allora porre $x = 0$ e $x + 1 = 1$: in questo caso la sequenza sarà 0, 1, 0, 1, 0 e così via (aritmetica modulo 2).

Poiché i numeri di sequenza devono essere adatti sia per numerare i pacchetti dati sia per indicare i riscontri, si utilizza la seguente convenzione: il numero di riscontro indica sempre il numero di sequenza del prossimo pacchetto atteso dal destinatario. Se per esempio il destinatario ha ricevuto correttamente il pacchetto 0, invia un riscontro con valore 1 (che significa che il prossimo pacchetto

atteso è quello con numero di sequenza 1). Se il destinatario ha ricevuto correttamente il pacchetto 1, invia il riscontro 0 (che significa che si attende il pacchetto 0).

Nel meccanismo Stop and Wait il numero di riscontro indica, in aritmetica modulo 2, il numero di sequenza del prossimo pacchetto atteso dal destinatario.

FSM

Viene illustrato le FSM per il meccanismo Stop and Wait. Essendo orientato alla connessione, entrambi i lati devono essere nello stato established prima di poter trasmettere pacchetti dati. Gli stati indicati sono in realtà nidificati nello stato established. Il mittente è inizialmente nello stato ready (pronto), ma si può muovere fra gli stati ready e blocking (bloccato). La variabile S viene inizializzata a 0.

- *Stato ready.* Il mittente in questo stato è in attesa di qualche evento. Se arriva una richiesta dal livello applicazione, crea un pacchetto con numero di sequenza S , memorizza una copia del pacchetto, lo invia, inizializza il timer e passa nello stato blocking.
- *Stato blocking.* Quando il mittente è in questo stato possono avvenire tre eventi.
 1. Se arriva un ACK integro (senza errori) con ackNo (numero di riscontro) uguale al prossimo pacchetto da inviare, ovvero con $\text{ackNo} = (S + 1) \bmod 2$, il timer viene interrotto, la finestra viene fatta scorrere in avanti calcolando $S = (S + 1) \bmod 2$ e il mittente passa nello stato ready.
 2. Se viene ricevuto un ACK corrotto o integro ma con $\text{ackNo} \neq (S + 1) \bmod 2$, esso viene scartato.
 3. Se interviene un time-out (scade il timer), il mittente invia l'unico pacchetto ancora in sospeso (ancora non confermato) e fa ripartire il timer.

Il destinatario rimane sempre nello stato ready, dove possono avvenire tre eventi.

1. Se arriva un pacchetto integro con $\text{seqNo} = R$, il messaggio nel pacchetto viene consegnato al livello applicazione, la finestra viene fatta scorrere calcolando $R = (R + 1) \bmod 2$ e viene inviato un riscontro con $\text{ackNo} = R$.
2. Se arriva un pacchetto integro con $\text{seqNo} \neq R$, il pacchetto viene scartato ma viene inviato un riscontro con $\text{ackNo} = R$.
3. Se arriva un pacchetto corrotto, viene scartato.

Efficienza

Il meccanismo Stop and Wait è molto inefficiente se il canale ha una velocità di trasmissione elevata (una banda passante elevata) e un lungo ritardo (tempo di andata e ritorno). Il prodotto di questi due valori è chiamato prodotto banda-ritardo.

Esempio 5.3.2 – Efficienza di Stop and Wait

In un sistema che utilizza un meccanismo Stop and Wait, l'ampiezza di banda del canale è di 1 Mbps; il tempo necessario a 1 bit per fare un viaggio di andata e ritorno è di 20 millisecondi. Quanto vale il prodotto banda-ritardo? Se i pacchetti dati del sistema hanno dimensione di 1000 bit, qual è la percentuale di utilizzo del canale?

Il prodotto banda-ritardo è pari a $(1 \times 10^6) * (20 \times 10^{-3}) = 20000 \text{ bit}$. Il sistema potrebbe inviare 20000 bit nel tempo necessario ai dati per andare dal mittente al ricevente e per ottenerne il riscontro, tuttavia invia solamente 1000 bit. Si può affermare che il coefficiente di utilizzo del canale è solamente di $1000/20000$, ovvero del 5%. Per questo motivo l'impiego del meccanismo Stop and Wait in un canale con elevato ritardo o con elevata velocità è inefficiente.

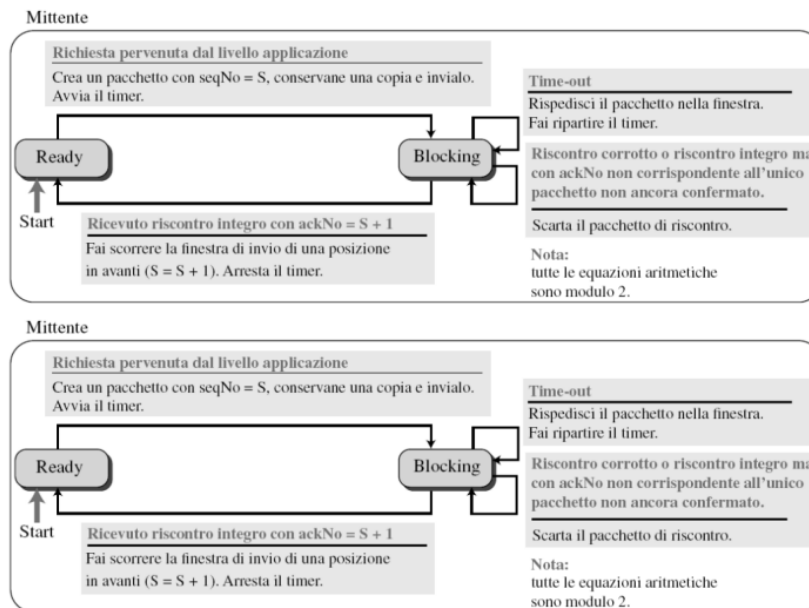


Figura 5.19: FSM per il mittente e destinatario Stop and Wait

5.3.3 Go-Back-N (GBN)

Nel meccanismo di Stop and Wait il mittente deve attendere che il pacchetto precedente abbia raggiunto il destinatario e che il relativo riscontro sia stato ricevuto, prima di poter inviare il pacchetto successivo.

Tuttavia, per migliorare l'efficienza di trasmissione si devono avere più pacchetti in transizione mentre il mittente è in attesa dei riscontri. Ovvero è necessario consentire di lasciare in sospeso (in attesa di riscontro) più di un pacchetto, mantenendo il canale occupato mentre il mittente è in attesa dei riscontri. Un primo meccanismo che consente di raggiungere questo obiettivo, è chiamato Go-Back-N (GBN) (riparti da N). Questo meccanismo adotta la tecnica del pipeling, ovvero permette di inviare più pacchetti senza dover attendere il riscontro ai primi pacchetti inviati.

Il concetto alla base del Go-Back-N è proprio la possibilità di inviare diversi pacchetti prima di ricevere dei riscontri, ma il destinatario può bufferizzare un solo pacchetto. Il mittente mantiene una copia dei pacchetti inviati fino a quando non ne riceve un riscontro. Si noti che più pacchetti dati e i relativi riscontri possono essere contemporaneamente nel canale.

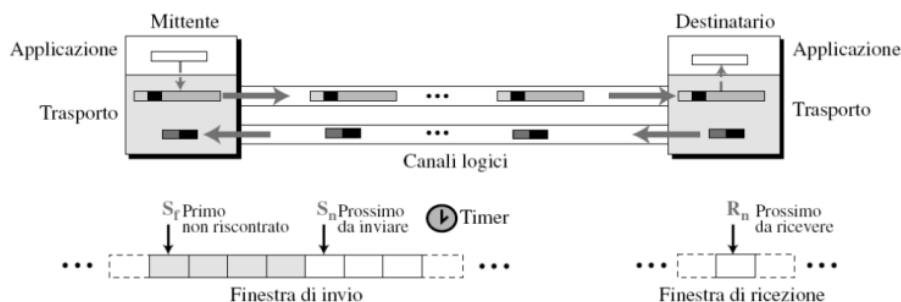


Figura 5.20: Schema generale del meccanismo Go-Back-N

Numeri di sequenza e di riscontro

In Go-Back-N i numeri di sequenza sono calcolati modulo 2^m , dove m è la dimensione del campo sequence number in bit.

Il numero di riscontro in questo caso è cumulativo e indica il numero di sequenza del prossimo pacchetto atteso. Per esempio, se il numero di riscontro (ackNo) vale 7 significa che tutti i pacchetti

fino al 6 sono stati ricevuti correttamente e che il destinatario si attende di ricevere il pacchetto con numero di sequenza 7.

Finestra di invio

La finestra di invio è una scatola immaginaria che ricopre i numeri di sequenza dei pacchetti che possono essere in transito o che possono essere spediti. Per ciascuna posizione di questa finestra, alcuni dei numeri di sequenza corrispondono ai pacchetti inviati, altri corrispondono a quelli che possono essere inviati. La dimensione massima della finestra è $2^m - 1$.

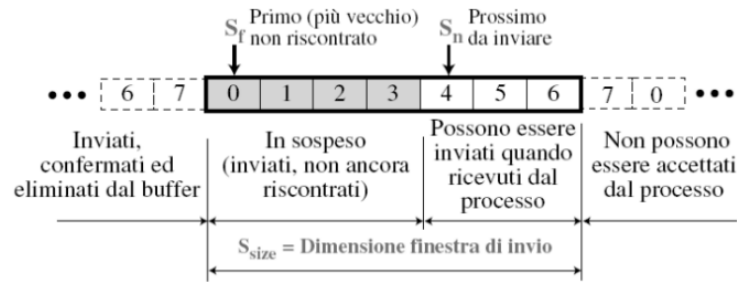


Figura 5.21: Finestra di invio per il meccanismo Go-Back-N

La finestra di invio in ogni istante divide i possibili numeri di sequenza in quattro regioni. La prima, alla sinistra della finestra, corrisponde ai numeri di sequenza dei pacchetti già confermati. Il mittente non si preoccupa di questi pacchetti e non ne mantiene copia. La seconda regione, su sfondo grigio, definisce l'intervallo dei numeri di sequenza dei pacchetti che sono stati inviati ma con stato ancora ignoto. Il mittente deve attendere per scoprire se sono stati ricevuti correttamente o se si sono persi; possono essere chiamati pacchetti in sospeso, o in attesa di riscontro. Il terzo intervallo, bianco nella figura, definisce l'intervallo dei numeri di sequenza dei pacchetti che possono essere inviati; tuttavia i dati corrispondenti non sono ancora stati ricevuti dal livello applicazione. Infine la quarta regione, alla destra della finestra, definisce i numeri di sequenza che non possono essere utilizzati fino a quando la finestra non avanza.

La finestra è ovviamente un concetto astratto: tre variabili ne definiscono le dimensioni e la posizione in ogni istante. Le variabili sono: S_f , (da Send First) ovvero primo pacchetto in attesa di riscontro nella finestra di invio, S_n (da Send Next) ovvero prossimo pacchetto da inviare nella finestra di invio, e S_{size} (da Send Size) ovvero dimensione della finestra di invio. La variabile S_f definisce il numero di sequenza del primo (il più vecchio) pacchetto in attesa di riscontro. La variabile S_n definisce il numero di sequenza che verrà assegnato al prossimo pacchetto da inviare. S_{size} definisce la dimensione della finestra, che si suppone fissa nel protocollo in esame.

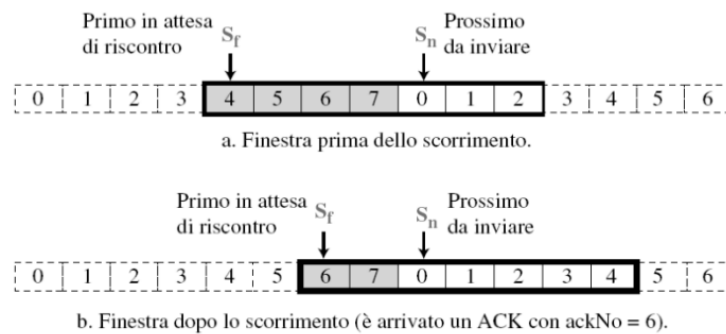


Figura 5.22: Scorrimento della finestra di invio

La finestra di invio può scorrere una o più posizioni quando viene ricevuto un riscontro privo di errori con $ackNo$ maggiore o uguale a S_f e minore di S_n (in aritmetica modulare).

Finestra di ricezione

La finestra di ricezione permette di ricevere i pacchetti dati corretti e inviare i relativi riscontri. Nel meccanismo Go-Back-N, la dimensione della finestra di ricezione è sempre 1. Il destinatario è sempre in attesa dell'arrivo di un pacchetto specifico. Qualsiasi pacchetto arrivato fuori sequenza viene scartato e deve essere rispedito. Si noti che per definire lo stato della finestra è necessaria una sola variabile, R_n , (da Receive Next) ovvero prossimo pacchetto atteso nella finestra di ricezione. I numeri di sequenza alla sinistra della finestra appartengono ai pacchetti già ricevuti e riscontrati; i numeri di sequenza alla destra della finestra indicano i pacchetti che non possono ancora essere ricevuti. Qualsiasi pacchetto ricevuto in queste due regioni viene scartato. Viene accettato e riscontrato solo il pacchetto con numero di sequenza uguale a R_n . Anche la finestra di ricezione scorre, ma di una sola posizione. Quando viene ricevuto un pacchetto corretto, la finestra scorre: $R_n = (R_n + 1) \bmod 2^m$.

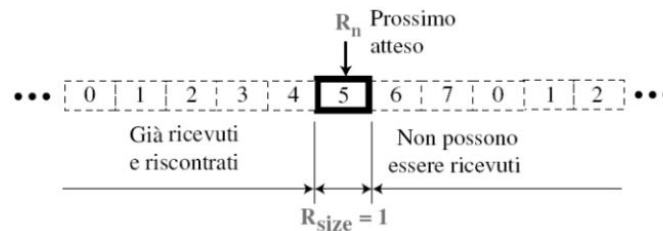


Figura 5.23: Finestra di ricezione per il meccanismo Go-Back-N

Timer e rispeditizione dei pacchetti

Sebbene vi possa essere un timer per ciascun pacchetto inviato, nel meccanismo in esame se ne usa uno solo. Il motivo è dovuto al fatto che il timer del primo pacchetto in attesa di riscontro scade sempre per primo. Quando questo timer scade, si rispediscono tutti i pacchetti in attesa di riscontro.

Per esempio, si supponga che il mittente abbia già inviato il pacchetto 6 ($S_n = 7$) e che scada l'unico timer. Se $S_f = 3$ significa che i pacchetti 3, 4, 5 e 6 non sono stati riscontrati; il mittente torna indietro e rispedisce i pacchetti 3, 4, 5 e 6. Questo è il motivo per cui questo meccanismo viene chiamato Go-Back-N: allo scadere di un timeout la macchina torna indietro di N posizioni e rispedisce tutti i pacchetti.

FSM

Il mittente inizia nello stato ready ma può successivamente trovarsi in due stati: ready o blocking. Le due variabili sono solitamente inizializzate a 0 ($S_f = S_n = 0$).

- *Stato ready.* Quando il mittente è in questo stato, possono avvenire quattro eventi.
 1. Se arriva una richiesta dal livello applicazione, il mittente crea un pacchetto con numero di sequenza impostato a S_n . Il pacchetto viene memorizzato e inviato. Il mittente avvia anche l'unico timer (se questo non fosse già stato attivato). Viene quindi incrementato il valore di S_n : $S_n = (S_n + 1) \bmod 2^m$. Se la finestra è tutta occupata, ovvero $S_n = (S_f + S_n) \bmod 2^m$, il mittente si sposta nello stato bloccato.
 2. Se arriva un riscontro integro con `ackNo` relativo a uno dei pacchetti in attesa di riscontro, il mittente fa scorrere la finestra (imposta $S_f = \text{ackNo}$); se tutti i pacchetti in attesa di riscontro sono confermati ($\text{ackNo} = S_n$) il timer viene interrotto; se c'è almeno un pacchetto in attesa di riscontro, il timer viene riavviato.
 3. Se viene ricevuto un riscontro corrotto o con `ackNo` non corrispondente ad alcun pacchetto in attesa di riscontro, il riscontro viene scartato.
 4. Se avviene un timeout, il mittente rispedisce tutti i pacchetti in attesa di riscontro e riavvia il timer.
- *Stato blocking.* In questo caso possono avvenire tre eventi.

1. Se arriva un riscontro intero con ackNo corrispondente a uno dei pacchetti in attesa di riscontro, il mittente fa scorrere la finestra (imposta $S_f = \text{ackNo}$) e se tutti i pacchetti in attesa di riscontro sono confermati ($\text{ackNo} = S_n$) il timer viene arrestato. Se non tutti i pacchetti in attesa di riscontro risultano confermati, viene riavviato il timer. Il mittente passa quindi allo stato ready.
2. Se si riceve un pacchetto di conferma corrotto o con ackNo non corrispondente ad alcun pacchetto in attesa di riscontro, la conferma viene scartata.
3. Se avviene un timeout, il mittente rispedisce tutti i pacchetti in attesa di riscontro e riavvia il timer.

Il destinatario rimane sempre nello stato ready. L'unica variabile R_n , viene inizializzata a 0. Possono avvenire tre eventi.

1. Se viene ricevuto un pacchetto intero con $\text{seqNo} = R_n$, il messaggio contenuto nel pacchetto viene consegnato al livello applicazione e la finestra viene fatta scorrere: $R_n = (R_n + 1) \bmod 2^m$. Infine viene inviato un riscontro con $\text{ackNo} = R_n$.
2. Se viene ricevuto un pacchetto con seqNo all'esterno della finestra, il pacchetto viene scartato e viene inviato un riscontro con $\text{ackNo} = R_n$.
3. Se viene ricevuto un pacchetto corrotto, viene semplicemente scartato.

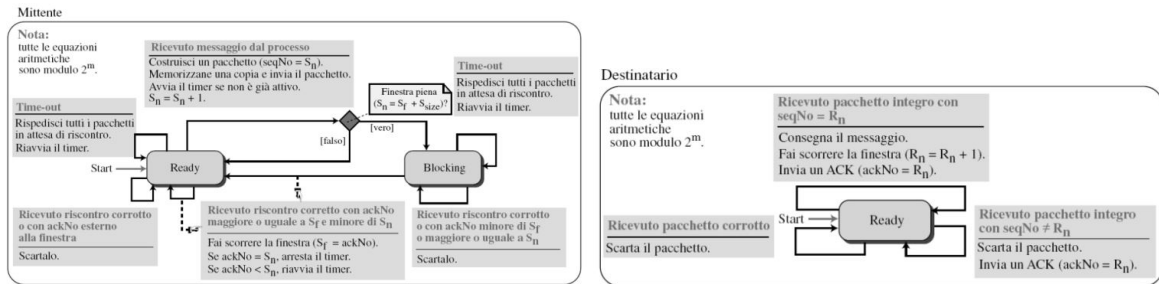
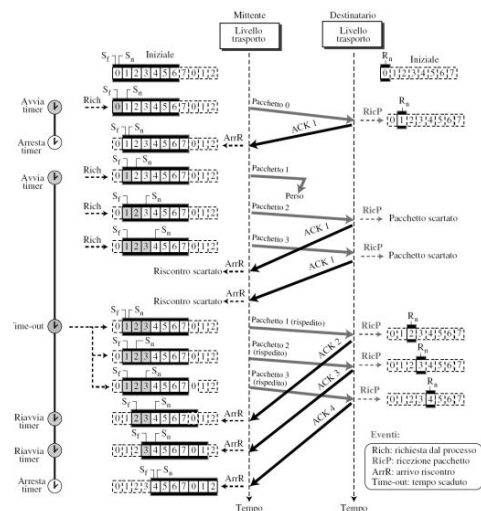


Figura 5.24: FMS per il meccanismo GBN

Esempio 5.3.3 – Go-Back-N in azione (ack cumulativo)

Viene illustrato ciò che avviene quando si perde un pacchetto.

I pacchetti 0, 1, 2 e 3 vengono inviati ma viene smarrito il pacchetto 1. Il destinatario riceve i pacchetti 2 e 3, ma questi vengono scartati poiché ricevuti fuori sequenza (il pacchetto atteso è il numero 1). Quando il destinatario riceve i pacchetti 2 e 3, invia un riscontro ACK 1 per indicare che si attende il pacchetto 1. Questi riscontri non sono utili al mittente poiché l' ackNo è uguale a S_f , non è strettamente maggiore di S_f ; quindi vengono ignorati. Quando scade il timeout, il mittente rispedisce i pacchetti 1, 2 e 3, che vengono riscontrati.



Dimensione della finestra di invio

Si può ora dimostrare il motivo per cui la dimensione della finestra di invio deve essere inferiore a 2^m . Si scelga per esempio $m = 2$, che significa che la dimensione della finestra può essere $2^m - 1$, ovvero

3. Se la dimensione della finestra è 3 (minore di 2^m) e tutti e tre i riscontri vengono persi, l'unico timer scade e vengono rispediti tutti e tre i pacchetti. Il destinatario si aspetta ora il pacchetto 3, non il pacchetto 0, così che il pacchetto duplicato viene correttamente scartato. Se invece la dimensione della finestra fosse 4 (ovvero uguale a 2^m) e tutti i riscontri venissero persi, il mittente invierebbe un duplicato del pacchetto 0. Tuttavia questa volta la finestra del destinatario si aspetta di ricevere il pacchetto 0 (nel ciclo successivo), quindi accetta il pacchetto 0 non come duplicato, ma come primo pacchetto del nuovo ciclo. Evidentemente si tratta di un errore: questo dimostra che la dimensione della finestra di invio deve quindi essere inferiore a 2^m .

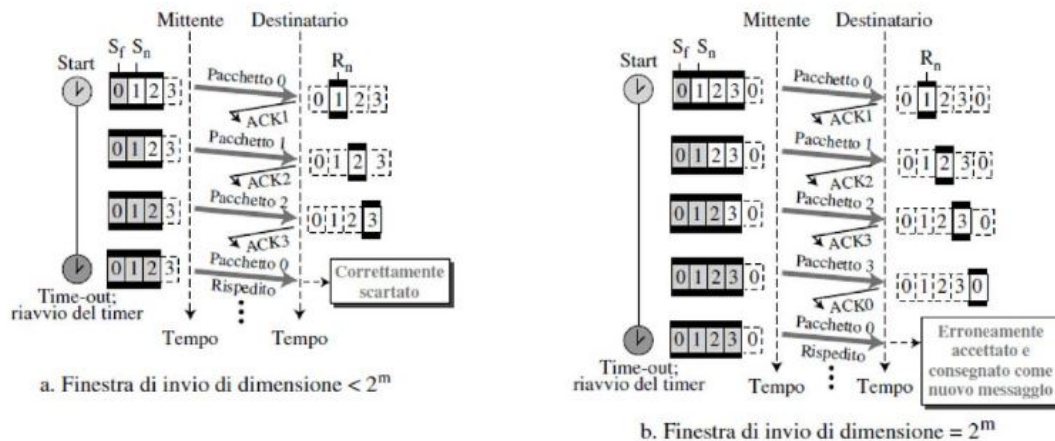


Figura 5.25: Dimensione della finestra di invio per Go-Back-N

Nel meccanismo Go-Back-N, la dimensione della finestra di invio deve essere inferiore a 2^m ; la dimensione della finestra di ricezione è sempre 1.

5.3.4 Ripetizione selettiva

Il meccanismo Go-Back-N semplifica i compiti del destinatario, che tiene traccia di una sola variabile e non deve bufferizzare i pacchetti fuori sequenza: questi vengono semplicemente ignorati. Questo meccanismo è tuttavia inefficiente se il livello rete sottostante perde numerosi pacchetti. Ogni volta che un singolo pacchetto risulta corrotto o smarrito, il mittente deve inviare tutti i pacchetti in attesa di riscontro anche se alcuni di questi sono già stati ricevuti correttamente ma fuori sequenza. Se il livello rete perde pacchetti in numero elevato a causa della congestione nella rete, la rispeditura di tutti i pacchetti in attesa di riscontro peggiora la congestione, che alla fine comporta la perdita di un numero ancora superiore di pacchetti, portando al collasso totale della rete.

E' stato sviluppato un altro meccanismo, chiamato Selective-Repeat (SR) o ripetizione selettiva, che, come indicato dal nome, rispedisce i pacchetti selettivamente, ovvero solo quelli che sono realmente stati smarriti.

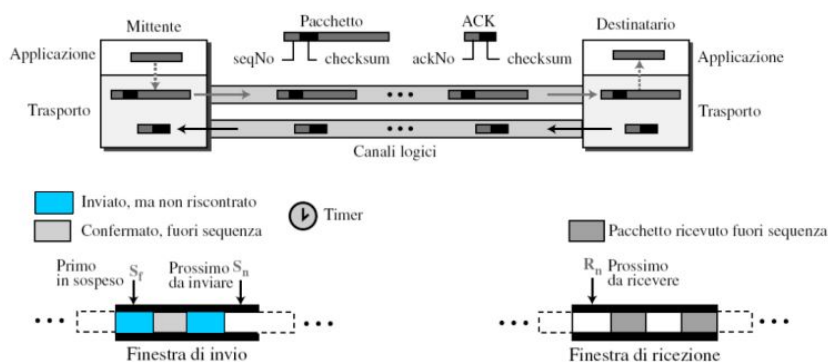


Figura 5.26: Schema generale del meccanismo Selective-Repeat

Finestre di invio e ricezione

Anche il meccanismo Selective-Repeat utilizza due finestre: una di invio e l'altra di ricezione. Tuttavia vi sono delle differenze con le finestre del Go-Back-N. Per prima cosa la dimensione massima della finestra di invio è molto più piccola, 2^{m-1} . Secondo, la dimensione della finestra di ricezione è uguale alla dimensione della finestra di invio.

La dimensione massima della finestra di invio è 2^{m-1} . Per esempio, se $m = 4$, i numeri di sequenza vanno da 0 a 15, ma la dimensione massima della finestra è 8 (mentre sarebbe 15 con il Go-Back-N).

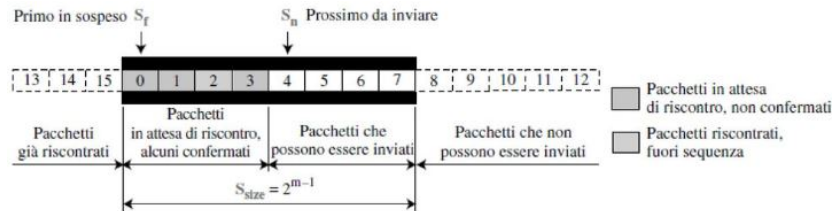


Figura 5.27: Finestra di invio del meccanismo Selective-Repeat

La finestra di ricezione nel Selective-Repeat è totalmente differente da quella nel Go-Back-N. La dimensione della finestra di ricezione è la stessa della finestra di invio (massimo 2^{m-1}). Il meccanismo Selective-Repeat permette a un numero di pacchetti uguale alla dimensione della finestra di arrivare fuori sequenza; i pacchetti vengono memorizzati nella finestra fino a quando si forma un blocco di pacchetti consecutivi da passare al livello applicazione. Poiché la dimensione delle finestre di invio e di ricezione è la stessa, tutti i pacchetti nella finestra di invio possono arrivare fuori sequenza ed essere memorizzati fino a quando possono essere consegnati. Nella finestra, le celle ombreggiate indicano i pacchetti che sono arrivati fuori sequenza e che sono in attesa dell'arrivo dei pacchetti trasmessi in precedenza prima di poter essere passati al livello applicazione.



Figura 5.28: Finestra di ricezione del meccanismo Selective-Repeat

Timer

Teoricamente il Selective-Repeat usa un timer per ciascun pacchetto in attesa di riscontro. Quando un timer scade ne viene rispedito solo il relativo pacchetto. A differenza di GBN che gestisce i pacchetti in attesa di riscontro come un singolo gruppo, SR li gestisce individualmente. La maggior parte dei protocolli di livello trasporto che implementano SR usa tuttavia un solo timer: per questo motivo anche in questo testo si usa un solo timer.

Riscontri

Vi è un'ulteriore differenza fra i due meccanismi. Nel GBN il riscontro è cumulativo, indica il numero di sequenza del pacchetto successivo atteso, confermando che tutti i pacchetti precedenti sono stati ricevuti correttamente. La semantica dei riscontri in SR è invece differente: il numero di riscontro indica il numero di sequenza di un singolo pacchetto ricevuto correttamente, non può fornire feedback per gli altri.

Nel meccanismo Selective-Repeat il numero di riscontro indica il numero di sequenza di uno specifico pacchetto ricevuto senza errori.

FSM

Le FSM per il meccanismo Selective-Repeat sono simili a quelle per GBN, ma vi sono delle differenze.

Il mittente inizia nello stato ready, ma successivamente può trovarsi negli stati ready o blocking. Ecco gli eventi e le azioni corrispondenti in ciascuno stato.

- *Stato ready.* In questo caso possono avvenire quattro eventi.
 1. Se arriva una richiesta dal livello applicazione, il mittente crea un pacchetto con numero di sequenza impostato a S_n . Il pacchetto viene inviato dopo averne memorizzato una copia. Se il timer non è già attivo, viene inizializzato. Il valore di S_n , viene incrementato: $S_n = (S_n + 1) \bmod 2^m$. Se la finestra è completa, ovvero vale $S_n = (S_f + S_{size}) \bmod 2^m$, il mittente passa nello stato blocked.
 2. Se viene ricevuto un riscontro intero con ackNo relativo a uno dei pacchetti in attesa di riscontro, quel pacchetto viene contrassegnato come riscontrato. Se $\text{ackNo} = S_f$ la finestra scorre a destra fino a quando S_f , punta al primo pacchetto non riscontrato (tutti i pacchetti consecutivi riscontrati sono ora all'esterno della finestra). Se vi sono ancora pacchetti in attesa di riscontro viene fatto ripartire il timer, altrimenti viene arrestato.
 3. Se viene ricevuto un riscontro corrotto o con ackNo non relativo a uno dei pacchetti in attesa di riscontro, viene scartato.
 4. Se scade un timer, il mittente rispedisce tutti i pacchetti non confermati nella finestra e riavvia il timer.
- *Stato blocking.* In questo caso possono avvenire tre eventi.
 1. Se viene ricevuto un riscontro corretto con ackNo relativo a uno dei pacchetti in attesa di riscontro, quel pacchetto viene contrassegnato come riscontrato. Inoltre, se $\text{ackNo} = S_f$, la finestra viene fatta scorrere alla destra fino a che S_f , punta al primo pacchetto non riscontrato (tutti i pacchetti riscontrati consecutivi sono ora all'esterno della finestra). In questo ultimo caso il mittente passa nello stato ready.
 2. Se viene ricevuto un riscontro corrotto o con ackNo non relativo a uno dei pacchetti in attesa di riscontro, il riscontro viene ignorato.
 3. Se scade un timer, il mittente rispedisce tutti i pacchetti non confermati nella finestra e riavvia il timer.

Il destinatario è sempre nello stato ready, nel quale possono avvenire tre eventi.

1. Se viene ricevuto un pacchetto con seqNo nella finestra, esso viene memorizzato e viene inviato un riscontro con $\text{ackNo} = \text{seqNo}$. Inoltre, se $\text{seqNo} = R_n$, il pacchetto e tutti gli altri consecutivi arrivati in precedenza vengono passati al livello applicazione: la finestra viene fatta scorrere in modo che R_n , punti alla prima cella libera.
2. Se viene ricevuto un pacchetto integro ma con seqNo all'esterno della finestra, il pacchetto viene scartato ma viene inviato al mittente un riscontro con $\text{ackNo} = R_n$. Questo è necessario per consentire al mittente di far scorrere la propria finestra nel caso qualche riscontro ai pacchetti con $\text{seqNo} < R_n$, fosse andato perso.
3. Se viene ricevuto un pacchetto corrotto, esso viene scartato.

Esempio 5.3.4 – Selective-Repeat in azione

Viene illustrato il comportamento del Selective-Repeat.

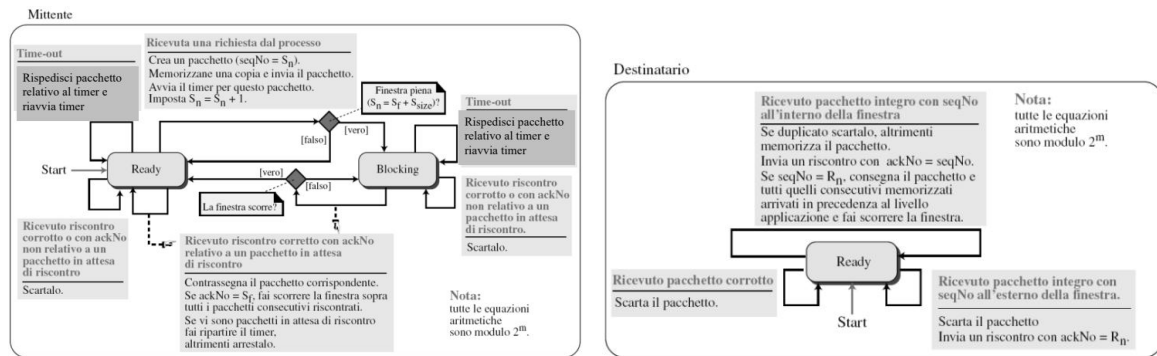
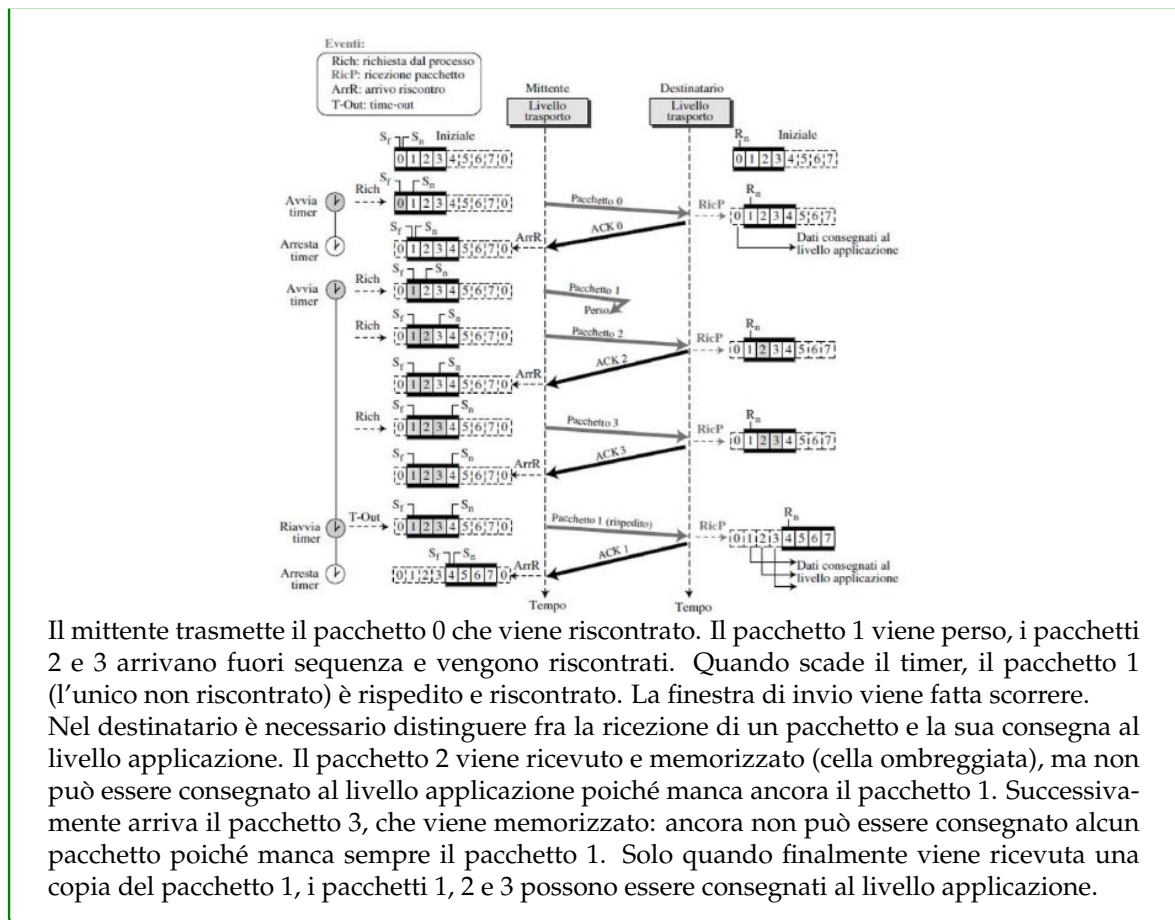


Figura 5.29: FSM per il meccanismo Selective-Repeat



Il mittente trasmette il pacchetto 0 che viene riscotrato. Il pacchetto 1 viene perso, i pacchetti 2 e 3 arrivano fuori sequenza e vengono riscotrati. Quando scade il timer, il pacchetto 1 (l'unico non riscotrato) è rispedito e riscotrato. La finestra di invio viene fatta scorrere. Nel destinatario è necessario distinguere fra la ricezione di un pacchetto e la sua consegna al livello applicazione. Il pacchetto 2 viene ricevuto e memorizzato (cella ombreggiata), ma non può essere consegnato al livello applicazione poiché manca ancora il pacchetto 1. Successivamente arriva il pacchetto 3, che viene memorizzato: ancora non può essere consegnato alcun pacchetto poiché manca sempre il pacchetto 1. Solo quando finalmente viene ricevuta una copia del pacchetto 1, i pacchetti 1, 2 e 3 possono essere consegnati al livello applicazione.

I pacchetti possono essere consegnati al livello applicazione se si verificano due condizioni. Per prima cosa deve essere stato ricevuto un insieme di pacchetti consecutivi. Secondo, l'insieme deve partire dall'inizio della finestra. Un livello trasporto affidabile consegna quindi i pacchetti secondo la sequenza di invio.

Dimensione delle finestre di invio e ricezione

E' ora possibile comprendere il motivo per cui la dimensione delle finestre di invio e di ricezione può essere al massimo la metà di 2^m . Si scelga come esempio $m = 2$, che significa che la dimensione della finestra è $2^m / 2$ ovvero $2^{(m-1)} = 2$.

Se la dimensione della finestra vale 2 e tutti i riscotri vengono persi, il timer del pacchetto 0 scade e il pacchetto viene quindi rispedito. Tuttavia la finestra del destinatario è in attesa del pacchetto 2, non del pacchetto 0 - quindi il pacchetto ricevuto viene correttamente scartato (il numero di sequenza 0 non è nella finestra). Quando la dimensione della finestra è 3 e tutti i riscotri vengono persi,

il mittente invia un duplicato del pacchetto 0: questa volta la finestra del destinatario si aspetta di ricevere il pacchetto 0 (0 fa parte della finestra), quindi accetta questo pacchetto 0 non come duplicato, ma come se fosse un pacchetto del ciclo successivo. Evidentemente si tratta di un errore.

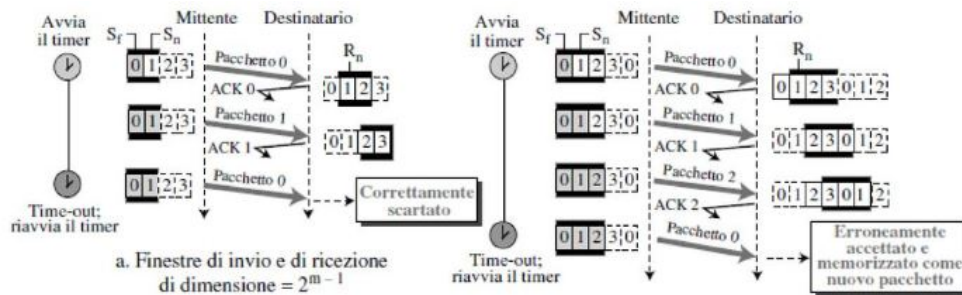


Figura 5.30: Dimensione della finestra nel meccanismo Selective-Repeat

Nel meccanismo Selective-Repeat la dimensione delle finestre di invio e di ricezione può essere al massimo la metà di 2^m .

5.3.5 Protocolli bidirezionali: piggybacking

I quattro meccanismi discussi in precedenza sono tutti unidirezionali: i pacchetti dati scorrono solo in una direzione e i riscontri nella direzione opposta. I pacchetti dati vengono in realtà trasmessi solitamente in entrambe le direzioni: dal client al server e dal server al client. Questo significa che anche i riscontri devono viaggiare in entrambe le direzioni. Per migliorare l'efficienza dei protocolli bidirezionali viene utilizzata una tecnica chiamata piggybacking ("viaggiare in spalla a qualcuno"). Quando un pacchetto trasporta dei dati da A a B, può trasportare anche i riscontri relativi ai pacchetti ricevuti da B; viceversa, quando un pacchetto trasporta dei dati da B ad A, può trasportare anche i riscontri relativi ai pacchetti ricevuti da A.

In questo sistema il client e il server usano entrambi due finestre indipendenti di invio e di ricezione.

5.3.6 Segmenti TCP

Sebbene la bufferizzazione possa gestire la differenza di velocità fra i processi produttore e consumatore, è necessario un passo ulteriore per poter trasmettere i dati. Il livello rete, quale fornitore di servizi al protocollo di livello trasporto (in questo caso TCP), deve inviare i dati in pacchetti, e non come flusso di byte. A livello trasporto, TCP raggruppa quindi un certo numero di byte in unità chiamate segmenti. TCP aggiunge un'intestazione formata da varie informazioni di controllo a ciascun segmento, che consegna al livello rete per la trasmissione. I segmenti vengono incapsulati in datagrammi IP e trasmessi. Tutte queste operazioni risultano essere perfettamente trasparenti al processo ricevente.

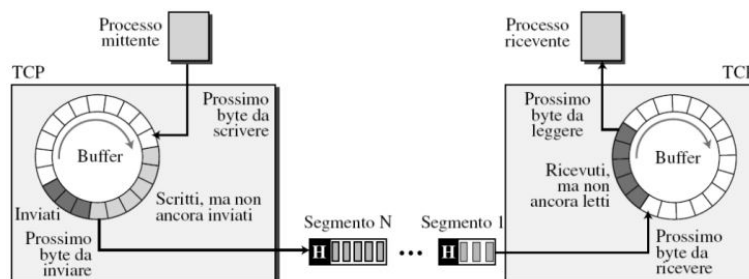


Figura 5.31: Segmenti TCP

Il segmento consiste di un'intestazione di dimensione compresa tra i 20 e i 60 byte, seguito dai dati provenienti dal programma applicativo. La dimensione dell'intestazione è di 20 byte in assenza di opzioni, fino a 60 byte altrimenti.

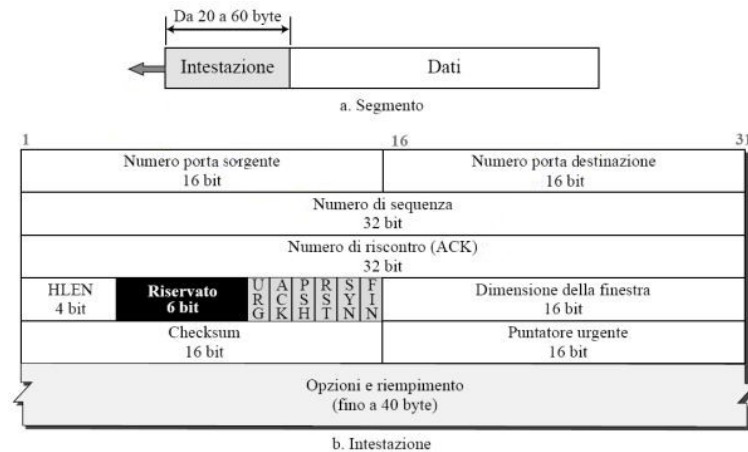


Figura 5.32: Formato dei segmenti TCP

- *Numero di porta sorgente.* Campo a sedici bit contenente il numero di porta del processo mittente sull'host che invia il segmento.
- *Numero di porta destinazione.* Campo a sedici bit contenente il numero di porta del processo destinatario sull'host che riceve il segmento.
- *Numero di sequenza.* Campo a 32 bit contenente il numero attribuito al primo byte di dati contenuto nel segmento. Più precisamente il numero di sequenza indica al destinatario quale dei byte inviati dal processo mittente è stato inserito come primo byte del segmento.
- *Numero di riscontro (acknowledgement number).* Campo a 32 bit che contiene il numero di sequenza del byte che il destinatario si aspetta di ricevere. Per esempio se il destinatario ha ricevuto il byte con numero di sequenza x , restituirà il numero di riscontro pari a $x + 1$.
- *Lunghezza dell'intestazione (HLEN).* Campo a quattro bit che indica il numero di parole di quattro byte presenti nell'intestazione TCP.
- *Flag di controllo.* Campo contenente i 6 bit di controllo (flag) elencati nella Figura 5.33; è possibile che più di un bit sia attivo (valga 1) simultaneamente. Questi bit intervengono nel controllo di flusso, nell'apertura e nella chiusura delle connessioni, nella determinazione della modalità del trasferimento di dati.

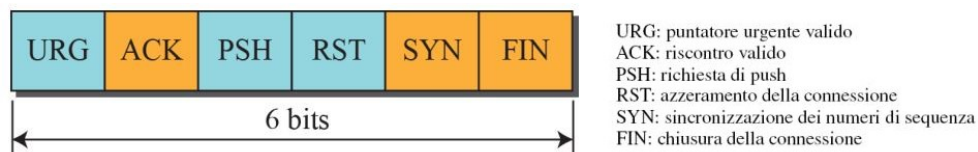


Figura 5.33: Flag di controllo

- *Dimensione della finestra.* Campo a 16 bit che indica la dimensione della finestra di cui l'altro host coinvolto nella trasmissione deve disporre.
- *Checksum.* Campo a 16 bit contenente il checksum.
- *Puntatore urgente.* Campo a 16 bit operativo se il flag URG (urgente) è attivato. Il campo contiene il numero che deve essere sommato al numero di sequenza per ottenere il numero dell'ultimo byte urgente nella sezione dati del segmento.
- *Opzioni.* Vi possono essere da 0 a 40 byte di informazioni opzionali nell'intestazione del pacchetto TCP.

5.3.7 Connessione TCP

Il protocollo TCP è orientato alla connessione, ovvero stabilisce un percorso virtuale fra il mittente e il destinatario. Tutti i segmenti appartenenti a un messaggio vengono spediti lungo questo percorso logico. Utilizzare un percorso virtuale per l'intero messaggio semplifica il processo di conferma e di ritrasmissione dei segmenti smarriti o danneggiati. Ci si potrebbe chiedere come TCP possa essere orientato alla connessione dato che utilizza i servizi di IP, un protocollo privo di connessioni. In realtà la connessione TCP è virtuale, non fisica: TCP opera a un livello più alto, utilizza i servizi di IP per consegnare i singoli segmenti al destinatario, ma è il TCP stesso che controlla la connessione. Un segmento smarrito o danneggiato viene ritrasmesso da TCP, ma IP è inconsapevole di tale ritrasmissione. Un segmento ricevuto fuori sequenza viene bufferizzato da TCP fino a quando riceve il segmento mancante: IP è ignaro di questo riordinamento.

La trasmissione orientata alla connessione nel protocollo TCP richiede tre fasi: apertura della connessione, trasmissione dei dati e chiusura della connessione.

Apertura della connessione

TCP trasmette i dati in modalità full-duplex: due entità TCP connesse in due macchine differenti sono in grado di trasmettere dati l'una all'altra simultaneamente. Questo comporta che ciascuna entità deve inizializzare la comunicazione e ottenere l'approvazione dell'entità corrispondente prima di poter trasferire qualsiasi dato.

La procedura di apertura della connessione in TCP è detta three way handshake (stretta di mano a tre vie). Consideriamo un esempio in cui un programma applicativo, chiamato client, vuole collegarsi a un altro programma applicativo, chiamato server, usando il protocollo di livello trasporto TCP. Il processo server deve essere attivo, e pronto ad accettare connessioni mediante il livello TCP, per cui deve richiedere al TCP una apertura passiva. Nonostante il server TCP sia in grado di aprire connessioni con un qualsiasi dispositivo connesso alla rete mondiale, non può aprire la connessione da solo, ma dietro richiesta di un processo applicativo. La richiesta fatta dal processo client, invece, è detta richiesta di apertura attiva: un processo client che voglia aprire una connessione con un processo server su un particolare dispositivo ne informa il suo TCP, il quale dà inizio alla procedura three way handshake.

Per illustrare la procedura si utilizzano due linee temporali, una per ciascuna postazione. Ogni segmento ha i valori appropriati per tutti i campi dell'intestazione (ed eventualmente anche per alcuni dei suoi campi opzionali): il numero di sequenza, il numero di riscontro, i flag di controllo (solo quelli utilizzati) e la dimensione della finestra (quando significativa). La procedura comporta i tre passi seguenti.

1. Il client invia il primo segmento, un segmento SYN nel quale viene impostato il solo flag SYN, che serve per la sincronizzazione dei numeri di sequenza. Il client sceglie un numero random come primo numero di sequenza, chiamato numero di sequenza iniziale, o ISN (Initial Sequence Number), e lo invia al server. Si noti che questo segmento non contiene alcun numero di riscontro e non definisce la dimensione della finestra: quest'ultima è significativa solo nel caso in cui il segmento include un riscontro. Si noti che il segmento SYN è un segmento di controllo e non contiene dati utente, tuttavia usa un numero di sequenza: quando inizierà il trasferimento dei dati, il numero di sequenza verrà incrementato di un'unità. Si potrebbe dire che il segmento SYN non trasporta dati reali, ma solo un byte immaginario.
2. Il server invia il secondo segmento, un segmento con i due flag SYN e ACK attivati. Il suo scopo è duplice: in primo luogo è un segmento SYN per la comunicazione nell'altra direzione. Il server utilizza questo segmento per inizializzare il numero di sequenza necessario per numerare i byte inviati dal server al client. In secondo luogo il server usa il segmento per riscontrare la ricezione del segmento SYN dal client impostando il flag ACK e indicando il prossimo numero di sequenza che si aspetta di ricevere dal client. Dato che contiene un riscontro, deve anche definire la dimensione della finestra di ricezione (rwnd) che il client dovrà utilizzare. Essendo un segmento SYN esso deve essere confermato, quindi usa un numero di sequenza.
3. Il client invia il terzo segmento, un segmento ACK, che conferma l'avvenuta ricezione del secondo segmento mediante il flag ACK. Si noti che il segmento ACK non usa alcun numero di sequenza se non contiene dati utenti, ma alcune implementazioni consentono a questo terzo

segmento, nella fase di apertura della connessione, di trasportare i primi dati dal client: in questo caso si deve utilizzare un numero di sequenza per indicare il numero del primo byte nei dati.

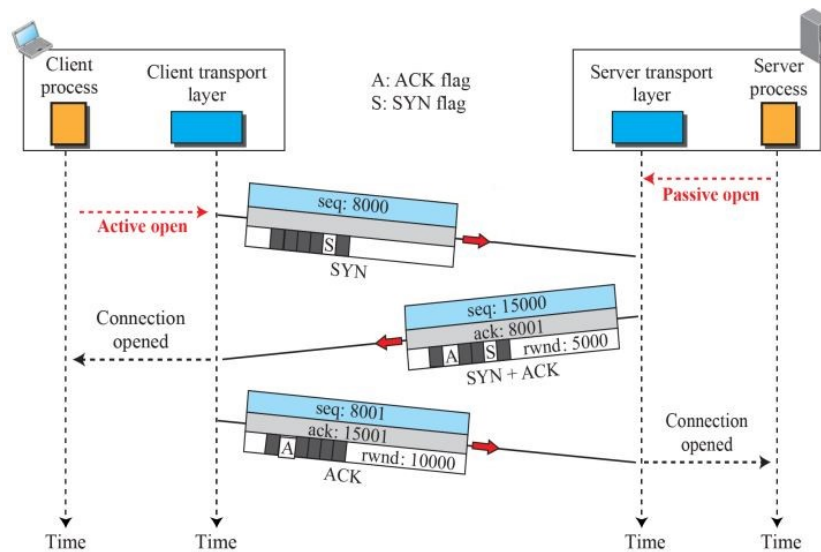


Figura 5.34: Apertura della connessione mediante il tree-way handshake

Trasferimento dei dati

Dopo aver stabilito la connessione è possibile trasferire i dati. Il client e il server possono inviare dati e riscontri in entrambe le direzioni. I dati che viaggiano nella medesima direzione dei riscontri sono trasportati nel medesimo segmento (in piggybacking). Viene illustrato un esempio, in cui il client, dopo aver aperto la connessione, invia 2000 byte di dati in due segmenti. Il server invia successivamente 2000 byte in un segmento; il client invia infine un nuovo segmento. I primi tre segmenti trasportano sia dati che riscontri, mentre l'ultimo segmento trasporta solo un riscontro perché non vi sono altri dati da trasmettere. Si notino i valori dei numeri di sequenza e di riscontro. I segmenti dati inviati dal client hanno il flag PSH (push - spingi) impostato, così che il server TCP sa di dover consegnare i dati al processo server non appena li riceve. Il segmento dal server, invece, non ha il flag PSH impostato.

- *Pushing dei dati.* Come si è già visto, il protocollo TCP mittente memorizza in un buffer i dati provenienti dal programma applicativo; la quantità di dati del buffer che vengono inseriti in un segmento è scelta a discrezione del protocollo TCP. Anche il protocollo TCP ricevente memorizza i dati ricevuti in un buffer e li trasmette al programma applicativo quando questo è pronto alla ricezione oppure quando il TCP lo ritiene conveniente. Questo modo d'operare così flessibile permette di migliorare l'efficienza delle procedure di trasmissione.

Vi sono casi, però, in cui il programma applicativo non può operare in regime di flessibilità: si supponga che un processo interagisca direttamente con un altro processo all'altro capo della connessione. Il primo può voler inviare un comando al secondo e aspettarsi una risposta immediata; ritardi nella trasmissione o nella consegna della risposta possono non essere tollerati da questo programma.

Il TCP gestisce queste operazioni con la funzione push che può essere richiesta dal mittente. In questo caso il TCP mittente non deve attendere che la finestra sia piena: deve, invece, creare e inviare immediatamente il segmento. Il TCP mittente, inoltre, deve attivare il bit PSH per segnalare al TCP ricevente che i dati contenuti nel segmento devono essere consegnati al più presto possibile al processo destinatario.

- *Dati urgenti.* TCP è un protocollo orientato al flusso. I dati provenienti dal processo giungono al protocollo TCP come un flusso di caratteri e ogni byte di dati occupa una determinata posizione all'interno del flusso. Vi sono casi, però, in cui un processo ha la necessità di inviare dati

urgenti; in altri termini il processo desidera che una porzione dei dati venga elaborata dal TCP prima degli altri indipendentemente dalla sua posizione nel flusso. La soluzione consiste nell'invio di un segmento con il bit URG attivato: il processo mittente notifica al TCP mittente che i dati sono urgenti; questi crea un segmento e inserisce i dati urgenti all'inizio del campo dati. Il resto del segmento può contenere dati normali dal buffer. Il campo puntatore urgente nell'intestazione indica la fine dei dati urgenti e l'inizio di quelli normali. Per esempio se il numero di sequenza del segmento vale 15000 e il valore del campo puntatore urgente è 200, il primo byte dei dati urgenti è il byte 15000 e l'ultimo byte è il 15200. I byte restanti nel segmento, se ve ne sono, sono dati non urgenti.

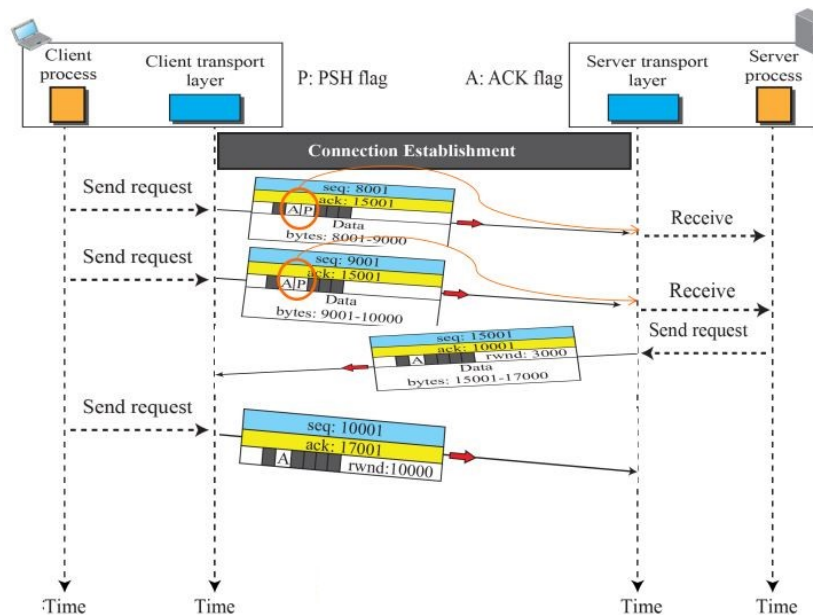


Figura 5.35: Trasferimento dati

Chiusura della connessione

Ciascuna delle due parti coinvolta nello scambio dei dati (client o server) può chiudere la connessione, sebbene la chiusura sia solitamente iniziata dal client. La maggior parte delle implementazioni attuali fornisce due opzioni per la chiusura della connessione: handshake a tre vie e handshake a quattro vie con l'opzione di half-close. Con handshake a tre vie:

1. In questo caso, il client TCP, dopo aver ricevuto un comando di chiusura dal processo applicativo client, invia un primo segmento FIN nel quale viene impostato il flag FIN. Si noti che il segmento FIN può contenere l'ultima parte dei dati inviati dal client o può essere un semplice segmento di controllo. Se si tratta di un segmento di controllo, usa un solo numero di sequenza perché richiede un riscontro.
2. Il server TCP, dopo aver ricevuto il segmento FIN, notifica la situazione al suo processo applicativo e invia il segmento FIN + ACK, per riscontrare la ricezione del segmento FIN al client e per annunciare la chiusura della connessione nella direzione opposta. Questo segmento può anche contenere gli ultimi dati da parte del server, se non trasporta dati, consuma solo un numero di sequenza perché è necessario riscontrarne la ricezione.
3. Il client TCP invia l'ultimo segmento dell'handshake, un segmento ACK, per riscontrare la ricezione del segmento FIN dal server TCP. Questo segmento contiene il numero di riscontro, che vale uno più il numero di sequenza ricevuto nel segmento FIN dal server, non può trasportare dati e non consuma numeri di sequenza.

Con half-close (chiusura a metà), in TCP, uno dei due processi può smettere di inviare dati mentre ne sta ancora ricevendo. Sebbene entrambe le entità possano eseguire la half-close, essa è solitamente

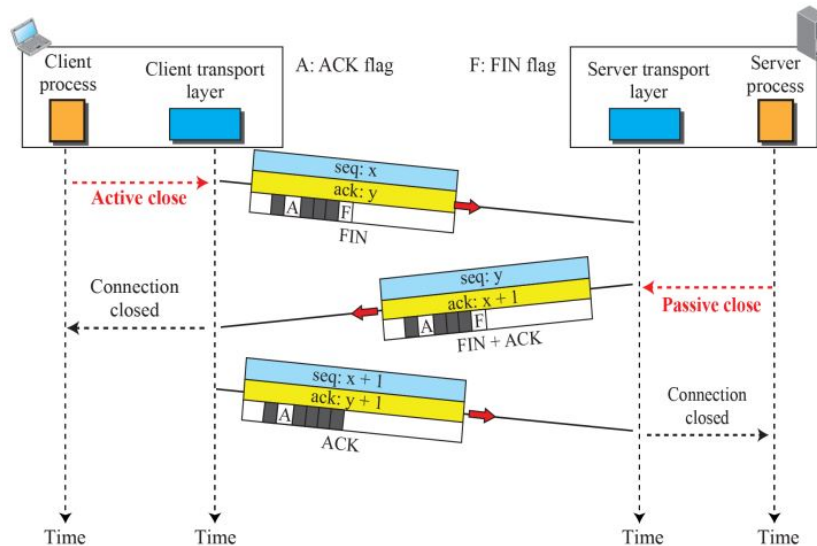


Figura 5.36: Chiusura della connessione mediante three-way handshake

iniziata dal client. Questo può avvenire quando il server ha bisogno di tutti i dati prima di poter procedere alla loro elaborazione. Il client richiede la half-close della connessione inviando un segmento FIN. Il server accetta la richiesta di chiusura inviando un segmento ACK. Il trasferimento dei dati dal client al server termina, ma il server può continuare a inviare dati. Quando il server ha terminato l'invio dei dati elaborati, invia un segmento FIN, che viene riscontrato da un segmento ACK dal client. Dopo il completamento della half-close è ancora possibile trasmettere i dati dal server al client e i riscontri dal client al server, ma il client non può inviare altri dati.

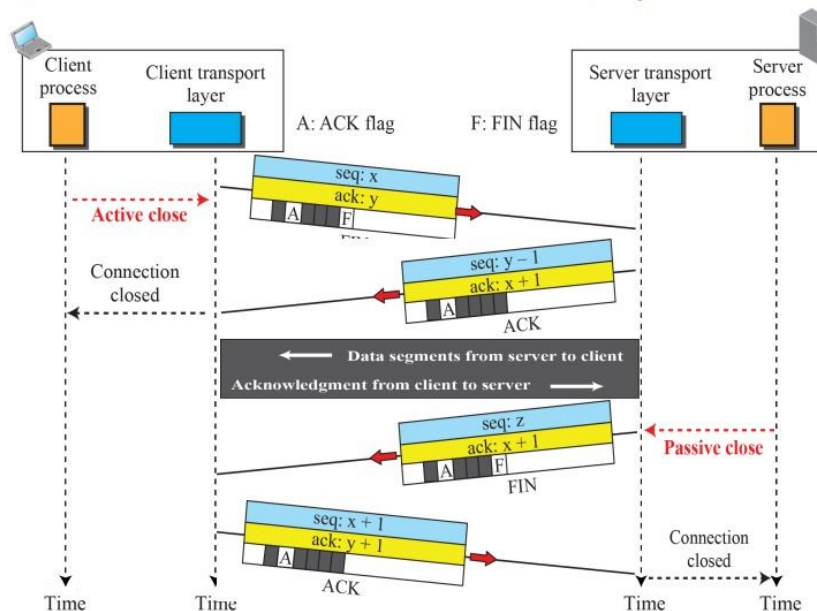


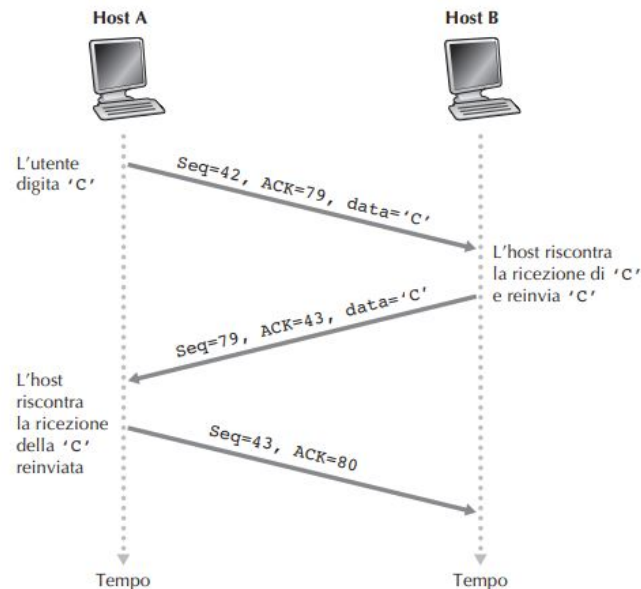
Figura 5.37: Half-close

5.3.8 Controllo degli errori

Presentiamo ora un esempio che illustra bene i numeri di sequenza e di acknowledgment di TCP.

Esempio 5.3.5 – Numeri di sequenza e di acknowledgment di TCP

Supponiamo che l'Host A inizi una sessione Telnet con l'Host B. Il primo viene etichettato come client, il secondo come server. Ogni carattere immesso dall'utente (nel client) verrà spedito all'host remoto; quest'ultimo restituirà una copia di ciascun carattere, che sarà mostrata sullo schermo dell'utente. Questo "eco" viene utilizzato per assicurarsi che i caratteri visibili all'utente siano già stati ricevuti ed elaborati nel sito remoto. Quindi, nel lasso di tempo che intercorre tra la digitazione sul tasto e la visualizzazione sul monitor, ciascun carattere attraversa la rete due volte.



Supponiamo ora che l'utente digiti la lettera 'C' ... e poi vada a bere un caffè. Esaminiamo i segmenti TCP scambiati tra client e server. Ipotizziamo che i numeri di sequenza iniziali siano rispettivamente 42 e 79 per client e server. Ricordiamo che il numero di sequenza di un segmento è il numero di sequenza del primo byte nel campo dati. Pertanto, il primo segmento inviato dal client avrà numero di sequenza 42 e quello inviato dal server 79. Ricordiamo che il numero di acknowledgment è il numero di sequenza del successivo byte di dati che l'host sta aspettando. Dopo l'instaurazione della connessione TCP, ma prima dell'invio dei dati, il client è in attesa del byte 79 e il server del byte 42. Vengono spediti tre segmenti. Il primo è inviato dal client al server e contiene nel campo dati il byte ASCII per la lettera 'C'. Il campo numero di sequenza del primo segmento contiene 42. Inoltre, visto che il client non ha ancora ricevuto dati dal server, questo primo segmento avrà 79 nel proprio campo numero di acknowledgment. Il secondo segmento spedito dal server al client ha un duplice scopo. Innanzitutto, fa un acknowledgment dei dati ricevuti dal server. Ponendo 43 nel campo acknowledgment, il server indica al client di aver ricevuto con successo i primi 42 byte e di attendere i byte da 43 in poi. L'altra finalità di questo segmento è di mandare indietro un "eco" della lettera 'C'. Di conseguenza, il secondo segmento presenta nel proprio campo dati la lettera 'C', in ASCII, e ha numero di sequenza 79, ossia il numero iniziale del flusso di dati da server, in questa connessione TCP, dato che si tratta proprio del primo byte di dati spedito dal server. Notiamo che il riscontro dei dati dal client al server viene trasportato in un segmento che a sua volta trasporta dati dal server al client; in gergo, si dice che tale acknowledgment è piggybacked (o anche in piggyback) sul segmento dati dal server al client. Il terzo segmento viene inviato dal client al server e ha come unico scopo dare un acknowledgment ai dati inviati dal server. Ricordiamo che il secondo segmento conteneva dati, la lettera 'C' dal server al client. Al contrario, il campo dati di questo segmento è vuoto (in altre parole, l'acknowledgment non è in piggyback ad alcun dato da client al server). Il segmento ha 80 nel suo campo numero di acknowledgment, in quanto il client ha ricevuto il flusso di byte fino al numero di sequenza 79 e ora è in attesa dei byte dall'80 in avanti. Si potrebbe considerare strano che questo segmento abbia un numero di sequenza pur senza contenere dati. In ogni caso, dato che TCP prevede un campo numero di sequenza, il segmento deve averne uno.

Il protocollo TCP è un protocollo di trasporto affidabile; ovvero garantisce al livello applica-

zione che i dati inviati verranno consegnati nel giusto ordine, senza errori, senza smarrimenti o duplicazioni.

Il controllo degli errori garantisce affidabilità e prevede meccanismi per l'individuazione e la rispeditura dei segmenti corrotti, per la rispeditura dei segmenti smarriti, per l'identificazione e lo scarto dei segmenti duplicati e per la memorizzazione dei segmenti ricevuti fuori sequenza fino a quando non arrivano i segmenti mancanti. I tre semplici strumenti usati dal protocollo TCP per individuare gli errori di trasmissione sono il checksum, i messaggi di riscontro e il timeout.

Checksum

Il protocollo TCP prevede che ciascun segmento contenga un campo checksum di 16 bit, utilizzato per identificare i segmenti corrotti. Se un segmento è corrotto, questo viene ignorato dal destinatario e considerato smarrito. TCP utilizza un checksum di 16 bit, obbligatorio in ogni segmento.

Riscontri

TCP utilizza i riscontri per confermare la ricezione dei segmenti dati e dei segmenti di controllo che non contengono dati ma che usano un numero di sequenza. I segmenti di riscontro (ACK) non sono mai riscontrati.

Quando, il destinatario, genera i riscontri? Nell'evoluzione di TCP sono state definite varie regole, utilizzate poi da varie implementazioni. Di seguito si riportano le regole più comuni:

1. Quando un'entità invia un segmento dati all'entità corrispondente, deve includere (piggybacking) un riscontro che fornisca il prossimo numero di byte che prevede di ricevere. Questa regola diminuisce il numero di segmenti necessari e riduce quindi il traffico.
2. Quando il destinatario non ha dati da inviare e riceve un segmento nell'ordine corretto (ovvero con il numero di sequenza atteso) e il segmento precedente è già stato riscontrato, ritarda l'invio del segmento ACK fino a quando non arriva un altro segmento o fino a quando non scade un tempo massimo predefinito (normalmente di 500ms). In altre parole il destinatario ritarda l'invio di un segmento ACK se ha un unico segmento nella sequenza corretta da riscontrare. Anche questa regola (detta ACK posticipato o delayed) evita che i segmenti ACK creino un traffico eccessivo, e fa uso di un timer di ACK-posticipato.
3. Quando arriva un segmento con numero di sequenza atteso e il segmento precedente non è stato riscontrato, il destinatario invia immediatamente un segmento ACK. In altre parole, non ci devono mai essere più di due segmenti nell'ordine corretto non riscontrati. Questo serve a evitare l'inutile ritrasmissione dei segmenti, che potrebbe produrre congestione nella rete.
4. Quando arriva un segmento fuori sequenza, il destinatario invia immediatamente un segmento ACK notificando il numero di sequenza atteso nel prossimo segmento. Questo consente la ritrasmissione rapida (fast retransmission) di eventuali segmenti smarriti.
5. Quando arriva un segmento mancante, il destinatario invia un segmento ACK per notificare il prossimo numero di sequenza atteso. Questo serve a notificare al mittente l'avvenuta ricezione di un segmento precedentemente annunciato come smarrito.
6. Quando arriva un segmento duplicato, il destinatario lo scarta e invia immediatamente un riscontro con il numero di sequenza del prossimo segmento atteso. Questo risolve alcuni problemi legati alla perdita dei segmenti di conferma.

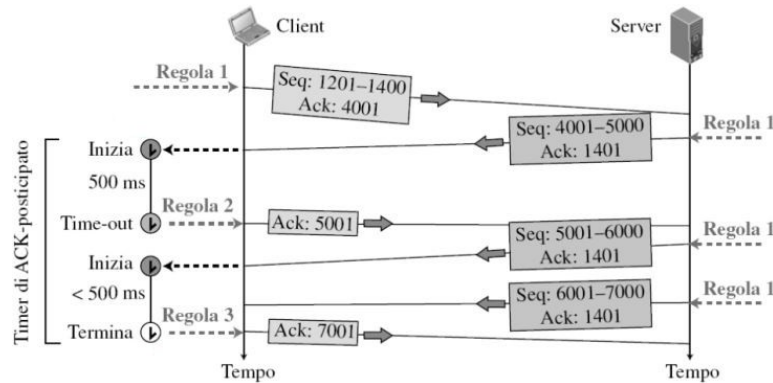
Ritrasmissione dei segmenti

Il cuore del meccanismo di controllo degli errori è la ritrasmissione dei segmenti: quando un segmento viene inviato, viene memorizzato in una coda in attesa di essere riscontrato. Alla scadenza del timer di ritrasmissione o quando il mittente riceve tre ACK duplicati per il precedente segmento nella coda, quel segmento viene ritrasmesso. Analizziamo ora i due casi nel dettaglio.

Il TCP mittente inizializza un timer di ritrasmissione, o RTO (Retransmission Time-Out), per ogni segmento inviato. Allo scadere del tempo limite senza averne ricevuto un riscontro, il segmento all'inizio della coda (il segmento con il numero di sequenza inferiore) viene ritrasmesso e si la ripartire il timer. Si noti che si ipotizza sempre che $S_f < S_n$. RTT è il tempo necessario a un segmento

Esempio 5.3.6 – Operatività normale

Il primo scenario illustra il trasferimento di dati bidirezionali fra due sistemi. Il client TCP invia un segmento, il server TCP ne invia tre.

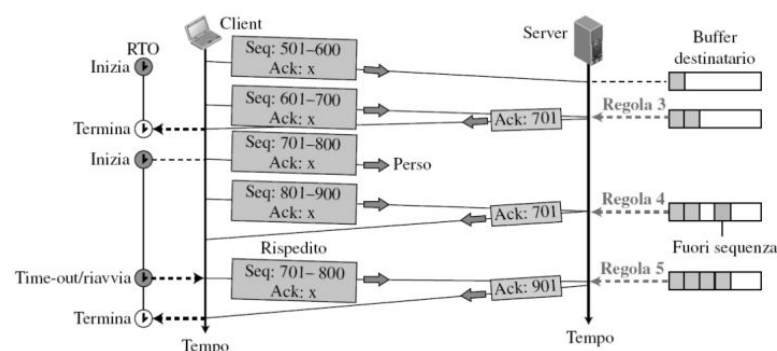


Per il primo segmento del client e tutti e tre i segmenti del server si applica la prima regola: vi sono dati da inviare, quindi il segmento indica il prossimo byte atteso. Quando il client riceve il primo segmento dal server, non ha altri dati da inviare: invia dunque un segmento ACK. Tuttavia la seconda regola stabilisce che il riscontro sia ritardato (posticipato) di 500ms in attesa di eventuali altri segmenti. Alla scadenza del timer viene inviata la conferma, dato che il client non può sapere se stanno arrivando altri segmenti e non può ritardare indefinitamente il riscontro. All'arrivo del segmento successivo viene inizializzato un altro timer di ACK posticipato. Questa volta, prima che scada il tempo impostato, arriva il terzo segmento che comporta l'invio del riscontro in conformità alla terza regola. Il timer RTO non è stato considerato, poiché nessun segmento è stato smarrito o ritardato. Si assume semplicemente che il timer RTO sia gestito correttamente.

Esempio 5.3.7 – Segmento smarrito

In questo scenario si illustra il caso in cui un segmento viene smarrito o corrotto.

I casi di segmento smarrito o corrotto vengono trattati nello stesso modo dal destinatario: il segmento smarrito viene perso da qualche parte nella rete, il segmento corrotto viene eliminato dal destinatario stesso. Entrambi vengono considerati persi. La figura illustra il caso in cui il segmento viene smarrito o eliminato da qualche router nella rete, eventualmente a causa di congestione.



Si suppone che il trasferimento dei dati sia unidirezionale: una postazione trasmette, l'altra riceve. In questo scenario il mittente invia i segmenti 1 e 2 che sono immediatamente riscontrati con un ACK (regola 3). Il terzo segmento viene smarrito. Il destinatario riceve il segmento 4 che risulta fuori sequenza, ne memorizza i dati nel suo buffer ma lascia uno spazio per indicare che non vi è continuità nei dati. Invia immediatamente un riscontro al mittente indicando il prossimo byte atteso (regola 4). Si noti che l'entità TCP destinataria memorizza i byte da 801 a 900, ma non li consegna all'applicazione fino a quando non riceve i dati mancanti.

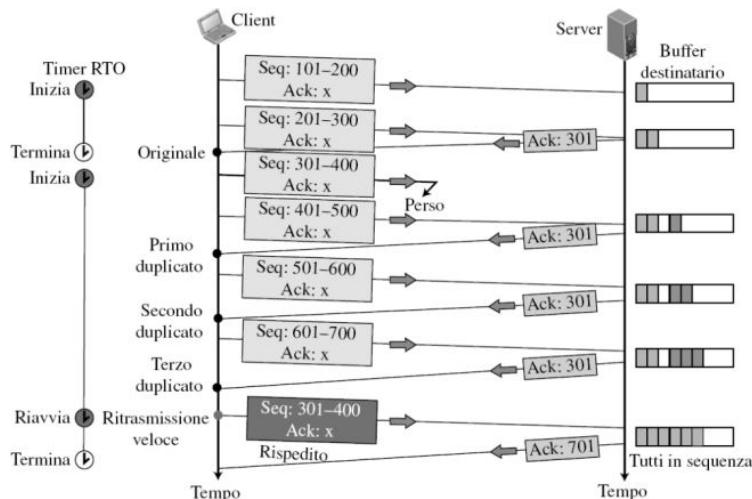
L'entità TCP destinataria consegna al processo applicativo solo dati correttamente ordinati. Il TCP mittente mantiene un timer RTO per tutto il periodo di connessione. Alla scadenza del timer il TCP mittente ritrasmette il segmento 3, che questa volta arriva correttamente e viene

riscontrato (regola 5).

Esempio 5.3.8 – Ritrasmissione veloce

In questo scenario si vuole esemplificare il concetto di ritrasmissione veloce.

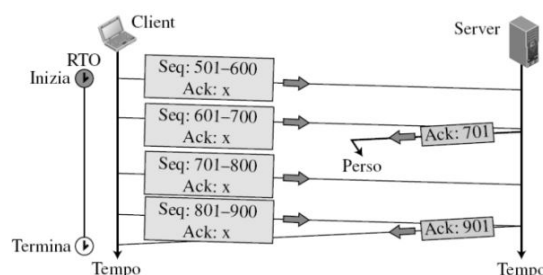
Questo scenario è simile al precedente ad eccezione del valore di RTO che in questo caso è superiore.



Quando il destinatario riceve un qualsiasi segmento successivo a quello atteso (in questo caso il terzo segmento) invia un riscontro indicante il segmento atteso (regola 4). Il mittente riceve quattro riscontri con il medesimo valore (tre duplicati). Sebbene il timer non sia ancora scaduto, la regola di ritrasmissione veloce richiede che il segmento 3, il segmento indicato come atteso da tutti e quattro i riscontri, sia immediatamente ritrasmesso. Una volta ritrasmesso il segmento, il timer viene fatto ripartire.

Esempio 5.3.9 – Risccontro smarrito senza ritrasmissione

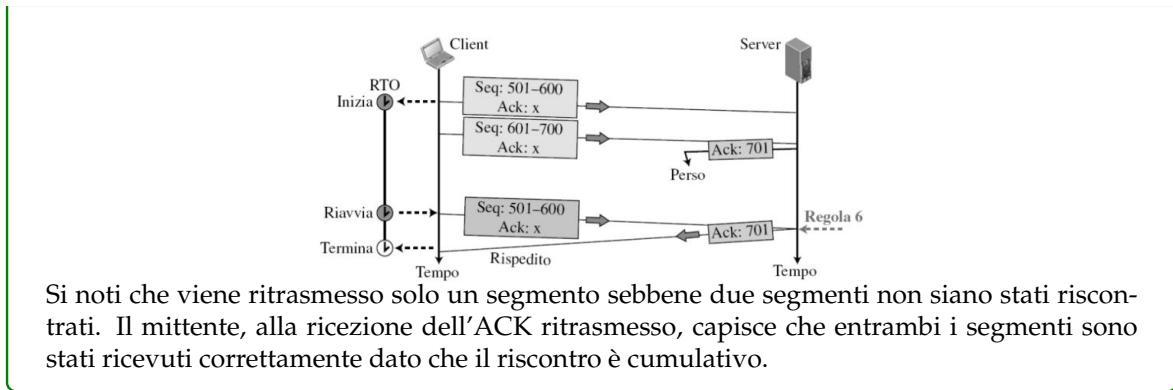
Questo scenario illustra il caso in cui un riscontro smarrito viene automaticamente rimpiazzato dal successivo.



Con la tecnica di riscontro cumulativo utilizzata da TCP, la perdita di una conferma può passare inosservata al TCP mittente: il riscontro successivo corregge automaticamente lo smarrimento del riscontro.

Esempio 5.3.10 – Risccontro smarrito con ritrasmissione

Se il riscontro successivo subisce un lungo ritardo o non vi è riscontro successivo (il riscontro smarrito è l'ultimo), il timer RTO scade senza che si sia ricevuto alcun riscontro. Il mittente ritrasmette quindi il segmento, che risulta duplicato. Il destinatario ignora il segmento che riceve duplicato ma ritrasmette immediatamente l'ultimo ACK per informare il mittente che il segmento o i segmenti sono arrivati a buon fine.



5.3.9 Controllo del flusso

Come già discusso, il controllo di flusso consente di bilanciare la velocità con cui un produttore genera i dati con la velocità con la quale un consumatore li utilizza. Il protocollo TCP separa il controllo di flusso dal controllo degli errori: si assume dunque che il canale logico fra il mittente e il destinatario TCP sia privo di errori.



Figura 5.39: Flusso dei dati e feedback del controllo del flusso in TCP

La Figura 5.39 mostra che i dati generati dal processo applicativo mittente vengono prima passati (pushed) al TCP mittente, da questo trasmessi al TCP destinatario e infine da quest'ultimo al processo applicativo ricevente (percorsi 1, 2 e 3). I feedback del controllo di flusso, invece, vengono trasmessi dal TCP ricevente al TCP mittente e da questo al processo applicativo mittente (percorsi 4 e 5). La maggior parte delle implementazioni TCP non fornisce feedback del flusso di controllo dal processo destinatario al TCP destinatario, ma consente al processo destinatario di richiedere a suo piacimento i dati dal TCP destinatario. In altri termini il TCP destinatario controlla il TCP mittente, il quale a sua volta controlla il processo mittente.

Il feedback del controllo di flusso dal TCP mittente al processo applicativo mittente (percorso 5) è ottenuto tramite il semplice rifiuto dei dati da parte del TCP mittente quando la sua finestra è completa. La descrizione del controllo di flusso si concentra quindi sul feedback inviato dal TCP destinatario al TCP mittente (percorso 4).

Apertura e chiusura delle finestre

Per controllare il flusso, il TCP forza il mittente e il destinatario a regolare la dimensione delle proprie finestre, sebbene la dimensione del buffer di entrambe le parti venga fissata all'apertura della connessione. La finestra di ricezione si chiude (sposta la sua parete sinistra verso la destra) quando arrivano altri byte dal mittente, si apre (sposta la sua parete destra verso destra) quando vengono richiesti altri byte dal processo. Si ipotizza che non venga mai ridotta (ovvero che la parete destra non si sposti verso sinistra).

L'apertura, la chiusura e la riduzione della finestra di invio sono controllate dal destinatario. La finestra di invio si chiude (sposta la sua parete sinistra verso destra) quando ciò viene consentito da un nuovo riscontro, si apre (sposta la sua parete destra verso destra) quando la dimensione della finestra di ricezione (rwnd) segnalata dal destinatario lo consente (nuovo ackNo + nuovo rwnd > ultimo rwnd). La finestra di invio viene ridotta quando non si verifica questa situazione.

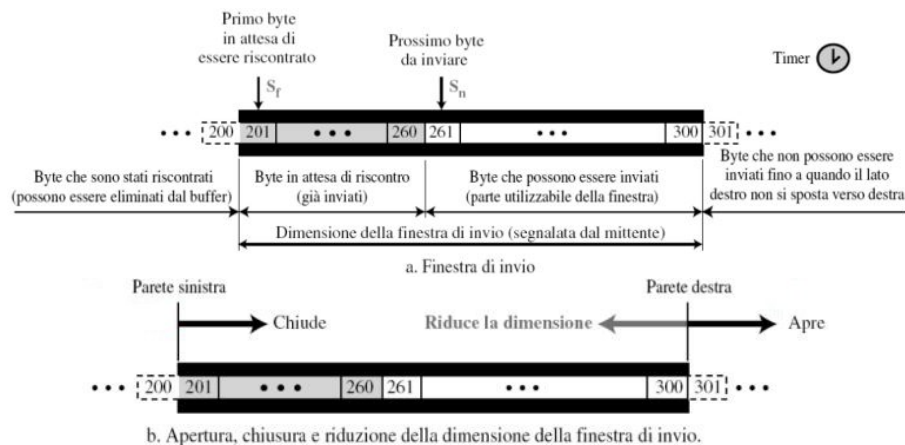
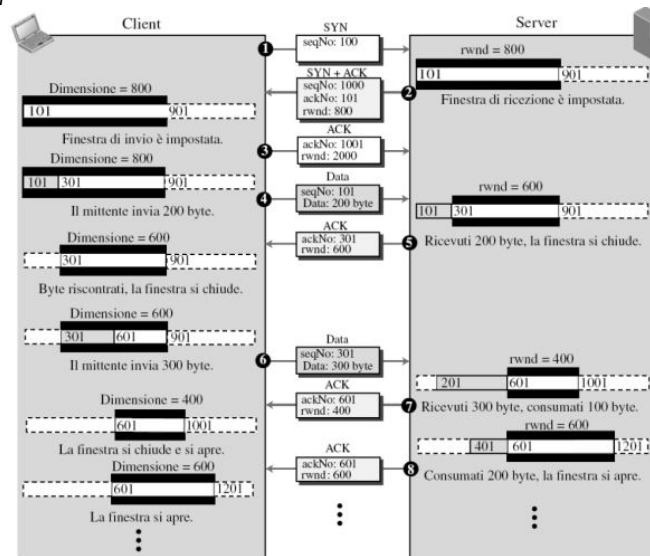


Figura 5.40: Finestra di invio nel protocollo TCP

Si descrive ora come le finestre di invio e di ricezione vengano impostate nella fase di apertura della connessione e come vengano modificate nella fase di trasferimento dati. A tal proposito viene illustrato un semplice esempio.

Esempio 5.3.11 – Finestre di invio e ricezione

Si illustra un semplice esempio di trasferimento dati unidirezionale (dal client al server). Per il momento si ignora il controllo degli errori, ipotizzando che nessun segmento venga corrotto, smarrito o arrivi duplicato o fuori sequenza.



Si noti che si sono mostrate due sole finestre per il trasferimento dati unidirezionale. Sebbene il client definisca la dimensione della finestra del server di 2000 byte nel terzo segmento, tale finestra non è schematizzata nella figura poiché si tratta di comunicazione unidirezionale. Vengono trasmessi otto segmenti fra client e server.

1. Il primo segmento è dal client al server (un segmento SYN) per richiedere l'apertura di una connessione. Il client annuncia il proprio numero di sequenza iniziale $seqNo=100$. Quando il server riceve questo segmento, alloca un buffer di dimensione 800 (per ipotesi) e imposta la propria finestra per utilizzare l'intero buffer ($rwnd=800$). Si noti che il numero del byte atteso successivo è il 101.
2. Il secondo segmento, un SYN+ACK, è dal server al client. Utilizza $ackNo=101$ per indicare che si attende di ricevere i byte a partire dal 101. Annuncia anche che il client può impostare una dimensione del buffer di 800 byte.

3. Il terzo segmento è un ACK dal client al server. Si noti che il client ha impostato la dimensione della finestra di ricezione con `rwnd=2000`, ma nella figura questa finestra non viene indicata poiché si tratta esclusivamente il caso di comunicazione unidirezionale.
4. Dopo che il client ha impostato la dimensione della propria finestra (800), come indicato dal server, il processo richiede la trasmissione di 200 byte di dati. Il client TCP numera questi byte da 101 a 300, crea un segmento e lo invia al server. Il segmento segnala che il numero del primo byte è 101 e che trasporta 200 byte di dati. La finestra del client viene quindi impostata in modo tale da indicare che sono stati inviati 200 byte di dati, che sono ancora da riscontrare. Il server quando riceve il segmento memorizza i dati e la finestra di ricezione si chiude per indicare che il byte successivo atteso è il numero 301; i byte memorizzati occupano 200 byte del buffer.
5. Il quinto segmento è il feedback dal server al client, con il quale il server riscontra i byte fino al 300 incluso (aspettandosi di ricevere il 301). Il segmento include anche la nuova dimensione della finestra di ricezione dopo il decremento (600). Il client, ricevuto questo segmento, elimina i byte confermati dalla propria finestra chiudendola, indicando che il byte successivo atteso è il 301. La dimensione della finestra, tuttavia, viene ridotta a 600 byte. Sebbene il buffer allocato possa contenere 800 byte, la finestra non si può aprire (spostare la parete destra verso destra) poiché il destinatario non lo consente.
6. Il segmento 6 viene inviato dal client dopo che il suo processo chiede la trasmissione di altri 300 byte. Il segmento definisce `seqNo=301` e contiene 300 byte. Il server, quando riceve questo segmento, ne memorizza i dati e deve ridurre la dimensione della propria finestra. Dopo che il proprio processo ha richiesto 100 byte di dati, la finestra chiude dalla sinistra per un ammontare di 300 byte, ma apre alla destra per un ammontare di 100 byte. Come risultato la sua dimensione viene ridotta di soli 200 byte, al valore attuale di 400 byte.
7. Nel segmento 7 il server riscontra la ricezione dei dati e annuncia che la dimensione della propria finestra è 400. Quando il client riceve questo segmento, deve ridurre nuovamente la propria finestra e impostare la sua dimensione al valore `rwnd=400` come indicato dal server. La finestra di invio si chiude a sinistra di 300 byte e si apre a destra di 100 byte.
8. Anche il segmento 8 viene inviato dal server, dopo che il proprio processo ha richiesto altri 200 byte. Il server incrementa la dimensione della propria finestra (`rwnd` vale ora 600) e informa il client che si attende ancora il byte 601, ma che la dimensione della propria finestra è ora di 600 byte. E' necessario precisare che la spedizione di questo segmento dipende dalle varie implementazioni. Alcune non consentono l'annuncio di `rwnd` in questo momento: il server deve ricevere altri dati prima di poterlo fare. Il client, ricevuto questo segmento, apre la propria finestra di 200 byte senza chiuderla. Come risultato la dimensione della propria finestra diviene 600.

Come già detto la finestra di ricezione non può essere ridotta. La finestra di invio, invece, può essere ridotta quando il destinatario definisce un valore di `rwnd` inferiore al precedente. Alcune implementazioni non permettono comunque la riduzione della finestra di invio: questa limitazione non consente di spostare la parete destra della finestra di invio verso sinistra. In altri termini il destinatario deve rispettare la seguente relazione fra l'ultimo riscontro e il precedente e fra il precedente e il nuovo valore di `rwnd` per evitare la riduzione della finestra di invio:

$$\text{nuovo ackNo} + \text{nuovo rwnd} \geq \text{ultimo ackNo} + \text{ultimo rwnd}$$

La parte sinistra della disuguaglianza rappresenta la nuova posizione della parete destra rispetto allo spazio dei numeri di sequenza; la parte destra indica la posizione precedente della parete destra. La relazione indica che la parete destra non deve spostarsi a sinistra. La disuguaglianza è necessaria al destinatario per controllare i propri annunci. Si noti tuttavia che la disuguaglianza è valida solo se $S_f < S_n$: si deve ricordare che tutti i calcoli vengono effettuati modulo 2^{32} .

5.3.10 Controllo della congestione

TCP utilizza diverse strategie per gestire la congestione nella rete.

Finestra di congestione

Quando si è descritto il controllo di flusso nel protocollo TCP, si è detto che la dimensione della finestra di invio è controllata dal destinatario tramite il valore di *rwnd*, che viene indicato in ogni segmento trasmesso nella direzione opposta. L'impiego di questa strategia garantisce che la finestra del ricevente non venga mai sovraccaricata con i dati ricevuti (non esiste congestione della postazione destinataria). Questo tuttavia non garantisce che i buffer intermedi, i buffer nei router, non si congestionino. Un router potrebbe ricevere dati da più di un mittente. Indipendentemente dalla capacità del buffer di un router, è sempre possibile che questo venga saturato e che si trovi costretto a scartare i segmenti di qualche specifico mittente TCP. In altre parole non vi è congestione agli estremi, ma vi può essere congestione nei nodi intermedi. TCP deve preoccuparsi della possibile congestione della rete poiché la perdita di un gran numero di segmenti può influire significativamente sul meccanismo di controllo degli errori. La perdita di numerosi segmenti comporta la loro rispeditura, peggiorando così la congestione della rete e portando alla fine al collasso della comunicazione.

TCP è un protocollo end-to-end che sfrutta i servizi di IP. La congestione nei router è un problema che riguarda IP e che dovrebbe essere gestito a livello rete. Tuttavia IP è un semplice protocollo senza controllo della congestione. TCP è dunque necessariamente costretto a occuparsi del problema.

TCP non può ignorare la congestione nella rete: non può inviare un numero di segmenti eccessivo alla rete, altrimenti ne verrebbe penalizzato. Ma TCP non può nemmeno essere troppo conservativo, nel senso di inviare un numero troppo limitato di segmenti nell'unità di tempo, poiché non sfrutterebbe al meglio la capacità della rete. TCP deve quindi prevedere delle strategie per accelerare la trasmissione dei dati quando non vi è congestione e ridurre la trasmissione quando rileva la congestione.

Per controllare il numero di segmenti da trasmettere, TCP utilizza una seconda variabile chiamata *cwnd* (congestion window - finestra di congestione), il cui valore dipende dal livello di congestione nella rete. Le due variabili *cwnd* e *rwnd* congiuntamente definiscono la dimensione della finestra di invio di TCP. La prima variabile è legata alla congestione della rete, la seconda alla congestione delle postazioni terminali. La dimensione finale della finestra corrisponde al minimo dei due valori.

$$\text{Dimensione della finestra} = \min(rwnd, cwnd)$$

Rilevare la congestione

Prima di analizzare come il valore di *cwnd* debba essere impostato e aggiornato, è necessario descrivere come un mittente TCP possa identificare il possibile verificarsi della congestione nella rete. Il mittente interpreta come sintomi di congestione due eventi: il timeout e la ricezione di tre riscontri duplicati.

Il primo evento è dunque il timeout. Se un mittente TCP non riceve un riscontro per un segmento o un gruppo di segmenti prima dello scadere del timeout, suppone che tale segmento o i segmenti siano stati smarriti a causa della congestione.

Il secondo evento è la ricezione di tre riscontri duplicati (quattro ACK con il medesimo numero di riscontro). Si ricordi che un destinatario TCP invia un riscontro duplicato per segnalare un ritardo nella ricezione di un segmento, ma tre riscontri duplicati sono un'indicazione di segmento smarrito, che può essere dovuto a congestione nella rete. La congestione nel caso dei tre riscontri duplicati è tuttavia probabilmente meno critica che nel caso del timeout. Se il destinatario invia tre riscontri duplicati, significa che un segmento è stato smarrito, ma anche che altri tre segmenti sono stati ricevuti: la rete è al limite della congestione o è appena rientrata da una congestione.

E' interessante notare che il mittente TCP usa solo un tipo di feedback dal destinatario per rilevare la congestione: i riscontri. L'assenza della ricezione rapida e regolare dei riscontri, evidenziata dallo scadere del timeout, è il sintomo di una congestione severa; la ricezione di tre riscontri duplicati è il sintomo di una congestione debole.

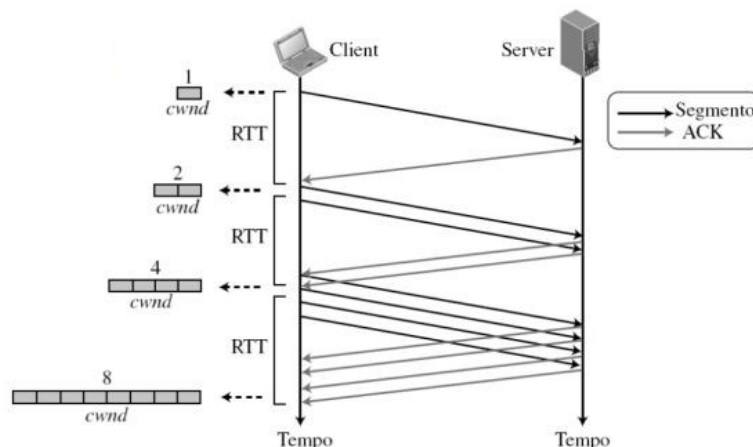
Strategie di gestione della congestione

La strategia generale del protocollo TCP per gestire la congestione si basa su tre fasi: slow start (partenza lenta), congestion avoidance (prevenzione della congestione) e fast recovery (recupero veloce). Si descrive per prima ciascun algoritmo, per poi mostrare come il protocollo TCP passi dall'uno all'altro durante una connessione.

Algoritmo 5.3.1 – Slow start: incremento esponenziale

L'algoritmo slow start (partenza lenta) si basa sull'idea che la dimensione della finestra di congestione (*cwnd*) viene inizializzata con la dimensione del segmento massimale (MSS - Maximum Segment Size), ma viene aumentata di un valore corrispondente a MSS a ogni segmento riscontrato. Come già visto, l'MSS viene determinato all'apertura della connessione utilizzando l'opzione omonima.

Il nome dell'algoritmo è decisamente fuorviante: l'algoritmo parte lentamente ma procede con velocità esponenzialmente crescente. Si è supposto che *rwnd* sia molto più elevato di *cwnd*, in modo tale che la finestra del mittente sia sempre uguale a *cwnd*. Si ipotizza anche che tutti i segmenti siano della stessa dimensione di MSS byte. Per semplicità si ignora anche la strategia dei riscontri in ritardo e si ipotizza che ogni segmento sia riscontrato individualmente.



Il mittente inizia con *cwnd*=1, quindi può inviare un solo segmento. Ricevuto il primo riscontro, il segmento riscontrato viene eliminato dalla finestra di invio: nella finestra si è quindi liberato spazio per un segmento. Anche la dimensione della finestra di congestione è incrementata di 1 perché la ricezione del riscontro è un buon sintomo di assenza di congestione nella rete. La dimensione della finestra vale ora 2 ed è possibile inviare due segmenti. Quando questi vengono confermati, la dimensione della finestra viene aumentata di 1 per ciascun riscontro ricevuto, quindi *cwnd* vale ora 4 e così via. In altri termini, la dimensione della finestra di congestione con questo algoritmo è una funzione del numero di riscontri ricevuti e può essere determinata come segue:

$$\text{Se arriva un riscontro: } cwnd = cwnd + 1$$

Valutando la dimensione di *cwnd* in funzione del tempo di andata e ritorno (RTT), si trova che la velocità di crescita è esponenziale, dunque molto aggressiva, in termini di ciascun RTT:

Inizio	→	$cwnd = 1 \rightarrow 2^0$
Dopo1RTT	→	$cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$
Dopo2RTT	→	$cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$
Dopo3RTT	→	$cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

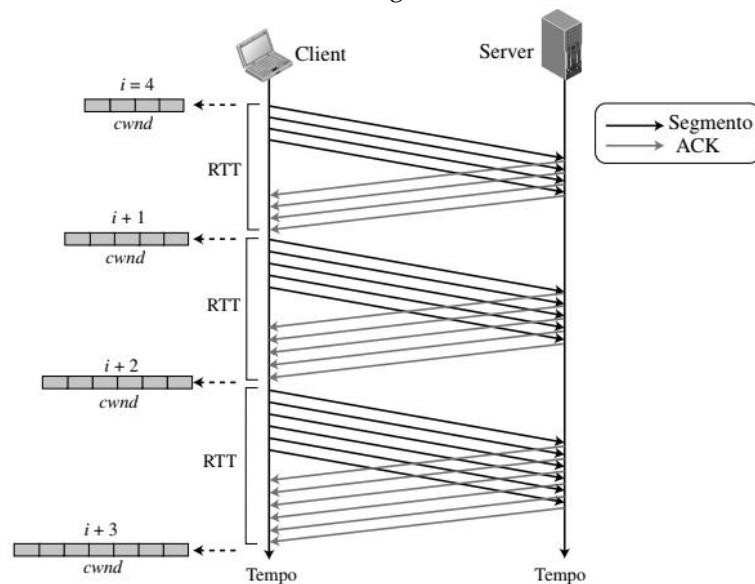
La progressione dell'algoritmo non può continuare indefinitamente: è necessaria una soglia per arrestare questa fase. Il mittente tiene traccia di una variabile, chiamata *sssthresh* (slow start threshold o soglia di slow start): quando la dimensione della finestra espressa in byte raggiunge questa soglia, slow start termina e inizia la fase successiva.

E' necessario menzionare il fatto che la progressione dell'algoritmo slow start è più lenta nel caso di riscontri posticipati. Si ricordi che $cwnd$ è incrementato di 1 per ciascun riscontro. Di conseguenza, se due segmenti vengono riscontrati cumulativamente, il valore di $cwnd$ si incrementa solo di 1, non di 2. La crescita è ancora esponenziale, ma non è una potenza di 2.

La dimensione della finestra di congestione aumenta esponenzialmente nella fase di slow start, ma per evitare la congestione il ritmo di crescita deve essere rallentato. TCP prevede un altro algoritmo, chiamato congestion avoidance, che incrementa in modo lineare anziché esponenziale il valore di $cwnd$.

Algoritmo 5.3.2 – Congestion avoidance: additive increase

Quando la dimensione della finestra di congestione raggiunge la soglia di slow start ($ssthresh$) dove $cwnd=i$, la fase di slow start si arresta e inizia la fase di incremento lineare (additive increase). Con questo algoritmo, ogni volta che viene riscontrata l'intera finestra di segmenti la dimensione della finestra di congestione viene incrementata di 1.



Il mittente inizia con $cwnd=4$. Questo significa che il mittente può inviare solo quattro segmenti, prima di ricevere un riscontro. In questo caso, dopo che il mittente ha ricevuto i riscontri per tutti i segmenti della finestra, la dimensione della finestra viene aumentata di 1. La dimensione della finestra è quindi 5. Dopo aver inviato 5 segmenti e aver ricevuto i 5 riscontri corrispondenti, la dimensione della finestra di congestione diviene 6 e così via.

Anche con questo algoritmo la dimensione della finestra di congestione è una funzione del numero di riscontri ricevuti e viene determinata come segue:

$$\text{Se arriva un riscontro: } cwnd = cwnd + (1/cwnd)$$

In altri termini, la dimensione della finestra cresce solamente di una porzione $1/cwnd$ di un MSS (in byte). Ovvero, tutti i segmenti nella finestra precedente devono essere riscontrati per aumentare la finestra di 1 MSS. Valutando la dimensione di $cwnd$ in termini del tempo di andata e ritorno (RTT), si trova che la velocità aumenta in modo lineare, ovvero molto più lentamente che nello slow start.

Inizio	→	$cwnd = i$
Dopo 1RTT	→	$cwnd = i + 1$
Dopo 2RTT	→	$cwnd = i + 2$
Dopo 3RTT	→	$cwnd = i + 3$

Con l'algoritmo congestion avoidance, la dimensione della finestra di congestione viene aumentata linearmente fino alla rilevazione della congestione.

Algoritmo 5.3.3 – Fast recovery

L'algoritmo fast recovery (recupero veloce) è opzionale in TCP. Esso inizia quando arrivano tre riscontri duplicati che vengono interpretati come indizio di leggera congestione nella rete. Come il congestion avoidance, anche questo algoritmo incrementa linearmente, ma incrementa la dimensione della finestra di congestione quando riceve un riscontro duplicato (dopo i tre riscontri duplicati che fanno partire questo algoritmo). Si può dire:

Se arriva un riscontro duplicato: $cwnd = cwnd + (1/cwnd)$

Sono state presentate tre tecniche di gestione della congestione in TCP. Si vuole ora capire quando viene utilizzata ciascuna di queste e quando TCP decide di passare dall'una all'altra. Per far questo è necessario fare riferimento a due versioni di TCP: Tahoe e Reno.

TCP Tahoe

La prima versione di TCP, chiamata TCP Tahoe, usava solo due algoritmi per la gestione della congestione: slow start e congestion avoidance.

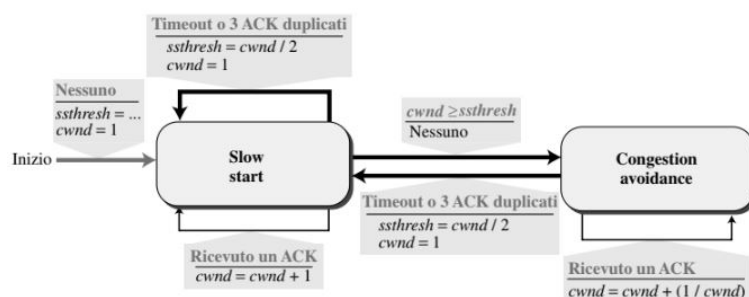
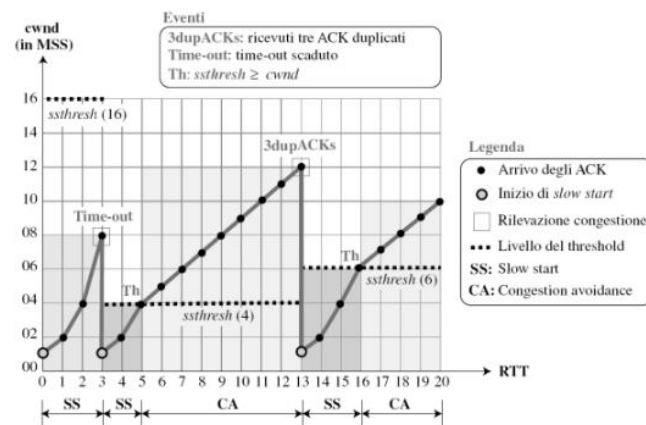


Figura 5.41: FSM per il meccanismo di controllo della congestione di TCP Tahoe

TCP Tahoe reagisce ai due sintomi di congestione, il timeout e i tre riscontri duplicati, nello stesso modo. In questa versione, una volta aperta la connessione, TCP avvia l'algoritmo slow start e imposta la variabile *ssthresh* a un valore preconcordato (solitamente multiplo di MSS) e *cwnd* a 1 MSS. In questo stato, come già detto, ogni volta che arriva un riscontro la dimensione della finestra di congestione viene aumentata di 1. E' evidente che si tratta di una strategia molto aggressiva che incrementa esponenzialmente la dimensione della finestra, cosa che può comportare una congestione. Se viene rilevata la congestione (scadenza del timeout o ricezione di tre riscontri duplicati), TCP interrompe immediatamente la crescita esponenziale, fa ripartire l'algoritmo slow start con nuovo threshold uguale alla metà del valore corrente di *cwnd* e reimposta la dimensione della finestra a 1. In altri termini, non solo TCP riparte dall'inizio, ma aggiorna anche il threshold. Se non viene rilevata congestione fino al raggiungimento del threshold, TCP considera di avere raggiunto il massimo delle possibili ambizioni e di non potere continuare a ingrandire la finestra a tale velocità. Si sposta quindi nello stato di congestion avoidance, dove la dimensione della finestra viene incrementata di 1 ogni volta che riceve un numero di riscontri pari alla dimensione corrente della finestra. Per esempio, se la dimensione della finestra è di 5 MSS, è necessario ricevere altri 5 riscontri prima di poterla incrementare a 6 MSS. Si noti che non vi è un limite alla dimensione della finestra che può essere raggiunta in questo stato: la crescita lineare continua fino alla chiusura della connessione o fino a quando viene rilevata la congestione. Se la congestione viene rilevata in questo stato, TCP reimposta il valore di *ssthresh* alla metà del valore corrente di *cwnd* e si sposta nuovamente nello stato slow start.

Sebbene in questa versione di TCP la dimensione di *ssthresh* venga continuamente aggiornata a ogni rilevazione di congestione, essa non diviene necessariamente minore del valore precedente. Per esempio, se il valore originale di *ssthresh* fosse 8 MSS e la congestione fosse rilevata quando TCP è nello stato di congestion avoidance con valore di *cwnd*=20, il nuovo valore di *ssthresh* diverrebbe 10, ovvero verrebbe aumentato.

Esempio 5.3.12 – TCP Thao

TCP inizia il trasferimento dei dati e imposta la variabile $ssthresh$ all'ambizioso valore di 16 MSS. Lo stato iniziale è lo slow start (SS) con $cwnd=1$. La finestra di congestione cresce esponenzialmente ma avviene un timeout dopo il terzo RTT (prima di raggiungere il threshold). TCP assume che la rete sia congestionata, imposta il valore di $ssthresh$ a 4 MSS (la metà del valore corrente di $cwnd$, che vale 8) e inizia un nuovo stato slow start (SS) con $cwnd=1$ MSS. La finestra di congestione cresce esponenzialmente fino a quando raggiunge il nuovo valore di threshold. A questo punto il TCP passa nello stato congestion avoidance (CA), dove la finestra di congestione cresce linearmente fino a quando raggiunge $cwnd=12$ MSS. A questo punto vengono ricevuti tre riscontri duplicati, un altro sintomo di congestione della rete. TCP dimezza nuovamente il valore di $ssthresh$ a 6 MSS e passa nello stato slow start (SS). La crescita esponenziale di $cwnd$ prosegue. Dopo RTT 15, la dimensione di $cwnd$ è 4 MSS. Dopo avere inviato quattro segmenti e ricevuto due soli riscontri, la dimensione della finestra raggiunge il valore di $ssthresh(6)$, quindi TCP passa nello stato congestion avoidance. Il trasferimento dei dati continua in questo stato (CA) fino a quando viene chiusa la connessione, in corrispondenza di RTT 20.

TCP Reno

In una versione più recente di TCP, chiamata Reno, è stato aggiunto un nuovo stato alla FSM per il controllo della congestione, chiamato stato di fast recovery (recupero veloce). Questa versione tratta i due sintomi della congestione, il timeout e la ricezione dei tre riscontri duplicati, in modo differente. Quando avviene un timeout, TCP passa allo stato slow start (o riparte in questo stato se già vi si trova). Quando riceve tre riscontri duplicati, TCP passa allo stato fast recovery e vi rimane fintantoché arrivano altri riscontri duplicati. Lo stato fast recovery è in un certo senso uno stato intermedio fra gli stati slow start e congestion avoidance. Si comporta come lo slow start, nel senso che $cwnd$ cresce esponenzialmente, ma il valore iniziale di $cwnd$ è uguale a $ssthresh$ più 3 MSS (anziché 1). Quando TCP passa nello stato fast recovery, possono avvenire tre eventi principali.

1. Se continuano ad arrivare riscontri duplicati, TCP rimane nello stato ma $cwnd$ cresce esponenzialmente.
2. Se avviene un timeout, TCP assume che vi sia una reale congestione nella rete e passa allo stato slow start.
3. Se arriva un nuovo riscontro (non duplicato), TCP passa allo stato congestion avoidance, ma riduce il valore di $cwnd$ al valore di $ssthresh$ come se i tre riscontri duplicati non fossero stati ricevuti e la transizione fosse dallo stato slow start allo stato congestion avoidance.

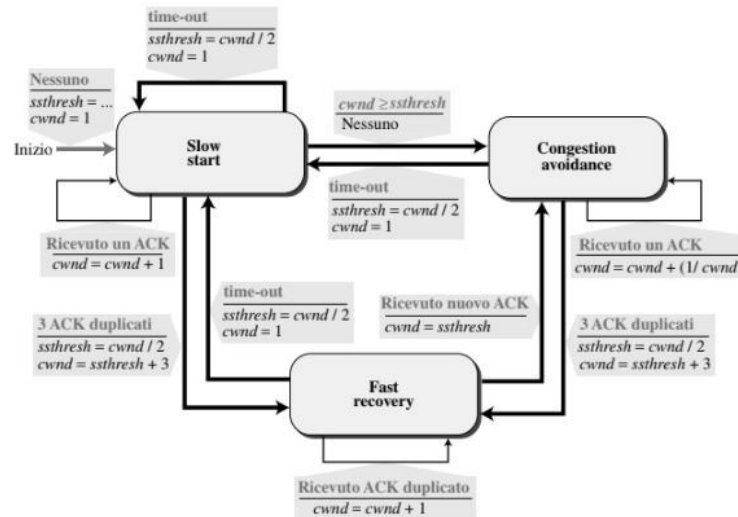
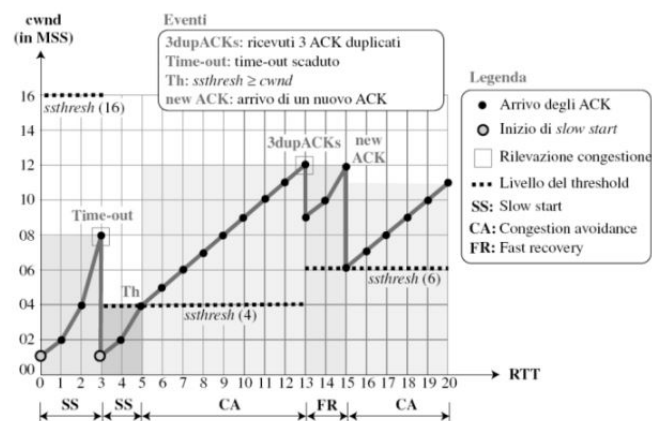


Figura 5.42: FSM per il TCP Reno

Esempio 5.3.13 – TCP Reno



La dimensione della finestra di congestione cambia nello stesso modo fino a RTT 13, quando vengono ricevuti 3 riscontri duplicati. A questo punto TCP Reno riduce $ssthresh$ a 6 MSS (come nel caso del TCP Tahoe) ma imposta il valore di $cwnd$ a un valore molto più alto ($ssthresh + 3 = 9$ MSS) anziché 1 MSS. TCP Reno passa ora nello stato di fast recovery. Si ipotizza che arrivino due altri ACK duplicati fino a RTT 15, mentre $cwnd$ cresce esponenzialmente. A questo punto arriva un nuovo riscontro (non duplicato) che segnala la ricezione del segmento smarrito. TCP Reno passa ora nello stato congestion avoidance, ma riduce prima la finestra di congestione a 6 MSS (il valore di $ssthresh$), ignorando l'intero stato fast recovery e ritornando nella traccia precedente.

5.3.11 Timeout e stima del tempo di andata e ritorno

TCP utilizza un meccanismo di timeout e ritrasmissione per recuperare i segmenti persi. Sebbene tutto questo sia concettualmente semplice, l'implementazione di tale meccanismo in un protocollo come TCP crea alcuni problemi. Forse la questione più ovvia è la durata degli intervalli di timeout. Chiaramente, il timeout dovrebbe essere più grande del tempo di andata e ritorno sulla connessione (RTT), ossia del tempo trascorso da quando si invia un segmento a quando se ne riceve l'acknowledgment, altrimenti ci sarebbero delle ritrasmissioni inutili. Ma di quanto deve essere maggiore? E, innanzitutto, come dovrebbe essere stimato RTT? Si dovrebbe associare un timer a ogni segmento non riscontrato?

Stima del tempo di andata e ritorno

Cominciamo lo studio della gestione dei timer TCP considerando come il protocollo stimi il tempo di andata e ritorno tra mittente e destinatario. L'RTT misurato di un segmento, denotato come *SampleRTT*, è la quantità di tempo che intercorre tra l'istante di invio del segmento (ossia quando viene passato a IP) e quello di ricezione dell'acknowledgment del segmento. Anziché misurare un *SampleRTT* per ogni segmento, la maggior parte delle implementazioni TCP effettua una sola misurazione di *SampleRTT* alla volta. Ossia, in ogni istante di tempo, *SampleRTT* viene valutato per uno solo dei segmenti trasmessi e per cui non si è ancora ricevuto acknowledgment, il che comporta approssimativamente la misurazione di un nuovo valore di *SampleRTT* a ogni RTT. Inoltre, TCP non calcola mai il *SampleRTT* per i segmenti ritrasmessi, cioè lo calcola soltanto per i segmenti trasmessi una volta sola. Ovviamente, i campioni variano da segmento a segmento in base alla congestione nei router e al diverso carico sui sistemi periferici. A causa di tale fluttuazione, ogni valore di *SampleRTT* può essere atipico. Per effettuare una stima risulta naturale calcolare una media dei valori di *SampleRTT*, che TCP chiama *EstimatedRTT*. Quando si ottiene un nuovo *SampleRTT*, TCP aggiorna *EstimatedRTT* secondo la formula:

$$EstimatedRTT = (1-\alpha) * EstimatedRTT + \alpha * SampleRTT$$

Il nuovo valore di *EstimatedRTT* è una combinazione ponderata del suo precedente valore e del nuovo valore di *SampleRTT*. Il valore raccomandato per α è 0,125 (ossia 1/8), nel qual caso la formula precedente diventa:

$$EstimatedRTT = 0,875 * EstimatedRTT + 0,125 * SampleRTT$$

Si noti che *EstimatedRTT* è una media ponderata dei valori *SampleRTT*. Come discusso in un problema alla fine di questo capitolo, tale media attribuisce maggiore importanza ai campioni recenti rispetto a quelli vecchi. Questo è naturale, dato che i primi riflettono meglio la congestione attuale della rete. In statistica, una media costruita in tal modo è detta media mobile esponenziale ponderata (EWMA, exponential weighted moving average). La parola “esponenziale” compare nell'acronimo EWMA in quanto il peso dei campioni decresce esponenzialmente al procedere degli aggiornamenti.

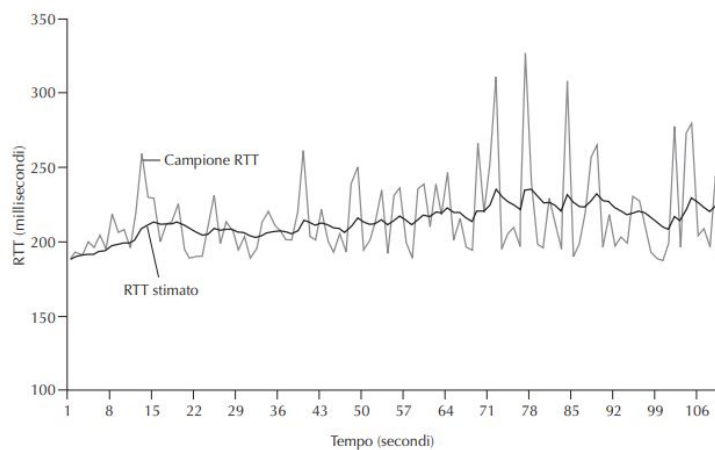


Figura 5.43: Campioni e stima RTT

La Figura 5.43 mostra i valori *SampleRTT* ed *EstimatedRTT* con $\alpha = 1/8$ per una connessione TCP tra *gaia.cs.umass.edu* (ad Amherst, Massachusetts) e *fantasia.eurecom.fr* (nel sud della Francia). Come si può vedere, le variazioni di *SampleRTT* vengono “smussate” dal calcolo di *EstimatedRTT*. Oltre ad avere una stima di RTT, è anche importante possedere la misura della sua variabilità, definisce la variazione RTT, *DevRTT*, come una stima di quanto *SampleRTT* generalmente si discosta da *EstimatedRTT*:

$$DevRTT = (1-\beta) * DevRTT + \beta * |SampleRTT - EstimatedRTT|$$

Si noti che *DevRTT* è un EWMA della differenza tra *SampleRTT* e *EstimatedRTT*. Se i valori di *SampleRTT* presentano fluttuazioni limitate, allora *DevRTT* sarà piccolo; in caso di notevoli fluttuazioni, *DevRTT* sarà grande. Il valore suggerito per β è 0,25.

Impostazione e gestione del timeout di ritrasmissione

Dati i valori di *EstimatedRTT* e *DevRTT*, quali valori andrebbero usati per l'intervallo di timeout di TCP? Chiaramente, l'intervallo non può essere inferiore a quello di *EstimatedRTT*, altrimenti verrebbero inviate ritrasmissioni non necessarie. Ma l'intervallo stesso non dovrebbe essere molto maggiore di *EstimatedRTT*, altrimenti TCP non ritrasmetterebbe rapidamente il segmento perduto, il che comporterebbe gravi ritardi sul trasferimento dei dati. E' pertanto auspicabile impostare il timeout a *EstimatedRTT* più un certo margine che dovrebbe essere grande quando c'è molta fluttuazione nei valori di *SampleRTT* e piccolo in caso contrario. Qui entra in gioco il valore di *DevRTT*. Tutti questi aspetti vengono presi in considerazione dal modo in cui TCP determina l'intervallo di timeout di ritrasmissione:

$$TimeoutInterval = EstimatedRTT + 4 * DevRTT$$

Viene raccomandato un valore iniziale di *TimeoutInterval* pari a 1 secondo. Inoltre, quando si verifica un timeout, *TimeoutInterval* viene raddoppiato per evitare un timeout prematuro riferito a un segmento successivo per cui si riceverà presto un *acknowledgment*. Tuttavia, appena viene ricevuto un segmento ed *EstimatedRTT* viene aggiornato, *TimeoutInterval* viene ricalcolato secondo la formula precedente.

Capitolo 6

Livello di rete

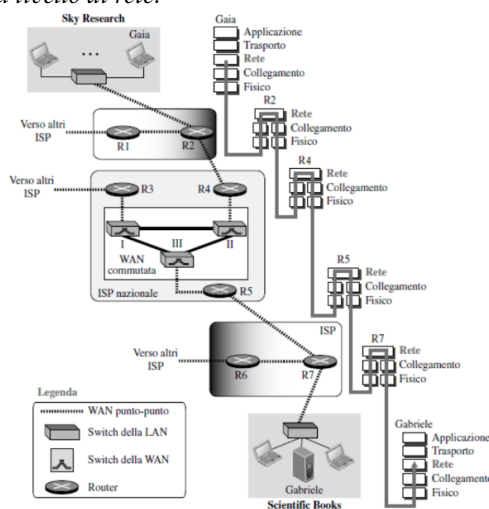
Il livello di rete nello stack TCP/IP è responsabile della consegna dei datagrammi tra gli host. Questo procedimento comprende numerosi aspetti, come ad esempio la progettazione degli indirizzi logici necessari per identificare in maniera univoca gli host connessi ad Internet. Inoltre, richiede anche degli opportuni protocolli di instradamento che permettano ai pacchetti del livello di rete di giungere dalla sorgente fino alla destinazione.

6.1 Introduzione

Esempio 6.1.1 – Comunicazione a livello rete

La Figura mostra la comunicazione tra Gaia e Gabriele a livello di rete.

Come viene mostrato, il livello di rete è presente nell'host sorgente, nell'host destinazione e in tutti i router lungo il percorso (R2, R4, R5 e R7). Nell'host sorgente (Gaia), il livello di rete accetta un pacchetto (più precisamente un segmento o un datagramma utente) dal livello trasporto, incapsula il pacchetto in un datagramma del livello di rete e lo consegna al livello di collegamento. Nell'host di destinazione (Gabriele), il datagramma viene decapsulato, il pacchetto viene estratto e consegnato al livello trasporto. Sebbene gli host sorgente e destinazione siano coinvolti in tutti e cinque i livelli della pila di protocolli TCP/IP, i router utilizzano solamente tre livelli durante l'attività vera e propria di instradamento dei pacchetti. Tuttavia, come vedremo in seguito, i router possono aver bisogno dei livelli di trasporto ed applicazione per alcuni scopi particolari (ad esempio per coordinarsi con altri router). Un router presente lungo il percorso tra sorgente e destinazione normalmente viene rappresentato con almeno due livelli di collegamento e due livelli fisici, in quanto riceve un pacchetto da una rete e lo invia su un'altra rete.



6.1.1 Servizi a livello rete

Il ruolo principale del livello di rete è quindi piuttosto semplice: trasferire pacchetti da un host a un altro. *Ma quando un router riceve un pacchetto come fa a sapere a chi deve inviarlo?*

Instradamento (routing)

Un compito del livello di rete è l'instradamento (routing). Il compito del livello di rete è quello di instradare il pacchetto dalla sua sorgente alla destinazione. Una rete fisica non è altro che la combinazione di varie altre reti (LAN e WAN) e dei router che le collegano. Ciò significa che, normalmente,

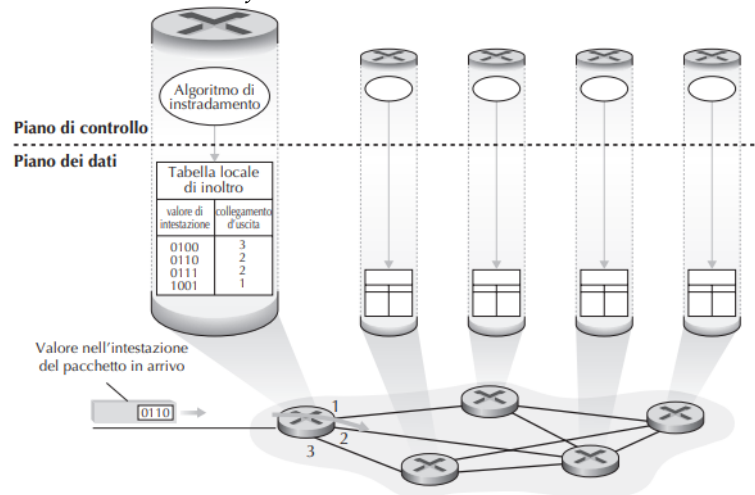
c'è più di un percorso che va dalla sorgente alla destinazione. Il livello di rete deve trovare il migliore tra tali possibili percorsi. La scelta del percorso migliore deve avvenire per mezzo di opportune strategie (algoritmi di routing). Nella Internet di oggi questo avviene tramite l'esecuzione di alcuni protocolli di routing (protocolli di instradamento), che hanno lo scopo di aiutare i router a condividere e coordinare la loro conoscenza sulla rete. Il risultato è la creazione di tabelle di instradamento che verranno utilizzate per decidere come instradare i pacchetti al momento del loro arrivo nei router.

Inoltro (forwarding)

Se routing significa applicare strategie ed eseguire protocolli per creare tabelle di instradamento in ogni router, l'inoltro (forwarding) si può definire come l'azione eseguita dai router quando un pacchetto arriva ad una delle sue interfacce e lo deve trasferire sull'appropriato collegamento di uscita. La tabella d'inoltro (forwarding table) che viene normalmente utilizzata da un router per completare tale azione viene talvolta definita, in modo piuttosto ambiguo, routing table. Quando un router riceve un pacchetto da una delle reti a cui è collegato direttamente, deve inoltrare il pacchetto ad un'altra delle reti a cui è collegato (nell'instradamento unicast) o a più reti a cui è collegato (nel caso di instradamento multicast). Per prendere questa decisione il router utilizza alcune informazioni che si trovano nell'intestazione del pacchetto, ovvero l'indirizzo di destinazione o un'etichetta, per trovare la giusta interfaccia di output all'interno della tabella d'inoltro.

Esempio 6.1.2 – Tabella di inoltro

Il pacchetto con valore del campo di intestazione pari a 0110 giunge a un router che, tramite la propria tabella di inoltro, individua che l'interfaccia in uscita è la 2 a cui lo inoltra internamente.

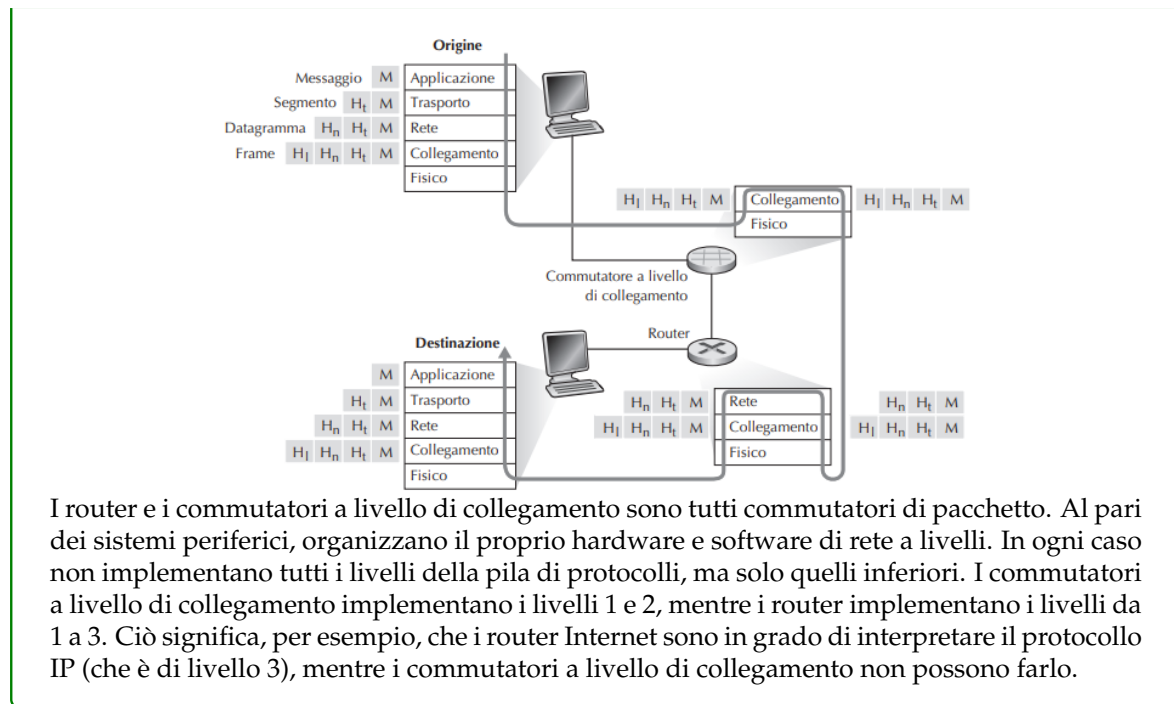


6.1.2 Packet switching

Dalla discussione su instradamento e inoltro deduciamo che a livello di rete avviene una qualche forma di switching (spostamento). Riserveremo il termine commutatore di pacchetto (packet switch o semplicemente switch) per indicare un generico dispositivo che si occupa del trasferimento dei pacchetti da un'interfaccia in ingresso a quella in uscita, in base al valore dei campi nell'intestazione del pacchetto. Alcuni commutatori di pacchetti, chiamati commutatori a livello di collegamento (link-layer switch), stabiliscono l'inoltro in relazione al valore del campo del frame a livello di collegamento (livello 2). Altri, chiamati router, prendono le decisioni di inoltro basandosi sul valore nel campo a livello di rete (livello 3). Un router infatti è un commutatore di rete che crea un collegamento tra una porta di input e una di output (o un insieme di porte di output).

Esempio 6.1.3 – Host, router e commutatori a livello di collegamento

La figura mostra il percorso seguito dai dati scendendo lungo la pila di protocolli del sistema mittente, risalendo e scendendo lungo le pile di protocolli dei commutatori e dei router che intervengono a livello di collegamento e, infine, risalendo la pila nel sistema ricevente.



Anche se nella comunicazione dei dati le tecniche di switching sono divise in due ampie categorie, il circuit switching (commutazione di circuito) e il packet switching (commutazione di pacchetto), solo quest'ultimo viene utilizzato a livello di rete in quanto l'unità dei dati, a questo livello, è il pacchetto, mentre il circuit switching viene usato soprattutto al livello fisico. A livello di rete un pacchetto proveniente dal livello superiore (un segmento o un datagramma utente) viene diviso in pacchetti (detti anche datagrammi) di dimensione gestibile ed ognuno di questi viene inviato tramite la rete. La sorgente invia i datagrammi uno alla volta: la destinazione li riceve uno per uno. Visto che durante il percorso è possibile che i datagrammi vengano frammentati in datagrammi più piccoli, la destinazione attende di aver ricevuto tutti i frammenti che fanno parte dello stesso datagramma prima di consegnare il contenuto al livello superiore. I dispositivi di interconnessione, in una rete basata su commutazione di pacchetto, dovranno anche determinare come instradare i datagrammi dalla sorgente alla destinazione finale. Oggi una rete basata su packet switching può utilizzare due approcci diversi per lo switching: l'approccio a datagramma e l'approccio a circuito virtuale.

Approccio a datagramma: servizio senza connessione

Con l'obiettivo della semplicità, Internet è stata progettata con un livello di rete in grado di fornire un servizio senza connessione, dove il protocollo di rete tratta ogni datagramma in modo del tutto indipendente ed i singoli datagrammi non hanno alcuna relazione con gli altri. L'idea alla base di questa progettazione è che il livello di rete deve essere solamente in grado di veicolare i datagrammi dalla sorgente alla destinazione. Secondo questo approccio i datagrammi che formano un messaggio (di livello applicazione) possono viaggiare tutti lungo lo stesso percorso verso la destinazione oppure no. Quando il livello di rete fornisce un servizio senza connessione, ogni datagramma che viaggia in Internet è un'entità indipendente, non c'è alcuna relazione tra i datagrammi che appartengono allo stesso messaggio. I commutatori in questo tipo di rete sono normalmente chiamati router.

Ogni datagramma viene instradato sulla base delle sole informazioni contenute nella sua intestazione: gli indirizzi sorgente e destinazione. L'indirizzo destinazione definisce dove il datagramma deve andare, l'indirizzo sorgente da dove viene. Il router in questo caso instrada il datagramma solo sulla base dell'indirizzo di destinazione. L'indirizzo sorgente può essere utilizzato per inviare un messaggio d'errore alla sorgente nel caso il datagramma venga scartato.

Approccio a circuiti virtuali: servizio orientato alla connessione

In un servizio orientato alla connessione (chiamato anche approccio a circuiti virtuali) c'è una relazione tra tutti i datagrammi che appartengono a un messaggio (di livello applicazione). Prima che tutti i datagrammi di un messaggio possano essere inviati, è necessario impostare una connessione virtuale

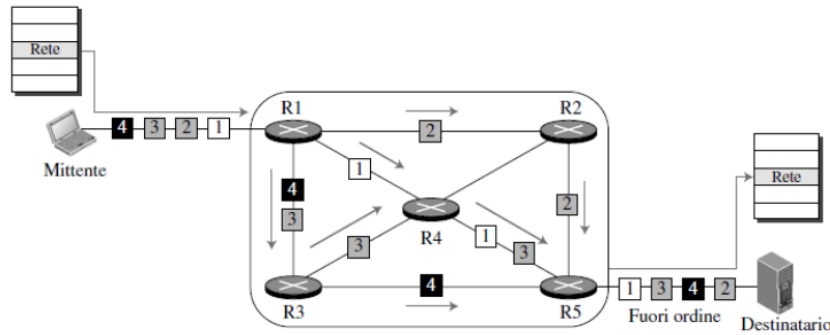


Figura 6.1: Una rete basata su packet switching senza connessione

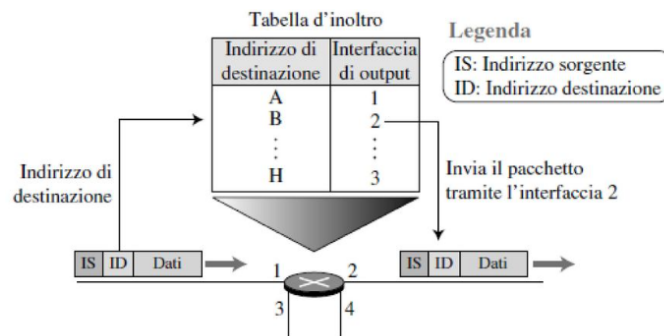


Figura 6.2: Processo d'inoltro di un router in una rete priva di connessione

per definire il percorso dei datagrammi. Dopo aver impostato la connessione, i datagrammi seguono tutti lo stesso percorso. In questo tipo di servizio, non solo il datagramma deve contenere gli indirizzi sorgente e destinazione, ma deve anche contenere un'etichetta di flusso (flow label), che identifichi il circuito virtuale che definisce il percorso virtuale che il datagramma deve seguire. Anche se può apparire che l'uso dell'etichetta renda inutili gli indirizzi sorgente e destinazione durante la fase di trasferimento dati, bene ricordare che nel caso di Internet il livello di rete continua ad utilizzare tali indirizzi. Una ragione di questa scelta è che parte del percorso del datagramma può dover utilizzare un servizio senza connessione dove gli indirizzi di sorgente e destinazione rimangono essenziali.

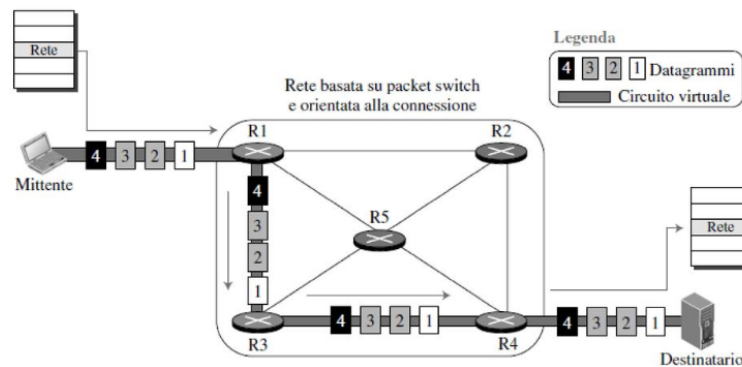


Figura 6.3: rete basata su circuiti virtuali

Ogni datagramma viene inoltrato sulla base dell'etichetta presente nella sua intestazione. Per procedere nel nostro esempio di Internet che utilizza un livello di rete basato su connessione supponiamo che il datagramma abbia già un'etichetta quando raggiunge il router. In questo caso la decisione d'inoltro si basa sul valore dell'etichetta, detta spesso virtual circuit identifier (identificatore del circuito virtuale).

Per implementare un servizio orientato alla connessione viene utilizzato un processo composto da tre fasi: setup (creazione del circuito virtuale), data transfer (trasferimento dei dati) e teardown (distruzione del circuito virtuale). Nella fase di setup gli indirizzi sorgente e destinazione del mittente-

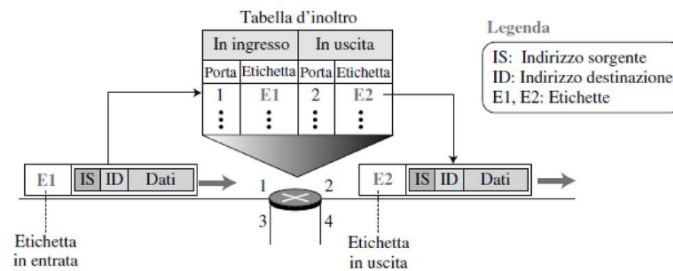


Figura 6.4: Inoltro di un router in una rete a circuito virtuale

te e del ricevente vengono utilizzati per configurare nei router le tabelle necessarie per l'impostazione dei circuiti virtuali. Nella fase di teardown la sorgente e la destinazione informano i router di cancellare tutte le informazioni relative al circuito che si sta eliminando. Il trasferimento dei dati avviene necessariamente tra queste due fasi.

6.1.3 Struttura di un router

Nella nostra discussione circa l'inoltro e l'instradamento abbiamo rappresentato il router come una scatola nera che accetta pacchetti in entrata da una delle porte di input (interfacce), usa una tabella d'inoltro per trovare la porta di output e da questa invia il pacchetto. La nostra attenzione sarà specificamente rivolta alla funzione di inoltro: ossia, alle architetture dei router per trasferire i pacchetti dai collegamenti in ingresso a quelli in uscita.

Componenti

Possiamo individuare quattro componenti principali: porte di input, porte di output, processore di routing e switching fabric.

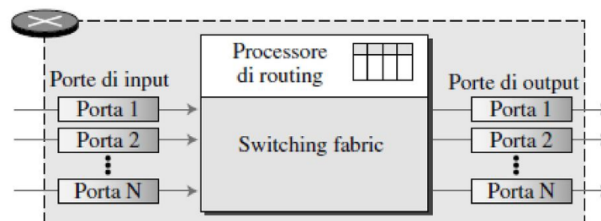


Figura 6.5: Componenti di un router

Porte di input

Una porta di input implementa le funzionalità del livello fisico e di collegamento del router. I bit vengono ricostruiti a partire dal segnale ricevuto. Il datagramma viene estratto dal frame che lo ha trasportato a livello collegamento, viene controllata la presenza di errori e viene quindi scartato se risulta danneggiato. Se il datagramma risulta integro, è pronto per essere processato dal livello di rete. In aggiunta a un processore del livello fisico e un processore del livello di collegamento, la porta di input ha dei buffer (ovvero delle code) per memorizzare i pacchetti in attesa di essere veicolati alla switching fabric.

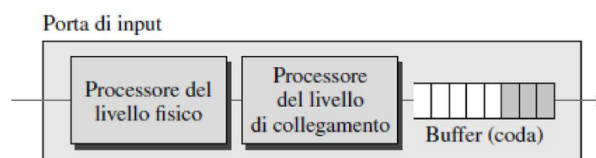


Figura 6.6: Porta di input di un router

Porte di output

Una porta di output svolge le stesse funzioni di quella di input, ma in ordine inverso. Per prima cosa i datagrammi in uscita vengono accodati, poi ogni datagramma viene incapsulato in un frame e alla fine per mezzo del livello fisico viene tradotto in un segnale da trasmettere.

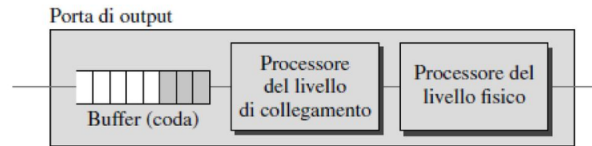


Figura 6.7: Porta di input di un router

Processore di routing

Il processore di routing implementa le funzionalità del livello di rete. L'indirizzo di destinazione viene usato per trovare l'indirizzo del salto (hop) successivo, cioè il prossimo apparato a cui inviare il pacchetto e il numero della porta di output da cui il datagramma deve essere spedito. A volte ci si riferisce a tale attività come "ricerca nella tabella" (table lookup) in quanto il processore di routing effettua una ricerca nella tabella d'inoltro. Nei router più recenti, questa funzione del processore di routing viene implementata nelle porte di input per semplificare e rendere più veloce il processo.

Switching fabric

Il compito più difficile all'interno di un router è spostare i datagrammi dalla coda di input a quella di output. La velocità con cui ciò viene fatto influisce sulla dimensione delle coda di input/output e sul ritardo complessivo nella consegna del datagramma. In passato, quando un router non era nulla più di un computer dedicato a questo scopo, la memoria del computer, o un bus, venivano usati come infrastruttura interna di comunicazione (la cosiddetta switching fabric). In pratica, la porta di input memorizzava il pacchetto nella memoria del calcolatore e quella di output invece prendeva il pacchetto dalla memoria. Oggi i router usano una varietà di switching fabric progettate per questo specifico scopo.

6.1.4 Elaborazione alle porte di ingresso e inoltro

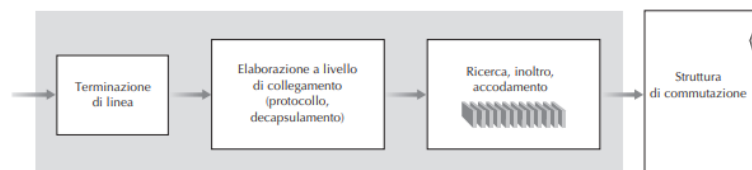


Figura 6.8: Elaborazione alle porte di ingresso

Come già detto, la funzione di terminazione della linea e l'elaborazione a livello di collegamento implementano rispettivamente il livello fisico e di collegamento associati a un singolo collegamento di ingresso al router. L'elaborazione effettuata alla porta di ingresso è centrale per la funzionalità del router: è qui che, utilizzando le informazioni della tabella di inoltro, viene determinata la porta di uscita a cui dirigere un pacchetto attraverso la struttura di commutazione. La tabella di inoltro viene elaborata e aggiornata dal processore di instradamento o ricevuta da un controller SDN remoto. Una sua copia conforme è di solito memorizzata su ciascuna porta di ingresso. La tabella di inoltro viene copiata sulla line card dal processore di instradamento tramite un bus separato. Essendoci copie locali della tabella di inoltro, la decisione relativa può essere presa dalle porte di ingresso, senza invocare il processore di instradamento centralizzato.

Esempio 6.1.4 – Tabella di inoltro

Consideriamo il caso “più semplice” in cui l’inoltro è basato sull’indirizzo di destinazione. Supponiamo che tutti gli indirizzi di destinazione siano a 32 bit (dimensione che è la lunghezza dell’indirizzo di destinazione nei datagrammi IPv4). Un’implementazione elementare della tabella di inoltro presenterebbe una riga per ogni possibile indirizzo di destinazione ma, dato che esistono più di 4 miliardi di possibili indirizzi, questa opzione non viene nemmeno presa in considerazione.

Supponiamo inoltre che il nostro router abbia quattro collegamenti, numerati da 0 a 3, e che i pacchetti debbano essere inoltrati verso le interfacce di collegamento come segue:

Intervallo degli indirizzi di destinazione		Interfaccia
da	11001000 00010111 00010000 00000000	0
a	11001000 00010111 00010111 11111111	
da	11001000 00010111 00011000 00000000	1
a	11001000 00010111 00011000 11111111	
da	11001000 00010111 00011001 00000000	2
a	11001000 00010111 00011111 11111111	
altrimenti		3

Chiaramente, in questo esempio, non è necessario avere 4 miliardi di righe nelle tabelle di inoltro dei router. Potremmo, per esempio, avere la seguente tabella, con appena quattro voci:

Corrispondenza di prefisso	Interfaccia
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
altrimenti	3

Con questa struttura il router confronta un prefisso dell’indirizzo di destinazione del pacchetto con una riga della tabella; se c’è corrispondenza, il router inoltra il pacchetto al collegamento associato. Per esempio, supponiamo che l’indirizzo di destinazione del pacchetto sia 11001000 00010111 00010110 10100001; dato che il prefisso a 21 bit di questo indirizzo rispecchia la prima riga nella tabella, il router inoltra il pacchetto all’interfaccia di collegamento 0. Se un prefisso non corrisponde alle prime tre righe, il router lo inoltra all’interfaccia 3.

Sebbene tutto questo sembri sufficientemente semplice, in realtà nasconde un accorgimento ingegnoso. Avrete notato che un indirizzo di destinazione può corrispondere a più di una riga. Per esempio, i primi 24 bit dell’indirizzo 11001000 00010111 00011000 10101010 corrispondono alla seconda riga della tabella e i primi 21 alla terza riga della tabella. Quando si verificano corrispondenze multiple, il router adotta la regola di corrispondenza a prefisso più lungo; in altre parole, viene determinata la corrispondenza più lunga all’interno della tabella e i pacchetti vengono inoltrati all’interfaccia di collegamento associata.

6.1.5 Struttura di commutazione

La struttura di commutazione (switching fabric) rappresenta il vero e proprio cuore dei router, attraverso il quale i pacchetti vengono commutati (ossia inoltrati) dalla porta di ingresso alla porta di uscita. La commutazione può essere ottenuta in vari modi.

Commutazione in memoria

I primi e più semplici router erano in genere calcolatori tradizionali, e la commutazione tra porte di ingresso e di uscita veniva effettuata sotto il controllo diretto della CPU (processore di instradamento). Le porte di ingresso e di uscita funzionavano come tradizionali dispositivi di I/O. Quando sopraggiungeva un pacchetto, la porta di ingresso ne segnalava l’arrivo tramite interrupt e quindi lo copiava nella memoria del processore di instradamento che procedeva a estrarre dall’indirizzo di destinazione. Quindi, individuava tramite la tabella di inoltro l’appropriata porta di uscita nel cui buffer copiava il pacchetto. Notiamo che se l’ampiezza di banda della memoria è tale da potervi scrivere o leggere B pacchetti al secondo, allora il throughput complessivo di inoltro (ossia

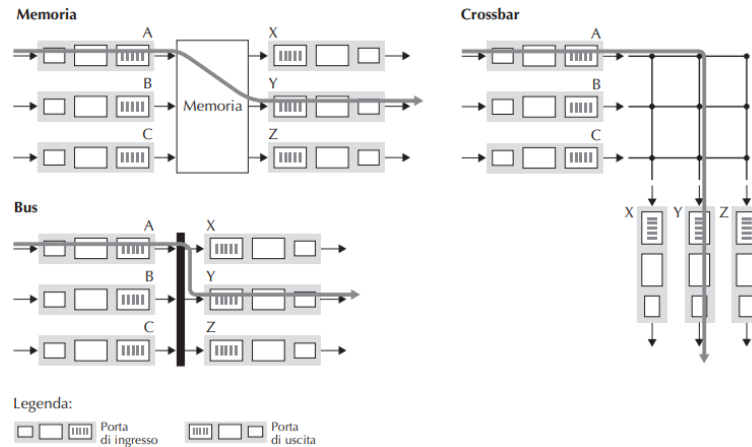


Figura 6.9: Tre tecniche di commutazione

la velocità massima alla quale i pacchetti vengono trasferiti dalle porte di ingresso a quelle di uscita) è necessariamente inferiore a $B/2$. Notiamo inoltre che due pacchetti non possono essere inoltrati contemporaneamente, anche se hanno differenti porte di destinazione, perché può essere effettuata solo un'operazione alla volta di scrittura/lettura in memoria tramite il bus di sistema.

Anche alcuni router attuali effettuano la commutazione in memoria. Esiste però una differenza sostanziale rispetto ai primi router: la ricerca dell'indirizzo di destinazione e la memorizzazione del pacchetto nella locazione di memoria opportuna vengono effettuate dai processori direttamente sulle linee card di ingresso. In alcuni casi, i router che effettuano la commutazione in memoria assomigliano molto a sistemi multiprocessore a memoria condivisa dove l'elaborazione su una linea card scrive direttamente i pacchetti nella memoria della porta di uscita appropriata.

Commutazione tramite bus

In questo approccio le porte di ingresso trasferiscono un pacchetto direttamente alle porte di uscita tramite un bus condiviso e senza intervento da parte del processore di instradamento. Questo viene tipicamente fatto aggiungendo un'etichetta interna di commutazione (intestazione) al pacchetto che indica la porta locale di output alla quale il pacchetto deve essere trasferito quando viene trasmesso sul bus. Il pacchetto viene ricevuto da tutte le porte di output, ma solo la porta corrispondente all'etichetta lo raccoglierà. L'etichetta viene quindi rimossa dalla porta di output in quanto usata solo all'interno della struttura di commutazione per attraversare il bus. Se più pacchetti arrivano contemporaneamente al router, ognuno su una porta di input diversa, tutti tranne uno dovranno aspettare, dato che sul bus si può trasferire soltanto un pacchetto alla volta. Poiché ciascun pacchetto deve attraversare il bus, la larghezza di banda della commutazione è limitata da quella del bus. Questo tipo di commutazione tramite bus è comunque spesso sufficiente per router che operano in reti di accesso e in quelle aziendali.

Commutazione attraverso rete di interconnessione

Un modo per superare la limitazione di banda di un singolo bus condiviso è l'utilizzo di una rete di interconnessione più sofisticata, quale quella usata in passato nelle architetture multiprocessore. Una matrice di commutazione (crossbar switch) è una rete di interconnessione che consiste di $2n$ bus che collegano n porte di ingresso a n porte di uscita. Ogni bus verticale interseca tutti i bus orizzontali a un punto di incrocio che può essere in qualsiasi momento aperto o chiuso dal controller della struttura di commutazione (la cui logica è parte stessa della struttura). Quando un pacchetto giunge a una porta di ingresso A e deve essere inoltrato alla porta Y, il controller chiude l'incrocio di A e Y e la porta A invia il pacchetto sul suo bus e solo il bus Y lo riceverà. Si noti che un pacchetto dalla porta B può essere inoltrato a X nello stesso tempo, perché i pacchetti A-Y e B-X usano bus di input e output diversi. Quindi, al contrario degli altri due approcci, la commutazione attraverso rete di interconnessione è in grado di inoltrare più pacchetti in parallelo. Una matrice di commutazione è non-blocking: un pacchetto in via di inoltrare verso una porta di uscita non viene bloccato a meno che esista un altro pacchetto in via di inoltrare sulla stessa porta di uscita. Tuttavia, se due pacchetti pro-

venienti da due diverse porte di input sono destinati alla stessa porta di output, uno dovrà accodarsi alla porta di input, perché solo un pacchetto alla volta può essere inoltrato su uno specifico bus.

6.1.6 Elaborazione alle porte di uscita

L'elaborazione alle porte di uscita prende i pacchetti dalla memoria della porta di uscita e li trasmette sul collegamento di uscita. Questo comprende selezionare e togliere dalla coda i pacchetti per la trasmissione e operare le necessarie funzioni a livello di collegamento e fisico.

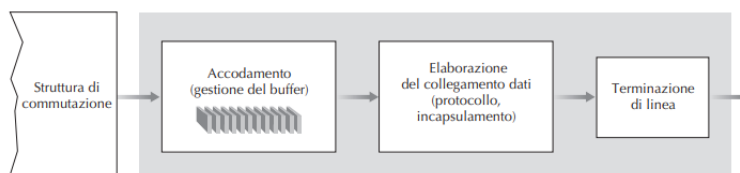


Figura 6.10: Elaborazione alle porte di uscita

6.1.7 Dove si verifica l'accodamento?

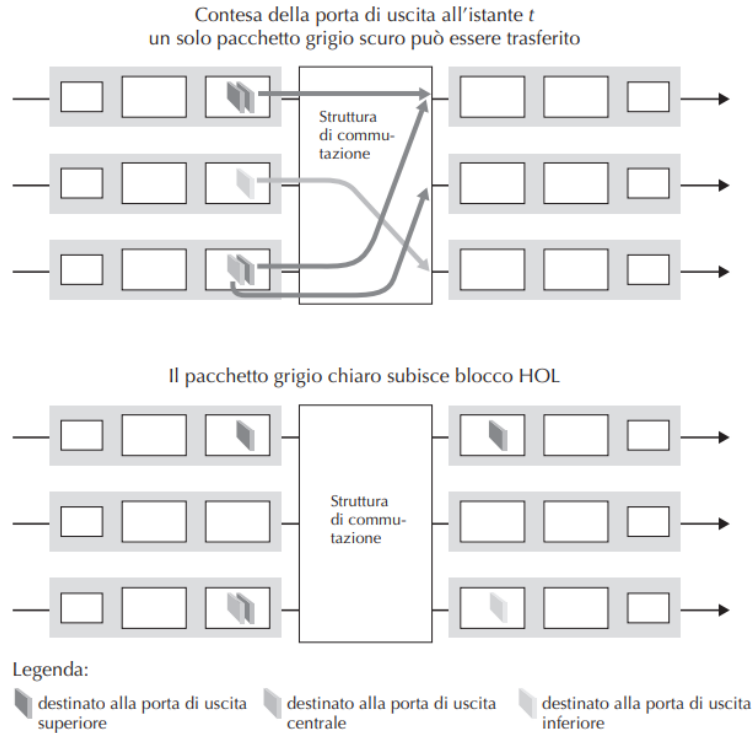
E' evidente che si possono formare code di pacchetti sia presso le porte di ingresso sia presso quelle di uscita. Il luogo e la lunghezza della coda (sia alle porte di input che di output) dipendono dalla quantità di traffico di rete, dalle velocità relative della struttura di commutazione e dalla linea. Quando queste code crescono, la memoria del router può esaurirsi e quindi può avvenire una perdita di pacchetti nel caso non vi sia memoria per immagazzinare quelli in arrivo.

Accodamento in ingresso

Ma che cosa avviene se la struttura di commutazione non è sufficientemente rapida (rispetto alle linee in ingresso) nel trasferire tutti i pacchetti in arrivo senza ritardo? In questo caso può verificarsi accodamento anche alle porte di ingresso. Per illustrare un'importante conseguenza di tale accodamento, consideriamo una struttura di commutazione facente uso di una rete di interconnessione, e supponiamo che (1) le velocità di tutti i collegamenti siano identiche, (2) un pacchetto possa essere trasferito da qualsiasi porta di ingresso a una data porta di uscita nello stesso intervallo di tempo richiesto perché un pacchetto sia ricevuto su un collegamento di ingresso, e (3) che i pacchetti vengano trasferiti da una coda di ingresso alla relativa coda di uscita desiderata con procedura FCFS (first-come-first-served, primo arrivato primo servito). E' possibile trasferire più pacchetti in parallelo quando le relative porte di uscita sono differenti. Tuttavia, se due pacchetti in testa a due code di ingresso sono destinati alla stessa coda di uscita, allora uno dei pacchetti sarà bloccato e dovrà attendere: istante per istante, la struttura di commutazione può trasferire solo un pacchetto a una certa porta di uscita.

Esempio 6.1.5 – HOL: Head Of the Line blocking

La figura mostra un esempio in cui due pacchetti (contraddistinti in grigio scuro) in testa alle rispettive code di ingresso sono destinati alla stessa porta di uscita, in alto a destra.



Supponiamo che la struttura di commutazione scelga di trasferire il primo pacchetto della coda in alto a sinistra. In questo caso, il pacchetto nella coda in basso a sinistra deve attendere, così come quello grigio più chiaro, che si trova dietro di lui, anche se non si verifica contesa per la porta di uscita centrale (che rappresenta la destinazione del pacchetto di colore chiaro). Questo fenomeno è noto come blocco in testa alla coda (HOL, head-of-the-line blocking): un pacchetto nella coda di ingresso deve attendere il trasferimento attraverso la struttura (anche se la propria porta di destinazione è libera) in quanto risulta bloccato da un altro pacchetto che lo precede.

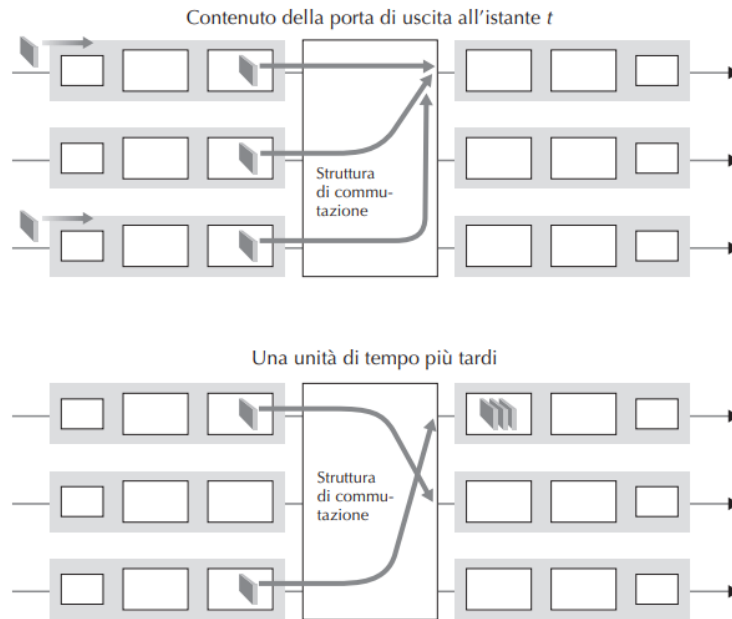
Accodamento in uscita

Dato che la porta di uscita può trasmettere un solo pacchetto in un intervallo prestabilito (tempo di trasmissione del pacchetto), gli N pacchetti in arrivo dovranno mettersi in coda per la trasmissione sul collegamento in uscita. Di conseguenza, è possibile che giungano altri N pacchetti nel periodo necessario per trasmettere solo uno degli N pacchetti già accodati, e così via. Quindi è possibile che si formino code di pacchetti alle porte di uscita e il numero di pacchetti in coda può continuare a crescere fino a esaurire lo spazio di memoria sulla porta di uscita.

In assenza di sufficiente memoria per inserire nel buffer il nuovo pacchetto in ingresso, occorrerà stabilire se scartarlo (politica nota come drop-tail, eliminazione in coda) o se rimuoverne uno o più, fra quelli già in coda, per far posto al nuovo arrivato. In alcuni casi può risultare vantaggioso eliminare un pacchetto (o marcarne l'intestazione) prima che il buffer sia pieno, al fine di fornire al mittente un segnale di congestione.

Esempio 6.1.6 – Accodamento alla porta di uscita

All'istante t , a ciascuna delle porte di ingresso giunge un pacchetto destinato alla porta in uscita superiore. I tre pacchetti originali sono stati trasferiti sulla porta in uscita e sono accodati in attesa di trasmissione.



Nell'unità di tempo successiva verrà trasmesso uno di questi tre pacchetti. Nel nostro esempio, due nuovi pacchetti sono giunti al commutatore; uno di questi pacchetti è destinato alla porta di uscita superiore. Quando vi sono più pacchetti accodati sulle porte di uscita, uno schedulatore di pacchetti (packet scheduler) deve stabilire in quale ordine trasmetterli.

Capacità dei buffer

Dato che i buffer nei router sono necessari per assorbire le fluttuazioni del traffico, sorge la domanda su quanto debba essere la loro capacità. Per molti anni la regola empirica usata per dimensionare i buffer era che la quantità di memoria riservata al buffer (B) dovesse essere uguale a un RTT medio (per esempio $250ms$) per la capacità del collegamento (C). Questo risultato si basa sull'analisi delle dinamiche di accodamento di un numero relativamente piccolo di flussi TCP. Quindi, un collegamento a $10Gbps$ con un RTT di $250ms$ avrebbe bisogno di un buffer pari a $B = RTT * C = 2,5Gbit$. Recenti studi teorici e sperimentali, tuttavia, suggeriscono che quando c'è un gran numero di flussi TCP (N) che passano attraverso un collegamento, la quantità di buffer necessaria sia $B = \frac{RTT * C}{\sqrt{N}}$. Con un gran numero di flussi, che tipicamente attraversano i grossi collegamenti dei router di dorsale, il valore di N può essere grande e la diminuzione nella quantità di buffer necessaria diventa abbastanza significativa.

6.2 Protocolli di livello rete

Nel corso del tempo i protocolli di livello rete hanno avuto molte versioni differenti.

Nella versione 4 il livello di rete può essere visto come formato da un protocollo principale e da tre protocolli ausiliari. Il protocollo principale, l'Internet Protocol versione 4 (IPv4) è responsabile della suddivisione in pacchetti, dell'inoltro (forwarding) e della consegna (delivery) dei datagrammi a livello di rete. L'Internet Control Message Protocol versione 4 (ICMPv4) aiuta l'IPv4 a gestire alcuni errori che possono avvenire nella consegna a livello di rete. L'Internet Group Management Protocol (IGMP) viene utilizzato per supportare l'IPv4 nella gestione del multicasting. Infine, l'Address Resolution Protocol (ARP) viene usato per far interagire il livello di rete e quello di collegamento. Più in dettaglio, ARP ha il compito specifico di permettere l'associazione tra indirizzi di livello di rete ed indirizzi di livello di collegamento.

L'IPv4 è un protocollo inaffidabile e senza connessione, basato su datagrammi. IPv4 offre un servizio di consegna di tipo best-effort, ovvero il protocollo fa del suo meglio per consegnare i dati che sono stati spediti ma non offre alcuna garanzia. Con il termine best-effort si intende che i datagrammi IPv4 possono essere danneggiati, persi, arrivare fuori ordine o in ritardo e possono anche generare congestione nella rete. Se è importante l'affidabilità allora è necessario associare ad IPv4 un

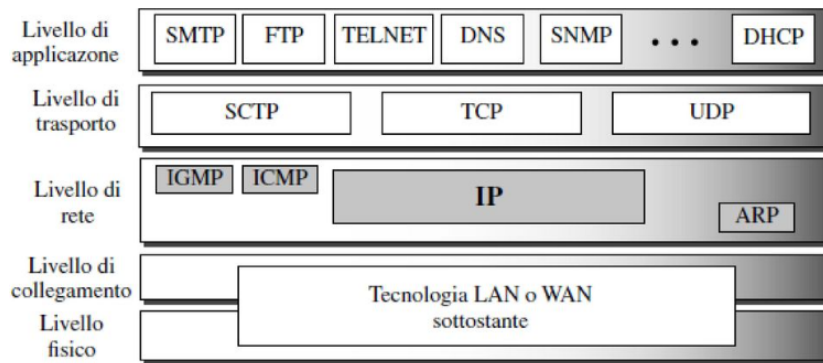


Figura 6.11: Struttura logica dello stack TCP/IP

protocollo di livello trasporto che sia in grado di garantirla aggiungendo alcune funzionalità, come ad esempio il TCP. Un esempio piuttosto semplice di servizio best-effort è l'ufficio postale: fa del suo meglio per consegnare la posta ordinaria ma non sempre ci riesce. Se una lettera non raccomandata viene persa, tocca al mittente (o al potenziale destinatario) scoprire la perdita e risolvere il problema. L'ufficio postale in sé non tiene traccia di ogni lettera e non può notificare al mittente la perdita o il danneggiamento.

6.2.1 Formato di datagrammi IPv4

I pacchetti usati dal protocollo IP vengono definiti datagrammi IP. Un datagramma è un pacchetto di lunghezza variabile composto da due parti: un'intestazione (header) e un campo dati (payload). L'intestazione è lunga da 20 a 60 byte e contiene le informazioni essenziali per il routing e la consegna dei datagrammi. Per comodità di lettura normalmente si rappresenta l'intestazione TCP/IP sotto forma di righe di 4 byte (32 bit) ciascuna.

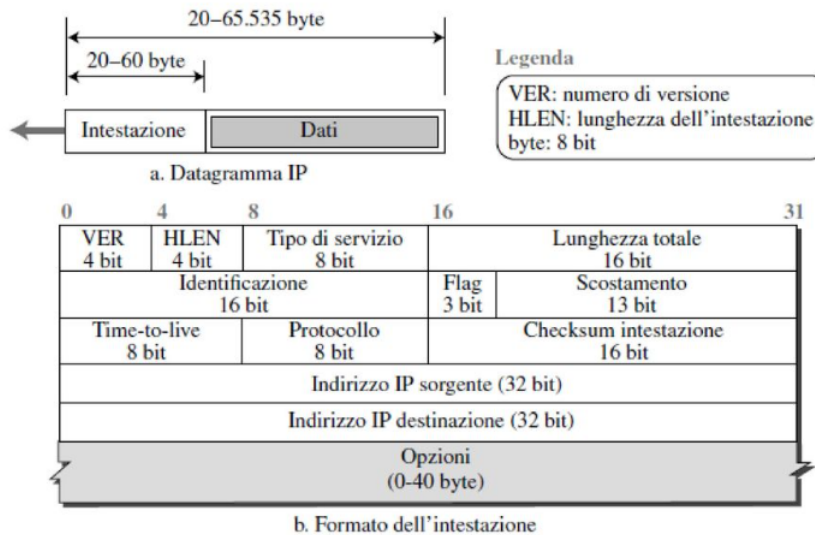


Figura 6.12: Datagramma IP

- *Numero versione.* Questo campo (VER), formato da 4 bit, definisce la versione del protocollo IP. Visto che si sta trattando IPv4 ovviamente il valore sarà 4.
- *Lunghezza dell'intestazione.* Il campo lunghezza intestazione (HLEN), formato da 4 bit, definisce la lunghezza totale dell'intestazione del datagramma in parole formate da 4 byte. Il datagramma IPv4 ha un'intestazione di lunghezza variabile. Quando un dispositivo riceve un datagramma deve sapere dove finisce l'intestazione e dove iniziano i dati incapsulati nel datagramma. La lunghezza dell'intestazione, come numero di byte, deve essere rappresentata facendo uso

dei soli 4 bit che formano questo campo. Per questo motivo, si ricorre ad una rappresentazione sotto forma di parole da 4 byte. Questo significa che la lunghezza totale dell'intestazione viene divisa per 4 e il valore inserito nel campo. Il ricevente ovviamente deve moltiplicare il valore di questo campo per 4 per trovare l'effettiva lunghezza totale.

- *Tipo servizio.* Secondo la progettazione originaria dell'intestazione IP, questo campo doveva specificare il tipo di servizio (Type Of Service, TOS) ovvero il modo in cui il datagramma doveva essere gestito dai router lungo il percorso. Alla fine degli anni 90, l'IETF ha ridefinito il campo per fornire i così detti servizi differenziati (Differentiated Services, DiffServ).
- *Lunghezza totale.* Questo campo, da 16 bit, definisce la lunghezza totale (intestazione più dati), espressa in byte, del datagramma IP. Un campo di 16 bit può rappresentare una lunghezza massima di 65.535 byte (nel caso in cui tutti i bit siano impostati a 1). Tuttavia la dimensione dei datagrammi normalmente è molto inferiore a questo massimo. Questo campo è utile al dispositivo ricevente per determinare se il pacchetto in ricezione è arrivato completamente.
- *Identificazione, flag e scostamento di frammentazione (offset).* Questi tre campi riguardano la frammentazione dei datagrammi IP che avviene quando la loro dimensione è maggiore rispetto a quella che la tecnologia di livello collegamento sottostante è in grado di trasportare.
- *Time-to-live (TTL, tempo di vita).* A causa di alcuni malfunzionamenti dei protocolli di instradamento, può accadere che un datagramma circoli su Internet, visitando alcune reti più di una volta, ma senza raggiungere la destinazione finale. Ovviamente questo porta alla creazione di traffico del tutto inutile. Il campo TTL viene usato per controllare il numero massimo di salti (hop), cioè il numero di router visitati dal datagramma. Quando un host sorgente invia il datagramma, esso memorizza un valore iniziale in questo campo. Questo valore è approssimativamente pari a due volte il numero massimo di salti tra due host qualsiasi presenti su Internet. Ogni router che elabora il datagramma decrementa di una unità questo numero. Se il TTL, dopo essere stato diminuito, giunge a zero allora il router scarta il datagramma.
- *Protocollo.* In TCP/IP, la sezione relativa ai dati di un pacchetto, chiamata payload, incapsula l'intero pacchetto di un altro protocollo (solitamente di livello superiore). Un datagramma, per esempio, può portare un pacchetto che appartiene a un protocollo di livello trasporto come UDP (quindi un datagramma utente) o TCP (quindi un segmento). Un datagramma può anche portare un pacchetto di un altro protocollo che usa direttamente il servizio offerto da IP, come ad esempio alcuni protocolli di instradamento o alcuni protocolli ausiliari. Gli enti di gestione Internet hanno assegnato ad ogni protocollo che fa uso del servizio IP un numero univoco composto da 8 bit, numero che viene di volta in volta inserito nel campo protocollo all'interno dell'intestazione. Quando il payload viene incapsulato in un datagramma nella sorgente, il numero di protocollo corrispondente viene inserito in questo campo; quando il datagramma arriva a destinazione, il valore di questo campo permette di definire a quale protocollo va consegnato il payload estratto dal datagramma. In altre parole, questo campo fornisce il multiplexing alla sorgente e il demultiplexing a destinazione.

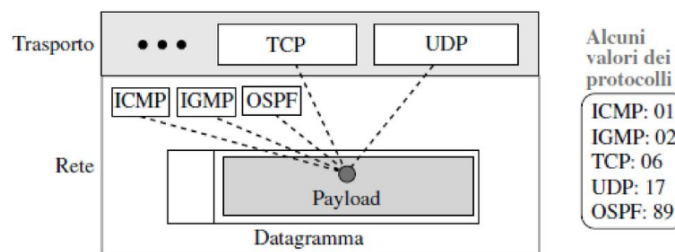


Figura 6.13: Multiplexing e demultiplexing utilizzando il valore del campo protocollo

- *Checksum dell'intestazione.* IP è un protocollo non affidabile, ad esempio non controlla se i dati incapsulati in un datagramma vengono danneggiati durante la trasmissione. IP lascia l'onere del controllo degli errori nei dati trasmessi al protocollo che è proprietario del payload, come ad esempio UDP o TCP. L'intestazione del datagramma, tuttavia, viene aggiunta dall'IP e il controllo degli errori è quindi di sua responsabilità. Gli errori nell'intestazione IP possono

essere un grosso problema. Se l'indirizzo IP di destinazione è danneggiato, il pacchetto sarà consegnato all'host sbagliato. Se il campo protocollo è danneggiato, il payload sarà consegnato al protocollo errato. Se i campi relativi alla frammentazione sono danneggiati, il datagramma non sarà riassembleto correttamente a destinazione, e così via. Per tali ragioni IP aggiunge un campo checksum che riguarda la sola intestazione ma non il payload. E' bene ricordare che, siccome il valore di alcuni campi, come il TTL, la frammentazione e le opzioni, possono cambiare da router a router, il checksum deve essere ricalcolato in ogni router. Il checksum, che viene inserito in un campo da 16 bit, è ottenuto come il complemento della somma degli altri campi dell'intestazione, il tutto utilizzando l'aritmetica complemento a uno.

- *Indirizzi sorgente e destinazione.* Questi campi, lunghi 32 bit, definiscono rispettivamente l'indirizzo IP della sorgente e quello della destinazione. L'host sorgente dovrebbe conoscere a priori il suo indirizzo IP. L'indirizzo IP di destinazione è invece noto al protocollo che utilizza il servizio offerto da IP oppure viene ottenuto per mezzo del DNS. Da notare che il valore di questi campi deve rimanere immutato durante tutto il tragitto del datagramma IP dall'host sorgente a quello di destinazione.
- *Opzioni.* L'intestazione di un datagramma può avere fino a 40 byte di opzioni. Le opzioni possono essere usate per il test o il debug della rete. Anche se le opzioni non sono una parte necessaria dell'intestazione IP, è richiesto che tutte le implementazioni di IP siano in grado di gestirle, quando presenti.
- *Payload (dati).* I dati trasportati ovviamente sono la ragione principale per la creazione del datagramma. Il payload è il pacchetto che deriva dagli altri protocolli che usano il servizio dell'IP, siano questi di livello superiore (per esempio, TCP e UDP) o di pari livello (per esempio, ICMP) nello stack TCP/IP. Equiparando un datagramma ad un pacchetto postale, il payload è il contenuto del pacchetto, mentre l'intestazione è solo l'informazione scritta sul pacco.

6.2.2 Frammentazione

Per giungere alla destinazione, un datagramma IP può dover viaggiare attraverso varie reti, ognuna con caratteristiche differenti. Ogni router toglie il datagramma dal frame di livello collegamento che ha ricevuto, lo elabora e poi lo incapsula in un nuovo frame. Il formato e la dimensione del frame ricevuto dipendono dalla tecnologia fisica e da quella di livello di collegamento utilizzati per trasportare il frame. Viceversa, il formato e le dimensioni del frame inviato dipendono dalla tecnologia fisica e da quella di livello di collegamento della rete su cui il frame verrà inviato. Per esempio se un router collega una LAN ad una WAN, riceverà un frame in formato LAN e invierà un nuovo frame in formato WAN.

Maximum Transfer Unit (MTU)

Ogni protocollo di livello collegamento ha il proprio formato di frame. Una delle caratteristiche di ciascun formato è la dimensione massima del payload che può essere incapsulato nel frame. In altre parole, quando un datagramma è racchiuso in un frame, la dimensione totale del datagramma deve essere inferiore rispetto a questa misura massima, che è definita dalle restrizioni imposte dall'hardware e dal software usati nella rete. Il valore dell'MTU differisce tra i diversi protocolli di livello collegamento e fisico. Per esempio il valore per una LAN normalmente è di 1.500 byte, ma per una WAN può essere maggiore o inferiore.

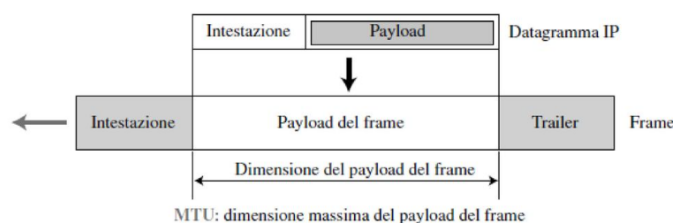


Figura 6.14: Maximum Transfer Unit (MTU)

Per rendere il protocollo IP indipendente dai livelli sottostanti, i progettisti hanno deciso di fissare la lunghezza massima del datagramma IP a 65.535 byte. Questo rende la trasmissione efficiente nel caso si usi un protocollo di livello collegamento con una MTU di tali dimensioni. Tuttavia, altre tecnologie di rete richiedono che il datagramma sia suddiviso in parti più piccole per poter essere trasportato. Questo procedimento viene chiamato frammentazione.

Quando un datagramma IP viene frammentato ogni frammento ha la propria intestazione, nella maggior parte dei casi i campi dell'intestazione sono uguali rispetto al datagramma originario ma alcuni devono essere cambiati. Un datagramma frammentato può essere a sua volta frammentato se incontra una rete con una MTU ancora più piccola. In altre parole un datagramma può essere frammentato più volte prima di raggiungere la sua destinazione finale.

Un datagramma può essere frammentato dall'host sorgente o da qualsiasi altro router lungo il percorso verso la destinazione. Il riassemblaggio del datagramma, tuttavia, viene fatto solo dall'host di destinazione, in quanto ogni frammento diventa un datagramma indipendente.

I diversi frammenti del datagramma originario possono viaggiare lungo percorsi differenti: questi percorsi non sono prevedibili o controllabili a priori ma, alla fine, tutti i frammenti che appartengono allo stesso datagramma dovrebbero arrivare all'host destinazione. Quindi è logico che il riassemblaggio avvenga solamente alla destinazione finale. Oltretutto, il riassemblaggio dei pacchetti durante il percorso provocherebbe una significativa perdita di efficienza.

Quando parliamo di frammentazione non intendiamo solamente che il payload del datagramma IP viene frammentato. Infatti, la maggior parte dell'intestazione, ad eccezione di alcune opzioni, deve essere copiata nell'intestazione di tutti i frammenti. L'host o il router che frammenta un datagramma IP deve cambiare i valori di tre campi dell'intestazione: i flag, lo scostamento di frammentazione e la lunghezza totale. Il resto dei campi dell'intestazione deve invece essere solamente copiato. Naturalmente il valore del checksum va ricalcolato ad ogni hop indipendentemente dalla frammentazione.

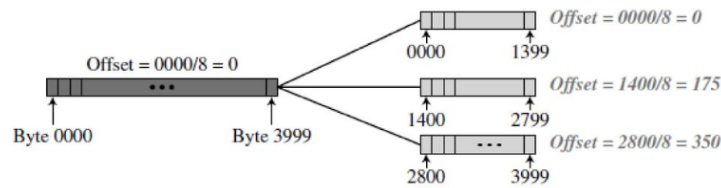
Campi relativi alla frammentazione

Abbiamo detto in precedenza che tre campi nell'intestazione del datagramma IP si riferiscono alla frammentazione: identificazione, flag e scostamento di frammentazione (offset).

- Il campo identificazione, da 16 bit, identifica un datagramma che ha origine da un host sorgente. La combinazione tra indirizzo IP sorgente e campo identificazione deve definire in modo unico un datagramma quando lascia l'host sorgente. Per garantire l'unicità, il protocollo IP utilizza un contatore per etichettare i datagrammi. Il contatore viene inizializzato a un numero positivo. Quando il protocollo IP invia un datagramma, copia il valore corrente del contatore nel campo identificazione ed incrementa il contatore di uno. Quando un datagramma viene frammentato il valore nel campo identificazione viene copiato in tutti i frammenti ottenuti. In altre parole tutti i frammenti hanno lo stesso numero di identificazione, che è anche lo stesso del datagramma originale. Il numero d'identificazione aiuta la destinazione a riassemblare il datagramma. Esso sa che tutti i frammenti con lo stesso valore d'identificazione vanno assemblati in un unico datagramma IP.
- Il campo flag, da 3 bit, in realtà definisce tre flag distinti, ognuno composto da un singolo bit. Il bit all'estrema sinistra è riservato (non viene attualmente usato). Il secondo bit (bit D) è definito do not fragment (non frammentare). Se il suo valore è 1 allora il dispositivo non deve frammentare il datagramma. Se non è in grado di trasmettere il datagramma attraverso nessuna delle reti fisiche allora lo scarta ed invia un messaggio d'errore ICMP all'host sorgente. Se il suo valore è 0, se necessario, il datagramma può essere frammentato. Il terzo bit (bit M) detto more fragments, nel caso sia impostato a 1 indica che questo datagramma non è l'ultimo frammento della serie e che ci sono altri frammenti dopo di lui. In altre parole non si tratta dell'ultimo frammento di un datagramma originario. Se il suo valore è 0 significa che è l'ultimo (o l'unico) frammento.
- Il campo scostamento di frammentazione (offset), da 13 bit, mostra la posizione relativa di questo frammento rispetto all'intero datagramma. E' l'offset dei dati nel datagramma originario misurato in unità da 8 byte.

Esempio 6.2.1 – Frammentazione: offset

La Figura mostra il payload di datagramma con una dimensione di 4000 byte suddiviso in tre frammenti.

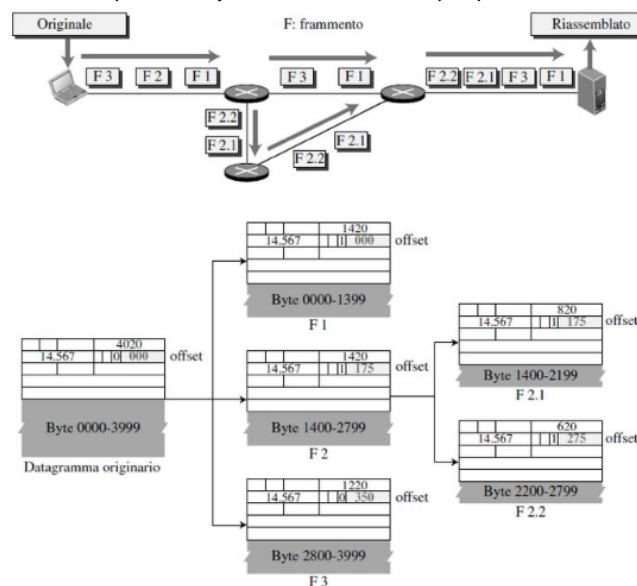


I dati nel datagramma originario sono numerati da 0 a 3999. Il primo frammento porta i byte da 0 a 1399. L'offset per questo datagramma è $0/8 = 0$. Il secondo frammento porta i byte da 1400 a 2799, il valore offset per questo frammento è $1400/8 = 175$. Infine il terzo frammento porta i byte da 2800 a 3999. Il valore offset per questo frammento è $2800/8 = 350$.

E' necessario ricordare che il valore dell'offset viene misurato in unità da 8 byte. Questo avviene perché la lunghezza del campo offset è di soli 13 bit e non può rappresentare una sequenza di byte maggiore di 8191. Ciò costringe gli host o i router che frammentano i datagrammi a scegliere la dimensione dei frammenti in modo che sia divisibile per 8, ad esclusione ovviamente dell'ultimo frammento originato dalla suddivisione di un datagramma.

Esempio 6.2.2 – Frammentazione

La figura mostra una visione espansa dei frammenti dell'esempio precedente.



Il pacchetto originario viene generato da un host client, i frammenti vengono riassemblati nel server. Il valore del campo identificazione è lo stesso in tutti i frammenti, e il valore del flag more fragments è impostato a 1 in tutti i frammenti ad esclusione dell'ultimo. Inoltre nella figura viene mostrato il valore del campo offset per ogni frammento. Da notare che anche se i frammenti sono arrivati fuori ordine alla destinazione, possono essere riassemblati correttamente.

La figura mostra inoltre cosa avviene se un frammento viene anch'esso frammentato. In questo caso il valore del campo offset si riferisce sempre al datagramma originario. Per esempio nella figura il secondo frammento è a sua volta frammentato in due frammenti da 800 e da 600 byte, ma l'offset mostra la posizione relativa dei frammenti rispetto ai dati originali. E' ovvio che anche se ogni frammento segue un percorso differente ed arriva in disordine, l'host di destinazione finale può riassemblare il datagramma originario dai frammenti ricevuti (se nessuno di essi è andato perso), utilizzando la seguente strategia:

1. Il primo frammento ha un valore del campo offset pari a 0.

2. Dividere per 8 la lunghezza del primo frammento. Il secondo frammento ha un valore offset pari a tale risultato.
3. Dividere per 8 la somma della lunghezza del primo e del secondo frammento. Il terzo frammento ha un valore offset pari a tale risultato.
4. Continuare il procedimento. L'ultimo frammento ha il suo bit M impostato a 0.

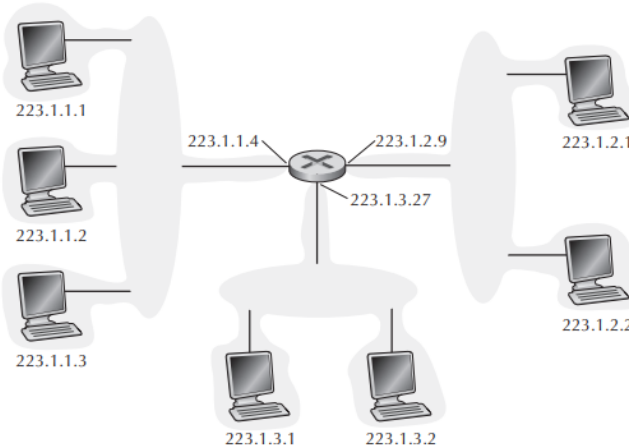
6.2.3 Indirizzi IPv4

Generalmente un host ha un solo collegamento con la rete; quando l'implementazione di IP dell'host vuole inviare un datagramma, lo fa su tale collegamento. Il confine tra host e collegamento fisico viene detto interfaccia. Invece, dato che il compito di un router è ricevere datagrammi da un collegamento e inoltrarli su un altro, questo deve necessariamente essere connesso ad almeno due collegamenti. Anche il confine tra un router e i suoi collegamenti è chiamato interfaccia. Il router presenta più interfacce, una su ciascuno dei suoi collegamenti. Dato che host e router sono in grado di inviare e ricevere datagrammi, IP richiede che tutte le interfacce abbiano un proprio indirizzo IP. Pertanto, l'indirizzo IP è tecnicamente associato a un'interfaccia, anziché all'host o al router che la contiene.

Gli indirizzi IP sono lunghi 32 bit (4 byte). Tali indirizzi sono solitamente scritti nella cosiddetta notazione decimale puntata (dotted-decimal notation), in cui ciascun byte dell'indirizzo viene indicato in forma decimale ed è separato con un punto dagli altri byte dell'indirizzo. Ogni interfaccia di host e router di Internet ha un indirizzo IP globalmente univoco. Una parte dell'indirizzo di un'interfaccia è determinata dalla sottorete cui è collegata.

Esempio 6.2.3 – Indirizzi delle interfacce e sottoreti

La figura mostra un router (con tre interfacce) che connette sette host.



I tre a sinistra e l'interfaccia del router cui sono connessi hanno un indirizzo IP della forma 223.1.1.xxx: ossia, i 24 bit più a sinistra nell'indirizzo IP sono identici. Le quattro interfacce sono interconnesse da una rete che non contiene router. Per adesso rappresentiamo la rete priva di router che connette questi host come una nuvola.

Per IP, questa rete che interconnette tre interfacce di host e l'interfaccia di un router forma una sottorete. IP assegna a questa sottorete l'indirizzo 223.1.1.0/24, dove la notazione /24, detta anche maschera di sottorete (subnet mask), indica che i 24 bit più a sinistra dell'indirizzo definiscono l'indirizzo della sottorete. Di conseguenza, la sottorete 223.1.1.0/24 consiste di tre interfacce di host (223.1.1.1, 223.1.1.2, 223.1.1.3) e di un'interfaccia di router (223.1.1.4). Ogni altro host connesso alla sottorete 223.1.1.0/24 deve avere un indirizzo della forma 223.1.1.xxx.

Spazio degli indirizzi (address space)

Un protocollo come l'IPv4 che utilizza degli indirizzi ha bisogno di definire uno spazio di indirizzi, cioè il numero totale di indirizzi utilizzati dal protocollo. Se un protocollo usa b bit per rappresentare un indirizzo, lo spazio degli indirizzi è pari a 2^b perché ogni bit può assumere solo due valori distinti (0 o 1). IPv4 usa indirizzi a 32 bit, il che significa che lo spazio degli indirizzi è 2^{32} o 4.294.967.296 (più di quattro miliardi). Se non ci fosse alcuna restrizione, oltre 4 miliardi di dispositivi potrebbero essere collegati a Internet.

Esistono tre notazioni comunemente usate per rappresentare gli indirizzi IPv4: la notazione binaria (base 2), la notazione decimale puntata (base 256) e quella esadecimale (base 16). Nella notazione binaria un indirizzo IPv4 viene rappresentato con 32 bit. Per facilitare la lettura dell'indirizzo normalmente vengono inseriti uno o più spazi tra ogni ottetto (8 bit). Spesso ci si riferisce a ciascun ottetto come ad un byte. Per rendere l'indirizzo IPv4 più compatto e facile da leggere normalmente viene scritto in forma decimale, con un punto che separa i byte. Questo formato viene definito notazione decimale puntata (dotted-decimal notation). Da notare che siccome ogni byte (ottetto) è formato da soli 8 bit, ogni numero nella notazione decimale puntata è compreso tra 0 e 255. A volte è possibile trovare un indirizzo IPv4 espresso in notazione esadecimale. In questo caso ogni cifra esadecimale equivale a quattro bit. Ciò significa che un indirizzo a 32 bit ha 8 cifre esadecimali. Tale notazione viene spesso utilizzata nella programmazione di rete.

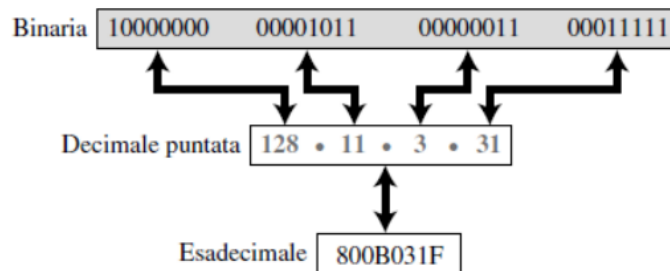


Figura 6.15: Tre diverse notazioni per l'indirizzamento IPv4

In ogni rete di comunicazione in cui è prevista la consegna di un messaggio da un mittente ad un destinatario il meccanismo di indirizzamento è gerarchico. Anche gli indirizzi IPv4 sono gerarchici, ma divisi in sole due parti. La prima parte dell'indirizzo, chiamata prefisso (prefix), identifica la rete, mentre la seconda parte dell'indirizzo, chiamata suffisso (suffix), identifica il nodo nella rete (o meglio il collegamento di un dispositivo a Internet).

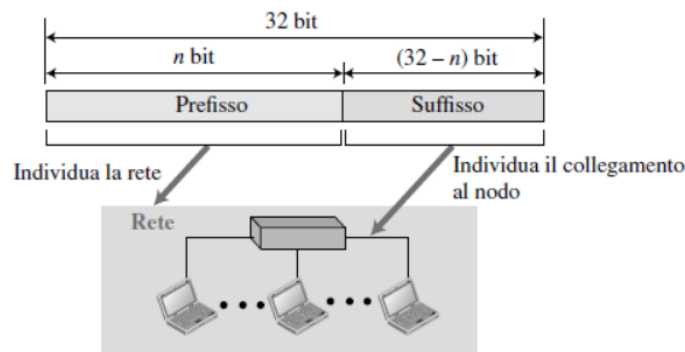


Figura 6.16: Gerarchia dell'indirizzamento

Un prefisso può avere una lunghezza fissa o variabile. Il sistema di identificazione delle reti in IPv4 è stato inizialmente progettato come prefisso a lunghezza fissa. Questo schema, che ormai è obsoleto, viene indicato come indirizzamento con classi (classful addressing). Il nuovo schema, chiamato indirizzamento senza classi (classless addressing), usa un prefisso di rete di lunghezza variabile.

Indirizzamento con classi

Come detto in precedenza, inizialmente l'indirizzamento IPv4 era stato progettato con un prefisso di lunghezza fissa, ma vista la necessità di supportare sia reti piccole che grandi, al posto di una sola lunghezza di prefisso ne erano state previste tre ($n = 8$, $n = 16$ e $n = 24$). L'intero spazio degli indirizzi era stato diviso in cinque classi (classe A, B, C, D ed E).

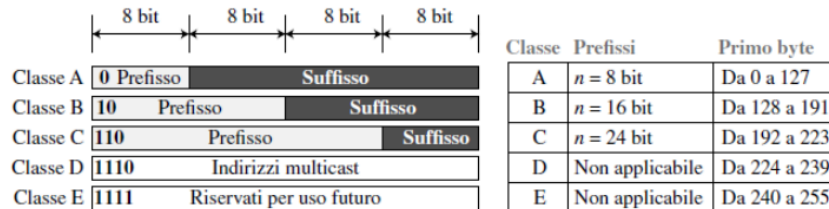


Figura 6.17: Indirizzamento con classi

Nella classe A la lunghezza della parte di rete è di 8 bit, ma siccome il primo bit, che è 0, identifica il tipo di classe, possiamo avere solo sette bit per l'identificazione delle reti. Ciò significa che ci sono solo $2^7 = 128$ reti al mondo che possono avere un indirizzo di classe A. Nella classe B la lunghezza della parte di rete è di 16 bit, ma siccome i primi due bit, che sono (10) 2 identificano la classe, possiamo avere solo 14 bit per identificare le reti. Ciò significa che ci sono solo $2^{14} = 16.384$ reti al mondo che possono avere un indirizzo di classe B.

Tutti gli indirizzi che iniziano con (110), appartengono alla classe C, nella quale la lunghezza della parte di rete è di 24 bit, ma siccome tre bit definiscono la classe, possiamo avere solo 21 bit per identificare le reti. Ciò significa che ci sono $2^{21} = 2.097$ reti al mondo che possono avere un indirizzo di classe C.

Gli indirizzi che iniziano con (1110), appartengono alla classe D, una classe che non è divisa in prefisso e suffisso ed è usata per gli indirizzi di tipo multicast. Infine, la classe E raccoglie tutti gli indirizzi che iniziano con (1111). Come per la classe D, la classe E non è divisa in prefisso e suffisso ed è riservata per uso futuro.

La ragione per cui l'indirizzamento con classi è diventato obsoleto è l'esaurimento degli indirizzi. Siccome gli indirizzi non sono stati assegnati in modo appropriato, Internet ha dovuto affrontare il problema dell'esaurimento degli indirizzi disponibili per società ed individui che volevano collegarsi in rete. Per comprendere meglio il problema pensiamo alla classe A. Tale classe può essere assegnata solo a 128 organizzazioni al mondo, ma a causa della struttura della classe A ognuna deve avere una singola rete con 16.777.216 nodi (computer collocati in questa unica rete). Siccome ci sono ben poche organizzazioni così grandi, la maggior parte degli indirizzi in questa classe è andata sprecata (gran parte degli indirizzi erano assegnati ma rimanevano inutilizzati). Gli indirizzi di classe B sono stati progettati per le organizzazioni di media dimensione, ma anche in questo caso molti degli indirizzi sono rimasti inutilizzati. Gli indirizzi di classe C hanno un difetto di progettazione completamente diverso. Il numero di indirizzi disponibile in ogni rete (256) era talmente piccolo che la maggior parte delle organizzazioni non lo trovavano adeguato per le proprie esigenze. Infine, gli indirizzi di classe E erano riservati per uso futuro, sprecando l'intera classe.

Indirizzamento senza classi

Con la crescita di Internet era chiaro che, come soluzione a lungo termine, sarebbe servito uno spazio degli indirizzi più grande. Tuttavia, questa soluzione richiede necessariamente che la lunghezza degli indirizzi IP venga aumentata, il che significa anche che il formato dei datagrammi IP deve essere modificato. Anche se la soluzione a lungo termine è già stata ideata ed implementata (si chiama IPv6), è stata studiata anche una soluzione a breve termine. In questa soluzione si continua ad usare lo stesso spazio degli indirizzi ma si cambia la loro distribuzione, attraverso un approccio più equo. La soluzione a breve termine è ancora basata sugli indirizzi IPv4 ma non fa più uso della divisione in classi. In altre parole la suddivisione in classi è stata rimossa per compensare la scarsità di indirizzi disponibili.

Gli enti di gestione di Internet hanno annunciato una nuova struttura chiamata indirizzamento senza classi. In questa forma di indirizzamento vengono utilizzati dei blocchi di lunghezza variabile che non appartengono a nessuna classe. Possiamo avere un blocco da 1, 2, 4, 128 indirizzi e così via.

Il prefisso in un indirizzo definisce il blocco (individua la rete): il suffisso definisce il nodo (individua il dispositivo). In teoria possiamo avere un blocco di $2^0, 2^1, 2^2 \dots$ indirizzi. Ad un'organizzazione si può quindi assegnare un blocco di indirizzi di dimensione adeguata alle sue esigenze.

A differenza dell'indirizzamento con classi, la lunghezza del prefisso nell'indirizzamento senza classi è variabile. Possiamo avere una lunghezza di prefisso che va da 0 a 32 bit. La dimensione della parte di rete è inversamente proporzionale alla lunghezza del prefisso. Un prefisso piccolo implica una rete più grande (con molti nodi al suo interno), un prefisso grande implica una rete più piccola (con pochi nodi).

Siccome la lunghezza del prefisso non è più parte integrante dell'indirizzo è necessario aggiungere questa informazione in modo separato. In questo caso la lunghezza del prefisso, n , viene aggiunta all'indirizzo separata da una barra (slash). La strategia di assegnazione degli indirizzi Internet è detta classless interdomain routing. CIDR generalizza la nozione di indirizzamento di sottorete. L'indirizzo IP viene diviso in due parti e mantiene la forma decimale puntata $a.b.c.d/x$, dove x indica il numero di bit nella prima parte dell'indirizzo. Gli x bit più a sinistra di un indirizzo della forma $a.b.c.d/x$ costituiscono la porzione di rete dell'indirizzo IP e sono spesso detti prefisso (di rete) dell'indirizzo.

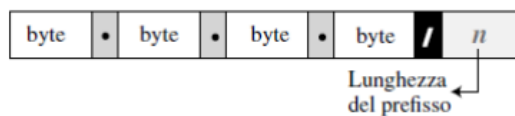


Figura 6.18: Numerazione slash (CIDR)

A un'organizzazione viene generalmente assegnato un blocco di indirizzi contigui con un prefisso comune per tutti gli indirizzi IP dei dispositivi che si trovano al suo interno. I rimanenti $32 - x$ bit di un indirizzo possono essere usati per distinguere i dispositivi interni dell'organizzazione, che hanno tutti lo stesso prefisso di rete.

Dato un indirizzo IP appartenente ad un blocco di indirizzi, normalmente vogliamo ricavare tre informazioni riguardo il suo blocco di appartenenza: il numero di indirizzi che contiene, il primo indirizzo del blocco e l'ultimo. Siccome la lunghezza del prefisso, n , è nota, possiamo facilmente ottenere queste tre informazioni.

1. Il numero di indirizzi nel blocco è dato da $N = 2^{32-n}$.
2. Per trovare il primo indirizzo, teniamo invariati i primi n bit partendo da sinistra e impostiamo a 0 tutti i bit restanti a destra (sono $32 - n$).
3. Per trovare l'ultimo indirizzo, teniamo invariati i primi n bit partendo da sinistra e impostiamo a 1 tutti i bit restanti a destra (anche in questo caso sono $32 - n$).

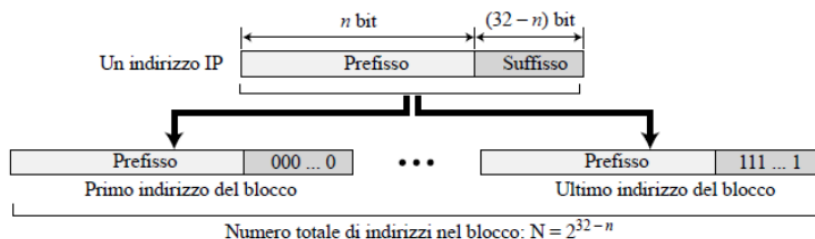


Figura 6.19: Estrazione delle informazioni nell'indirizzamento senza classi

Un altro modo per trovare il primo e l'ultimo indirizzo del blocco è usare la maschera dell'indirizzo (address mask), cioè un numero composto da 32 bit in cui i primi n bit a sinistra sono impostati a 1 e il resto dei bit ($32 - n$) sono impostati a 0. Il motivo principale per utilizzare una notazione di questo tipo è che può essere usata da un programma per calcolare in modo efficiente le informazioni di un blocco, usando solamente tre semplici operatori sui bit: NOT, AND e OR.

1. Il numero degli indirizzi nel blocco è $N = \text{NOT}(\text{maschera}) + 1$.
2. Il primo indirizzo nel blocco = (qualsiasi indirizzo del blocco) AND (maschera)

3. L'ultimo indirizzo nel blocco = (qualsiasi indirizzo nel blocco) OR (NOT (maschera)).

Indirizzo di rete (network address)

Il primo indirizzo, detto indirizzo di rete (network address), è particolarmente importante in quanto è usato nell'instradamento dei datagrammi verso la rete di destinazione. Per il momento supponiamo che Internet sia composta da m reti e da un router con m interfacce. Quando un datagramma giunge al router da un qualsiasi host sorgente, il router deve sapere a quale rete va inviato il datagramma e quindi da quale interfaccia va inviato. Quando il datagramma arriva alla rete di destinazione, raggiungerà l'host di destinazione utilizzando un'altra strategia. Dopo che il network address è stato trovato, il router consulta la sua tabella d'inoltro per trovare l'interfaccia corrispondente dalla quale inviare il datagramma. L'indirizzo di rete in realtà è lo strumento per l'identificazione delle reti: ogni rete è identificata per mezzo del suo network address.

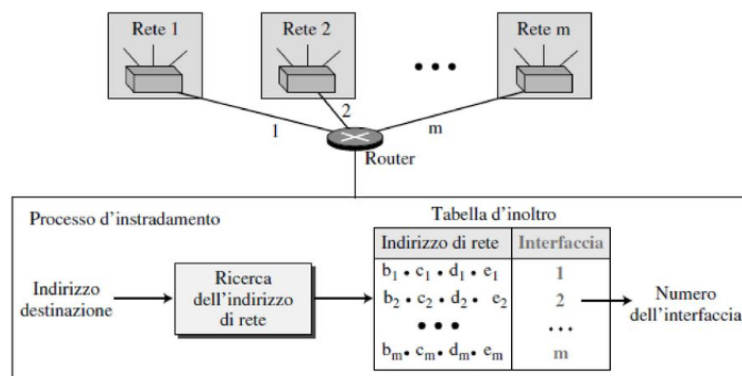


Figura 6.20: Indirizzo di rete (network address)

Prima di terminare l'argomento degli indirizzi IPv4 dobbiamo parlare di cinque indirizzi speciali che vengono usati per scopi particolari: l'indirizzo this-host, l'indirizzo limited-broadcast, l'indirizzo di loopback.

- *Indirizzo this-host.* L'unico indirizzo del blocco 0.0.0.0/32 viene chiamato indirizzo this-host (questo host). Viene usato ogni volta che un host ha la necessità di inviare un datagramma IP ma non conosce il proprio indirizzo da usare come indirizzo sorgente.
- *Indirizzo limited-broadcast.* L'unico indirizzo nel blocco 255.255.255.255/32 viene chiamato indirizzo limited-broadcast. Viene usato ogni volta che un router o un host devono inviare un datagramma a tutti i dispositivi che si trovano all'interno della rete. I router della rete, bloccano i datagrammi che hanno questo indirizzo come destinazione, in questo modo la sua diffusione sarà limitata alla rete locale.
- *Indirizzo di loopback.* Il blocco 127.0.0.0/8 viene chiamato indirizzo di loopback. Un datagramma con uno degli indirizzi in questo blocco come indirizzo di destinazione non lascia mai l'host di spedizione, rimane al suo interno. Questi indirizzi vengono usati per il test e debug del software in esecuzione sull'host locale.

Ottenere indirizzi IP

Per ottenere un blocco di indirizzi IP da usare in una sottorete, un amministratore di rete deve innanzitutto contattare il proprio ISP e ottenere un blocco di indirizzi contigui con un prefisso comune. I router esterni alla rete dell'organizzazione considerano solo gli x bit del prefisso, cioè, quando un router all'esterno dell'organizzazione inoltra un datagramma avente un indirizzo di destinazione che è interno, dovrà considerare solo i primi x bit dell'indirizzo. Questo riduce in modo considerevole la dimensione della tabella di inoltro dei router, dato che una sola riga della forma $a.b.c.d/x$ è sufficiente per far pervenire i pacchetti all'organizzazione. I rimanenti $32 - x$ bit di un indirizzo possono essere usati per distinguere i dispositivi interni dell'organizzazione, che hanno tutti lo stesso prefisso di rete. Saranno quindi i router della rete interna che utilizzeranno i restanti bit dell'indirizzo per indirizzarli al dispositivo destinatario. Tali bit potrebbero presentare un'aggiuntiva struttura di sottorete.

Rivolgersi a un ISP è solo un modo per ottenere un blocco di indirizzi, ma non è l'unico. Anche perché lo stesso ISP deve richiedere, a sua volta, un blocco di indirizzi. Esiste un'autorità globale che ha la responsabilità ultima di gestire lo spazio di indirizzamento IP e di allocare i blocchi di indirizzi: l'Internet Corporation for Assigned Names and Numbers (ICANN). Il ruolo di ICANN non è solo allocare indirizzi IP, ma anche gestire i root DNS server. Ha anche il compito, assai controverso, di assegnare e risolvere dispute sui nomi di dominio.

Dynamic Host Configuration Protocol (DHCP)

Un'organizzazione che ha ottenuto un blocco di indirizzi li può assegnare individualmente alle interfacce di host e router nella propria struttura. Per gli indirizzi delle interfacce dei router, l'amministratore di sistema configura manualmente gli indirizzi IP nel router (spesso da remoto, tramite uno strumento di gestione della rete). Gli indirizzi degli host possono essere configurati manualmente, ma più spesso questo compito è svolto utilizzando il dynamic host configuration protocol (DHCP). DHCP consente a un host di ottenere un indirizzo IP in modo automatico. L'amministratore di rete può configurare DHCP di modo che un dato host riceva un indirizzo IP persistente, in modo che ogni volta che l'host entra in rete gli venga assegnato sempre lo stesso indirizzo IP, oppure in modo da assegnare a ciascun host che si connette un indirizzo IP temporaneo, che sarà diverso tutte le volte che l'host si collega alla rete.

DHCP viene spesso detto protocollo plug-and-play o zero-conf (zero-configuration) per la sua capacità di automatizzare la connessione degli host alla rete. Questa peculiarità lo rende molto interessante per gli amministratori di rete che, altrimenti, dovrebbero svolgere questi compiti manualmente. DHCP è anche largamente usato nelle reti residenziali di accesso a Internet e nelle LAN wireless, dove gli host arrivano e se ne vanno con estrema frequenza dalla rete.

DHCP è un protocollo client-server. Un client è di solito un host appena connesso che desidera ottenere informazioni sulla configurazione della rete, non soltanto su uno specifico indirizzo IP. Nel caso più semplice, ogni sottorete dispone di un server DHCP. In caso contrario, è necessario un agente di relay DHCP (generalmente implementato in un router), che conosca l'indirizzo di un server DHCP per quella rete. In aggiunta al suo indirizzo IP, un computer necessita anche di conoscere il proprio prefisso di rete (o la maschera di rete dell'indirizzo). La maggior parte dei computer ha bisogno anche di altre due informazioni: l'indirizzo di un router di default che deve essere usato per comunicare con le altre reti (oltre a quella locale) e l'indirizzo di almeno un server per la risoluzione dei nomi. Riassumendo, normalmente servono quattro informazioni fondamentali: l'indirizzo del computer, il prefisso, l'indirizzo del default router e l'indirizzo IP del server dei nomi di dominio. DHCP può essere usato per fornire tutte queste informazioni all'host.

Per i nuovi host, il protocollo DHCP si articola in quattro punti:

1. *Individuazione del server DHCP.* Il primo compito di un host appena collegato è l'identificazione del server DHCP con il quale interagire. Questa operazione è svolta utilizzando un messaggio DHCP discover, che un client invia in un pacchetto UDP attraverso la porta 67. Il pacchetto UDP è incapsulato in un datagramma IP. Ma a chi dovrebbe essere inviato questo datagramma? L'host non conosce ancora l'indirizzo IP della rete alla quale è collegato e ancor meno l'indirizzo di un server DHCP per quella rete. Detto ciò, il client DHCP crea un datagramma IP contenente il suo messaggio DHCP con l'indirizzo IP di destinazione broadcast di 255.255.255.255 e l'indirizzo IP sorgente di 0.0.0.0, cioè "questo host". Il client DHCP inoltra il datagramma IP al suo livello di collegamento, che invia il frame in broadcast a tutti i nodi collegati alla sottorete.
2. *Offerta del server DHCP.* Un server DHCP, che riceve un messaggio di identificazione, risponde al client con un messaggio DHCP offer, che viene inviato in broadcast a tutti i nodi della sottorete, usando di nuovo l'indirizzo IP broadcast 255.255.255.255. Dato che in una sottorete possono essere presenti diversi server DHCP, il client dovrebbe trovarsi nell'invidiabile posizione di essere in condizione di scegliere tra le diverse "offerte" disponibili. Ciascun messaggio di offerta server contiene l'ID di transazione del messaggio di identificazione ricevuto, l'indirizzo IP proposto al client, la maschera di sottorete e la durata della concessione (lease time) dell'indirizzo IP (il lasso di tempo durante il quale l'indirizzo IP sarà valido). Tale valore è comunemente dell'ordine delle ore o dei giorni.
3. *Richiesta DHCP.* Il client appena collegato sceglierà tra le offerte dei server e risponderà con un messaggio DHCP request, che riporta i parametri di configurazione.

4. *Conferma DHCP.* Il server risponde al messaggio di richiesta DHCP con un messaggio DHCP ACK, che conferma i parametri richiesti.

Il DHCP è un protocollo client/server nel quale il client invia un messaggio di richiesta e il server invia un messaggio di risposta. Nella Figura 6.21 viene mostrato il formato generale dei messaggi DHCP.

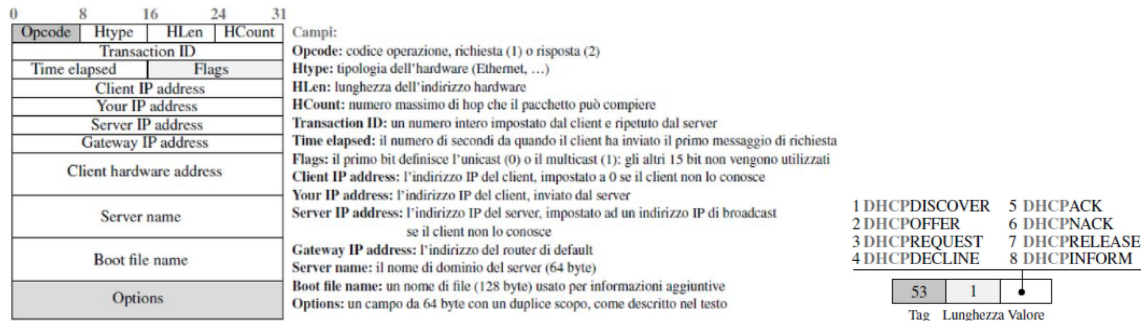
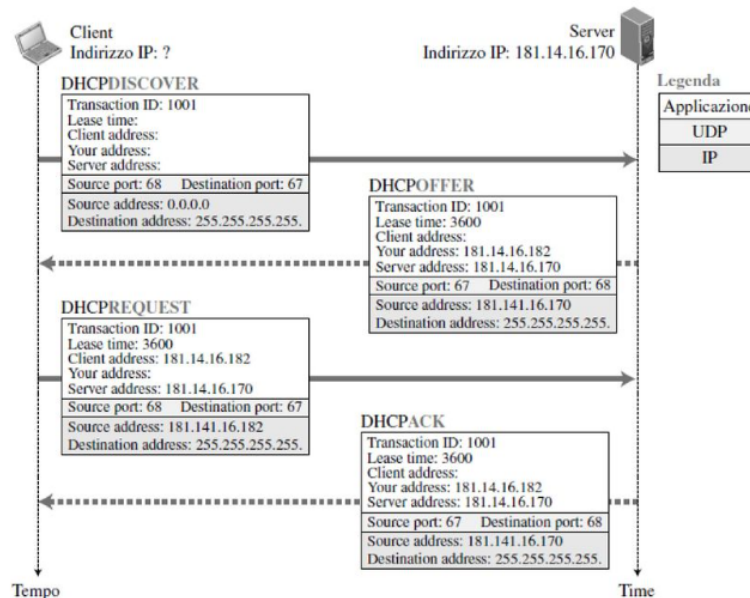


Figura 6.21: Formato dei messaggi e delle opzioni DHCP

Il campo Options (composto da 64 bit) ha un duplice scopo. Può trasportare informazioni aggiuntive oppure la specifica di opzioni DHCP. Per scegliere quest'ultima possibilità, il server utilizza un numero, chiamato magic cookie, sotto forma di un indirizzo IP con valore 99.130.83.99, ed inserisce questo valore nei primi 4 byte del campo opzioni del pacchetto di risposta. Quando il cliente legge una risposta cerca il magic cookie nel campo opzioni del pacchetto ricevuto. Se è presente allora i 60 byte successivi sono opzioni. Un'opzione è composta da tre campi: un campo tag da 1 byte, un campo lunghezza da 1 byte e un campo valore di lunghezza variabile. Ci sono numerosi campi tag che vengono usati e possono anche differire a seconda delle implementazioni. Se il campo tag è 53 allora il campo valore definisce uno degli 8 tipi di messaggio illustrati nella Figura 6.21.

Esempio 6.2.4 – Funzionamento del DHCP

Viene illustrato un semplice scenario di funzionamento del DHCP.



1. L'host che vuole entrare in rete crea un messaggio di tipo DHCPDISCOVER nel quale solo il campo transaction-ID viene impostato usando un numero generato casualmente. L'host non può impostare alcun altro campo in quanto non ha le informazioni necessarie per farlo. Questo messaggio viene incapsulato in un datagramma utente UDP con porta sorgente impostata a 68 e la porta destinazione impostata a 67, entrambe sono porte note. Il datagramma utente viene quindi incapsulato in un datagramma IP con

indirizzo sorgente impostato a 0.0.0.0 ("this-host") e l'indirizzo di destinazione impostato a 255.255.255.255 (indirizzo broadcast). Il motivo è semplice, l'host al momento non conosce né il proprio indirizzo né quello del server.

2. Il server o i server DHCP (ce ne possono essere più di uno) rispondono con un messaggio di tipo DHCPOFFER nel quale il campo your-IP-address contiene l'indirizzo IP offerto per l'host che ne ha fatto richiesta e il server-IP-address comprende l'indirizzo IP del server. Il messaggio contiene anche un lease time (tempo di concessione), cioè per quanto tempo l'host potrà far uso di quell'indirizzo IP. Questo messaggio di risposta viene incapsulato in un datagramma utente con gli stessi numeri di porta della richiesta, ma ovviamente in ordine inverso. Il datagramma utente a sua volta viene incapsulato in un datagramma IP con l'indirizzo del server come indirizzo IP sorgente, ma l'indirizzo di destinazione è ancora un indirizzo broadcast. Questo è utile anche per consentire agli altri server DHCP di ricevere l'offerta e fornire un'offerta migliore, qualora possano.
3. L'host richiedente riceve una o più offerte e seleziona la migliore. A questo punto l'host deve inviare un messaggio di tipo DHCPREQUEST al server che ha inviato l'offerta migliore. Tutti i campi con valore noto vengono impostati. Il messaggio viene incapsulato in un datagramma utente con numeri di porta uguali al primo messaggio di richiesta. Il datagramma utente viene incapsulato in un datagramma IP con l'indirizzo sorgente impostato al nuovo indirizzo assegnato al client, ma l'indirizzo di destinazione è ancora impostato all'indirizzo broadcast. Questo è necessario per annunciare a tutti gli altri server che la loro offerta non è stata accettata.
4. Infine, il server selezionato risponde con un messaggio di tipo DHCPACK al client se l'indirizzo IP offerto è valido. Se il server non è più in grado di mantenere la sua offerta (per esempio se nel frattempo l'indirizzo è stato offerto ad un altro host), il server invia un messaggio di tipo DHCPNACK e il client deve ripetere il procedimento. Anche questo messaggio è broadcast per far sapere agli altri server se la richiesta è stata accettata o rifiutata.

NAT

La distribuzione degli indirizzi per mezzo degli ISP ha creato un nuovo problema. Supponiamo che un ISP abbia assegnato un blocco di indirizzi di dimensione molto limitata ad una piccola azienda o a una famiglia. Se l'azienda cresce o la famiglia necessita di un blocco più ampio, l'ISP potrebbe non essere in grado di soddisfare la richiesta in quanto gli indirizzi che precedono o seguono il blocco possono essere già stati assegnati ad altre reti.

La proliferazione di sottoreti "small office, home office" (o SOHO, piccoli uffici in ambiente domestico), sembrerebbe implicare che ogni volta che si voglia installare una, l'ISP debba allocare un intervallo di indirizzi per coprire tutte le macchine della sottorete. Di conseguenza, al crescere della LAN dovrebbe esserle allocato un blocco maggiore di indirizzi. Ma che cosa avverrebbe se l'ISP avesse già allocato la parte contigua all'intervallo di indirizzi dell'attuale rete SOHO? E che cosa dovrebbe sapere il normale utente per gestire gli indirizzi IP? Fortunatamente, esiste un approccio più semplice e sempre più usato: il NAT (network address translation). Questa tecnologia consente di usare una serie di indirizzi privati per la comunicazione interna e una serie di indirizzi Internet globali (almeno uno) per la comunicazione con il resto del mondo. In questo modo, la rete interna necessita di un solo collegamento all'Internet globale, per mezzo di un router in grado di effettuare le operazioni di NAT.

Come mostrato in Figura 6.22, la rete privata utilizza solamente indirizzi IP privati. Il router che collega la rete interna ad Internet utilizza due indirizzi IP, uno privato e uno globale (pubblico). E' bene notare che la rete privata risulta invisibile al resto di Internet, dall'esterno è possibile rilevare solamente il router NAT con indirizzo pubblico 200.24.5.8.

Tutti i pacchetti in uscita dalla rete privata passano tramite il router NAT, che sostituisce l'indirizzo IP sorgente presente nel datagramma con l'indirizzo NAT globale del router (l'indirizzo pubblico). Anche tutti i pacchetti in entrata passano tramite il router NAT, che in questo caso sostituisce l'indirizzo destinazione nel pacchetto (l'indirizzo globale del router NAT) con l'indirizzo privato appropriato (quello dell'host nella rete interna).

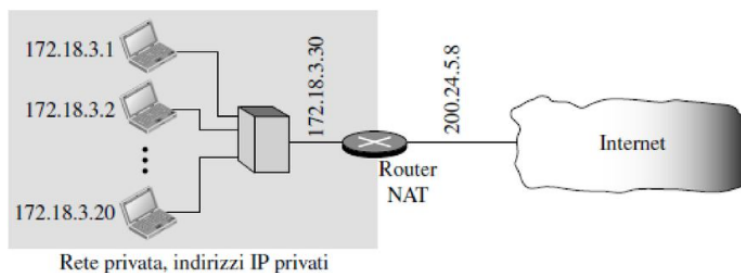


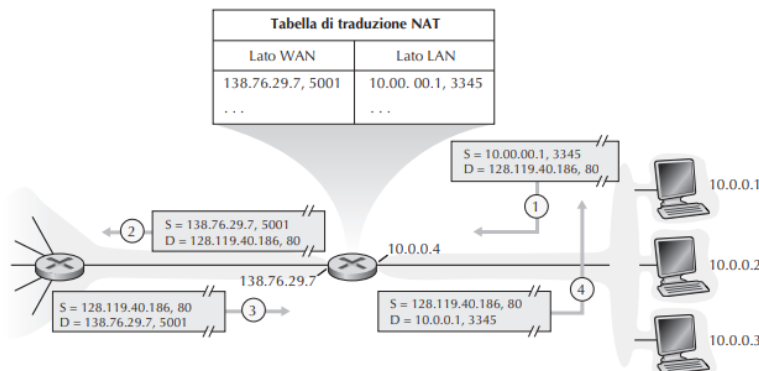
Figura 6.22: Network Address Translation (NAT)

Ma se gli indirizzi privati hanno significato solo all'interno di una data rete, come viene gestito l'indirizzamento dei pacchetti relativi all'Internet globale, in cui gli indirizzi sono necessariamente univoci? La risposta è il NAT. I router abilitati al NAT non appaiono come router al mondo esterno, ma si comportano come un unico dispositivo con un unico indirizzo IP.

Contestualmente, ci si potrebbe chiedere dove i calcolatori della rete domestica ottengano i propri indirizzi e dove il router acquisisca il proprio indirizzo IP. Spesso la risposta è DHCP. Il router ottiene il proprio indirizzo dal server DHCP dell'ISP e manda in esecuzione un server DHCP per fornire gli indirizzi ai calcolatori all'interno dello spazio di indirizzamento della rete domestica. Se tutti i datagrammi in arrivo al router NAT dalla rete geografica hanno lo stesso indirizzo IP di destinazione (nello specifico, quello dell'interfaccia sul lato WAN del router NAT), allora come apprende il router a quale host interno dovrebbe essere inoltrato un determinato datagramma? Il trucco consiste nell'utilizzare una tabella di traduzione NAT (NAT translation table) nel router NAT e nell'includere nelle righe di tale tabella i numeri di porta oltre che gli indirizzi IP.

Esempio 6.2.5 – NAT

La figura mostra l'attività di un router abilitato al NAT, con un'interfaccia che fa parte della rete domestica (sulla destra della figura).



Come visto in precedenza, le quattro interfacce della rete domestica hanno lo stesso indirizzo di sottorete, 10.0.0.0/24. Lo spazio di indirizzamento 10.0.0.0/8 è una delle tre parti dello spazio di indirizzi IP riservato alle reti private, o reame (realm) con indirizzi privati: ossia, una rete i cui indirizzi hanno significato solo per i dispositivi interni. In effetti, esistono centinaia di migliaia di reti private, molte delle quali usano un identico spazio di indirizzamento, 10.0.0.0/24, per scambiare pacchetti fra i loro dispositivi.

Nella figura tutto il traffico che lascia il router domestico verso Internet ha l'indirizzo IP di origine 138.76.29.7 e tutto il traffico in entrata deve avere lo stesso indirizzo come destinazione. In sostanza, il router abilitato al NAT nasconde i dettagli della rete domestica al mondo esterno.

Supponiamo che un utente che si trovi nella rete domestica dietro l'host 10.0.0.1 richieda una pagina web da un server (porta 80) con indirizzo IP 128.119.40.186. L'host 10.0.0.1 assegna il numero di porta di origine (arbitrario) 3345 e invia il datagramma nella rete locale. Il router NAT riceve il datagramma, genera per esso un nuovo numero di porta di origine 5001, sostituisce l'indirizzo IP sorgente con il proprio indirizzo IP sul lato WAN 138.76.29.7 e sostituisce il numero di porta di origine iniziale 3345 con il nuovo numero 5001. Quando

genera un nuovo numero di porta di origine, il router NAT può selezionare qualsiasi numero di porta che attualmente non si trova nella tabella di traduzione NAT. Notiamo che, essendo il campo numero di porta lungo 16 bit, il protocollo NAT può supportare più di 60.000 connessioni simultanee con un solo indirizzo IP sul lato WAN relativo al router. Il NAT del router aggiunge inoltre una riga alla propria tabella di traduzione NAT. Il web server, ignaro della manipolazione subita dal datagramma in arrivo con la richiesta HTTP, risponde con un datagramma con l'indirizzo IP del router NAT come destinazione e il cui numero di porta di destinazione è 5001. Quando questo datagramma arriva al router NAT, quest'ultimo consulta la tabella di traduzione NAT usando l'indirizzo IP di destinazione e il numero di porta di destinazione per ottenere l'appropriato l'indirizzo IP (10.0.0.1) e il corretto numero di porta di destinazione (3345) del browser nella rete domestica. Il router quindi riscrive l'indirizzo di destinazione del datagramma e il suo numero di porta di destinazione e inoltra il datagramma nella rete domestica.

6.2.4 Inoltro dei datagrammi IP

Come abbiamo detto in precedenza, inoltrare significa collocare il pacchetto nel giusto percorso che lo porterà a destinazione. Dato che oggi Internet è costituita da una combinazione di collegamenti (reti), inoltrare significa inviare il pacchetto al salto (hop) successivo (che può essere la destinazione finale o un dispositivo d'interconnessione intermedio).

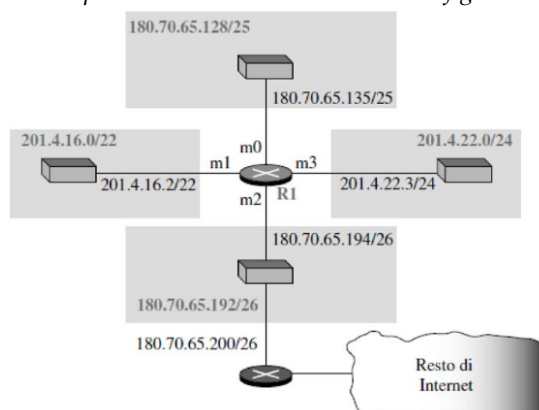
Inoltro basato sull'indirizzo di destinazione

Per prima cosa ci occuperemo dell'inoltro basato sull'indirizzo di destinazione. Questo è l'approccio tradizionale, e oggi è quello prevalente. Questo approccio richiede che l'host o il router che deve effettuare il forwarding abbia una tabella d'inoltro. Quando un host ha un datagramma da inviare o quando un router ha ricevuto un datagramma da inoltrare, esso fa riferimento a questa tabella per trovare il salto successivo a cui inviare il datagramma.

Il compito del modulo di forwarding è quello di effettuare le ricerche nella tabella. In ciascuna riga, gli n bit a sinistra nell'indirizzo di destinazione (prefisso) sono lasciati invariati e il resto dei bit (suffisso) sono impostati a 0. Se l'indirizzo risultante (chiamato indirizzo di rete) combacia con l'indirizzo nella prima colonna, allora l'informazione nelle successive due colonne viene estratta. In caso contrario la ricerca continua. Normalmente l'ultima riga ha un valore di default nella prima colonna che rappresenta tutti gli indirizzi di destinazione che non combaciano con le righe precedenti (in generale si dice quindi che è una "riga di default").

Esempio 6.2.6 – Tabella d'inoltro

Realizzare una tabella d'inoltro per il router R1 utilizzando la configurazione mostrata in figura.



La tabella risultante è la seguente:

<i>Indirizzo di rete/maschera</i>	<i>Salto successivo</i>	<i>Interfaccia</i>
180.70.65.192/26	-	m2
180.70.65.128/25	-	m0
201.4.22.0/24	-	m3
201.4.16.0/22	-	m1
Default	180.70.65.200	m2

Viene illustrata anche la stessa tabella d'inoltro usando la rappresentazione in bit del prefisso:

<i>Indirizzo di rete/maschera</i>	<i>Salto successivo</i>	<i>Interfaccia</i>
10110100 01000110 01000001 11	-	m2
10110100 01000110 01000001 1	-	m0
11001001 00000100 00011100	-	m3
11001001 00000100 000100	-	m1
Default	180.70.65.200	m2

Quando arriva un datagramma in cui i 26 bit a sinistra nell'indirizzo di destinazione combaciano con i bit della prima riga, il pacchetto viene inviato attraverso l'interfaccia m2. Quando arriva un pacchetto i cui 25 bit nell'indirizzo di destinazione combaciano con i bit della seconda riga, il pacchetto viene inviato attraverso l'interfaccia m0, e così via. La tabella mostra chiaramente che la prima riga ha il prefisso più lungo e la quarta riga ha il prefisso più corto. Il prefisso più lungo indica un intervallo di indirizzi più piccolo, il prefisso più corto indica invece un intervallo di indirizzi più grande.

Mostrare il processo d'inoltro di un datagramma, con indirizzo di destinazione 180.70.65.140, nel caso arrivi a R1.

Il router esegue i seguenti passaggi:

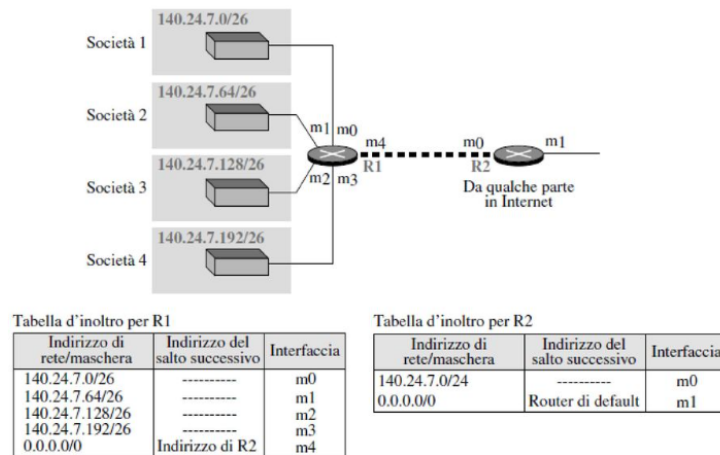
1. La prima maschera (/26) è applicata all'indirizzo di destinazione. Il risultato è 180.70.65.128, che non combacia con l'indirizzo di rete corrispondente.
2. La seconda maschera (/25) è applicata all'indirizzo di destinazione. Il risultato è 180.70.65.128 che combacia con l'indirizzo di rete corrispondente. L'indirizzo del salto successivo e il numero di interfaccia m0 vengono estratti dalla tabella e usati per inoltrare il datagramma.

Aggregazione degli indirizzi

Quando viene utilizzato l'indirizzamento con classi, nella tabella d'inoltro è presente solamente una riga per ciascuna rete esterna all'organizzazione a cui appartiene il router. Quando un datagramma arriva al router, questo verifica la riga corrispondente ed effettua l'inoltro in base alle informazioni ricavate. Quando invece si utilizza l'indirizzamento senza classi, si assiste ad un aumento del numero delle righe nella tabella d'inoltro. Questo perché lo scopo dell'indirizzamento senza classi è proprio quello di dividere lo spazio degli indirizzi in blocchi più piccoli e gestibili. L'aumento nel numero delle righe da controllare porta ad un maggior tempo necessario per le ricerche nella tabella d'inoltro. Per attenuare questo problema è stato ideato il meccanismo di aggregazione degli indirizzi.

Esempio 6.2.7 – Aggregazione degli indirizzi

La figura mostra un esempio dove sono presenti due router. R1 è connesso alle reti di quattro organizzazioni, ciascuna con un blocco da 64 indirizzi, R2 è da qualche parte in Internet, lontano da R1.



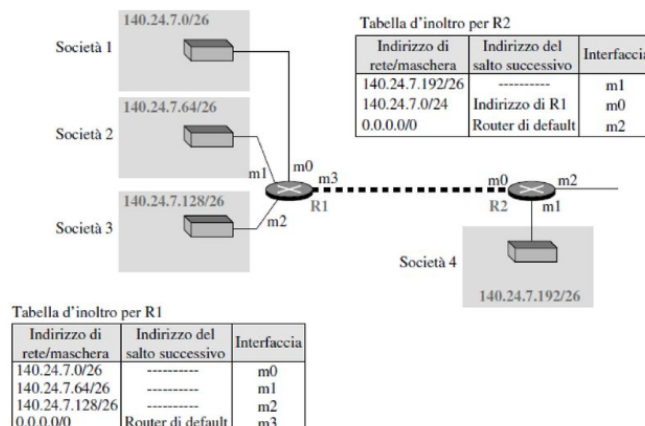
E' evidente che R1 ha una tabella d'ingresso più lunga di R2 poiché ciascun datagramma che gli arriva deve essere indirizzato correttamente all'organizzazione appropriata. R2, invece, può avere una tabella d'ingresso molto piccola. Per R2, qualunque datagramma con indirizzo di destinazione compreso da 140.24.7.0 a 140.24.7.255 è inviato attraverso l'interfaccia m0, senza considerare in alcun modo il destinatario specifico (a quale organizzazione dovrà alla fine essere consegnato il datagramma). Questo meccanismo è chiamato aggregazione degli indirizzi, poiché i blocchi degli indirizzi di quattro società sono stati aggregati in un solo blocco più grande. Nel caso ciascuna organizzazione avesse blocchi di indirizzi che non possono essere aggregati in un blocco più grande (ad esempio blocchi non contigui), R2 sarebbe costretto ad avere una tabella d'ingresso molto più lunga.

Corrispondenza con la maschera più lunga

Che cosa accade se una delle organizzazioni nella figura precedente non è geograficamente vicina alle altre tre? Ad esempio, se l'organizzazione 4 non può essere connessa al router R1 per qualche ragione, possiamo ancora utilizzare l'idea di aggregazione di indirizzo e assegnare il blocco 140.24.7.192/26 alla società 4? La risposta è sì poiché l'instradamento nell'indirizzamento senza classi sfrutta un altro principio, la corrispondenza con la maschera più lunga (longest mask matching). Questo principio afferma che la tabella d'ingresso viene ordinata dalla maschera più lunga alla maschera più corta. In altre parole, se ci sono 3 maschere, /27, /26 e /24, la maschera /27 deve essere nella prima riga e la /24 deve essere nell'ultima (senza considerare la regola di default).

Esempio 6.2.8 – Corrispondenza con la maschera più lunga

Vediamo se questo principio permette di risolvere la situazione nella quale l'organizzazione 4 è separata dalle altre tre.



Supponiamo che un pacchetto arrivi al router R2 con indirizzo di destinazione 140.24.7.200. La prima maschera nel router R2 è applicata e da l'indirizzo di rete 140.24.7.192. Il data-

gramma è quindi instradato correttamente all'interfaccia m1 e raggiunge l'organizzazione 4. Tuttavia, se la tabella d'inoltro non fosse memorizzata con il prefisso più lungo per primo, applicando la maschera /24 si avrebbe un instradamento sbagliato e l'invio del datagramma al router R1.

6.2.5 ICMP

L'IPv4 non implementa alcun meccanismo per segnalare gli errori o correggerli. Cosa accade se qualcosa va storto? Che cosa accade se un router deve scartare un datagramma perché non riesce a trovare un percorso per la destinazione finale o perché il campo time-to-live ha raggiunto il valore zero? Che cosa accade se un host di destinazione non ha ricevuto tutti i frammenti di un datagramma entro un determinato limite di tempo prestabilito? Queste sono tutte situazioni nelle quali si è verificato un errore, ma il protocollo IP non ha meccanismi integrati per renderlo noto all'host mittente.

Inoltre, il protocollo IP è sprovvisto di un meccanismo per effettuare richieste sullo stato di un sistema remoto. Ad esempio, un host, può a volte aver bisogno di determinare se un router o un altro host è attivo. Altre volte un amministratore di rete può aver bisogno di informazioni relative ad un host o un router.

Il protocollo ICMP (Internet Control Message Protocol) viene usato da host e router per scambiarsi informazioni a livello di rete: il suo uso più tipico è la notifica degli errori.

ICMP è spesso considerato parte di IP, ma dal punto di vista dell'architettura si trova esattamente sopra IP, dato che i suoi messaggi vengono trasportati nei datagrammi IP: ossia, i messaggi ICMP vengono trasportati come payload di IP, esattamente come i segmenti TCP o UDP. Allo stesso modo, se un host riceve un datagramma IP, che specifica ICMP come protocollo di livello superiore, allora effettua il demultiplexing dei contenuti del datagramma a ICMP, esattamente come farebbe per contenuti TCP o UDP.

Messaggi

I messaggi ICMP hanno un campo tipo e un campo codice e contengono l'intestazione e i primi 8 byte del datagramma IP che ha provocato la generazione del messaggio, in modo che il mittente possa determinare il datagramma che ha causato l'errore. Vengono mostrati alcuni tipi di messaggio ICMP:

Tipo ICMP	Codice	Descrizione
0	0	risposta echo (a ping)
3	0	rete destinazione irraggiungibile
3	1	host destinazione irraggiungibile
3	2	protocollo destinazione irraggiungibile
3	3	porta destinazione irraggiungibile
3	6	rete destinazione sconosciuta
3	7	host destinazione sconosciuto
4	0	riduzione (controllo di congestione)
8	0	richiesta echo
9	0	annuncio di un router
10	0	scoperta di un router
11	0	TTL scaduto
12	0	intestazione IP errata

Una delle applicazioni che un host può utilizzare per verificare il funzionamento di un altro host è il programma ping. Il programma ping si basa sui messaggi di richiesta e risposta eco dell'ICMP. Un host invia una richiesta eco (tipo 8, codice 0) a un altro host che, se attivo, può rispondere con una risposta eco (tipo 0, codice 0). In maniera molto grossolana, il programma ping può anche misurare l'affidabilità e la congestione del router tra due host inviando una sequenza di messaggi richiesta-risposta.

Esempio 6.2.9 – Ping

Si mostra come inviare un messaggio ping al sito `pads.cs.unibo.it`. Il programma effettua automaticamente la risoluzione DNS del nome di dominio e inizializza il numero di sequenza a partire da 0: questo numero viene incrementato di uno ogni volta che viene inviato un nuovo messaggio di richiesta. Si noti che ping calcola il tempo di round-trip.

```
$ping pads.cs.unibo.it
PING chernobog.pads.cs.unibo.it (130.136.132.11) 56(84) bytes of data.
64 bytes from chernobog.pads.cs.unibo.it (130.136.132.11): icmp_req=1 ttl=52
time=34.2 ms
64 bytes from chernobog.pads.cs.unibo.it (130.136.132.11): icmp_req=2 ttl=52
time=33.1 ms
64 bytes from chernobog.pads.cs.unibo.it (130.136.132.11): icmp_req=3 ttl=52
time=34.0 ms
64 bytes from chernobog.pads.cs.unibo.it (130.136.132.11): icmp_req=4 ttl=52
time=33.9 ms
64 bytes from chernobog.pads.cs.unibo.it (130.136.132.11): icmp_req=5 ttl=52
time=33.3 ms
--- chernobog.puds.cs.unibo.it ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms rtt
min/avg/max/mdev = 33.177/33.758/34.220/0.417 ms
```

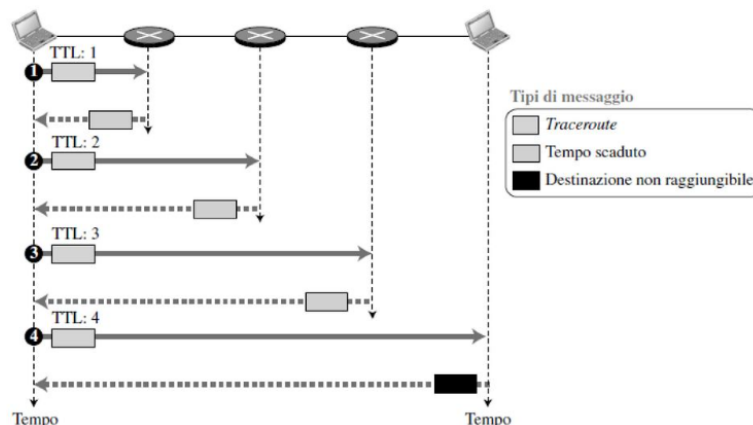
Traceroute

Il programma traceroute in UNIX o tracert in Windows può essere utilizzato per individuare il percorso di un datagramma dalla sorgente alla destinazione tramite l'identificazione dell'indirizzo IP di tutti i router che vengono visitati lungo il percorso. Solitamente il programma viene impostato per un massimo di 30 salti (router), che sono usualmente sufficienti per raggiungere la destinazione.

Il programma traceroute ha un funzionamento molto diverso da ping. Ping è basato su due messaggi query; traceroute è invece implementato per mezzo di due messaggi di segnalazione degli errori: tempo scaduto e destinazione non raggiungibile. Traceroute è un programma di livello di applicazione, ma in questo caso è necessario soltanto il programma client e non il server, poiché non è necessario raggiungere il livello di applicazione dell'host di destinazione. In altre parole non ci sono programmi server di traceroute. Il client traceroute genera un datagramma utente UDP usando intenzionalmente un numero di porta che, con alta probabilità, nel destinatario risulta inutilizzata. Se vi sono n router nel percorso, il programma traceroute invia $(n + 1)$ datagrammi. I primi n messaggi vengono scartati dagli n router, uno per ogni router, l'ultimo messaggio è scartato dall'host di destinazione. Il programma client traceroute utilizza $(n + 1)$ messaggi ICMP di segnalazione degli errori per trovare il percorso tra sorgente e destinazione.

Esempio 6.2.10 – Traceroute

Mostreremo che il programma traceroute non ha bisogno di conoscere il valore di n ; esso viene trovato automaticamente. La figura mostra un esempio nel quale $n = 3$.



Il primo messaggio traceroute viene inviato con valore time-to-live (TTL) impostato a 1, il messaggio viene scartato quindi dal primo router e di conseguenza viene inviato un messaggio di errore ICMP di tempo scaduto. Grazie a quest'ultimo traceroute è in grado di determinare l'indirizzo IP del primo router (l'indirizzo IP del mittente del messaggio di errore) e il nome del router (che può essere presente nella sezione dati del messaggio). Il secondo messaggio traceroute è inviato con TTL impostato a 2, e il mittente può quindi trovare l'indirizzo IP e il nome del secondo router. Analogamente, il terzo messaggio permette di trovare le informazioni relative al router 3. Anche il quarto messaggio viene scartato quando raggiunge l'host di destinazione, ma per un'altra ragione. L'host di destinazione non trova alcuna applicazione in ascolto sulla porta di destinazione specificata nel datagramma utente UDP. Questa volta l'ICMP invia un messaggio diverso, il messaggio destinazione non raggiungibile con codice 3 (porta non raggiungibile), per indicare che non è stato possibile consegnare il datagramma. Dopo aver ricevuto questo messaggio ICMP diverso dai precedenti, il programma traceroute sa che la destinazione finale è stata raggiunta. Può quindi utilizzare le informazioni presenti nell'ultimo messaggio ICMP ricevuto per trovare l'indirizzo IP e il nome della destinazione finale.

Il programma traceroute imposta inoltre un timer per trovare il tempo di round-trip di ciascun router e della destinazione. La maggior parte dei programmi traceroute invia tre messaggi a ogni dispositivo, con lo stesso valore di TTL, per poter effettuare una stima migliore del tempo di round-trip.

6.3 Routing

Instradare un pacchetto dalla sua sorgente alla sua destinazione significa pertanto instradare il pacchetto da un router sorgente (il router di default dell'host sorgente) a un router di destinazione (il router collegato alla rete di destinazione). Anche se un pacchetto deve passare attraverso i router sorgente e destinazione, la domanda è: quali altri router dovrà attraversare il pacchetto? In altre parole, poiché solitamente ci sono molti percorsi diversi tra la sorgente e la destinazione, quale percorso deve effettuare il pacchetto?

Per formulare i problemi di instradamento si utilizza un grafo. Ricordiamo che un grafo $G = (N, E)$ è un insieme N di nodi e un insieme E di archi (edge), ove ciascun arco collega una coppia di nodi di N . Nel contesto dell'instradamento a livello di rete, i nodi del grafo rappresentano i router – che prendono decisioni sull'inoltro dei pacchetti – e gli archi che connettono tali nodi rappresentano i collegamenti fisici tra i router.

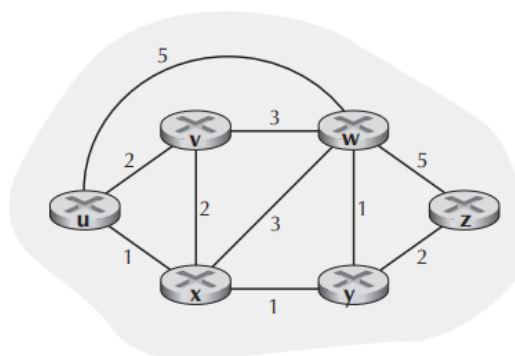


Figura 6.23: Modello astratto di un grafo di una rete di calcolatori

Un arco è anche associato un valore che ne indica il costo. In genere, questo può riflettere la lunghezza fisica del collegamento corrispondente (per esempio, un collegamento transoceanico può avere costo superiore rispetto a un collegamento terrestre a corto raggio), la velocità del collegamento o il suo prezzo. Per i nostri scopi, considereremo i costi degli archi come dati, senza preoccuparci di come siano stati determinati (spesso viene usato il termine metrica per indicare i costi associati agli archi di una rete). Per ogni arco (x, y) tra i nodi x e y denotiamo con $c(x, y)$ il suo costo. Se la coppia (x, y) non appartiene a E , poniamo $c(x, y) = +\infty$. Inoltre, consideriamo sempre e solo

grafi non orientati, cioè con archi bidirezionali, ragion per cui l'arco (x, y) è uguale all'arco (y, x) e $c(x, y) = c(y, x)$. Inoltre, un nodo y viene detto adiacente o vicino (neighbor) a un nodo x se (x, y) è un arco in E .

Dato che agli archi del grafo sono assegnati dei costi, obiettivo naturale di un algoritmo di instradamento è l'individuazione dei percorsi meno costosi tra sorgenti e destinazioni. Per rendere questo problema più preciso, ricordiamo che un percorso in un grafo $G = (N, E)$ è una sequenza di nodi $(x_1, x_2, \dots, x_p)$ tali che ciascuna delle coppie $(x_1, x_2), (x_2, x_3), \dots, (x_{p-1}, x_p)$ sia un arco appartenente a E . Il costo di un percorso $(x_1, x_2, \dots, x_p)$ è semplicemente la somma di tutti i costi degli archi lungo il percorso, ossia $c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$. Dati due nodi qualsiasi x e y , esistono più percorsi che li congiungono, ciascuno con il proprio costo. Uno o più di tali percorsi rappresenta un percorso a costo minimo (least-cost path). Il problema è quindi chiaro: determinare un percorso tra l'origine e la destinazione che abbia il costo minimo. Si noti che se tutti gli archi del grafo hanno lo stesso costo, il percorso a costo minimo rappresenta anche il percorso più breve (shortest path), ossia il percorso con il minor numero di collegamenti tra sorgente e destinazione.

6.3.1 Algoritmi d'instradamento

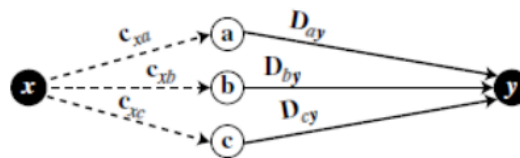
Algoritmo d'instradamento con vettore distanza (Distance Vector - DV)

L'instradamento distance-vector è iterativo, asincrono e distribuito. E' distribuito nel senso che ciascun nodo riceve parte dell'informazione da uno o più dei suoi vicini direttamente connessi, a cui, dopo aver effettuato il calcolo, restituisce i risultati. E' iterativo nel senso che questo processo si ripete fino a quando non avviene ulteriore scambio informativo tra vicini. E' asincrono nel senso che non richiede che tutti i nodi operino al passo con gli altri. Prima di presentare l'algoritmo usato dall'instradamento DV sarà bene discutere un'importante relazione esistente tra costi e percorsi a costo minimo. Sia $d_x(y)$ il costo del percorso a costo minimo dal nodo x al nodo y . Allora i costi minimi sono correlati dalla nota formula di Bellman-Ford:

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

dove $\min_v$ riguarda tutti i vicini di x . Tale relazione è piuttosto intuitiva. Infatti se, dopo aver viaggiato da x a v , consideriamo il percorso a costo minimo da v a y , il costo del percorso sarà $c(x, v) + d_v(y)$. Dato che dobbiamo iniziare viaggiando verso qualche vicino v , il costo minimo da x a y è il minimo di $c(x, v) + d_v(y)$ calcolato su tutti i nodi adiacenti v .

Esempio 6.3.1 – Rappresentazione grafica dell'equazione di Bellman-Ford



a. Caso generale con tre nodi intermedi

Possiamo pensare a $(a \rightarrow y)$, $(b \rightarrow y)$ e $(c \rightarrow y)$ come ai percorsi a costo minimo precedentemente stabiliti e a $(x \rightarrow y)$ come al nuovo percorso a costo minimo.

$$D_{xy} = \min\{(c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy})\}$$

Il concetto di vettore distanza è alla base del distance-vector routing. Un albero a costo minimo è una combinazione di percorsi a costo minimo dalla radice dell'albero verso tutte le destinazioni. Tali percorsi vengono collegati insieme per formare l'albero. Il routing a vettore distanza scinde questi percorsi e crea quello che viene chiamato un vettore distanza, cioè un array monodimensionale che rappresenta l'albero.

Da notare che il "nome" del vettore distanza definisce la radice, gli indici invece definiscono le destinazioni e il valore di ogni cella definisce il costo minimo dalla radice alla destinazione. Un vettore distanza non fornisce il percorso da seguire per giungere alla destinazione (informazione che invece l'albero è in grado di fornire); il vettore riporta solo i costi minimi per le destinazioni.

Sappiamo che un vettore distanza può rappresentare i percorsi a costo minimo di un albero a costo minimo, ma la questione è: in quale modo i nodi all'interno di una rete all'inizio creano il proprio

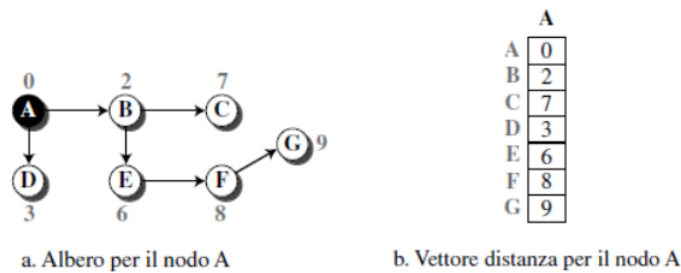
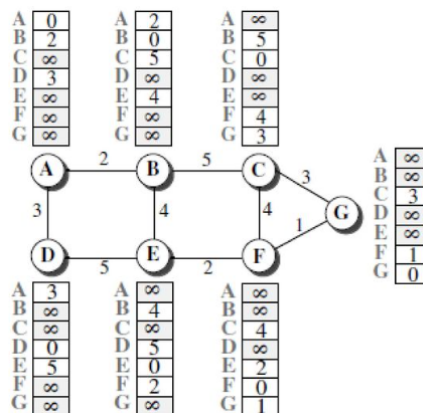


Figura 6.24: Il vettore a distanza che corrisponde ad un albero

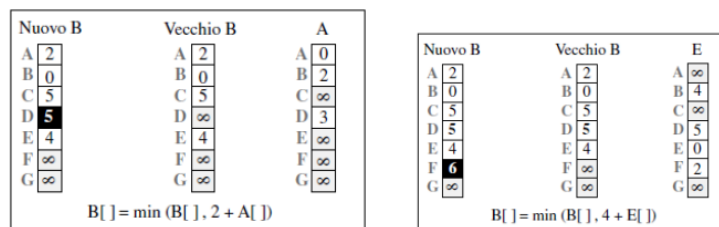
vettore distanza? Ogni nodo della rete, quando viene inizializzato, crea un vettore distanza molto rudimentale con le sole informazioni che il nodo riesce ad ottenere dai propri vicini (i nodi con cui è direttamente collegamento). Per fare questo, il nodo invia alcuni messaggi di benvenuto attraverso tutte le sue interfacce e scopre l'identità dei vicini e la sua distanza con ciascuno di essi. Quindi crea un semplice vettore distanza inserendo le distanze scoperte nelle celle corrispondenti e lascia il valore delle altre celle ad infinito. Questi vettori distanza rappresentano i percorsi a costo minimo elaborati sulla base delle sole informazioni (limitate) di cui dispone il nodo in questo momento. Quando si è a conoscenza di una sola distanza tra due nodi, questa può essere considerata il minimo. Per migliorare questi vettori, i nodi della rete devono aiutarsi scambiandosi informazioni. Dopo che ogni nodo ha creato il suo vettore ne invia una copia a tutti i suoi vicini. Quando un nodo riceve un vettore distanza da un vicino provvede ad aggiornare il suo vettore distanza applicando l'equazione di Bellman-Ford.

Esempio 6.3.2 – Vettori a distanza

Vengono mostrati i vettori a distanza iniziali della nostra rete:



Questi vettori iniziali non permettono veramente ad una rete di inoltrare i pacchetti. Per esempio il nodo A crede di non essere collegato al nodo G in quanto la cella corrispondente mostra un costo infinito.



La figura mostra due eventi asincroni, che avvengono uno dopo l'altro, a distanza di qualche tempo. Nel primo evento il nodo A ha inviato il suo vettore al nodo B. Il nodo B aggiorna il suo vettore usando il costo $c_{BA} = 2$. Nel secondo evento E ha inviato il suo vettore al nodo B. Il nodo B aggiorna il suo vettore usando il costo $c_{EA} = 4$. Grazie al primo evento, il nodo B ha un miglioramento nel suo vettore: il suo costo minimo verso il nodo D è ora cambiato da

infinito a 5 (passando attraverso il nodo A). Dopo il secondo evento il nodo B ha un ulteriore miglioramento del suo vettore: il suo costo minimo verso il nodo F è cambiato da infinito a 6 (passando attraverso il nodo E).

Ora possiamo vedere lo pseudocodice semplificato che descrive l'algoritmo di routing basato su vettore a distanza. L'algoritmo viene eseguito da ogni nodo in modo indipendente e asincrono.

Algoritmo 6.3.1 – Algoritmo Distance Vector

L'idea di base è la seguente. Ciascun nodo x inizia con $D_x(y)$, una stima del costo del percorso a costo minimo da se stesso al nodo y , per tutti i nodi in N . Sia $D_x = [D_x(y) : y \text{ in } N]$ il vettore delle distanze del nodo x , che è il vettore delle stime dei costi da x a tutti gli altri nodi, y , in N . Con l'algoritmo Bellman Ford, ciascun nodo x mantiene i seguenti dati di instradamento.

- Per ciascun vicino v , il costo $c(x, v)$ da x al vicino v .
- Il vettore delle distanze del nodo x , che è $D_x = [D_x(y) : y \text{ in } N]$, contenente la stima presso x del costo verso tutte le destinazioni, y , in N .
- I vettori delle distanze di ciascuno dei suoi vicini, ossia $D_v = [D_v(y) : y \text{ in } N]$, per ciascun vicino y di x .

In questo algoritmo distribuito e asincrono, di quando in quando un nodo invia una copia del proprio vettore delle distanze a ciascuno dei suoi vicini. Quando un nodo x riceve un nuovo vettore da qualcuno dei suoi vicini v , lo salva e quindi usa la formula di Bellman-Ford per aggiornare il proprio vettore come segue:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \text{ per ciascun nodo } y \text{ in } N$$

Se il vettore delle distanze del nodo x è cambiato per via di tale passo di aggiornamento, il nodo x manderà il proprio vettore aggiornato a ciascuno dei suoi vicini, i quali a loro volta aggiorneranno il proprio vettore.

A ciascun nodo x :

Inizializzazione

per tutte le destinazioni y in N :

se y è un vicino

$D_x(y) = c(x, y)$

else

$D_x(y) = \text{infinito}$

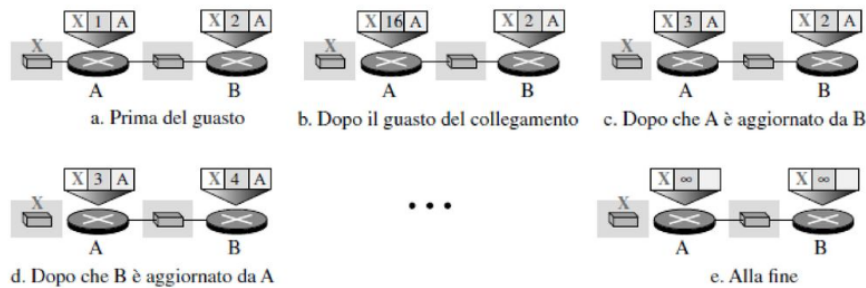
per ciascun vicino w

invia il vettore delle distanze $D_x = [D_x(y) : y \text{ in } N]$ a w

Un problema con il routing basato su vettore distanza è che i decrementi di costo (cioè le buone notizie) si diffondono rapidamente, mentre gli aumenti di costo (le cattive notizie) si propagano lentamente. Affinché un protocollo di routing lavori correttamente, se un collegamento è rotto (e quindi il costo del relativo arco diventa infinito), ogni altro router dovrebbe venirne a conoscenza immediatamente, ma nel routing basato su vettore distanza serve un po' di tempo. Questo problema è spesso chiamato conteggio all'infinito. A volte servono molti aggiornamenti prima che il costo di un collegamento rotto venga registrato come infinito da tutti i router.

Esempio 6.3.3 – Conteggio all'infinito: ciclo a due nodi

Un esempio di conteggio all'infinito è il problema del ciclo a due nodi. Per comprendere il problema, diamo un'occhiata allo scenario illustrato.



La figura illustra un sistema formato da tre nodi. Abbiamo mostrato solo le parti della tabella d'inoltro necessarie per la discussione. All'inizio sia il nodo A che il nodo B sanno come raggiungere il nodo X. Ma improvvisamente si guasta il collegamento tra A e X. Di conseguenza, il nodo A modifica la sua tabella. Se A invia immediatamente la sua tabella a B allora non c'è problema. Tuttavia, se B invia la sua tabella d'inoltro ad A prima ricevere quella di A, il sistema diventa instabile. Il nodo A riceve l'aggiornamento e, supponendo che B abbia trovato un modo per raggiungere X, aggiorna immediatamente la sua tabella d'inoltro. Ora A invia il suo nuovo aggiornamento a B. Quello che accade è che B è portato a credere che qualcosa sia cambiato vicino ad A e di conseguenza aggiorna la sua tabella d'inoltro. Il costo per raggiungere X aumenta gradualmente fino ad arrivare all'infinito. Ora, finalmente, sia A che B sanno che X non può essere raggiunto. Tuttavia, durante questo periodo di tempo il sistema non è stabile. Il nodo A crede che il percorso per raggiungere X passi da B e il nodo B crede che il percorso per raggiungere X passi attraverso A. Se A riceve un pacchetto destinato a X il pacchetto va a B e quindi ritorna ad A. Allo stesso modo, se B riceve un pacchetto destinato a X, va verso A e torna a B. I pacchetti rimbalzano tra A e B, creando un ciclo a due nodi.

Una soluzione all'instabilità viene chiamata *split horizon*. Con questa strategia invece di inviare la tabella attraverso ogni interfaccia, ciascun nodo invia solo una parte della sua tabella tramite le sue varie interfacce. Se, secondo tale tabella, il nodo B ritiene che il percorso ottimale per raggiungere X passi tramite A, allora non deve fornire questa informazione ad A: l'informazione è arrivata da A che quindi la conosce già. Il fatto che le informazioni vengano prese dal nodo A, vengano modificate ed inviate nuovamente al nodo A è ciò che crea confusione. Nel nostro scenario, il nodo B elimina l'ultima riga della sua tabella d'inoltro prima di inviarla ad A. In questo caso il nodo A mantiene il valore di anche il nodo B corregge la sua tabella. Il sistema diventa stabile subito dopo il primo aggiornamento: sia il nodo A che il nodo B sanno che X non è più raggiungibile.

L'utilizzo della strategia *split horizon* ha un effetto collaterale. Normalmente il protocollo che implementa l'algoritmo di routing utilizza un timer: se per una certa quantità di tempo non ci sono novità circa un percorso, il nodo lo elimina dalla sua tabella. Quando il nodo B, nello scenario precedente, elimina il percorso verso X dai suoi annunci ad A, il nodo A non può sapere se ciò è dovuto alla strategia *split horizon* (la sorgente delle informazioni era A) o se dipende dal fatto che B recentemente non ha ricevuto alcuna notizia di X. Nella strategia dell'inversione avvelenata, B può ancora rendere pubblico il valore per X, ma se la sorgente delle informazioni è A, allora sostituisce la distanza con valore infinito. In questo caso l'infinito viene usato come avvertimento: "Non usare questo valore, quello che so circa questo percorso viene da te".

Algoritmo d'instradamento a stato del collegamento (LS)

Un algoritmo di routing che deriva direttamente dalla nostra precedente discussione sulla creazione degli alberi a costo minimo e delle tabelle d'inoltro è il routing a stato del collegamento, Link-State (LS) routing.

Questo metodo utilizza il termine *link-state* (stato del collegamento) per definire le caratteristiche di un collegamento (un arco) che rappresenta una rete, parte di una internet. In questo algoritmo, il costo associato ad un arco definisce lo stato del collegamento. I collegamenti con costi inferiori sono preferibili rispetto a quelli con costi superiori; se il costo di un collegamento è infinito, significa che esso non esiste più o è stato interrotto.

Per creare un albero a costo minimo utilizzando questo metodo ogni nodo deve avere una mappa completa della rete, il che significa che deve conoscere lo stato di ciascun collegamento. La raccolta di stati per tutti i collegamenti viene chiamata Link-State Database (LSDB). Il LSDB è unico per l'intera

rete, ma ogni nodo deve averne un duplicato per poter essere in grado di creare l'albero a costo minimo. Il LSDB può essere rappresentato per mezzo di un array bidimensionale (matrice) in cui il valore di ogni cella definisce il costo del collegamento corrispondente.

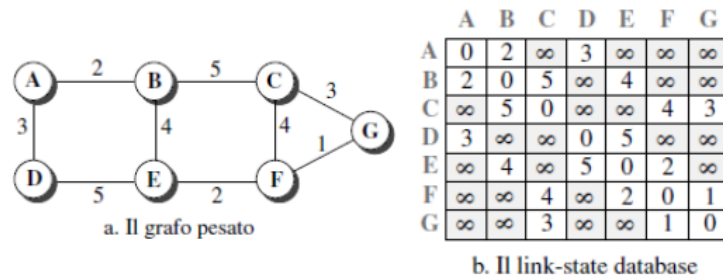


Figura 6.25: Esempio di un link-state database

Ora il problema è come può ogni nodo creare la sua copia del LSDB che contenga le informazioni necessarie sull'intera rete? Ciò può essere fatto tramite un procedimento chiamato flooding (inondazione). Ogni nodo può inviare dei messaggi di benvenuto a tutti i suoi vicini (i nodi ai quali è collegato direttamente) per raccogliere una coppia di informazioni circa ogni suo vicino: l'identità del nodo e il costo del collegamento. La combinazione di queste due informazioni viene chiamata pacchetto LS (LS packet, LSP). L'LSP viene inviato attraverso ogni interfaccia del nodo. Quando un nodo riceve un LSP da una delle sue interfacce e confronta il nuovo LSP con un'eventuale vecchia copia che potrebbe già avere. Se l'LSP appena arrivato è più vecchio di quello che ha già (questo è rilevabile confrontando i numeri di sequenza), l'LSP viene scartato. Se è più recente o è il primo ricevuto, il nodo scarta il vecchio LSP (se ce n'è uno) e conserva quello che ha appena ricevuto. A questo punto il nodo invia una copia dell'LSP da ogni sua interfaccia ad esclusione di quella da cui è arrivato il pacchetto. Questo garantisce che il flooding si interrompa da qualche parte nella rete (ad esempio dove un nodo ha una sola interfaccia). Dopo aver ricevuto tutti i nuovi LSP, ogni nodo è in grado di creare la sua copia dell'LSDB globale. Questo LSDB è esattamente uguale per ogni nodo e rappresenta l'intera mappa della rete.

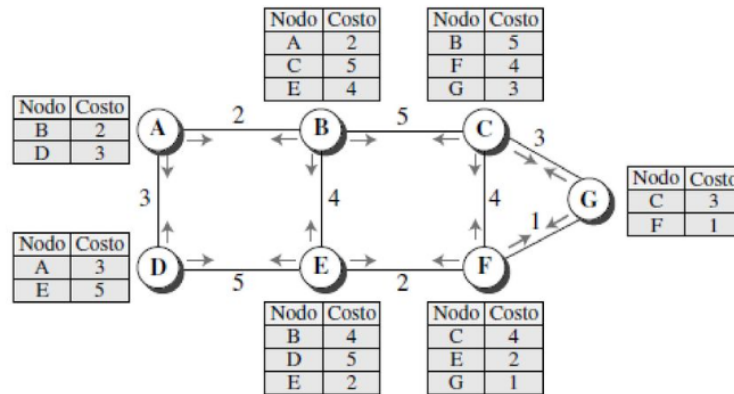


Figura 6.26: Gli LSP creati ed inviati da ciascun nodo per costruire l'LSDB

A questo punto possiamo confrontare l'algoritmo di routing a stato del collegamento con quello a vettore distanza. In quest'ultimo, ogni router comunica ai suoi vicini ciò che sa sull'intera rete, mentre nell'algoritmo di routing a stato del collegamento ogni router riferisce all'intera rete ciò che sa dei suoi vicini.

L'algoritmo di calcolo dei percorsi che presentiamo associato all'instradamento link state è noto come algoritmo di Dijkstra. L'algoritmo di Dijkstra calcola il percorso a costo minimo da un nodo (l'origine, che chiameremo u) a tutti gli altri nodi nella rete, è iterativo e ha le seguenti proprietà: dopo la k -esima iterazione, i percorsi a costo minimo sono noti a k nodi di destinazione e, tra i percorsi a costo minimo verso tutti i nodi di destinazione, questi k percorsi hanno i k costi più bassi.

Algoritmo 6.3.2 – Algoritmo di Dijkstra

Per costruire il suo albero a costo minimo utilizzando l'LSDB condiviso, ogni nodo deve eseguire l'algoritmo di Dijkstra. Questo algoritmo è composto dai seguenti passi:

- il nodo sceglie se stesso come radice dell'albero, crea un albero con un singolo nodo e imposta il costo totale di ogni nodo sulla base delle informazioni che trova nell'LSDB;
- il nodo seleziona un altro nodo, tra tutti quelli che non sono presenti nell'albero, in modo che sia il più vicino possibile alla radice, e lo aggiunge all'albero. Dopo che questo nodo è stato aggiunto all'albero, il costo dei nodi non presenti nell'albero deve essere aggiornato in quanto i percorsi potrebbero essere cambiati;
- il nodo ripete il passaggio 2 finché tutti i nodi non sono stati aggiunti all'albero. I tre passaggi sopra riportati alla fine creano un albero a costo minimo.

Adottiamo la seguente notazione:

- N : insieme dei nodi della rete.
- $c(x, y)$: costo del collegamento dal nodo x al nodo y .
- $D(v)$: costo minimo del percorso dal nodo origine alla destinazione v per quanto concerne l'iterazione corrente dell'algoritmo.
- $p(v)$: immediato predecessore di v lungo il percorso a costo minimo dall'origine a v .
- N' : sottoinsieme di nodi contenente tutti (e solo) i nodi v per cui il percorso a costo minimo dall'origine a v è definitivamente noto.

Inizializzazione:

```

 $N' = \{r\}$  /*  $r$  è la radice dell'albero */
per tutti i nodi  $n$ 
    se  $n$  è adiacente a  $r$ 
        allora  $D(n) = c(r, n)$ 
    altrimenti  $D(n) = \text{infinito}$ 
  
```

Ciclo

determina un n non in N' tale che $D(n)$ sia minimo

aggiungi n a N'

per ogni nodo a adiacente a n e non in N' aggiorna $D(a)$:

```

 $D(a) = \min(D(a), D(n) + c(n, a))$ 
  
```

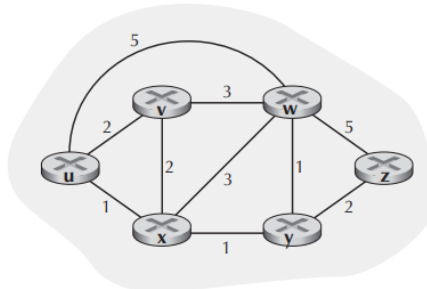
```

/* il nuovo costo verso  $a$  è il vecchio costo verso  $a$  oppure
   il costo del cammino minimo noto verso  $a$  più il costo da  $n$  a  $a$  */
  
```

Finché $N' = N$

Esempio 6.3.4 – Algoritmo di Dijkstra

Consideriamo la rete in figura e calcoliamo i percorsi a costo minimo da u a tutte le destinazioni.



I passi dell'algoritmo sono mostrati nella tabella, le cui righe forniscono i valori delle variabili

dell'algoritmo al termine dell'iterazione.

Passo	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	u	2, u	5, u	1, u	∞	∞
1	ux	2, u	4, x		2, x	∞
2	uxy	2, u	3, y			4, y
3	$uxyv$		3, y			4, y
4	$uxyvw$					4, y
5	$uxyvwz$					

Analizziamo in dettaglio i primissimi passi.

- Nel passo di inizializzazione i valori dei percorsi a costo minimo noti da u ai suoi nodi adiacenti, v , w e x , sono posti rispettivamente a 2, 5 e 1. In particolare il costo verso w vale 5 (anche se vedremo presto che di certo esiste un percorso a costo inferiore) dato che questo è il costo del collegamento diretto (un solo hop) da u a w . I costi verso y e z sono posti a infinito dato che tali nodi non sono adiacenti a u .
- Nella prima iterazione prendiamo in considerazione i nodi non ancora aggiunti all'insieme N' e determiniamo il nodo a costo minimo come alla fine della precedente iterazione. Tale nodo è x , di costo 1, e pertanto x viene aggiunto all'insieme N' . Viene poi aggiornato $D(v)$ per tutti i nodi, ottenendo i risultati mostrati nella seconda riga (Passo 1) della tabella. Il costo del percorso verso v non è cambiato. Il costo del percorso verso w (che era 5 al termine dell'inizializzazione) passando per il nodo x è diventato 4. Viene quindi selezionato questo percorso a costo inferiore e il predecessore di w lungo il percorso minimo da u diventa x . Analogamente, il costo verso y (attraverso x) viene calcolato essere 2, e la tabella viene aggiornata di conseguenza.
- Nella seconda iterazione si trova che i nodi v e y hanno percorsi a costo minimo (2), ne scegliamo arbitrariamente uno e aggiungiamo y all'insieme N' che ora conterrà u , x e y . I costi verso i nodi rimanenti non ancora in N' , ossia v , w e z , sono aggiornati dell'algoritmo Dijkstra, ottenendo i risultati mostrati nella terza riga della tabella.
- E così via...

Quando l'algoritmo Dijkstra termina, abbiamo per ciascun nodo il suo predecessore lungo il percorso a costo minimo dal nodo origine. Per ciascun predecessore abbiamo il rispettivo predecessore, e in questo modo riusciamo a costruire l'intero percorso dall'origine a tutte le destinazioni. La tabella di inoltro in un nodo, diciamo u , può pertanto essere costruita da queste informazioni memorizzando, per ciascuna destinazione, il nodo del successivo hop sul percorso a costo minimo da u alla destinazione.

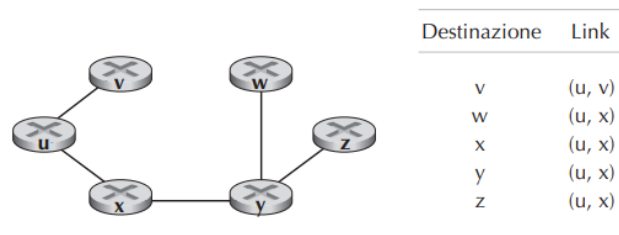


Figura 6.27: Percorso a costo minimo e tabella di inoltro per il nodo u

Confronto tra gli instradamenti LS e DV

I due meccanismi di instradamento studiati utilizzano approcci complementari nel calcolare i percorsi. Nel DV ciascun nodo dialoga solo con i vicini direttamente connessi, informandoli delle stime a costo minimo da sé stesso a tutti i nodi (che conosce) nella rete. Nel LS, ciascun nodo dialoga con tutti gli altri nodi via broadcast, ma comunica loro solo i costi dei collegamenti direttamente connessi. Concludiamo la nostra trattazione degli instradamenti LS e DV con un veloce confronto di alcuni

dei rispettivi attributi. Ricordiamo che N rappresenta l'insieme dei nodi (router) ed E l'insieme degli archi (collegamenti).

- *Complessità dei messaggi.* LS richiede che ciascun nodo conosca il costo di ogni collegamento nella rete. Ciò implica l'invio di $O(|N| * |E|)$ messaggi. Inoltre, ogniqualvolta cambia il costo di un collegamento, il nuovo costo deve essere comunicato a tutti i nodi. A ogni iterazione l'instradamento DV richiede scambi di messaggi tra nodi adiacenti. Abbiamo visto che il tempo richiesto perché l'algoritmo converga può dipendere da molti fattori. Quando cambiano i costi dei collegamenti, l'instradamento DV propaga i risultati dei costi cambiati se il nuovo costo ha causato la variazione del percorso a costo minimo per uno o più nodi connessi a tale collegamento.
- *Velocità di convergenza.* Abbiamo visto che la nostra implementazione di LS è un algoritmo $O(|N|^2)$ che richiede $O(|N| * |E|)$ messaggi. L'algoritmo del DV può convergere lentamente e può presentare cicli di instradamento. DV presenta anche il problema del conteggio all'infinito.
- *Robustezza.* Che cosa avviene se un router si guasta, funziona male o viene sabotato? Con LS, un router può comunicare via broadcast un costo sbagliato per uno dei suoi collegamenti connessi (ma non per altri). Un nodo può anche alterare o eliminare i pacchetti ricevuti in broadcast LS. Ma i nodi LS si occupano di calcolare soltanto le proprie tabelle di inoltro, e gli altri nodi effettuano calcoli simili per quanto li riguarda. Ciò significa che i calcoli di instradamento sono in qualche modo isolati, il che fornisce un certo grado di robustezza. Con DV, un nodo può comunicare percorsi a costo minimo errati a tutte le destinazioni. Più in generale notiamo che, a ogni iterazione, il calcolo di DV in un nodo viene comunicato ai suoi vicini e quindi ai vicini dei vicini alla successiva iterazione. In questo senso, un calcolo errato su un nodo si può diffondere per l'intera rete.

In conclusione, nessuno dei due meccanismi di instradamento è nettamente superiore all'altro ed entrambi vengono utilizzati in Internet.

Path-vector routing

Sia il routing a stato del collegamento che a vettore distanza si basano sul costo minimo. Tuttavia, ci sono dei casi in cui il costo non è l'obiettivo prioritario. Ad esempio, supponiamo che nella rete ci siano dei router tramite i quali un mittente non vuole che passino i suoi pacchetti. Per esempio un router può appartenere ad una società che non fornisce un sufficiente livello di sicurezza o che appartiene a un rivale commerciale del mittente che potrebbe analizzare i pacchetti alla ricerca di informazioni riservate. Il routing a costo minimo non può evitare che un pacchetto transiti in un'area se tale area si trova lungo il percorso a costo minimo. In altre parole, lo scopo del costo minimo applicato dal routing LS o DV, non consente a un mittente di applicare politiche specifiche al percorso dei suoi pacchetti.

Per soddisfare queste richieste è stato ideato un terzo algoritmo di routing, chiamato path vector (PV) routing (routing a vettore-distanza). Il routing PV non ha gli svantaggi del routing LS o DV sopra descritti, in quanto non si basa sul routing a costo minimo. Il percorso migliore viene determinato dalla sorgente utilizzando la politica che essa stessa decide di imporre al percorso. In altre parole la sorgente può controllare il percorso.

Nel path-vector routing, il percorso da una sorgente a tutte le destinazioni viene determinato dal miglior albero di copertura (spanning tree). Esso, tuttavia, non è l'albero a costo minimo bensì è quello determinato dalla sorgente quando impone la propria politica. Se ci sono più percorsi per la destinazione, la sorgente può scegliere il percorso che corrisponde meglio a tale politica. Una sorgente è libera di applicare politiche multiple allo stesso tempo. Una delle politiche più comuni ha lo scopo di minimizzare il numero di nodi visitati. Un'altra politica comune è quella di imporre che i propri pacchetti non passino mai da alcuni nodi intermedi durante il loro percorso.

Il path-vector routing, come quello a vettore distanza, è un algoritmo di instradamento asincrono e distribuito. Gli alberi di copertura vengono creati gradualmente e in modo asincrono da ciascun nodo. Quando un nodo viene inizializzato, esso crea un path vector (vettore percorso) basato sulle informazioni che riesce ad ottenere dai suoi vicini. Anche in questo caso, un nodo invia dei messaggi di benvenuto ai suoi vicini per raccogliere tali informazioni.

Ogni nodo, dopo la creazione del path-vector iniziale, lo invia a tutti i suoi vicini. Ogni nodo, quando riceve un path-vector da un vicino, aggiorna il suo path-vector tramite un'equazione simile

a quella di Bellman-Ford, ma applicando la sua politica invece del costo minimo. Possiamo definire tale equazione come

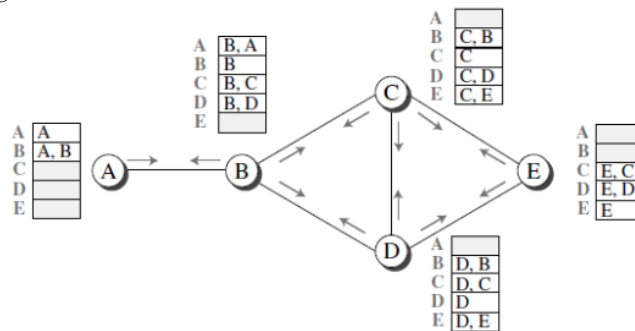
$$\text{percorso}(x, y) = \text{miglior}\{\text{percorso}(x, y), [(x + \text{percorso}(v, y))]\} \text{ per tutti } v \text{ nella rete}$$

In tale equazione l'operatore (+) significa che bisogna aggiungere x all'inizio del percorso. Dobbiamo anche essere cauti ed evitare di aggiungere un nodo a un percorso vuoto perché un percorso vuoto non è altro che un percorso inesistente.

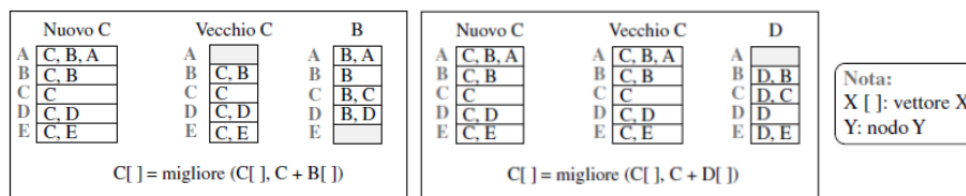
La politica viene definita selezionando il migliore tra vari percorsi. Il path-vector routing impone anche un'altra condizione a questa equazione: se il percorso (v, y) comprende x , tale percorso viene scartato per evitare di aggiungere un ciclo. In altre parole x non vuole visitare se stesso quando seleziona un percorso verso y .

Esempio 6.3.5 – Path vector routing

Viene mostrata una piccola rete con solo cinque nodi. Ogni sorgente ha creato il suo albero di copertura che corrisponde alla propria politica. La politica imposta da tutte le sorgenti è usare il numero minimo di nodi per raggiungere una destinazione.



E' bene notare che, anche in questo caso, non intendiamo dire che tutte queste tabelle vengono create simultaneamente; esse vengono create quando i vari nodi vengono inizializzati. La figura mostra anche come questi path-vector vengono inviati ai vicini subito dopo che sono stati creati (le frecce in figura).



Evento 1: C riceve una copia del vettore di B Evento 2: C riceve una copia del vettore di D

Viene mostrato il path-vector del nodo C dopo due eventi. Nel primo evento il nodo C riceve una copia del vettore B, che migliora il suo vettore: ora sa come raggiungere il nodo A. Nel secondo evento il nodo C riceve una copia del vettore D, che non cambia il suo vettore. Di fatto il vettore per il nodo C dopo il primo evento diviene stabile e serve da tabella d'indirizzo.

Algoritmo 6.3.3 – Algoritmo path-vector

Viene riportata una versione semplificata dell'algoritmo path-vector:

```

Path Vector.Routing ()
{
    // Inizializzazione
    for (y = 1 to N)
    {
        if (y e' me stesso)
            Path[y] = me.stesso
        else if (y e' un vicino)
            Path[y] = me.stesso + il_nodo_vicino
        else
    }

```

```

        Path[y] = vuoto
    }
    Spedisci il vettore (Path[1], Path[2], ..., Path[y])
                                a tutti i vicini

// Aggiornamento
repeat (sempre)
{
    wait (un vettore Path, da un vicino w)
    for (y=1 to N)
    {
        if (Path comprende me_stesso)
            scarta il percorso // Evita ogni ciclo
        else
            Path[y] = il_migliore_tra [Path[y],
                                    (me_stesso + Path [y])]
    }
    If (c'è un cambiamento nel vettore)
        Spedisci il vettore [Path[1], Path[2]..... Path[y]]
                                a tutti i vicini
    }
} // Fine del path-vector

```

6.3.2 Protocollo di routing

Un protocollo è qualcosa di più di un algoritmo. Un protocollo deve definire il suo ambito di funzionamento, i messaggi che vengono scambiati, la comunicazione tra router e l'interazione con i protocolli di altri ambiti.

Routing Information Protocol (RIP)

Il Routing Information Protocol (RIP) è uno dei protocolli di routing intra-dominio più ampiamente utilizzati ed è basato sull'algoritmo di instradamento a vettore distanza.

La gestione dei costi viene gestita in modo molto semplice ed indipendentemente da fattori quali le prestazioni dei router e dei collegamenti, come ad esempio: ritardi, ampiezza della banda ecc. Quindi, il costo viene definito come il numero di salti (hop), cioè il numero di sottoreti, che un pacchetto deve visitare per andare dal router sorgente all'host di destinazione. Da notare che la rete nel quale si trova l'host sorgente non viene considerata in questo calcolo. Questo perché l'host sorgente non usa alcuna tabella d'inoltro: il pacchetto viene semplicemente consegnato al router di default. In RIP il costo massimo di un percorso è di 15 hop, il che significa che il valore 16 rappresenta l'infinito (nessuna connessione).

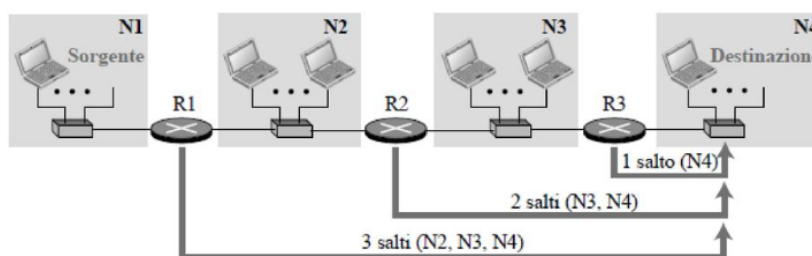


Figura 6.28: Conto degli hop in RIP

L'algoritmo a vettore distanza di cui abbiamo parlato ha lo scopo principale di scambiare vettori distanza tra i nodi vicini. Invece, in questo caso dei router il loro obiettivo è quello di costruire delle tabelle d'inoltro per far giungere i pacchetti alla loro rete di destinazione.

In RIP, una tabella d'inoltro è formata da tre colonne: nella prima c'è l'indirizzo della rete di destinazione, nella seconda l'indirizzo del prossimo router al quale il pacchetto deve essere inoltrato

e nella terza il costo (numero di hop) per raggiungere la rete di destinazione. Si noti che la prima e la terza colonna insieme contengono le stesse informazioni di un vettore distanza, ma con la differenza che in questo caso il costo mostra il numero di salti fino alla rete di destinazione.

Tabella d'inoltro per R1			Tabella d'inoltro per R2			Tabella d'inoltro per R3		
Rete di destinazione	Prossimo router	Costo (in hop)	Rete di destinazione	Prossimo router	Costo (in hop)	Rete di destinazione	Prossimo router	Costo (in hop)
N1	—	1	N1	R1	2	N1	R2	3
N2	—	1	N2	—	1	N2	R2	2
N3	R2	2	N3	—	1	N3	—	1
N4	R2	3	N4	R3	2	N4	—	1

Figura 6.29: Tabelle d'inoltro

Anche se una tabella d'inoltro RIP definisce solo il prossimo router (nella seconda colonna), questa informazione è sufficiente per ricostruire l'intero albero minimo. Ad esempio R1 definisce che il prossimo router per il percorso verso N4 è R2; R2 definisce che il prossimo router per N4 è R3; R3 definisce che non c'è alcun router successivo per tale percorso. Quindi l'albero è $R1 \rightarrow R2 \rightarrow R3 \rightarrow N4$.

Ci si potrebbe chiedere qual è l'uso della terza colonna della tabella d'inoltro. La terza colonna non è necessaria per inoltrare il pacchetto, ma serve per aggiornare la tabella d'inoltro quando ci sono modifiche del percorso.

RIP si basa su una coppia di processi, un client e un server, e sul loro scambio di messaggi. Una parte del messaggio, che chiamiamo voce (entry), può essere ripetuta tante volte quante necessarie. Ogni voce fornisce le informazioni relative a una riga della tabella d'inoltro del router che invia il messaggio.

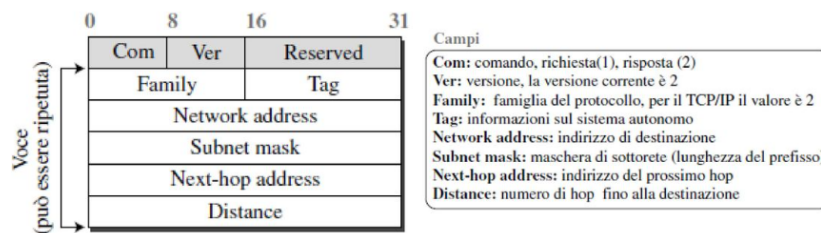


Figura 6.30: Formato dei messaggi RIP

Il RIP ha due tipi di messaggio: richieste e risposte. Un messaggio di richiesta viene inviato da un router appena acceso o da un router che ha delle voci scadute in memoria. I messaggi di richiesta possono riguardare voci specifiche oppure richiedere tutte le voci disponibili. Un messaggio di risposta (o aggiornamento) può essere sollecitato o meno. Un messaggio di risposta sollecitato viene inviato solo come risposta a un messaggio di richiesta. Esso contiene informazioni circa la destinazione specificata nel messaggio di richiesta corrispondente. Un messaggio di risposta non sollecitato, invece, viene inviato periodicamente (ogni 30 secondi) o quando c'è un cambiamento nella tabella d'inoltro.

Per il suo funzionamento, RIP utilizza tre timer.

- Il *timer periodico* controlla l'invio dei messaggi di aggiornamento. Ogni router ha un timer periodico che viene impostato a un valore scelto casualmente nell'intervallo tra 25 e 35 secondi (per evitare che tutti i router inviino i loro messaggi allo stesso tempo e creino un picco di traffico). Il timer effettua il conto alla rovescia; quando arriva a zero viene inviato un messaggio d'aggiornamento e il timer viene impostato nuovamente in modo casuale.
- Il *timer di scadenza* regola la validità dei percorsi. Quando un router riceve delle informazioni aggiornate per un percorso, il timer di scadenza viene impostato a 180 secondi (per quel particolare percorso). Ogni volta che viene ricevuto un nuovo aggiornamento, il timer del relativo percorso viene fatto ripartire dal valore iniziale. Se ci sono dei problemi in una rete e non viene ricevuto alcun aggiornamento nell'arco dei 180 secondi assegnati, il percorso viene considerato come scaduto e il suo conto dei salti viene impostato a 16 (il che significa che la destinazione è irraggiungibile).

- Il *timer per la garbage collection* viene usato per eliminare i percorsi dalla tabella d'inoltro. Quando le informazioni circa un percorso non sono più valide, un router non provvede immediatamente alla loro rimozione dalla sua tabella. Al contrario, il router continua a annunciare il percorso con un valore di 16. Allo stesso tempo, per quel percorso, un timer per la garbage collection viene impostato a 120 secondi. Quando il contatore raggiunge lo 0, il percorso viene definitivamente eliminato dalla tabella. Questo timer consente ai vicini di venire a conoscenza di un percorso che non è più valido, prima che sia eliminato.

Analizziamo brevemente le caratteristiche di RIP:

- *Split horizon with poisoned reverse (inversione avvelenata)*. Serve per evitare che un router invii rotte non valide al router da cui ha imparato la rotta (evitare cicli). Si mette a infinito (16) il costo della rotta che passa attraverso il vicino a cui si manda advertisement.
- *Triggered updates*. Riduce il problema della convergenza lenta. Quando cambia una rotta si inviano immediatamente informazioni ai vicini senza attendere il timeout.
- *Hold-down*. Fornisce robustezza. Quando si riceve una informazione di una rotta non più valida, si avvia un timer e tutti gli advertisement riguardanti quella rotta che arrivano entro il timeout vengono tralasciati.

Il RIP è implementato per mezzo di un processo demone (sempre attivo) che ascolta sulla porta nota 520 UDP. Nel caso di UNIX BSD tale processo viene chiamato *routed* (come abbreviazione di *route daemon*). Questo significa che anche se RIP è un protocollo di routing i messaggi RIP vengono incapsulati all'interno di datagrammi utente UDP, che a loro volta sono incapsulati in datagrammi IP. In altre parole, RIP lavora a livello di applicazione ma crea delle tabelle d'inoltro per IP (che si trova a livello di rete). Poiché RIP viene implementato come un processo a livello di applicazione, può inviare e ricevere messaggi su una socket standard e utilizzare un protocollo di trasporto standard.

Open Shortest Path First (OSPF)

Il termine *open* nell'acronimo indica che le specifiche del protocollo sono pubblicamente disponibili.

OSPF è un protocollo *link-state* che utilizza il *flooding* (inondazione) di informazioni riguardo lo stato dei collegamenti e l'algoritmo di Dijkstra per la determinazione del percorso a costo minimo. In OSPF, un router costruisce una mappa topologica, cioè un grafo, dell'intero sistema autonomo e manda in esecuzione (locale) l'algoritmo di Dijkstra per determinare un albero dei percorsi minimi verso tutte le sottoreti (albero in cui il router stesso rappresenta il nodo radice). I costi dei collegamenti vengono fissati dall'amministratore di rete.

In OSPF, ogni qualvolta che si verifica un cambiamento nello stato di un collegamento (per esempio, una variazione di costo o un cambiamento di disponibilità), il router manda informazioni di instradamento via broadcast a tutti gli altri router nel sistema autonomo. Inoltre, invia periodicamente lo stato dei collegamenti (almeno ogni 30 minuti), anche se questo non è cambiato. Gli annunci OSPF sono contenuti in messaggi OSPF che vengono trasportati direttamente da IP come un protocollo di livello superiore con identificativo 89.

OSPF è un protocollo molto complesso che utilizza ben cinque tipi diversi di messaggi.

- Il *messaggio hello* (ciao) viene usato dai router per annunciare la propria esistenza e per indicare tutti gli altri nodi che sono già noti.
- Il *messaggio database description* (descrizione database) viene normalmente inviato in risposta ad un messaggio hello per consentire a un router che si è appena connesso di ottenere l'intero LSDB.
- Il *messaggio link-state request* (richiesta stato del collegamento) viene inviato da un router che ha bisogno di informazioni su uno specifico collegamento.
- Il *messaggio link-state update* (aggiornamento stato del collegamento) è il messaggio principale usato da OSPF ed è essenziale per la costruzione dell'LSDB.
- Il *messaggio di riscontro link-state* (link-state ack) viene usato per rendere affidabile OSPF: ogni router che riceve un messaggio di aggiornamento link-state deve darne riscontro al mittente.

Struttura di Internet

Prima di parlare dei protocolli di routing unicast dobbiamo comprendere la struttura attuale di Internet. La rete è cambiata, passando da una topologia ad albero, con un'unica dorsale (backbone), a una topologia con dorsali multiple gestite da aziende private.

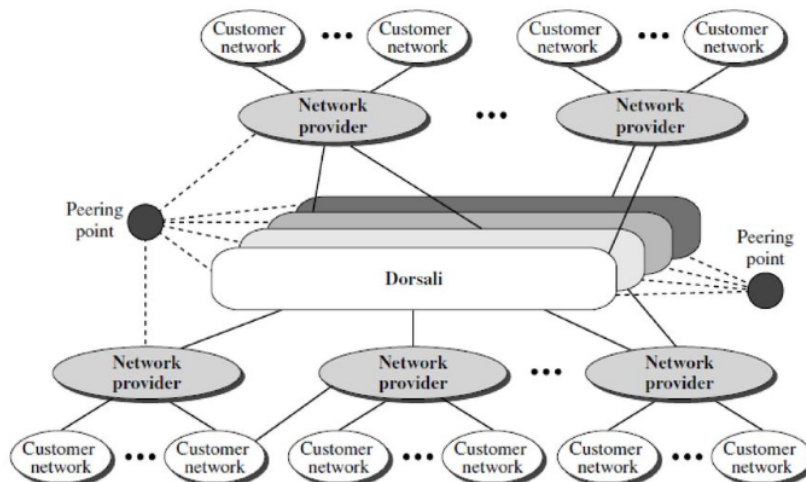


Figura 6.31: Struttura di internet

Ci sono numerose dorsali, gestite da società private di telecomunicazioni, che forniscono la connettività globale. Tali dorsali sono connesse tramite alcuni peering point che consentono la connettività tra dorsali. A livello inferiore, ci sono dei network provider (fornitori di rete) che utilizzano le dorsali per avere connettività globale ma principalmente forniscono servizi di connettività a clienti Internet. Infine ci sono le customer network (reti dei clienti) che usano i servizi forniti dai network provider. Tutte queste entità (dorsali, network provider o customer network) possono essere definite Internet Service Provider (ISP). In tutti e tre i casi forniscono dei servizi, ma a livelli differenti.

Nel nostro studio degli algoritmi di instradamento abbiamo visto la rete semplicemente come una collezione di router interconnessi. Ciascun router era indistinguibile dagli altri nel senso che tutti eseguivano lo stesso algoritmo per calcolare l'instradamento attraverso la rete. In pratica, questo modello con la sua visione omogenea dei router risulta un po' semplicistico per almeno due importanti motivi.

- *Scalabilità.* Al crescere del numero di router, il tempo richiesto per calcolare, memorizzare e comunicare le informazioni di instradamento diventa proibitivo. Attualmente, Internet è costituita da centinaia di milioni di host. Archiviare le informazioni di instradamento su ciascuno di essi richiederebbe chiaramente un'enorme quantità di memoria, e il traffico generato dalla comunicazione broadcast degli aggiornamenti non lascerebbe banda per i pacchetti di dati. Un algoritmo distance-vector con interazioni tra un così grande numero di router non convergerebbe mai. Ovviamente, si deve fare qualcosa per ridurre la complessità del calcolo dei percorsi nelle reti di grandi dimensioni.
- *Autonomia amministrativa.* Internet è una rete di ISP che generalmente desiderano gestire liberamente i propri router (per esempio, scegliendo l'algoritmo di instradamento preferito) o nascondere all'esterno aspetti dell'organizzazione interna della rete. L'ideale sarebbe che ciascuno fosse in grado di amministrare la propria rete nel modo auspicato, pur mantenendo la possibilità di connetterla alle reti esterne.

Implementare un routing gerarchico significa considerare ogni ISP come un sistema autonomo (Autonomous System, AS). Ogni AS può eseguire un protocollo di routing che soddisfa le sue esigenze. A livello di Internet globale, questo non è possibile: è necessario un protocollo di routing globale in grado di unire assieme tutti gli AS. Il protocollo di routing usato all'interno degli AS viene definito protocollo di routing intra-AS, protocollo di routing intradominio (intradomain routing protocol), o Interior Gateway Protocol (IGP); il protocollo di routing globale viene definito protocollo di routing inter-AS, protocollo di routing interdominio (interdomain routing protocol) o Exterior Gateway Protocol (EGP). Ci sono numerosi protocolli di routing intra-dominio, e ogni AS è libero

di sceglierne uno, ma dovrebbe essere chiaro che dobbiamo avere un solo protocollo inter-dominio che gestisce il routing tra queste entità. Attualmente i due protocolli di routing intra-dominio più comuni sono RIP e OSPF; l'unico protocollo di routing inter-dominio è il BGP.

Come si è detto in precedenza, ogni ISP è un sistema autonomo quando si tratta di gestire le reti ed i router sotto il suo controllo. Anche se possono esserci AS piccoli, di medie dimensioni e grandi, ad ogni AS viene assegnato un numero identificativo (autonomous number, ASN) dall'ICANN. L'ASN è un numero intero senza segno, di 16 bit, che identifica in modo univoco l'AS. I sistemi autonomi, tuttavia, non sono classificati rispetto alla loro dimensione; vengono classificati a seconda del modo in cui sono connessi agli altri AS. Abbiamo degli AS stub (terminali), degli AS multihomed (a collegamenti multipli) e degli AS di transito. Il tipo, come vedremo in seguito, influenza il funzionamento del protocollo di routing inter-dominio rispetto all'AS.

- *AS stub.* Un AS stub ha un solo collegamento verso un altro AS. Il traffico dati può essere generato da o destinato ad un AS stub ma non può accadere che i dati transitino attraverso l'AS. Un buon esempio di AS stub è la rete di una grande azienda che può essere solamente sorgente o destinazione dei dati.
- *AS multihomed.* Un AS multihomed ha più di una connessione con altri AS, ma non consente al traffico dei dati di passare attraverso di esso. Un buon esempio di questo tipo di AS è quello di un'azienda che può usare i servizi di più di un network provider ma come politica impone che gli AS a cui è collegata non possano sfruttare la sua rete per il transito dei loro dati.
- *AS di transito.* Un AS di transito è collegato a vari AS e consente anche il transito del traffico dati. I network provider e le dorsali sono dei buoni esempi di AS di transito.

Border Gateway Protocol (BGP)

Il border gateway protocol, detto BGP, rappresenta l'attuale standard de facto dei protocolli di instradamento tra sistemi autonomi in Internet. BGP è forse il più importante dei protocolli di Internet, al pari di IP, in quanto è la colla che tiene insieme le migliaia di ISP che formano Internet. E' un protocollo di tipo distance-vector decentralizzato e asincrono.

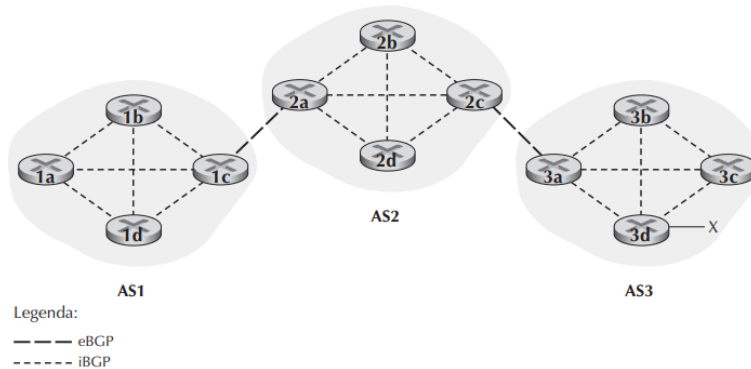
BGP mette a disposizione di ciascun router un modo per:

1. Ottenere informazioni sulla raggiungibilità dei prefissi di sottorete da parte dei sistemi confinanti. In particolare, BGP consente a ciascuna sottorete di comunicare la propria esistenza al resto di Internet. Basta che una sottorete urla "Esisto, son qui" e BGP si assicura che lo sappiano tutti i router di Internet. Se non fosse per BGP, ogni sottorete sarebbe isolata, sola, sconosciuta e irraggiungibile dal resto di Internet.
2. Determinare i percorsi "ottimi" verso le sottoreti. Un router può venire a conoscenza di più cammini verso uno specifico prefisso; per determinare il migliore, esegue localmente BGP, sulla base delle informazioni di raggiungibilità e delle politiche del sistema.

Per permettere ad ogni router di instradare correttamente i pacchetti, qualsiasi sia la destinazione, è necessario installare su tutti i router di confine (border router) dell'AS una variante del BGP chiamata BGP esterno (external BGP, eBGP). Tutti i router (non solamente quelli di confine) dovranno invece usare la seconda variante del BGP, chiamata BGP interno (internal BGP, iBGP). Questo significa che i router di confine devono eseguire ben tre protocolli d'instradamento (intra-dominio, eBGP e iBGP) e che tutti gli altri router ne eseguono due (intra-dominio e iBGP).

Esempio 6.3.6 – eBGP e iBGP

Si consideri la rete in figura con tre AS: AS1, AS2 e AS3, quest'ultimo con una sottorete di prefisso x .



Ogni router, in ogni AS, funge sia da router gateway, vale a dire un router di bordo direttamente connesso a uno o più router in altri AS, sia da router interno, connesso cioè solo a host e router interni all'AS. In AS1, per esempio, il router 1c è un router gateway, mentre 1a, 1b e 1d sono router interni.

Esaminiamo ora come BGP distribuirebbe le informazioni di raggiungibilità del prefisso x a tutti i router. In linea di principio è semplice. AS3 invia un messaggio BGP ad AS2, con l'annuncio dell'esistenza in AS3 di x; denotiamo tale messaggio con "AS3 x". Quindi AS2 invia un messaggio BGP a AS1 con l'annuncio che x esiste ed è raggiungibile passando prima da AS2 per poi arrivare a AS3: "AS2 AS3 x". In questo modo ogni AS verrà a conoscenza non solo dell'esistenza di x ma anche dei percorsi per raggiungerlo.

Benché questa sia l'idea generale, in realtà sono i router e non gli AS a inviare gli annunci. In BGP, coppie di router si scambiano informazioni di instradamento su connessioni TCP semi-permanenti usando la porta 179. Ogni connessione TCP, con tutti i messaggi BGP che vi vengono inviati, è detta sessione BGP. Nel caso in cui questa coinvolga due sistemi autonomi viene detta sessione BGP esterna (sessione eBGP), mentre quella tra router dello stesso sistema autonomo è chiamata sessione BGP interna (sessione iBGP). Generalmente, in BGP c'è una connessione TCP di questo tipo per ciascun collegamento che connette direttamente due router in due diversi sistemi autonomi; nella figura vediamo una connessione sessione eBGP tra i gateway 2a e 1c e tra i router 2c e 3a.

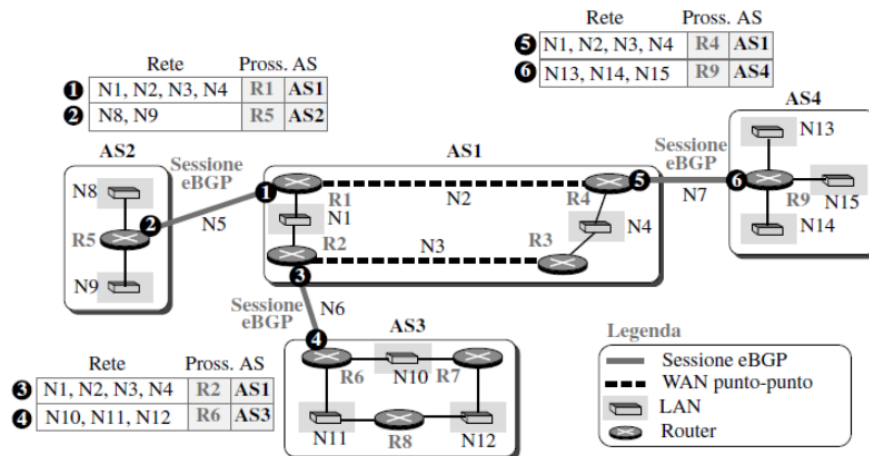
Esaminiamo ora come BGP distribuirebbe le informazioni di raggiungibilità del prefisso x ad AS1 e AS2. In router gateway 3a invia un messaggio eBGP "AS3 x" al router gateway 2c che a sua volta lo invia su una sessione iBGP a tutti i router di AS2 compreso il gateway 2a. Quest'ultimo quindi invia un messaggio eBGP "AS2 AS3 x" al router gateway 1c, che infine usa iBGP per inviare il messaggio "AS2 AS3 x" a tutti i router di AS1. Completato tale processo, ogni router di AS1 e AS2 è a conoscenza dell'esistenza di x e del percorso per raggiungerlo. Ovviamente in una rete reale esistono più percorsi da un router a una data destinazione che attraversano AS diversi.

Il BGP è un protocollo sostanzialmente punto-punto. Quando il software BGP viene installato in due router, essi cercano di creare una connessione TCP utilizzando la porta nota 179. In altre parole si forma una coppia client/server che continua a scambiarsi messaggi. I due router che eseguono i processi BGP vengono chiamati peer BGP oppure speaker BGP.

La variante eBGP del BGP permette a due router di confine che si trovano in due diversi AS di formare coppie di speaker eBGP e di scambiarsi messaggi. Ogni connessione logica nella terminologia BGP viene definita sessione.

Esempio 6.3.7 – Funzionamento di eBGP

Nel nostro esempio ci sono tre sessioni, come mostrato in figura.



La figura mostra anche i messaggi di aggiornamento (in versione molto semplificata) inviati dai router coinvolti nelle sessioni eBGP. Caso per caso, il numero cerchiato indica il router mittente del messaggio. Ad esempio, il messaggio numero 1 viene inviato dal router R1 e dice al router R5 che N1, N2, N3 ed N4 sono reti che possono essere raggiunte tramite il router R1 (R1 trae questa informazione dalla sua tabella d'inoltro intra-dominio). Il Router R5 ora può aggiungere queste informazioni in coda alla sua tabella d'inoltro. Quando R5 riceve un qualsiasi pacchetto destinato ad una di queste quattro reti può usare la sua tabella d'inoltro e scoprire che il prossimo router a cui inviare il pacchetto è R1.

E' facile notare che i messaggi scambiati durante le tre sessioni eBGP servono per indicare ad alcuni router come instradare i pacchetti destinati ad alcune reti nella internet, ma le informazioni di raggiungibilità non sono complete. Ci sono due problemi che devono ancora essere affrontati.

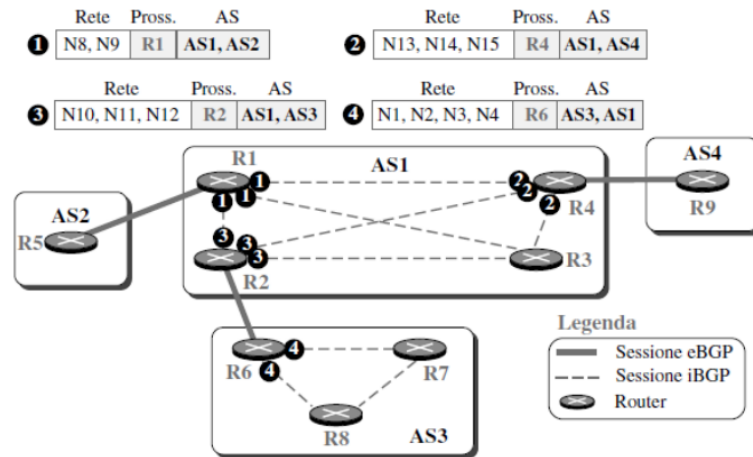
1. Alcuni router di confine non sanno come instradare un pacchetto destinato ad un AS che non è un vicino. Ad esempio, R5 non sa come instradare i pacchetti destinati alle reti in AS3 e AS4. I router R6 ed R9 si trovano nella stessa situazione di R5, infatti R6 non sa nulla delle reti in AS4 e R9 non sa nulla delle reti in AS3.
2. Nessuno dei router non di confine sa come instradare un pacchetto destinato alle reti che si trovano in altri AS.

Per risolvere questi due problemi dobbiamo vedere l'uso della seconda variante del protocollo BGP, l'iBGP, che a differenza dell'eBGP viene eseguito tra tutte le coppie di router (di confine e non).

Il protocollo iBGP è simile al protocollo eBGP per il fatto che utilizza anch'esso connessioni TCP su porta nota 179. Ma a differenza di eBGP crea una sessione tra ogni possibile coppia di router all'interno di un AS. Tuttavia vanno chiariti alcuni punti. Per prima cosa, se un AS ha un solo router ovviamente non può esserci alcuna sessione iBGP. Infatti, nella nostra rete d'esempio, non possiamo creare una sessione iBGP all'interno di AS2 o AS4. In secondo luogo, se ci sono n router in un AS, saranno generate $[n(n-1)/2]$ sessioni iBGP interne all'AS.

Esempio 6.3.8 – Combinazione di sessioni eBGP e iBGP

Viene mostrata la combinazione delle sessioni eBGP e iBGP nell'esempio.



Da notare che non è stato necessario mostrare in figura le reti fisiche all'interno degli AS visto che le sessioni BGP sfruttano normali connessioni TCP. Inoltre, è interessante notare anche che in questa fase vengono scambiati solo quattro messaggi. Il primo messaggio (numero 1) viene inviato da R1 e rende noto che le reti N8 e N9 sono raggiungibili tramite il percorso AS1-AS2, e il router successivo è R1. Il messaggio viene inviato, tramite sessioni separate, a R2, R3 ed R4. I router R2, R4 e R6 fanno la stessa cosa ma inviano messaggi diversi a destinazioni diverse. Il punto interessante è che, in questa fase, R3, R7 ed R8 creano delle sessioni con i loro peer, ma in realtà non hanno alcun messaggio da inviare.

Il processo d'aggiornamento non termina qui. Per esempio, dopo che R1 ha ricevuto il messaggio d'aggiornamento da R2, esso combina le informazioni circa la raggiungibilità di AS3 con quelle che già conosceva relativamente ad AS1 ed invia un nuovo messaggio d'aggiornamento a R5. Ora R5 sa come raggiungere le reti in AS1 e AS3. Il processo continua quando R1 riceve il messaggio d'aggiornamento da R4.

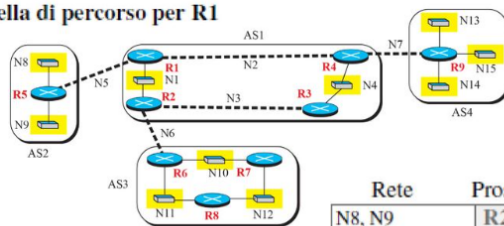
A un certo punto è necessario che non vi siano più modifiche agli aggiornamenti ricevuti in precedenza e che tutte le informazioni siano state propagate attraverso tutti gli AS. Ora ogni router può combinare le informazioni ricevute dall'eBGP e dall'iBGP e creare una tabella dei percorsi, ma solo aver applicato il criterio scelto per trovare il percorso migliore.

Esempio 6.3.9 – Tabelle di percorso

Vengono mostrate le tabelle di percorso per i router dell'esempio.

Rete	Pross.	Percorso
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Tabella di percorso per R1



Rete	Pross.	Percorso
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Tabella di percorso per R2

Rete	Pross.	Percorso
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

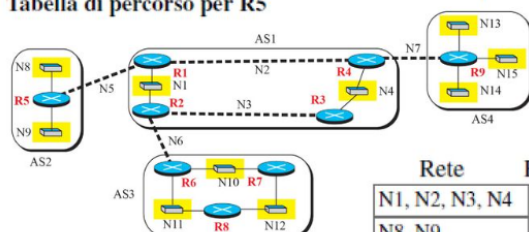
Tabella di percorso per R5

Rete	Pross.	Percorso
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Tabella di percorso per R3

Rete	Pross.	Percorso
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Tabella di percorso per R4



Rete	Pross.	Percorso
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Tabella di percorso per R6

Per esempio il router R1 ora sa che tutti i pacchetti destinati alle reti N8 o N9 dovranno passare attraverso AS1 e AS2 e il router successivo a cui inoltrare il pacchetto è il router R5. Allo stesso modo, il router R4 sa che ogni pacchetto destinato alle reti N10, N11 ed N12 deve passare tramite AS1 e AS3 e il router successivo a cui inoltrare questo pacchetto è il router R1, ecc.

Il ruolo di un protocollo d'instradamento inter-dominio come il BGP è quello di permettere ai router all'interno dell'AS di incrementare le loro informazioni di routing. In altre parole, le tabelle di percorso ottenute ed organizzate da BPG non vengono usate, di per sé, per l'instradamento dei pacchetti. Bensì sono inserite nelle tabelle d'inoltro intra-dominio (generate RIP - OSPF). Ciò può essere fatto in molti modi, a seconda del tipo di AS.

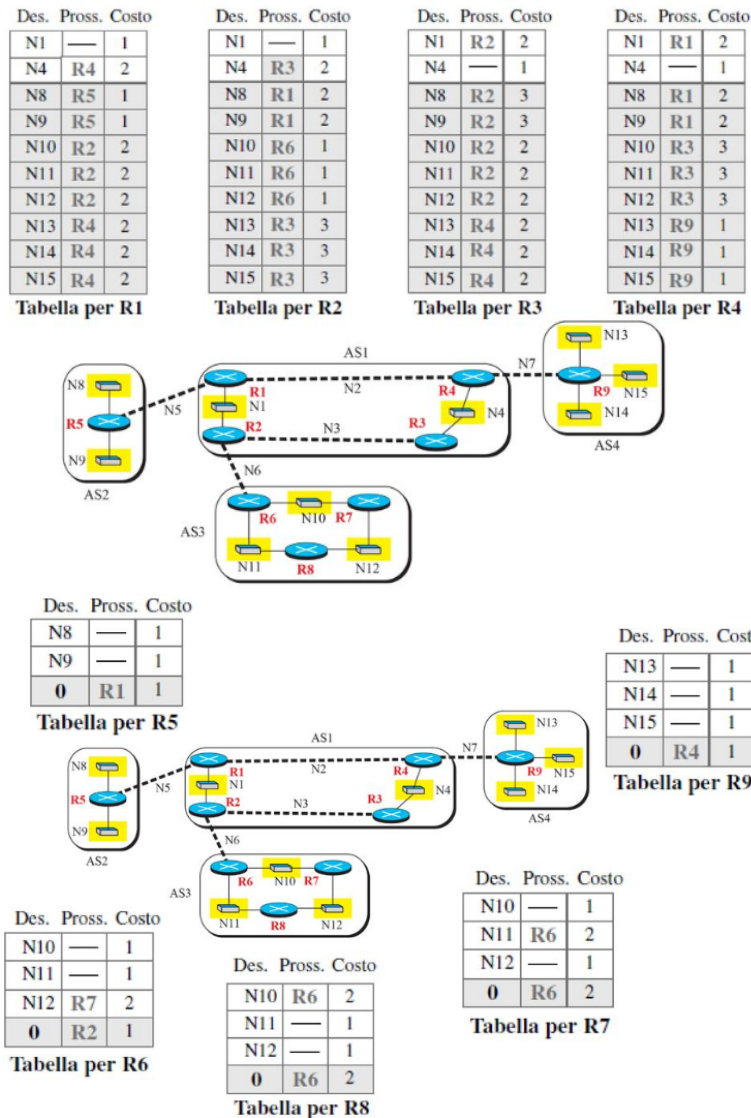
Nel caso di AS stub, l'unico router di confine dell'area aggiunge una regola di default alla fine della sua tabella d'inoltro e definisce come prossimo router quello (detto speaker) che si trova dall'altro lato della connessione eBGP. Nell'Esempio 6.3.2, R5 in AS2 definisce l'R1 come router di default per tutte le reti diverse da N8 ed N9. La situazione è la stessa per il router R9 in AS4 con il router di default impostato su R4. In AS3, R6 imposta R2 come suo router di default, mentre R7 e R8 impostano R6. Queste impostazioni avvengono in conformità con le tabelle di percorso descritte nell'Esempio 6.3.2 per tali router. In altre parole le tabelle di percorso vengono inserite nelle tabelle d'inoltro intra-dominio aggiungendo una sola ulteriore regola di default.

Nel caso di un AS di transito la situazione è più complicata. R1 in AS1 deve inserire l'intero contenuto della tabella di percorso per R1 (Esempio 6.3.2) nella sua tabella d'inoltro intra-dominio. La situazione è la stessa per R2, R3 ed R4.

Un problema da risolvere è il valore da associare al costo. Sappiamo che RIP e OSPF usano metriche differenti. Una soluzione molto comune è quella di impostare il costo di tutte le reti esterne allo stesso costo necessario per raggiungere il primo AS nel percorso. Ad esempio, il costo di R5 per raggiungere tutte le reti negli altri AS è pari al costo per raggiungere N5. Il costo di R1 per raggiungere le reti N10, N11 e N12 è il costo per raggiungere N6, e così via. Quindi, in questo caso, il costo viene preso direttamente dalle tabelle d'inoltro intra-dominio.

Esempio 6.3.10 – Tabelle d'inoltro inter-dominio

Vengono mostrate le tabelle d'inoltro inter-dominio.



Per semplicità assumiamo che tutti gli AS stiano utilizzando il RIP come protocollo di routing intra-dominio. Le aree ombreggiate sono le informazioni inserite dal protocollo BGP, le regole di default hanno il valore 0 come destinazione.

Quando un router annuncia un prefisso per una sessione BGP, include anche un certo numero di attributi BGP. In gergo, un prefisso assieme ai suoi attributi è detto rotta (route). Due dei più importanti attributi sono AS-PATH e NEXT-HOP.

AS-PATH elenca i sistemi autonomi attraverso i quali è passato l'annuncio del prefisso. Quando un prefisso attraversa un sistema autonomo, questi aggiunge il proprio ASN all'attributo AS-PATH. I router usano tale attributo per rilevare ed evitare gli annunci reiterati. Nello specifico, se un router vede che il proprio AS è contenuto nella lista di percorsi, rifiuta l'annuncio.

Nel fornire il delicato collegamento tra i protocolli di instradamento intra-AS e inter-AS, l'attributo NEXT-HOP ha un ruolo sottile, ma importante. In NEXT-HOP è riportata l'interfaccia del router che inizia l'AS-PATH.

L'input del processo di selezione è l'insieme di tutte le rotte apprese e accettate dal router. Se ne esistono due o più verso lo stesso prefisso, BGP invoca in sequenza le seguenti regole di eliminazione fino a individuare un'unica possibilità.

1. Alle rotte viene assegnato come attributo un valore di preferenza locale, che potrebbe essere stato impostato direttamente dal router o appreso da un altro router nello stesso AS. Si tratta di

una scelta che è lasciata all'amministratore di rete del sistema autonomo. Si selezionano quindi le rotte con i più alti valori di preferenza locale.

2. Tra le rotte con lo stesso valore di preferenza locale, si seleziona quella con AS-PATH più breve. Se questa fosse l'unica regola di selezione, allora BGP utilizzerebbe un algoritmo Bellman Ford per la determinazione del percorso, con metrica sulla distanza data dal numero di hop tra sistemi autonomi anziché il numero di hop tra router.
3. Tra le rotte con lo stesso valore di preferenza locale e la stessa lunghezza AS-PATH si seleziona quella il cui router di NEXT-HOP è più vicino. In questo caso, con "router più vicino" s'intende quello che presenta il percorso con costo minore, determinato dall'algoritmo intra-AS (instradamento a patata bollente).
4. Se rimane ancora più di una rotta, il router si basa sugli identificatori BGP.

Il protocollo BGP usa quattro tipi di messaggi per la comunicazione tra speaker, sia attraverso gli AS che all'interno di un AS: open (apertura), update (aggiornamento), keepalive (mantenimento in vita) e notification (notifica). Tutti i pacchetti BGP condividono la stessa intestazione comune.

- *Messaggio open.* Per collegarsi ai router vicini, un router BGP apre una connessione TCP con un vicino ed invia un messaggio di questo tipo.
- *Messaggio update.* Si tratta del cuore del protocollo BGP. I messaggi di update sono usati dai router per rimuovere le destinazioni che erano state precedentemente annunciate, per annunciare il percorso verso una nuova destinazione, o entrambe le cose. Da notare che in BGP, un singolo messaggio di update può rimuovere numerose destinazioni già annunciate in precedenza ma può annunciare solo una nuova destinazione (o destinazioni multiple con gli stessi attributi di percorso).
- *Messaggio keepalive.* I peer BGP si scambiano, a intervalli regolari di tempo, dei messaggi per verificare che il peer dall'altro lato della connessione sia ancora presente e funzionante.
- *Messaggio notification.* Un messaggio di notifica viene inviato da un router ogni volta che una condizione d'errore viene rilevata oppure se un router desidera chiudere la sessione.

6.3.3 Routing multicast

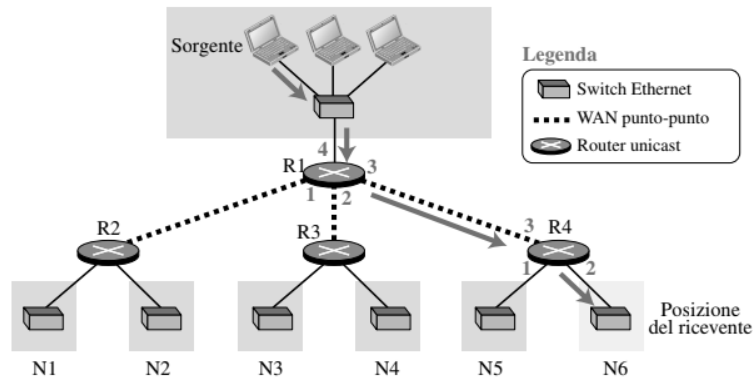
Nelle sezioni precedenti abbiamo appreso che l'inoltro di un datagramma da parte di un router è normalmente basato sul prefisso dell'indirizzo di destinazione presente nel datagramma, che definisce la rete alla quale l'host di destinazione è collegato. Una volta compreso il principio d'inoltro, possiamo ora definire più in dettaglio cosa intendiamo per unicast, multicast e broadcast. Analizziamo questi termini e il modo in cui riferiscono ad Internet.

Unicast

Nell'unicast è presente una rete sorgente e una di destinazione. Il rapporto tra la rete sorgente e quella di destinazione è uno a uno. Ogni router lungo il percorso del datagramma lo inoltra attraverso una sola delle sue interfacce.

Esempio 6.3.11 – Unicast

La figura mostra una rete di piccole dimensioni dove un pacchetto unicast deve essere consegnato da un host d'origine a un host di destinazione collegato allo switch Ethernet chiamato N6.



Il router R1 è responsabile dell'inoltro del datagramma attraverso l'interfaccia 3 e il router R4 deve inoltrare il datagramma attraverso l'interfaccia 2. Quando il datagramma arriva a N6, la consegna all'host di destinazione è di responsabilità della rete. Infatti, il datagramma può essere inviato in broadcast (a tutti gli host della rete) oppure lo switch Ethernet può consegnarlo al solo host di destinazione.

Broadcast

L'invio broadcast implica una comunicazione uno-a-tutti: quando un nodo trasmette un frame, il canale lo diffonde e tutti gli altri nodi ne ricevono una copia. Broadcast può essere eseguito come:

- *Uncontrolled flooding*: quando un nodo riceve un pacchetto broadcast, lo duplica e lo invia a tutti i nodi vicini (eccetto a quello da cui lo ha ricevuto). Se il grafo ha cicli, una o più copie del pacchetto cicleranno all'infinito nella rete.
- *Controlled flooding*: a sua volta diviso in Sequence number e Reverse path forwarding.

Con il sequence number non vengono forwardati i pacchetti già ricevuti e inoltrati. Ogni nodo tiene una lista di (*indirizzoIP, #seq*) dei pacchetti già ricevuti, duplicati, inoltrati. Quando riceve un pacchetto controlla nella lista, se già inoltrato lo scarta, altrimenti lo forwarda.

Con il reverse path forwarding (RPF) viene forwardato il pacchetto solo se è arrivato dal link che è sul suo shortest path (unicast) verso la sorgente. RPF elimina il problema di inondare la rete con troppi pacchetti ma non elimina completamente la trasmissione di pacchetti ridondanti. Come soluzione si può costruire lo spanning tree prima di inviare i pacchetti broadcast: si prende un nodo come centro (*E*); ogni nodo invia un messaggio di join in unicast verso il centro; i messaggi vengono inoltrati finché arrivano a un nodo che già appartiene all'albero o alla radice. I pacchetti vengono inoltrati solo sui link dell'albero, e ogni nodo riceve solo una copia del pacchetto.

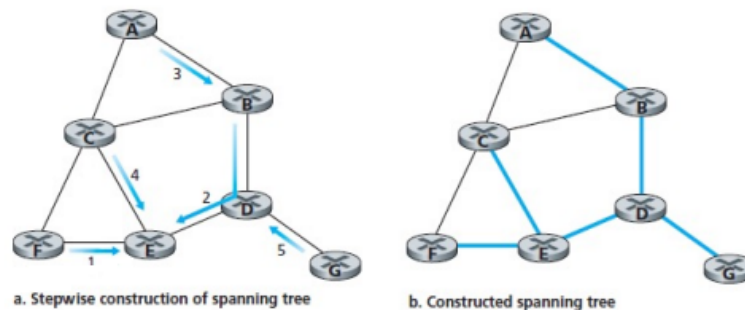


Figura 6.32: Broadcast sullo spanning tree

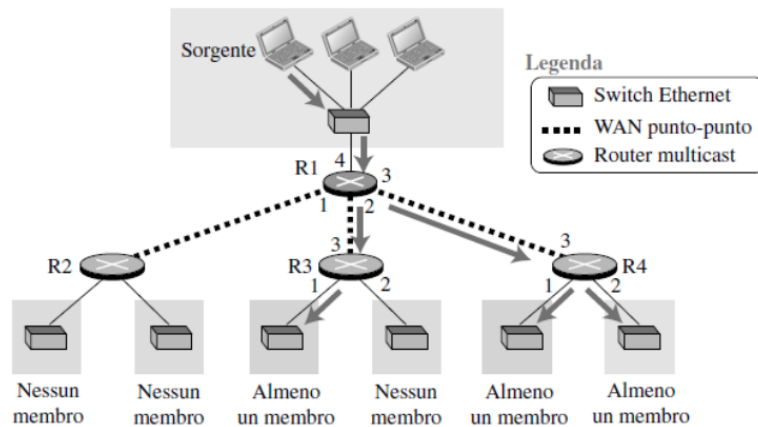
Multicast

Nel multicast è presente una sola sorgente e un gruppo di destinazioni. Il rapporto è uno a molti. In questo tipo di comunicazione, l'indirizzo della sorgente è un indirizzo unicast, mentre l'indirizzo di

destinazione è un indirizzo di gruppo, cioè un gruppo di una o più reti di destinazione nelle quali si trova almeno un host che è interessato a ricevere il datagramma multicast. L'indirizzo di gruppo definisce i membri del gruppo stesso.

Esempio 6.3.12 – Multicast

Nel multicast, un router potrebbe inviare copie multiple dello stesso datagramma attraverso più interfacce di rete.



Nella figura, il router R1 deve inviare il datagramma attraverso le interfacce 2 e 3. Allo stesso modo, il router R4 necessita di inviare il datagramma attraverso tutte le sue interfacce. Il router R3 invece sa che non ci sono membri che appartengono al gruppo multicast nell'area raggiunta dall'interfaccia 2 e quindi invia i datagrammi relativi al gruppo solamente attraverso l'interfaccia 1.

Occorre distinguere tra multicast e unicast multiplo. La Figura 6.33 illustra entrambi i concetti.

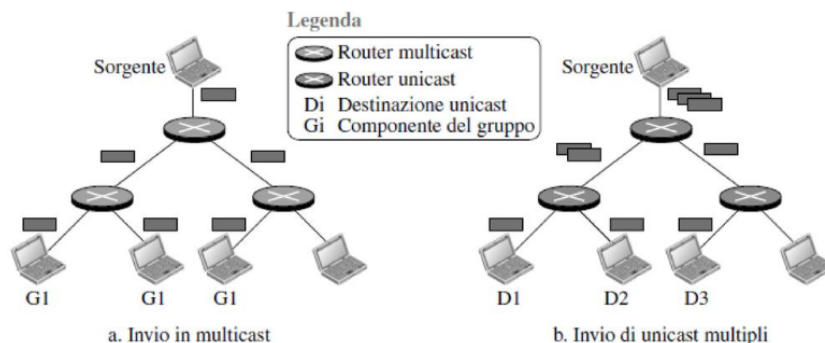


Figura 6.33: Confronto tra multicast e unicast multiplo

Il multicast ha inizio con un singolo datagramma spedito dalla sorgente che viene poi duplicato dai router. L'indirizzo di destinazione presente in ogni datagramma è lo stesso per tutti i duplicati. Da notare che tra ogni coppia di router viaggia una singola copia del datagramma.

Nel caso dell'unicast multiplo invece dalla sorgente partono numerosi datagrammi. Ad esempio, se ci sono tre destinazioni, la sorgente invia tre datagrammi distinti, ognuno con un differente indirizzo unicast di destinazione. In questo caso ci possono essere copie multiple del datagramma che viaggiano tra le coppie di router. Ad esempio, se una persona invia un'email a un gruppo di persone quello che avviene è un unicast multiplo. Il software usato per la spedizione della e-mail crea tante copie del messaggio quanti sono i destinatari e le invia una ad una.

Indirizzi multicast

Quando inviamo un datagramma unicast a una destinazione, l'indirizzo sorgente del datagramma definisce il mittente e l'indirizzo di destinazione definisce il ricevente. Nella comunicazione multicast, il mittente è solo uno, ma i riceventi sono tanti, a volte migliaia o potenzialmente milioni. Dovrebbe essere chiaro che non possiamo includere gli indirizzi di tutti i destinatari nell'intestazione

del datagramma. L'indirizzo di destinazione di un datagramma, come descritto nell'Internet Protocol (IP), deve essere uno solo. Per questa ragione servono degli specifici indirizzi multicast. Un indirizzo multicast definisce un gruppo di destinatari. In altre parole, un indirizzo multicast è l'identificatore di un gruppo. Quando viene formato un nuovo gruppo, composto da vari membri attivi, un'apposita autorità assegnerà un indirizzo multicast inutilizzato a questo nuovo gruppo, indirizzo che servirà per identificare in modo univoco il gruppo. Questo significa che l'indirizzo sorgente di un datagramma in una comunicazione multicast può essere un indirizzo unicast (che individua in modo univoco il mittente), ma l'indirizzo di destinazione può essere un indirizzo multicast (che individua un gruppo). In questo modo, un host che è membro di n gruppi ha in realtà $(n + 1)$ indirizzi: il suo indirizzo unicast che lo identifica come sorgente o destinazione nelle comunicazioni unicast ed n indirizzi multicast che sono usati solo come indirizzi di destinazione per l'invio di datagrammi a ciascun gruppo.

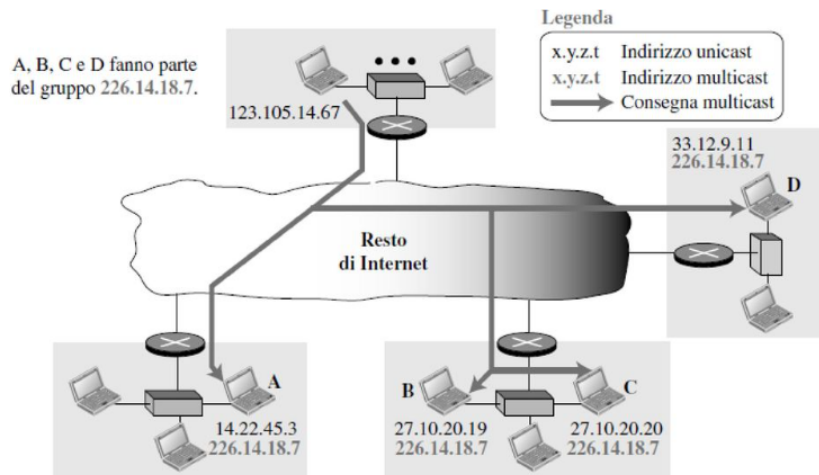


Figura 6.34: Esempio di utilizzo degli indirizzi multicast

Un router o un host di destinazione hanno la necessità di distinguere tra datagrammi unicast e multicast. IPv4 riserva un blocco di indirizzi esclusivamente per il multicast. Nell'indirizzamento con classi tutta la classe D era composta da indirizzi multicast. L'indirizzamento senza classi utilizza lo stesso blocco di indirizzi a cui ora si fa riferimento come 224.0.0.0/4 (da 224.0.0.0 a 239.255.255.255).

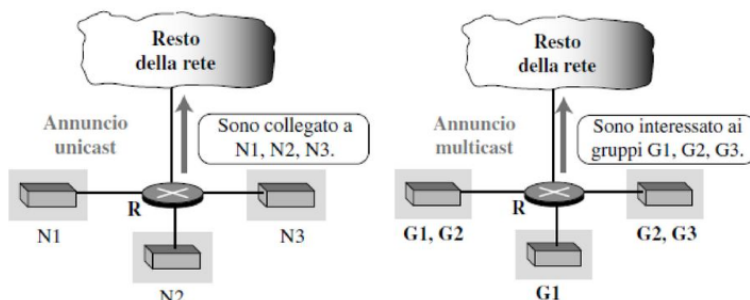


Figura 6.35: Un indirizzo multicast in notazione binaria

I primi quattro bit definiscono il blocco, tutti i successivi invece sono a disposizione per identificare gruppi multicast. E' bene notare come il numero di gruppi che possono essere definiti è molto elevato (225).

Gruppi multicast

I router non sono a conoscenza dell'appartenenza degli host ai vari gruppi multicast. L'appartenenza ad un gruppo infatti non ha alcuna relazione con il prefisso associato alla rete. L'appartenenza ad un gruppo multicast porta l'host ad avere un indirizzo multicast separato ed aggiuntivo rispetto a quello primario. In secondo luogo, l'appartenenza non è un attributo fisso dell'host; ciascun host può entrare a far parte di nuovi gruppi e lasciarne altri, anche in un periodo di tempo molto breve. Per questa ragione, un router ha bisogno di aiuto per scoprire quali gruppi sono presenti in ciascuna delle sue interfacce. Dopo aver raccolto queste informazioni, il router può propagare l'appartenenza al gruppo ad ogni altro router che faccia uso di un protocollo di routing multicast.

Esempio 6.3.13 – Annuncio unicast rispetto ad annuncio multicast

Nel caso dell'unicast, il router R sa che gli host con i prefissi N1, N2 e N3 sono connessi alle sue interfacce; esso propaga quindi queste informazioni al resto della rete. Nel caso del multicast, il router R necessita di conoscere se ciascun host fa parte di qualche gruppo multicast (G1, G2 o G3).

In altre parole, nel routing unicast è sufficiente avere un protocollo d'instradamento per diffondere le informazioni relative ai vari collegamenti. Invece, nel caso del multicast occorrono due protocolli: il primo per raccogliere le informazioni di appartenenza ai gruppi e il secondo per diffonderle.

Internet Group Management Protocol (IGMP)

Il protocollo che oggi viene usato per raccogliere informazioni relative all'appartenenza a gruppi è l'Internet Group Management Protocol (IGMP). L'IGMP è un protocollo di livello rete e così come l'ICMP si tratta anche in questo caso di un protocollo ausiliario, che viene considerato parte integrante di IP. Lavora tra un host e il router che gli è direttamente connesso e offre agli host il mezzo di informare i router ad essi connessi del fatto che un'applicazione in esecuzione vuole aderire ad uno specifico gruppo multicast. I messaggi IGMP, così come quelli ICMP, vengono incapsulati in datagrammi IP.

- **Membership query.** Periodicamente tutti gli host collegati ad un router vengono contattati per mezzo di un messaggio query. Il messaggio chiede a ciascun host di indicare il suo stato di appartenenza a gruppi multicast.
- **Membership report.** Viene inviato da un host al suo router, come risposta ad un messaggio query, per informare il router su un'adesione, anche non inseguita a una query (al momento dell'adesione). Il messaggio contiene una lista di record, in ciascun record l'host fornisce l'identificatore di un gruppo (il suo indirizzo multicast) e la lista di tutte le fonti da cui l'host è interessato a ricevere messaggi di gruppo (versione inclusiva) oppure la lista delle fonti da cui non è interessato a ricevere messaggi di gruppo (versione esclusiva).
- **Leave group.** Viene inviato da host al suo router quando si lascia un gruppo. Il leave group è opzionale: il router può capire che non ci sono più host associati a un gruppo quando non riceve report in risposta a query.

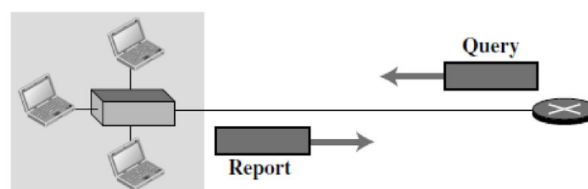


Figura 6.36: Funzionamento di IGMP

Se un host desidera entrare a far parte di gruppo aspetterà fino a quando non riceve un messaggio query per poi inviare la sua richiesta nel messaggio report di risposta. Se invece un host intende lasciare il gruppo, esso semplicemente non risponde al messaggio query. Nel caso nessun altro host risponda al messaggio corrispondente, il gruppo verrà eliminato dal database del router.

Un router multicast tiene una lista per ciascuna sottorete dei gruppi multicast con un timer per membership. La membership deve essere aggiornata da report inviati prima della scadenza del timer, può essere anche aggiornata tramite messaggi di leave espliciti.

Instradamento multicast

Nell'instradamento multicast, come così come in quello unicast, occorre creare tabelle di routing per instradare in modo ottimale i pacchetti dalla sorgente alla loro destinazione. Tuttavia, come abbiamo detto in precedenza, le decisioni sull'instradamento multicast di ciascun router dipendono non solo dalla destinazione del pacchetto ma anche dalla sorgente del pacchetto stesso. Il coinvolgimento della sorgente nel processo di instradamento rende il routing multicast molto più complesso rispetto a quello unicast. Fra la popolazione complessiva di router solo alcuni (quelli collegati a host del gruppo multicast) dovranno ricevere traffico multicast. E' necessario un protocollo che coordini i router multicast in Internet (instradare pacchetti multicast dalla sorgente alla destinazione). A grandi linee sono stati sviluppati due differenti approcci per l'instradamento multicast: quello definito Source-based Tree e quello chiamato Group-shared Tree.

- *Approccio Source-based Tree.* Nell'approccio Source-based Tree ciascun router crea un albero diverso per ogni combinazione di sorgente e gruppo. In altre parole, se in una rete ci sono m gruppi ed n sorgenti, un router necessita di creare $(m \times n)$ alberi d'instradamento. In ciascun albero, la sorgente è la radice, i membri del gruppo sono le foglie e il router stesso si trova da qualche parte nell'albero.
- *Approccio Group-shared Tree.* Nell'approccio Group-shared Tree si designa un router che finge di essere la sorgente di ciascun gruppo. Il router designato, che viene chiamato core o rendezvous-point router, agisce come rappresentante del gruppo. Qualsiasi sorgente che abbia un pacchetto da inviare a un membro di questo gruppo, lo invia al core router (usando unicast) e il core router è poi responsabile del multicast per gli altri componenti del gruppo. Il core router crea un singolo albero d'instradamento con se stesso come radice e gli altri router come foglie (purché abbiano host attivi come componenti del gruppo). In questo approccio, ci sono m core router (uno per ogni gruppo) e ogni core router costruisce un albero d'instradamento, per un totale di m alberi. Questo significa che il numero degli alberi d'instradamento viene ridotto da $(m \times n)$ dell'approccio Source-based Tree a m .

6.4 IPv6

L'esaurimento degli indirizzi e alcune limitazioni di IPv4, già nei primi anni '90, hanno portato allo sviluppo di una nuova versione del protocollo. Questa nuova versione, chiamata Internet Protocol versione 6 (IPv6) o IP di nuova generazione (IP new generation, IPng) è nata con lo scopo di aumentare lo spazio degli indirizzi rispetto a IPv4, ridisegnare il formato dei datagrammi IP e allo stesso tempo rivedere anche alcuni protocolli ausiliari come ICMP. Quanto segue mostra i principali cambiamenti nel protocollo IPv6.

- *Spazio degli indirizzi di dimensione maggiore.* Un indirizzo IPv6 è lungo 128 bit. Paragonato ai 32 bit dell'IPv4, è un enorme aumento (2% volte) dello spazio degli indirizzi.
- *Miglior formato dell'intestazione.* IPv6 utilizza un nuovo formato per l'intestazione (header) dei datagrammi. Nell'intestazione IPv6 le opzioni sono separate dall'intestazione di base e inserite, quando necessario, tra l'intestazione di base e i dati. Ciò semplifica e velocizza il processo di routing visto che la maggior parte delle opzioni non è necessario che sia controllata dai router.
- *Nuove opzioni.* IPv6 supporta nuove opzioni per consentire l'implementazione di funzionalità aggiuntive.
- *Possibilità di estensione.* L'IPv6 è progettato per consentire l'estensione del protocollo se richiesto dall'introduzione di nuove tecnologie o applicazioni.
- *Supporto per l'allocazione delle risorse.* In IPv6 è stato rimosso il campo tipo del servizio (Type Of Service, TOS) che è presente in IPv4. Al suo posto sono stati aggiunti due nuovi campi, classe di traffico ed etichetta di flusso, per consentire alla sorgente di richiedere un trattamento

speciale per il datagramma. Il meccanismo, se fosse supportato dai router, potrebbe migliorare la gestione del traffico audio o video in tempo reale.

- *Maggiore sicurezza.* Le opzioni relative a crittografia ed autenticazione permettono di migliorare la riservatezza e verificare l'integrità dei datagrammi.

L'adozione di IPv6 è stata lenta. Il motivo principale che ha portato al suo sviluppo, cioè il progressivo esaurimento degli indirizzi IPv4, è stato rallentato grazie a tre rimedi a breve termine: l'indirizzamento senza classi, l'uso del protocollo DHCP e soprattutto la diffusione del NAT. Il fatto che si tratti di soluzioni a breve termine, che non risolvono a fondo il problema, e l'uso sempre più ampio di nuovi servizi (per esempio, l'uso del protocollo IP in mobilità, la telefonia basata su IP ecc.) prima o poi richiederanno la sostituzione totale di IPv4 con IPv6. In passato si prevedeva che questa migrazione sarebbe avvenuta entro il 2010, cosa che in effetti non è successa. Le previsioni attuali parlano invece del 2020, ma probabilmente si andrà ben oltre.

6.4.1 Formato dei datagrammi IPv6

Ciascun datagramma è composto da un'intestazione di base (base header) seguita dai dati (payload). L'intestazione di base occupa 40 byte, mentre il payload può arrivare fino a 65.535 byte.

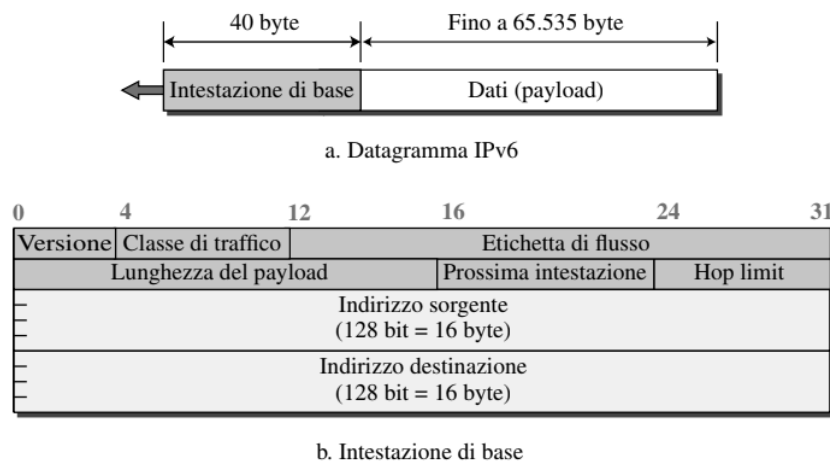


Figura 6.37: Formato dei datagrammi IPv6

- *Versione (version).* Il campo versione (4 bit) definisce il numero di versione del protocollo IP. Per l'IPv6, il valore è 6.
- *Classe di traffico (traffic class).* Il campo classe di traffico (8 bit) è utilizzato per distinguere il tipo di servizio di trasferimento a seconda della tipologia di dati contenuti nel payload. Esso sostituisce il campo tipo di servizio (Type Of Service, TOS) che è presente in IPv4.
- *Etichetta di flusso (flow label).* L'etichetta di flusso (20 bit) dovrebbe permettere una nuova gestione dei dati sotto forma di flusso di datagrammi, rispetto ai singoli datagrammi indipendenti come accade in IPv4.
- *Lunghezza del payload (payload length).* Il campo lunghezza del payload (2 byte) definisce la lunghezza del datagramma IP escludendo l'intestazione. Si noti che in IPv4 ci sono due campi legati dalla lunghezza: lunghezza dell'intestazione e lunghezza totale. Invece in IPv6, la lunghezza dell'intestazione di base è fissa (40 byte), quindi è necessario indicare solamente la lunghezza del payload.
- *Prossima intestazione (next header).* La prossima intestazione è un campo (8 bit) che definisce il tipo della prima intestazione estesa (extension header), se presente, o il tipo dei dati che seguono l'intestazione di base all'interno del payload. Questo campo è simile al campo "protocollo" che è presente in IPv4.
- *Hop limit (numero massimo di salti).* Il campo hop limit ha la stessa funzione del campo TTL in IPv4.

- *Indirizzo sorgente e indirizzo destinazione (source and destination address)*. I campi indirizzo sorgente e destinazione sono indirizzi Internet da 16 byte (128 bit) che identificano la sorgente originale del datagramma e la sua destinazione.
- *Dati (payload)*. Se confrontato con IPv4, il campo payload in IPv6 ha una forma e un significato differenti. Il campo dati in IPv6 può contenere una combinazione di zero o più intestazioni estese (usate per rappresentare le varie opzioni) seguite dai dati provenienti da altri protocolli (UDP, TCP e così via). In IPv6, le opzioni, che fanno parte dell'intestazione in IPv4, sono implementate sotto forma di intestazioni estese. Il campo dati può contenere tante intestazioni quante ne sono necessarie. Ogni intestazione è formata da due campi obbligatori, intestazione successiva (next header) e lunghezza (length), seguite dalle informazioni specifiche della varie opzioni. Si noti l'uso che viene fatto del campo "prossima intestazione" (next header); definisce il tipo della prossima intestazione quando si riferisce a un'opzione oppure identifica il protocollo incapsulato nel payload del datagramma.

6.4.2 Passaggio da IPv4 a IPv6

Anche se è pronta una nuova versione del protocollo IP, rimane da capire come può essere fatto il passaggio dalla versione corrente a quella nuova. La prima soluzione che viene in mente, la più banale, è definire un giorno di passaggio nel quale tutti gli host e router smetteranno di usare IPv4 ed inizieranno ad usare solo IPv6. Ovviamente la transizione non può essere fatta in questo modo. A causa dell'enorme numero di sistemi in rete la migrazione non può avvenire improvvisamente. Ci sarà bisogno di una quantità di tempo considerevole prima che tutti i sistemi siano aggiornati. La transizione dovrà essere studiata accuratamente per evitare problemi tra il vecchio sistema e quello nuovo. L'IETF ha studiato tre strategie che possono essere implementate in questa fase: il dual stack (doppia pila di protocolli), il tunneling e la traduzione dell'intestazione (header translation).

Dual stack

La strategia prevede che tutti gli host, durante la transizione, abbiano una doppia pila di protocolli per la comunicazione in rete. In altre parole, ciascun host (o router) deve eseguire IPv4 e IPv6 simultaneamente fino a quando tutta la rete non sarà passata ad IPv6.

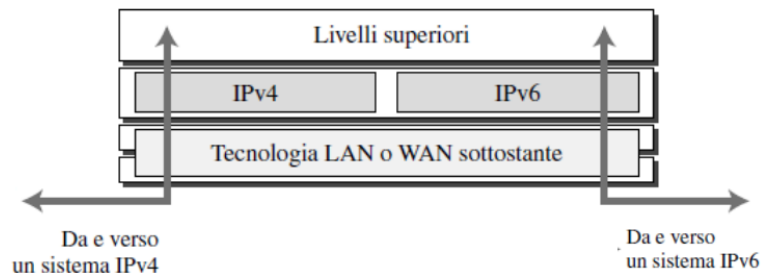


Figura 6.38: Dual stack

Per determinare quale versione utilizzare quando si invia un pacchetto a una destinazione, l'host sorgente interroga il DNS. Se il DNS restituisce un indirizzo IPv4 allora l'host invia un pacchetto IPv4. Se il DNS restituisce un indirizzo IPv6 allora l'host invia un pacchetto IPv6.

Tunneling

Il tunneling è una tecnica utilizzata quando due host che usano IPv6 vogliono comunicare tra loro ma i datagrammi devono attraversare una regione della rete che usa solo IPv4. Per passare attraverso questa regione i datagrammi devono necessariamente avere un indirizzo IPv4. La soluzione è quella di incapsulare ciascun datagramma IPv6 nel payload di un datagramma IPv4. Una volta superata la tratta IPv4, il datagramma IPv6 potrà essere nuovamente estratto. E' come se i datagrammi IPv6 viaggiassero all'interno di un tunnel tra due isole IPv6. Per rendere esplicito il fatto che il datagramma IPv4 sta trasportando un datagramma IPv6, il valore del protocollo nell'intestazione del datagramma IPv4 viene impostato a 41.

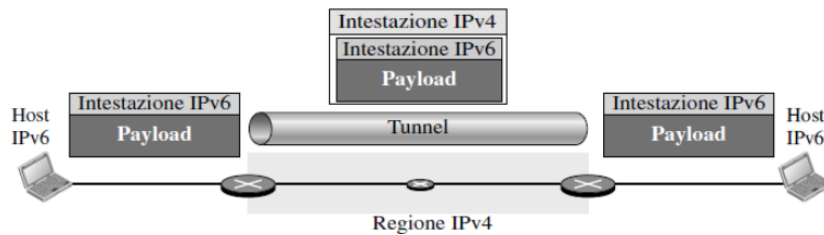


Figura 6.39: Strategia di tunneling

Traduzione dell'intestazione

La traduzione dell'intestazione (header translation) sarà necessaria quando la maggior parte di Internet sarà migrata ad IPv6, con alcuni sistemi residui ancora basati su IPv4. Supponiamo che il mittente intenda (o debba) usare IPv6, ma che il ricevente non sia in grado di supportarlo. In questo caso il meccanismo di tunneling non funziona. L'unica alternativa è quella di effettuare una traduzione completa dell'intestazione del datagramma, in modo di convertirla da IPv6 a IPv4, prima che il datagramma arrivi a destinazione.

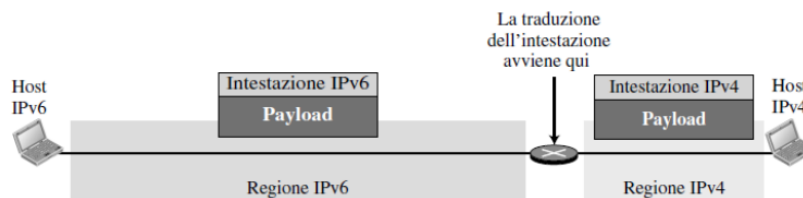


Figura 6.40: Strategia di traduzione dell'intestazione

Capitolo 7

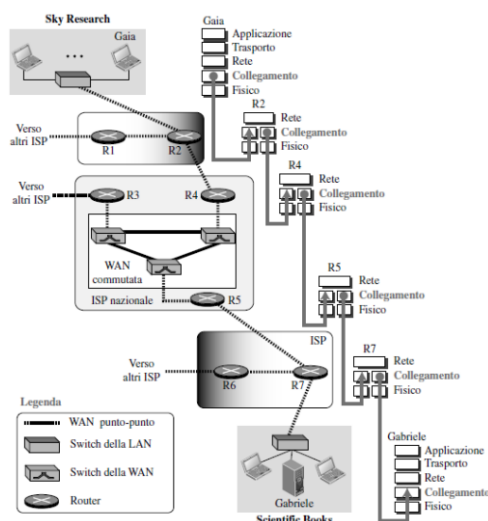
Livello di collegamento

7.1 Introduzione

Nel capitolo precedente si è visto che la comunicazione a livello di rete avviene tra host. Un datagramma è un'unità dati che può essere frammentata e riasssemblata ma che in generale viene inviata da un host, che si trova da qualche parte nel mondo, ad un altro host, che si trova in qualche altra zona. Internet non è altro che una combinazione di reti unite assieme da dispositivi di collegamento (per esempio, router e switch). Se un datagramma deve viaggiare da un host ad un altro, deve quindi passare necessariamente attraverso tali reti.

Esempio 7.1.1 – Comunicazione a livello di collegamento

La figura mostra la comunicazione tra Gaia e Gabriele. Tuttavia, in questo caso, la comunicazione a livello di collegamento è composta da cinque connessioni logiche separate tra i diversi livelli di collegamento lungo il percorso tra i due host. Il livello di collegamento sul computer di Gaia comunica con quello sul router R2. Il livello di collegamento sul router R2 comunica con quello sul router R4, e così via. Infine, il livello di collegamento sul router R7 comunica con quello del computer di Gabriele. Solo un livello di collegamento è coinvolto alla sorgente e alla destinazione, a differenza dei router dove ne sono coinvolti due. Questo avviene poiché i computer di Gaia e Gabriele sono collegati ciascuno ad una sola rete, i router invece ricevono da una rete e trasmettono su un'altra.



Anche se la comunicazione nei livelli di applicazione, trasporto e rete è end-to-end (cioè avviene, dal punto di vista logico, dall'host iniziale a quello finale), quella a livello di collegamento è invece da nodo a nodo. Come si è visto nei capitoli precedenti, un'unità di dati (cioè un pacchetto) trasmessa da un certo punto della rete per arrivare fino alla sua destinazione deve attraversare molte altre reti (LAN e WAN) collegate per mezzo di router. E' consuetudine riferirsi ai router e agli host alle due estremità come a nodi e alle reti nel mezzo come a collegamenti. Nella Figura 7.1 è possibile trovare la rappresentazione di un semplice percorso formato da soli sei nodi.

Il primo nodo è l'host sorgente e l'ultimo è l'host di destinazione. Gli altri quattro nodi intermedi sono dei router. Il primo, il terzo e il quinto collegamento rappresentano tre LAN, il secondo e il quarto due WAN.

I nodi all'interno delle reti sono fisicamente collegati da un mezzo trasmissivo, come un cavo o l'aria. Compito del livello di collegamento è controllare come viene usato tale mezzo. Possiamo avere un livello di collegamento che utilizza l'intera capacità di un mezzo oppure solo parte della sua capacità. In altre parole, possiamo avere un collegamento punto-punto o un collegamento broadcast. In un collegamento punto-punto, il collegamento è dedicato a due soli dispositivi (mittente e

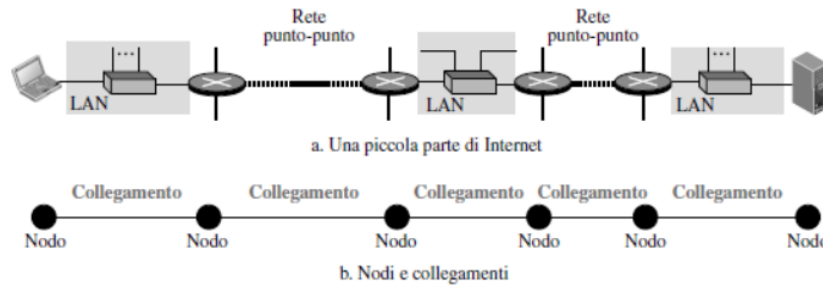


Figura 7.1: Nodi e collegamenti

ricevente). Viceversa, in un collegamento broadcast, il collegamento è condiviso tra varie coppie di dispositivi. Per esempio, quando due amici usano il telefono di casa per chiacchierare, stanno usando un collegamento punto-punto. Quando gli stessi due amici usano i loro telefoni cellulari, stanno usando un collegamento broadcast (visto che l'aria viene condivisa tra vari utilizzatori di cellulari).

Per comprendere meglio le funzionalità ed i servizi forniti dal livello di collegamento possiamo dividere questo livello in due sotto-livelli: il Data-Link Control (DLC) ed il controllo dell'accesso al mezzo trasmissivo (Media Access Control, MAC). Il sotto-livello di DLC si occupa di tutte le questioni comuni sia ai collegamenti punto-punto che a quelli broadcast. Il sotto-livello MAC si occupa invece solo degli aspetti specifici dei canali broadcast.

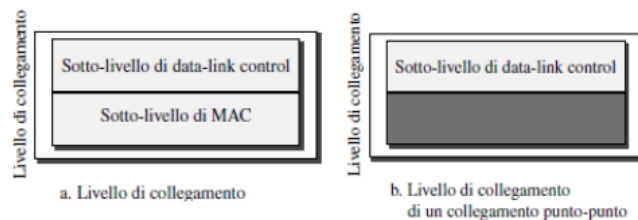


Figura 7.2: Divisione del livello di collegamento in due sotto-livelli

7.1.1 Servizi offerti dal livello di collegamento

Sebbene il servizio di base del livello di collegamento sia il trasporto di datagrammi da un nodo a quello adiacente lungo un singolo canale di comunicazione, i dettagli dei servizi forniti possono variare da un protocollo all'altro. I possibili servizi che possono essere offerti dai protocolli a livello di collegamento includono i seguenti.

- *Framing*. Quasi tutti i protocolli incapsulano i datagrammi del livello di rete all'interno di un frame a livello di collegamento, prima di trasmetterlo. I frame sono costituiti da un campo dati, nel quale è inserito il datagramma, e da vari campi di intestazione. La struttura del frame è specificata dal protocollo.
- *Accesso al collegamento*. Un protocollo che controlla l'accesso al mezzo trasmissivo (MAC, medium access control) specifica le regole con cui immettere i frame nel collegamento.
- *Consegna affidabile*. I protocolli a livello di collegamento che forniscono un servizio di consegna affidabile garantiscono il trasporto senza errori di ciascun datagramma. Abbiamo già visto che anche alcuni protocolli di trasporto (come TCP) forniscono un servizio di consegna affidabile. Analogamente a quello del livello di trasporto, anche il servizio di consegna affidabile del livello di collegamento può essere realizzato attraverso acknowledgment e ritrasmissioni. Il servizio di consegna affidabile è spesso utilizzato per i collegamenti soggetti a elevati tassi di errore (per esempio i collegamenti wireless), allo scopo di correggere l'errore localmente, sul collegamento dove è avvenuto l'errore, piuttosto che costringere i protocolli di trasporto o di applicazione a procedere alla ritrasmissione dei dati dalla sorgente alla destinazione. Tuttavia, la consegna affidabile del livello di collegamento può essere considerata non necessaria nei collegamenti che presentano un basso numero di errori sui bit, come nel caso di collegamenti con

fibra ottica, cavo coassiale e doppino, ragion per cui molti protocolli del livello di collegamento non implementano questo servizio.

- *Controllo di flusso.* Così come avviene a livello trasporto, il controllo di flusso coordina la quantità di dati che può essere inviata prima di ricevere un riscontro. I concetti di fondo del controllo di flusso a livello di collegamento sono gli stessi del livello trasporto.
- *Rilevazione e correzione degli errori.* Il nodo ricevente può decidere, erroneamente, che un bit in un frame sia 0 quando questo era stato trasmesso come 1, e viceversa. Gli errori di bit sono causati dall'attenuazione di segnale e dai disturbi elettromagnetici. Siccome non è utile inoltrare i datagrammi contenenti errori, molti protocolli del livello di collegamento forniscono un meccanismo per rilevarne la presenza. Ciò è possibile grazie all'inserimento, da parte del nodo trasmittente, di bit di controllo di errore all'interno del frame e all'esecuzione di un semplice controllo da parte del nodo ricevente. Anche il livello di trasporto e di rete di Internet fornisce una seppur limitata rilevazione degli errori: il checksum (Internet checksum). Il rilevamento degli errori a livello di collegamento è però solitamente più sofisticato, in quanto implementato in hardware. La correzione dell'errore è simile alla rilevazione degli errori, ma in più il nodo ricevente determina il punto preciso del frame in cui si è verificato l'errore (per poi correggerlo).

Ogni volta che i bit fluiscono da un punto ad un altro sono soggetti a cambiamenti imprevedibili, a causa delle interferenze. Queste interferenze possono cambiare la forma del segnale e quindi alterare la ricezione dei bit. Il termine errore su un singolo bit (single-bit error) indica che un solo bit in una determinata unità di dati (ad esempio: un byte, un carattere o un pacchetto) è stato cambiato da 1 a 0 o viceversa. Il termine errore a raffica (burst error) indica che 2 o più bit nell'unità di dati sono stati cambiati di valore.

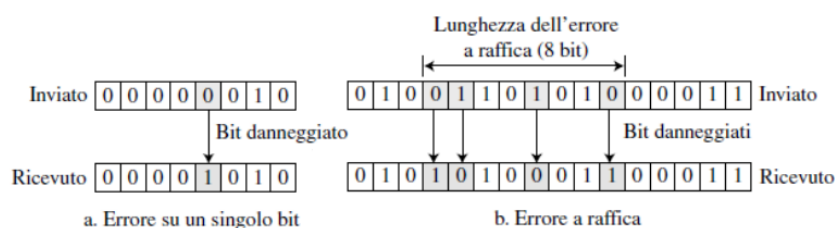


Figura 7.3: Errore su un singolo bit ed errore a raffica

La probabilità che avvenga un errore a raffica è più elevata rispetto a quella di un errore su un singolo bit, in quanto la durata dell'interferenza (detta anche rumore) normalmente è più lunga rispetto a quella di un solo bit. Questo significa che quando il rumore influisce sui dati, esso coinvolge un insieme di bit. Il numero di bit coinvolti dipende dalla velocità di trasferimento dei dati e dalla durata del rumore. Per esempio, se stiamo inviando dei dati a 1 kilobyte al secondo, un rumore di 1/100 di secondo può influire su 10 bit. Se invece stiamo inviando i dati a 1 Mbps, lo stesso rumore può influire su 10.000 bit.

7.1.2 Dov'è implementato il livello di collegamento?

Il livello di collegamento di un host è implementato in hardware o in software? Su una scheda o un chip separati? Come si interfaccia con le altre parti dell'hardware dell'host e con il suo sistema operativo?

Per un dato collegamento, il protocollo del livello di collegamento è sostanzialmente realizzato da un adattatore di rete (network adapter), noto anche come scheda di rete (NIC, network interface card). Il cuore della scheda di rete è il controller a livello di collegamento (link layer controller), che è di solito un chip dedicato, che implementa molti dei servizi a livello di collegamento (framing, accesso al collegamento, rilevazione degli errori). La maggior parte, quindi, delle funzionalità del controller è implementata in hardware. Fino agli anni '90, molti adattatori di rete erano su schede fisicamente separate ma ora sono sempre più spesso integrati sulla scheda madre del computer.

Lato mittente, il controller prende un datagramma, creato e memorizzato nella memoria dell'host dai livelli più alti della pila di protocolli, lo incapsula in un frame a livello di collegamento riempiendone i vari campi dell'intestazione, e lo trasmette sul canale di comunicazione, seguendo il protocollo

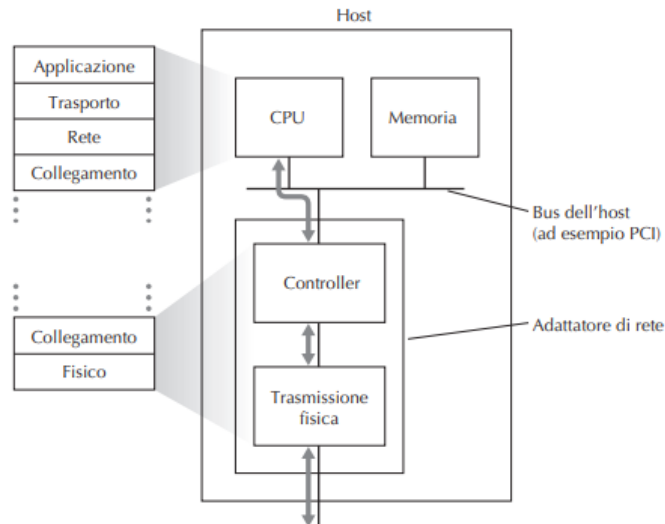


Figura 7.4: Adattatore di rete

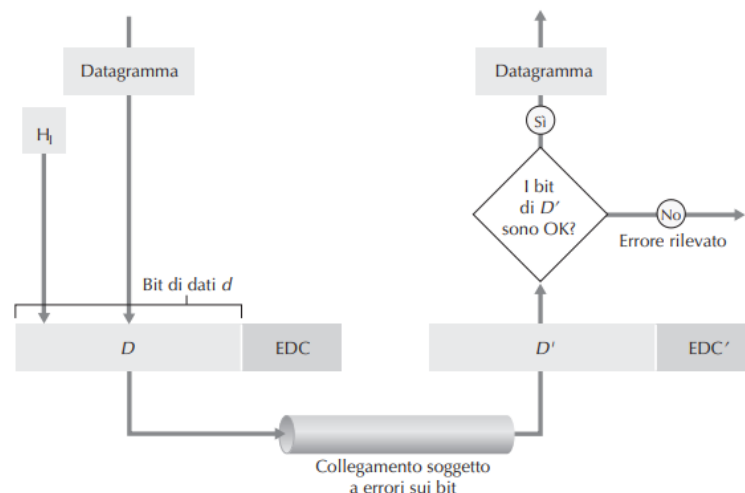
di accesso al canale. Dall'altro lato, un controller riceve l'intero frame, estrae il datagramma e lo consegna al livello di rete. Se il protocollo del livello di collegamento fornisce il servizio di rilevazione degli errori, allora è il controller trasmittente a impostare i bit di rilevazione degli errori ed è quello ricevente che esegue il controllo.

Lato ricevente, il software del livello di collegamento risponde agli interrupt del controller (per esempio, dovuti alla ricezione di uno o più frame), effettua la gestione di condizioni di errore e il passaggio del datagramma fino al livello di rete. Il livello di collegamento, quindi, è una combinazione di hardware e software: il luogo dove, nella pila di protocolli, il software incontra l'hardware.

7.1.3 Tecniche di rilevazione e correzione degli errori

Esempio 7.1.2 – Scenario di rilevazione e correzione degli errori

Nello scenario di riferimento, al nodo trasmittente, ai dati D che devono essere protetti da errori vengono aggiunti dei bit detti EDC (error detection and correction).



Generalmente, i dati che devono essere protetti non includono soltanto datagrammi trasferiti verso il basso dal livello di rete per la trasmissione, ma anche le informazioni relative agli indirizzi del livello di collegamento, i numeri di sequenza e altri campi nell'intestazione del frame. I dati D e i bit EDC sono inviati in un frame al nodo ricevente. Questo legge una sequenza di bit D' ed EDC' che può essere diversa dall'originale, come risultato della modifica dei bit in transito. Il nodo ricevente deve determinare se D' coincida con D , potendo contare soltanto su D' e su EDC'.

Le attuali tecniche non sempre consentono al nodo ricevente di rilevare se si sono verificati errori nei bit. Anche con l'utilizzo dei bit di rilevazione degli errori è possibile che ci siano degli errori non rilevati; vale a dire che il nodo ricevente potrebbe non accorgersi che le informazioni ricevute contengono errori. Di conseguenza, il ricevente potrebbe consegnare un datagramma errato al livello di rete o ignorare che il contenuto di alcuni campi nell'intestazione del frame sia stato alterato.

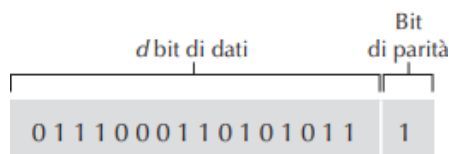
Dovremo quindi optare per uno schema per la rilevazione degli errori che riduca la probabilità di questo evento. Generalmente, le tecniche più sofisticate comportano un'elevata ridondanza, nel senso che sono necessari calcoli più complessi e la trasmissione di molti bit aggiuntivi.

Controllo di parità

La forma più semplice di rilevamento degli errori è quella che utilizza un unico bit di parità (parity bit). Supponiamo che le informazioni da inviare siano costituite da d bit. In uno schema di parità pari, il mittente include un bit addizionale e sceglie il suo valore in modo da rendere pari il numero totale di bit 1 nei $d + 1$ bit trasmessi (l'informazione originale più il bit di parità). Nello schema di parità dispari, il valore del bit di parità è scelto in modo che ci sia un numero dispari di bit 1.

Esempio 7.1.3 – Parità pari a un bit

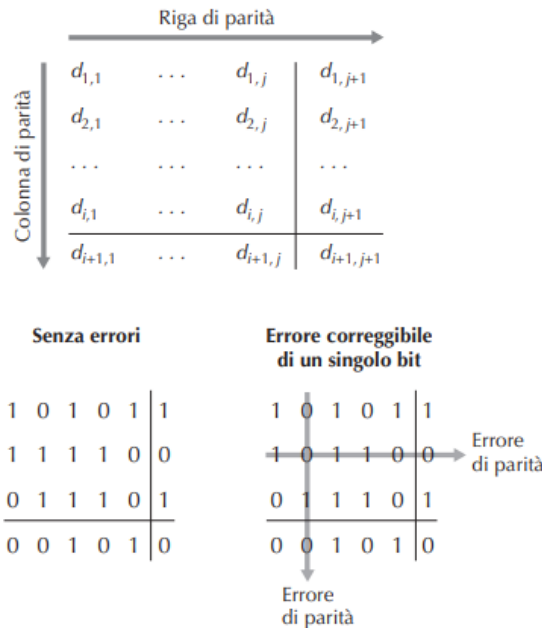
La figura mostra uno schema di parità pari, con un bit di parità inserito in un campo separato.



Con un solo bit di parità ciò che il ricevente deve fare è molto semplice: contare il numero di bit a 1 tra quelli ricevuti. Se trova un numero dispari di bit 1, sa che si è verificato almeno un errore in un bit (più precisamente, sa che si è verificato un numero dispari di errori nei bit). Tuttavia che cosa accade se si è verificato un numero pari di errori nei bit? In questo caso ci troveremmo in presenza di un errore non rilevato. Se la probabilità di errori nei bit è bassa e si può assumere che gli errori siano indipendenti, l'eventualità di errori multipli in un pacchetto è estremamente ridotta e un solo bit di parità potrebbe risultare sufficiente. Tuttavia, è stato statisticamente rilevato che gli errori tendono generalmente a verificarsi a raffiche (burst) piuttosto che in modo indipendente. La probabilità che non vengano rilevati errori a burst in un frame protetto da un solo bit di parità può avvicinarsi al 50%. E' evidente che occorre adottare una strategia più efficiente per la rilevazione degli errori.

Esempio 7.1.4 – Parità pari bidimensionale

La figura illustra una generalizzazione bidimensionale dello schema di parità con un bit.



In questo caso, i d bit del dato D sono suddivisi in i righe e j colonne per ognuna delle quali è stato calcolato un valore di parità. I risultanti $i + j + 1$ bit di parità contengono bit per la rilevazione dell'errore nei frame a livello di collegamento. Supponiamo ora che si verifichi un solo errore nei d bit originali. Con questo schema di parità bidimensionale i bit di parità della colonna e della riga contenenti il bit errato individueranno l'errore. Il ricevente può quindi non solo rilevare che si è verificato un errore, ma può utilizzare gli indici di colonna e di riga per identificare il bit alterato e correggerlo.

La figura mostra un esempio in cui il bit a 1 in posizione (2,2) è "corrotto" ed è diventato 0 (errore rilevabile e correggibile dal ricevente). Nonostante la nostra esposizione sia stata centrata sui d bit originali, anche un errore negli stessi bit di parità è rilevabile e correggibile. Questo schema può rilevare (ma non correggere) qualsiasi combinazione di due errori in un pacchetto.

7.2 Protocolli di accesso multiplo

Nell'introduzione di questo capitolo abbiamo osservato che esistono due tipi di collegamento di rete: punto a punto e broadcast. Il collegamento punto a punto è costituito da un trasmittente a un'estremità del collegamento e da un unico ricevente all'altra. Molti protocolli del livello di collegamento sono stati progettati per questi collegamenti, tra cui PPP (point-to-point protocol) e HDLC (high-level data link control). Il collegamento broadcast può avere più nodi trasmittenti e riceventi connessi allo stesso canale broadcast condiviso. Il termine broadcast indica che, quando un nodo trasmette un frame, il canale lo diffonde e tutti gli altri nodi ne ricevono una copia. Ethernet e Wireless LAN sono esempi di tecnologie con collegamenti broadcast.

Ipotizziamo una festa dove molte persone sono in una stanza (l'aria fornisce il canale broadcast) parlando e ascoltandosi a vicenda. Una seconda analogia è quella di una classe nella quale insegnanti e studenti condividono lo stesso singolo canale broadcast. Il problema principale in entrambi gli scenari descritti è determinare chi e quando debba parlare (cioè trasmettere). I protocolli impiegati sono estremamente vari.

Ecco qualche esempio: "Concedi a ciascuno la possibilità di parlare"; "Non parlare finché non sei interrogato"; "Non monopolizzare la conversazione"; "Alza la mano se devi porre una domanda"; "Non interrompere se qualcuno sta parlando"; "Non addormentarti quando qualcuno parla".

Analogamente, le reti di calcolatori usano protocolli simili – i cosiddetti protocolli di accesso multiplo – che fissano le modalità con cui i nodi regolano le loro trasmissioni sul canale condiviso. I protocolli di accesso multiplo sono richiesti in un'ampia varietà di configurazioni di rete, comprese le LAN cablate, quelle wireless e le reti satellitari. Anche se, tecnicamente, ciascun nodo accede al canale broadcast attraverso la propria scheda di rete, in questo paragrafo useremo il termine nodo per

indicare il dispositivo che trasmette e riceve. In pratica, centinaia o anche migliaia di nodi possono comunicare direttamente su un canale broadcast.

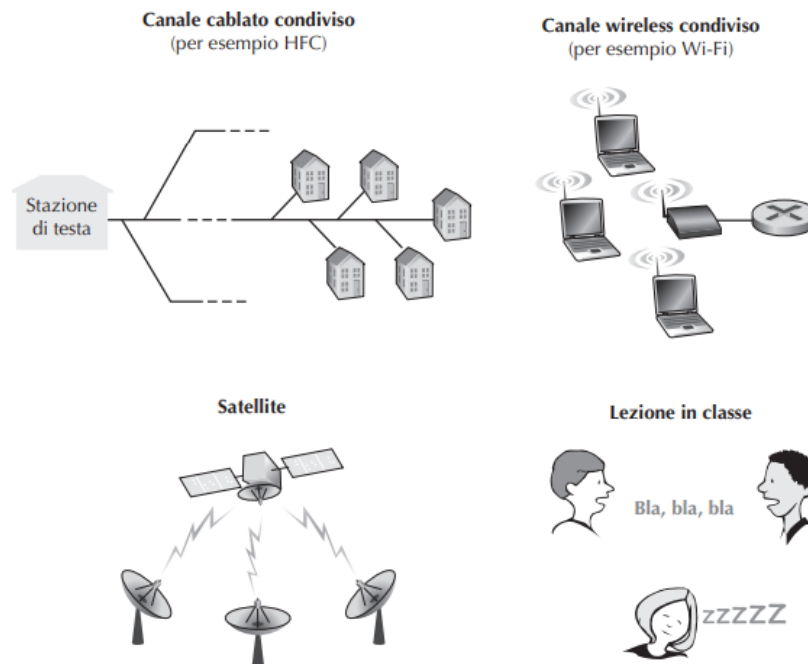


Figura 7.5: Canali ad accesso multiplo

Dato che tutti i nodi sono in grado di trasmettere frame, è possibile che due o più lo facciano nello stesso istante, per cui tutti i nodi riceveranno contemporaneamente più frame. Tra questi si genera una collisione a causa della quale nessuno dei nodi riceventi riuscirà a interpretare i frame che, in un certo senso, risultano ingarbugliati tra loro. Di conseguenza con la collisione si verifica una perdita di frame, mentre il canale rimane inutilizzato. E' chiaro che, se una situazione di questo genere dovesse ripetersi con eccessiva frequenza, gran parte della banda del canale risulterebbe sprecata. Per far sì che il canale broadcast venga utilizzato in modo efficiente, occorre coordinare le trasmissioni dei nodi attivi.

Concludiamo questa panoramica rilevando che, idealmente, un protocollo di accesso multiplo per un canale broadcast con velocità di R bit al secondo dovrebbe avere le seguenti caratteristiche:

1. Quando un solo nodo deve inviare dati, questo dispone di un throughput pari a R bps.
2. Quando M nodi devono inviare dati, questi dispongono di un throughput pari a R/M bps. Ciò non implica necessariamente che ciascuno degli M nodi abbia sempre una velocità istantanea di trasmissione di R/M , ma che ciascun nodo dovrebbe avere una velocità di trasmissione media di R/M in un dato, appropriato intervallo di tempo.
3. Il protocollo è decentralizzato; in altre parole, non ci sono nodi principali che qualora non funzionassero correttamente potrebbero rendere inattivo l'intero sistema.
4. Il protocollo è semplice, in modo che risulti economico da implementare.

In generale possiamo classificare praticamente tutti i protocolli di accesso multiplo in una di queste categorie: protocolli a suddivisione del canale (channel partitioning protocol), protocolli ad accesso casuale (random access protocol) e protocolli a rotazione (taking-turn protocol).

7.2.1 Protocolli a suddivisione del canale

Il multiplexing a divisione di tempo, TDM (time division multiplexing) e quello a divisione di frequenza, FDM (frequency division multiplexing), sono due tecniche che possono essere utilizzate per suddividere la larghezza di banda di un canale broadcast fra i nodi che lo condividono. Per esempio, supponiamo che il canale supporti N nodi e che la sua velocità di trasmissione sia di R bps. TDM

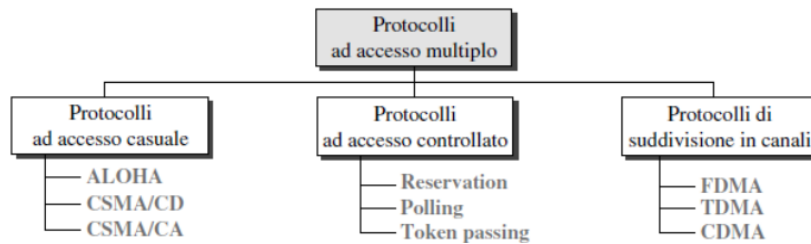


Figura 7.6: Tassonomia dei protocolli ad accesso multiplo

suddivide il tempo in intervalli di tempo (time frame) e poi divide ciascun intervallo di tempo in N slot temporali (time slot). Si noti che il time frame di TDM non deve essere confuso con l'unità di dati a livello di collegamento che le schede di rete si scambiano, che è anch'essa chiamata frame. Per ridurre la confusione, in questo paragrafo ci riferiremo all'unità di dati del livello di collegamento come a un pacchetto. Ogni slot è quindi assegnato a uno degli N nodi. Ogni volta che un nodo ha un pacchetto da inviare, trasmette i bit del pacchetto durante lo slot di tempo assegnatogli. Generalmente, le dimensioni dello slot sono stabilite in modo da consentire la trasmissione di un singolo pacchetto.

Ritornando alla nostra analogia con la festa, se questa è regolata da TDM, consente a un invitato di parlare per un intervallo di tempo stabilito. Lo stesso tempo spetterà quindi a un altro invitato, e così via. Una volta che tutti hanno avuto l'opportunità di parlare, la sequenza si ripete.

TDM viene utilizzato in quanto riesce a evitare le collisioni ed è perfettamente imparziale: ciascun nodo ottiene, durante ciascun intervallo di tempo, un tasso trasmissivo di R/N bps. In ogni caso, TDM presenta due specifici inconvenienti. In effetti anche quando non vi sono altri nodi che devono inviare un pacchetto, quello che vuole trasmettere è vincolato ad attendere il suo turno nella sequenza di trasmissione e a non superare il limite medio di R/N bps. Immaginiamo che ci sia un solo invitato che abbia qualcosa da dire e che questa sia una delle rare circostanze in cui tutti vogliano ascoltare. In questo particolare caso, è evidente che TDM risulta una pessima scelta di protocollo per l'accesso multiplo in questa particolare festa.

Mentre TDM suddivide il canale condiviso in intervalli di tempo, FDM lo divide in frequenze differenti (ciascuna con una larghezza di banda di R/N) e assegna ciascuna frequenza a un nodo. Quindi, a partire da un canale da R bps, FDM crea N canali di R/N bps. FDM condivide alcuni vantaggi e alcuni svantaggi del precedente protocollo. Anche FDM evita le collisioni e divide equamente la larghezza di banda tra gli N nodi. Tuttavia, anche con FDM la larghezza di banda è limitata a R/N , pure quando vi è un solo nodo che ha un pacchetto da spedire.

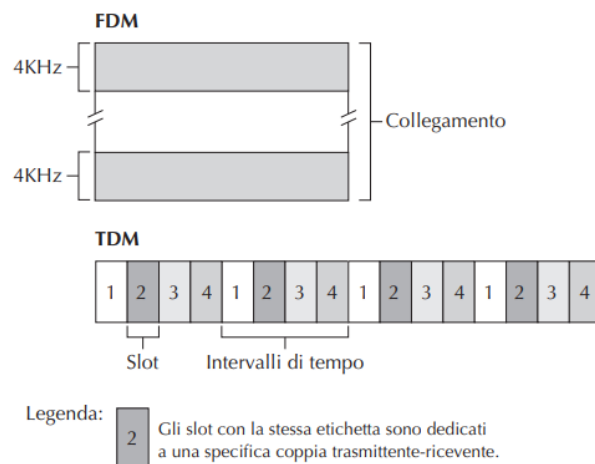


Figura 7.7: Esempi di TDM e FDM con quattro nodi

7.2.2 Protocolli ad accesso casuale

Nell'accesso casuale (random access) nessuna stazione è superiore alle altre e nessuna ha il controllo sulle altre. Ogni volta che una stazione ha dei dati da inviare, usa una procedura definita dal protocollo per decidere se spedire o se attendere. Tale decisione dipende dallo stato del mezzo trasmissivo (inattivo o occupato). In altre parole, ogni stazione può trasmettere quando vuole, a condizione che segua la procedura predefinita, compreso il test sullo stato del mezzo trasmissivo.

Due caratteristiche danno il nome a questo metodo. Primo, non c'è un tempo programmato nel quale la stazione deve trasmettere, la trasmissione tra le stazioni è casuale. Ecco perché questi metodi vengono chiamati ad accesso casuale. Secondo, nessuna regola specifica quale sarà la prossima stazione a trasmettere. Esse competono una con l'altra per accedere al mezzo trasmissivo. Ecco perché questi metodi sono anche chiamati di contesa (le stazioni si contendono il mezzo trasmissivo).

Nei protocolli ad accesso casuale ogni stazione ha il diritto di accedere al mezzo senza essere controllata da nessun'altra stazione. Tuttavia, se più di una stazione cerca di trasmettere in contemporanea, avviene un conflitto d'accesso, una collisione, e i frame trasmessi vengono distrutti o modificati. Per evitare il conflitto d'accesso, o per risolverlo quando avviene, ogni stazione segue una procedura basata sulle seguenti domande: quando la stazione può accedere al mezzo? Cosa può fare la stazione se il mezzo trasmissivo è occupato? Come può la stazione determinare il successo o il fallimento della trasmissione? Cosa può fare la stazione nel caso in cui ci sia stato un conflitto d'accesso?

ALOHA

ALOHA, il primo metodo ad accesso casuale, è stato sviluppato all'Università delle Hawaii nei primi anni 70. E' stato ideato per l'uso in una LAN radio (wireless) ma può essere utilizzato su qualsiasi mezzo condiviso.

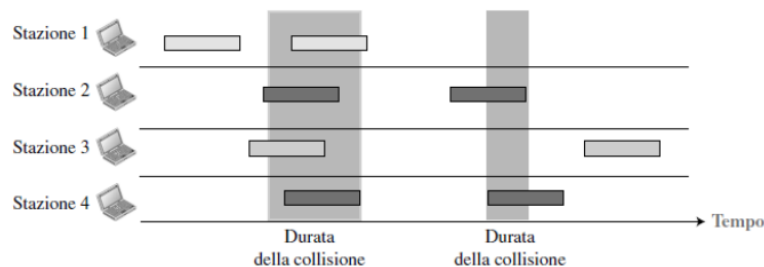
Dato che il mezzo trasmissivo è condiviso tra tutte le stazioni, è ovvio che ci possono essere delle collisioni. Quando una stazione invia dei dati, un'altra può tentare di fare la stessa cosa contemporaneamente. In questo caso avviene una collisione e i dati trasmessi sono alterati.

ALOHA puro

La versione originale del protocollo ALOHA viene chiamata ALOHA puro. E' un protocollo semplice ed elegante. L'idea è che ogni stazione può inviare un frame tutte le volte che ha qualcosa da inviare (accesso multiplo). Tuttavia, siccome il canale da condividere con le altre stazioni è unico, possono avvenire collisioni tra i frame delle diverse stazioni.

Esempio 7.2.1 – ALOHA puro

Nell'esempio ci sono quattro stazioni che si contendono l'accesso al canale condiviso. La figura mostra che ogni stazione invia due frame, quindi in totale nel mezzo condiviso ce ne sono otto. Alcuni dei frame collidono a causa della contesa per il canale condiviso.



La figura mostra che solo due frame sopravvivono: uno della stazione 1 e uno della stazione 3. E' bene notare che è sufficiente che due frame collidano anche solo per un singolo bit per portare alla distruzione entrambi i frame, nella loro totalità. In caso di collisione è necessario inviare nuovamente tutti i frame che sono stati distrutti durante la trasmissione.

Il protocollo ALOHA si basa sull'invio di un riscontro da parte del ricevente. Quando una stazione invia un frame, si aspetta che il ricevente invii un riscontro. Se esso non arriva dopo un certo periodo di tempo (timeout), la stazione assume che il frame (o il riscontro) siano andati distrutti e reagisce trasmettendo nuovamente il frame.

Una collisione coinvolge due o più stazioni. Se tutte queste stazioni tentano di reinviare i loro frame subito dopo il timeout, avverrà immediatamente una nuova collisione. Secondo le specifiche del protocollo ALOHA puro quando il periodo di timeout è trascorso, ogni stazione aspetta un ulteriore periodo di tempo di durata casuale prima di trasmettere nuovamente il suo frame. La casualità aiuterà ad evitare altre collisioni. Questo lasso di tempo viene chiamato tempo di back-off (back-off time) T_B .

L'ALOHA puro prevede un secondo metodo per evitare che i frame ritrasmessi congestionino il canale. Dopo un numero massimo di tentativi di ritrasmissione $K_{\max}$, una stazione deve interrompere i suoi tentativi e provare solamente più tardi.

Il periodo di timeout equivale al massimo ritardo di propagazione di un round-trip (andata e ritorno) tra le stazioni. Quindi è due volte la quantità di tempo necessaria per inviare un frame tra le due stazioni più lontane ($2 \times T_p$). Il tempo di back-off T_B , è un valore scelto casualmente che normalmente dipende da K (il numero di trasmissioni tentate senza successo). La formula usata per determinare T_B dipende dalla specifica implementazione. Una formula comune è il back-off esponenziale binario (binary exponential back-off). In questo approccio, ad ogni ritrasmissione, per trovare T_B , un moltiplicatore R , compreso tra 0 e $(2^K - 1)$, viene scelto in modo casuale e moltiplicato per T_p (il tempo massimo di propagazione) o per T_{fr} , (il tempo medio richiesto per inviare un frame). Con questa procedura l'intervallo in cui scegliere il numero casuale aumenta di dimensione dopo ogni collisione. Il valore di $K_{\max}$ normalmente scelto è 15.

Esempio 7.2.2 – Calcolo back-off

Le stazioni in una rete wireless ALOHA sono a una distanza massima di 600 km. Supponendo che i segnali si propaghino a 3×10^8 m/s, troviamo:

$$T_p = (600 \times 10^3) / (3 \times 10^8) = 2 \text{ ms}$$

Per $K = 2$ l'intervallo di R è $\{0, 1, 2, 3\}$. Ciò significa che T_B può essere 0, 2, 4 o 6 ms, sulla base del risultato della variabile casuale R .

Vediamo ora la lunghezza dell'intervallo, chiamato tempo di vulnerabilità, durante il quale c'è la possibilità che avvengano collisioni. Supponiamo che le stazioni inviino dei frame di lunghezza fissa con ogni frame che impiega T_{fr} secondi per essere inviato.

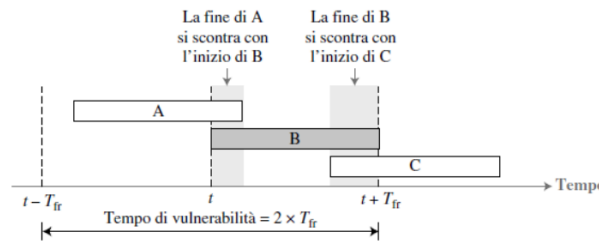


Figura 7.8: Tempo di vulnerabilità nel protocollo ALOHA puro

La stazione B inizia ad inviare un frame al tempo t . Ora, immaginiamo che la stazione A abbia iniziato ad inviare il suo frame dopo $t - T_{fr}$. Ciò porta a una collisione tra i frame della stazione B e quelli della stazione A . Inoltre, supponiamo che la stazione C inizi ad inviare un frame prima del tempo $t + T_{fr}$. Anche in questo caso ci sarà una collisione tra i frame della stazione B e quelli della stazione C .

Guardando la Figura 7.8 vediamo che il tempo di vulnerabilità, durante il quale in ALOHA puro potrebbe avvenire una collisione, è 2 volte il tempo di trasmissione del frame ($2 \times T_{fr}$).

Chiamiamo G il numero medio di frame generati dal sistema durante il tempo di trasmissione di un frame. Si può dimostrare che il numero medio di frame trasmessi con successo per ALOHA puro è $S = G \times e^{-2G}$. Il throughput massimo $S_{\max}$ è di 0,184, per $G = 1/2$. Questo valore può essere ricavato impostando a 0 la derivata di S rispetto a G . In altre parole, se metà di un frame viene generata durante il tempo di trasmissione di uno (un frame dura il tempo di trasmissione di due), allora il 18,4% di tali frame raggiunge la destinazione con successo. Ci si aspetta che $G = 1/2$ produca il throughput massimo in quanto il tempo di vulnerabilità è pari a 2 volte il tempo di trasmissione di un frame.

Quindi, se una stazione genera solo un frame nel suo tempo di vulnerabilità (e nessun'altra stazione ne genera uno in quel tempo), esso raggiungerà la sua destinazione con successo.

L'efficienza di un protocollo di accesso multiplo che fa uso di slot temporali è definita come la frazione di slot riusciti in presenza di un elevato numero di nodi attivi che hanno sempre un elevato numero di pacchetti da spedire. Notiamo che se non venisse utilizzata alcuna forma di controllo dell'accesso e se ciascun nodo ritrasmettesse subito dopo che si è effettuata la collisione, l'efficienza sarebbe zero.

Se un frame va in collisione, allora il nodo lo ritrasmette immediatamente (dopo aver completato la trasmissione del frame) con probabilità p . Altrimenti, attenderà il tempo di trasmissione del frame. Trascorso questo tempo, il nodo ritrasmetterà il frame con probabilità p o aspetterà, restando inattivo per un altro periodo di tempo, con probabilità $1-p$. Per determinare l'efficienza massima di ALOHA puro, analizziamo un singolo nodo. A ogni dato istante, la probabilità che un nodo stia trasmettendo è p . Supponiamo che la trasmissione di un frame inizi al tempo t_0 . Affinché questo frame sia trasmesso con esito positivo nessun altro nodo può cominciare la sua trasmissione nell'intervallo di tempo $(t_0-1, t_0]$, in quanto si sovrapporrebbe con l'inizio della trasmissione del frame del nodo i . La probabilità che tutti gli altri nodi non diano inizio a una trasmissione in questo intervallo è $(1-p)^{N-1}$. Analogamente, nessun altro nodo può iniziare la trasmissione mentre il nodo i sta trasmettendo, in quanto si sovrapporrebbe all'ultima parte della trasmissione di quel frame. La probabilità che tutti gli altri nodi non comincino a trasmettere in questo intervallo è anch'essa $(1-p)^{N-1}$. Di conseguenza, possiamo notare che la probabilità che un certo nodo abbia successo nella trasmissione è $p(1-p)^{2(N-1)}$. Ricavando il limite per N che tende all'infinito, otteniamo che l'efficienza massima del protocollo ALOHA puro è solo $1/(2e) = 0,18$. Questo è il prezzo che il protocollo ALOHA deve pagare per la completa decentralizzazione.

Slotted ALOHA

Lo Slotted ALOHA è stato inventato per migliorare l'efficienza dell'ALOHA puro. Nello Slotted ALOHA il tempo viene diviso in slot (intervalli di dimensione fissa) di T_{fr} secondi e le stazioni vengono forzate a trasmettere solo all'inizio degli slot.

Siccome le stazioni sono sincronizzate e possono trasmettere solo all'inizio degli slot, se si lasciano sfuggire tale momento, devono attendere fino all'inizio dello slot successivo. Ciò significa che, a questo punto, la stazione che ha avviato la trasmissione nello slot precedente avrà ormai già finito di inviare il suo frame. Naturalmente c'è ancora la possibilità di collisione se due stazioni cercano di trasmettere all'inizio dello stesso slot. I nodi devono essere d'accordo nel confine fra gli intervalli, e ciò può essere fatto facendo emettere da una attrezzatura speciale un breve segnale all'inizio di ogni intervallo.

Nella nostra descrizione assumeremo che: tutti i frame consistano esattamente di L bit; il tempo sia suddiviso in slot di L/R secondi (cioè che uno slot equivalga al tempo di trasmissione di un pacchetto); i nodi comincino la trasmissione dei frame solo all'inizio degli slot; i nodi siano sincronizzati in modo che tutti sappiano quando iniziano gli slot; qualora in uno slot due o più frame collidano, tutti i nodi della rete rilevino l'evento prima del termine dello slot. Indichiamo con p una probabilità, vale a dire un numero tra 0 e 1. Le operazioni dei nodi slotted ALOHA sono semplici.

- Quando un nodo ha un nuovo frame da spedire, attende fino all'inizio dello slot successivo e poi trasmette l'intero frame.
- Se non si verifica una collisione, l'operazione ha avuto successo, quindi non occorre effettuare una ritrasmissione e il nodo può predisporre l'invio di un nuovo frame.
- Se si verifica una collisione, il nodo la rileva prima del termine dello slot e ritrasmette con probabilità p il suo frame durante gli slot successivi, fino a quando l'operazione non ha successo.

Questo protocollo sembra possedere grandi vantaggi. A differenza della suddivisione del canale, slotted ALOHA consente a un singolo nodo di trasmettere continuamente pacchetti alla massima velocità del canale, quando è il solo nodo attivo. Un nodo si dice attivo se ha un frame da trasmettere. Slotted ALOHA è anche fortemente decentralizzato, in quanto ciascun nodo rileva le collisioni e decide indipendentemente quando ritrasmettere, anche se è comunque necessario che gli slot siano sincronizzati ai nodi. Slotted ALOHA funziona bene quando è attivo un solo nodo, ma è altrettanto efficiente in presenza di molti nodi attivi? Esistono in questo caso due possibili problemi. In primo

luogo, una certa frazione degli slot presenterà collisioni e di conseguenza andrà “sprecata”. Il secondo problema è che una grande frazione degli slot risulterà vuota, perché tutti i nodi attivi terminano la trasmissione in conseguenza della politica di trasmissione probabilistica. I soli slot non sprecati saranno esattamente quelli utilizzati da un solo nodo per trasmettere.

Supponiamo che ciascun nodo abbia sempre un frame da spedire e che il nodo trasmetta con probabilità p un nuovo frame o uno che ha già subito una collisione. In primo luogo, ipotizziamo di avere N nodi. In questo caso la probabilità che un dato slot sia vincente è data dalla probabilità che un solo nodo trasmetta, mentre i rimanenti $N-1$ rimangono inattivi. La probabilità che un dato nodo trasmetta è p ; la probabilità che i rimanenti nodi non trasmettano è $(1-p)^{N-1}$. Quindi la probabilità di successo di un dato nodo è $p(1-p)^{N-1}$. Poiché ci sono N nodi, la probabilità che un nodo arbitrario abbia successo è $Np(1-p)^{N-1}$. Di conseguenza, con N nodi attivi, l'efficienza dello slotted ALOHA è $Np(1-p)^{N-1}$. Per ottenere la massima efficienza con N nodi attivi bisogna trovare il valore p^* che massimizza questa espressione. Per ottenere la massima efficienza per un elevato numero di nodi attivi, ricaviamo il limite di $Np^*(1-p^*)^{N-1}$ per N che tende all'infinito. Dopo aver svolto questi calcoli, otterremo che l'efficienza massima del protocollo è data da $1/e = 0,37$. Vale a dire che quando un gran numero di nodi ha molti pacchetti da trasmettere, allora (nel caso migliore) solo il 37% degli slot compie lavoro utile. Quindi, l'effettiva velocità di trasmissione del canale non è R bps, ma soltanto $0,37R$ bps!

CSMA: accesso multiplo con rilevamento della portante

Nei due protocolli ALOHA i nodi prendono la decisione di trasmettere indipendentemente dall'attività degli altri nodi collegati al canale broadcast. In particolare, un nodo non presta attenzione al fatto che vi sia un altro nodo che sta trasmettendo né arresta la trasmissione se un altro nodo inizia a interferire con la sua trasmissione. Nella nostra analogia con il comportamento umano, i protocolli ALOHA corrispondono a un maleducato che continua a parlare senza preoccuparsi del fatto che altri stiano conversando tra loro.

Disponiamo di “protocolli” che ci consentono non solo di comportarci più educatamente, ma anche di diminuire il tempo di “collisione” tra le diverse conversazioni e, di conseguenza, di aumentare la quantità di dati che viene scambiata. In particolare, esistono due regole importanti per dialogare in modo civile, che sono le seguenti.

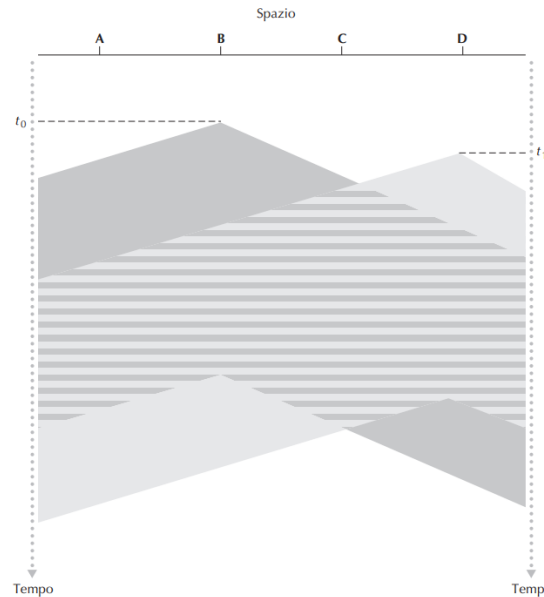
- Ascoltare prima di parlare. Se qualcun altro sta parlando, aspettare finché abbia concluso. Nel mondo delle reti, ciò è chiamato rilevamento della portante (carrier sensing): un nodo ascolta il canale prima di trasmettere. Se il canale sta già trasmettendo un frame, il nodo aspetta finché rileva che il canale è libero per un intervallo di tempo e quindi inizia la trasmissione.
- Se qualcun altro comincia a parlare insieme a voi, smettete di parlare. Nel mondo delle reti, ciò si chiama rilevamento della collisione (collision detection): il nodo che sta trasmettendo rimane contemporaneamente in ascolto del canale. Se osserva che un altro nodo sta trasmettendo un frame che interferisce col suo, arresta la propria trasmissione, aspetta un intervallo di tempo casuale e poi ripete il processo.

Queste due regole sono alla base dei protocolli CSMA (carrier sense multiple access, accesso multiplo con rilevamento della portante).

La prima domanda è: perché, se tutti i nodi eseguono il rilevamento della portante, si verificano collisioni? Dopo tutto, i nodi rinunciano alla trasmissione quando rilevano che qualcun altro sta trasmettendo. La risposta alla domanda può essere meglio spiegata mediante diagrammi spazio-temporali.

Esempio 7.2.3 – Diagramma spazio-temporale CSMA con trasmissioni in collisione

La figura ne illustra uno relativo a quattro nodi (A, B, C, D). L'asse orizzontale mostra la posizione dei nodi sul canale condiviso e quello verticale rappresenta il tempo.



All'istante t_0 , il nodo B rileva che il canale è inattivo (nessun altro nodo sta trasmettendo), comincia la trasmissione e i suoi bit si propagano in entrambe le direzioni lungo il mezzo trasmissivo. La propagazione verso valle dei bit di B al crescere del tempo indica che è necessario un intervallo di tempo non nullo per l'effettiva propagazione dei bit di B lungo il canale (benché questa avvenga a una velocità paragonabile a quella della luce). Al tempo $t_1 > t_0$, il nodo D ha un frame da spedire. Sebbene al tempo t_1 il nodo B stia ancora trasmettendo, i suoi bit non hanno ancora raggiunto D e quindi, in quell'istante, D ritiene libero il canale. Nel pieno rispetto del protocollo, D allora comincia a trasmettere il suo frame. Dopo un breve periodo, la trasmissione di B comincerà a interferire con quella di D.

Dalla figura risulta evidente che il ritardo di propagazione da un estremo all'altro di un canale broadcast (il tempo richiesto da un segnale per propagarsi da un nodo all'altro) avrà un ruolo cruciale nel determinare le sue prestazioni. Maggiore è questo ritardo, maggiore sarà la possibilità che il nodo, pur attento a rilevare la portante, non si accorga che è già cominciata la trasmissione da parte di un altro nodo.

Il tempo di vulnerabilità per il CSMA è pari al tempo di propagazione T . È il tempo necessario affinché un segnale si propaghi da un'estremità all'altra del mezzo. Quando una stazione invia un frame e un'altra cerca di inviarne un altro allo stesso tempo, avviene una collisione. Ma se il primo bit del frame raggiunge l'estremità del mezzo, ogni stazione ha già percepito il bit ed evita di trasmettere.

Cosa deve fare una stazione se il canale è occupato? Cosa deve fare se il canale è inattivo? Sono stati ideati tre metodi per rispondere a queste domande: 1-persistente, non persistente, p-persistente.

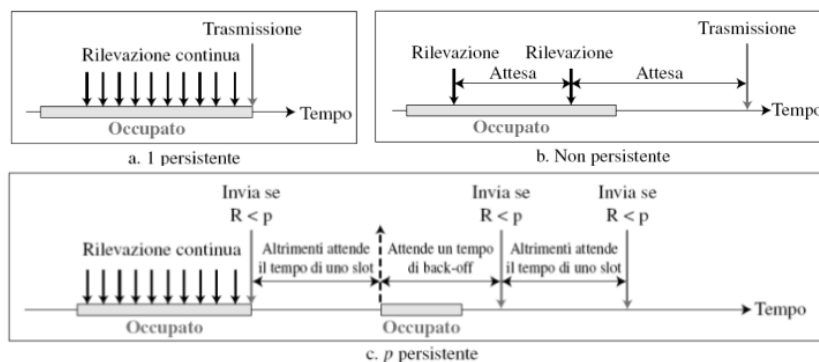


Figura 7.9: Comportamento dei tre metodi di persistenza

- *1 persistente*. Il metodo dell'1 persistente è semplice e chiaro. Secondo questo metodo, se una stazione trova il canale attivo continua a controllarlo fino a quando non diviene inattivo. Quan-

do finalmente il canale diviene inattivo la stazione invia immediatamente il suo frame (con probabilità 1). Questo metodo ha la più alta probabilità di generare collisioni in quanto due o più stazioni possono trovare il mezzo trasmissivo inattivo ed inviare immediatamente i loro frame.

- *Non persistente.* Nel metodo non persistente una stazione che deve inviare un frame rileva il mezzo trasmissivo. Se è inattivo, trasmette immediatamente. Se non è inattivo, attende un lasso di tempo casuale e poi effettua nuovamente la rilevazione. L'approccio non persistente riduce la probabilità di collisioni in quanto è improbabile che due o più stazioni attendano esattamente lo stesso lasso di tempo e cerchino di nuovo di trasmettere contemporaneamente. Tuttavia tale metodo riduce l'efficienza della rete in quanto il mezzo resta inattivo quando invece potrebbero esserci delle stazioni con dei frame in attesa di essere inviati.
- *p persistente.* Il metodo p persistente viene utilizzato se il canale è stato suddiviso in intervalli fissi di tempo (slot) dove ciascuno slot ha una durata che è uguale o maggiore rispetto al tempo massimo di propagazione. L'approccio p persistente unisce i vantaggi delle altre due strategie. Riduce la possibilità di collisione e migliora l'efficienza. Secondo tale metodo, dopo che la stazione ha trovato il mezzo trasmissivo inattivo, segue i seguenti passi:
 1. con probabilità p , la stazione invia il suo frame;
 2. con probabilità $q = 1 - p$, la stazione attende l'inizio del prossimo slot e controlla nuovamente il mezzo trasmissivo: se il mezzo è inattivo, torna al punto 1; se è occupato, agisce come se fosse avvenuta una collisione e utilizza la procedura di back-off.

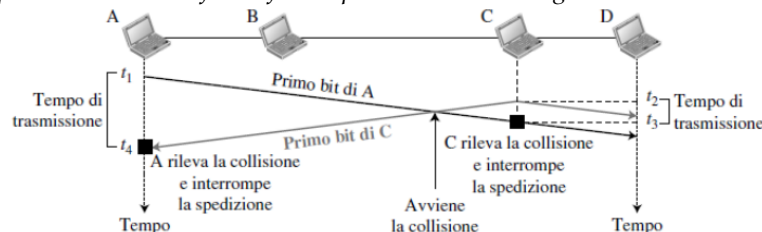
CSMA/CD

Il metodo CDMA non specifica la procedura da seguire a seguito di una collisione. Il Carrier Sense Multiple Access with Collision Detection (CSMA/CD) a partire dall'algoritmo CSMA aggiunge una parte relativa alla gestione delle collisioni.

In questo metodo una stazione, durante l'invio di un frame, continua a controllare il mezzo trasmissivo per verificare se la trasmissione ha avuto successo. Se è così la trasmissione è finita, in caso contrario (c'è stata una collisione) il frame dovrà essere nuovamente inviato.

Esempio 7.2.4 – Collisione nel CSMA/CD

Vediamo la figura dove due stazioni (A e C) sono coinvolte in una collisione. Entrambe le stazioni continuano a spedire i bit dei loro frame fino a quando non sono in grado di rilevare la collisione.



Al tempo t_1 , la stazione A ha effettuato la sua procedura di persistenza e inizia ad inviare i bit del suo frame. Al tempo t_2 , la stazione C non ha ancora rilevato il primo bit inviato da A. La stazione C effettua la sua procedura di persistenza ed inizia ad inviare i bit nel suo frame, che si propagano in tutte le direzioni. La collisione avviene in un momento successivo al tempo t_2 . La stazione C rileva una collisione al tempo t_3 , quando riceve il primo bit del frame di A. La stazione C interrompe la trasmissione immediatamente (oppure dopo un breve lasso di tempo, ma ora si assume che sia immediatamente). La stazione A rileva la collisione al tempo t_4 quando riceve il primo bit del frame di C. Come reazione, anche la stazione A interrompe immediatamente la trasmissione. Guardando la figura vediamo che A trasmette per la durata di $t_4 - t_1$ e C trasmette per la durata $t_3 - t_2$.

Affinché il CSMA/CD possa funzionare, è necessario aggiungere un vincolo alla dimensione del frame. Prima di inviare l'ultimo bit del frame la stazione mittente deve essere in grado di rilevare una collisione, nel caso ce ne sia una, ed interrompere la trasmissione. Il motivo è che la stazione, una volta che l'intero frame è stato inviato, non ne tiene copia ed inoltre non controlla il mezzo trasmissivo per rilevare eventuali collisioni. Quindi il tempo di trasmissione del frame T_{fr} deve essere almeno

due volte il tempo di propagazione massimo T_p . Per comprendere la ragione, pensiamo allo scenario del caso pessimo. Se due stazioni coinvolte in una collisione si trovano alla distanza massima, il segnale dalla prima necessita del tempo T_p per raggiungere la seconda e l'effetto della collisione necessita di un ulteriore tempo T_p raggiungere la prima. Quindi il requisito è che la prima stazione deve essere ancora trasmissione dopo $2T_p$.

Esempio 7.2.5 – CSMA/CD

Una rete che utilizza il CSMA/CD ha un'ampiezza di banda di 10 Mbps. Se il tempo di propagazione massimo (compresi i ritardi nei dispositivi) è $25,6 \mu s$, qual è la dimensione minima del frame?

Il tempo di trasmissione minimo del frame è $T_{fr} = 2T_p = 51,2 \mu s$. Ciò significa, nel peggiore dei casi, che una stazione deve trasmettere per un periodo di $51,2 \mu s$ per poter rilevare la collisione. La dimensione minima del frame è quindi $10 Mbps \times 51,2 \mu s = 512 \text{ bit}$ o 64 byte.

Quando un solo nodo ha un frame da inviare può trasmettere alla massima velocità del canale (10 Mbps, 100 Mbps o 1 Gbps per Ethernet). Se, invece, i nodi che vogliono trasmettere sono numerosi, l'effettiva velocità di trasmissione sul canale può risultare notevolmente inferiore. Definiamo efficienza di CSMA/CD la frazione di tempo media durante la quale i frame sono trasferiti sul canale senza collisioni in presenza di un alto numero di nodi attivi, con un'elevata quantità di frame da inviare.

7.2.3 Protocolli a rotazione

Ricordiamo che due proprietà auspicabili di un protocollo di accesso multiplo sono: (1) quando un solo nodo è attivo, deve avere un throughput di R bps, e (2) quando sono attivi M nodi, ciascuno deve avere un throughput vicino a R/M bps. I protocolli ALOHA e CSMA possiedono la prima proprietà, ma non la seconda. Questo ha incoraggiato i ricercatori a creare un'altra classe di protocolli: i protocolli a rotazione (taking-turn protocol). Esistono decine di questi protocolli, ciascuno con molte varianti; ne vedremo qui due tra le più importanti.

Protocollo polling

Il primo è il protocollo polling nel quale uno dei nodi, designato come principale, interPELLa "a turno" gli altri. In particolare, il nodo principale invia un messaggio al nodo 1, comunicandogli che può trasmettere fino a un dato numero massimo di frame. Dopo che il nodo 1 ha trasmesso dei frame, il nodo principale avvisa il nodo 2 che può iniziare a inviare un dato numero massimo di frame. Il nodo principale può determinare che un nodo ha terminato di inviare i propri frame, osservando la mancanza di segnale sul canale. La procedura prosegue in questo modo, con il nodo principale che interPELLa in modo ciclico tutti gli altri nodi.

Il protocollo polling elimina le collisioni e gli slot vuoti che si riscontrano nei protocolli ad accesso casuale e quindi può avere un'efficienza più elevata, ma presenta anche alcuni svantaggi. Il primo è l'introduzione del ritardo di polling: il tempo richiesto per notificare a un nodo il permesso di trasmettere. Se, per esempio, è attivo solo un nodo, allora questo trasmetterà a un tasso inferiore di R bps, in quanto il nodo principale deve contattare ciclicamente i nodi inattivi ogni volta che quello attivo ha terminato l'invio del suo numero massimo di frame. Il secondo inconveniente, potenzialmente molto più serio, è che, se il nodo principale si guasta, l'intero canale diventa inattivo.

Protocollo token-passing

Il secondo protocollo a rotazione è il protocollo token-passing (a passaggio di gettone). In questo caso non esiste un nodo principale, ma un frame, un messaggio di controllo detto token (gettone), che circola fra i nodi seguendo un ordine prefissato. Per esempio: il nodo 1 deve sempre spedire il token al nodo 2 che deve inviarlo al nodo 3 e così via fino a quando il nodo N lo restituirà al nodo 1 che riprenderà la procedura. Se il nodo che riceve il token non ha pacchetti da inviare, lo inoltra immediatamente al successivo. Altrimenti, procede a trasmettere il numero massimo di frame consentito, prima di inoltrare il token al nodo successivo.

Questo protocollo è decentralizzato e altamente efficiente, ma non è privo di problemi. Per esempio, il guasto di un nodo può mettere fuori servizio l'intero canale. Oppure, se un nodo non riesce a inoltrare il token, occorre invocare procedure di recupero, per rimetterlo in circolazione.

7.3 Indirizzamento di collegamento

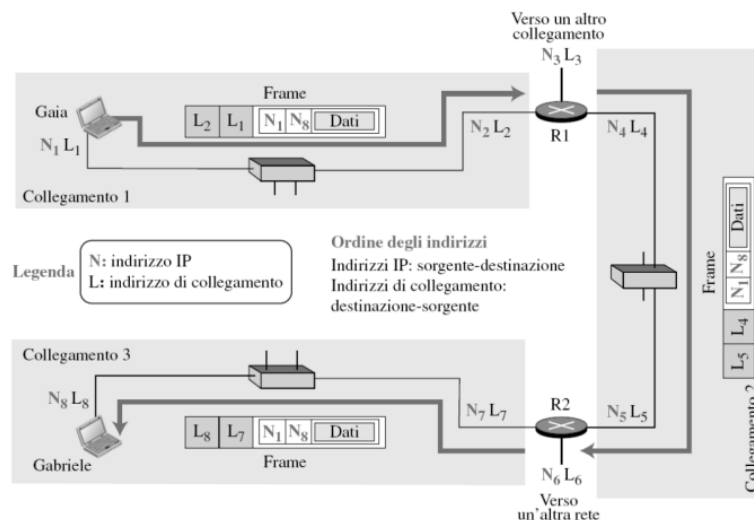
Host e router hanno indirizzi a livello di collegamento. Ciò potrebbe essere sorprendente visto che i nodi hanno anche un indirizzo a livello di rete. Dovreste quindi domandarvi perché abbiamo la necessità di avere indirizzi sia a livello di rete sia a livello di collegamento.

In una rete senza connessione come Internet non è possibile far sì che un datagramma raggiunga la sua destinazione solamente usando gli indirizzi IP. La ragione è che ogni datagramma in Internet, dallo stesso host sorgente allo stesso host destinazione, può prendere un percorso diverso. Gli indirizzi IP sorgente e destinazione definiscono le due estremità della rete, ma non dicono attraverso quali collegamenti deve passare il datagramma. In una rete senza connessione, è necessario avere un ulteriore meccanismo di indirizzamento: gli indirizzi di livello collegamento. Un indirizzo di livello collegamento (indirizzo di collegamento, link-layer address, link address) è spesso chiamato anche indirizzo fisico oppure indirizzo MAC (MAC address).

I collegamenti sono ovviamente controllati dal livello di collegamento e utilizzano indirizzi di questo livello. Quando un datagramma passa dal livello di rete al livello di collegamento, esso viene racchiuso all'interno di un frame e viene aggiunta un'intestazione che contiene due indirizzi di livello collegamento (sorgente e destinazione del frame). Questi due indirizzi vengono cambiati ogni volta che il frame passa da un collegamento ad un altro.

Esempio 7.3.1 – Indirizzi IP e indirizzi di livello collegamento in una piccola rete

Nella rete della figura ci sono 3 collegamenti e 2 router. Abbiamo mostrato solo 2 host: Gaia (sorgente) e Gabriele (destinazione). Per ogni host vengono indicati due indirizzi, gli indirizzi IP (N) e quelli di collegamento (L).



Da notare che un router ha tante coppie di indirizzi quanti sono i suoi collegamenti. In tutto abbiamo mostrato 3 frame, uno per ogni collegamento. Ogni frame trasporta lo stesso datagramma che contiene sempre gli stessi indirizzi di sorgente e destinazione ($N1$ e $N8$), ma gli indirizzi di collegamento del frame cambiano su ciascun collegamento. Nel collegamento 1, gli indirizzi di collegamento sono $L1$ ed $L2$. Nel collegamento 2, essi sono $L4$ e $L5$. Nel collegamento 3 sono $L7$ e $L8$. Da notare che gli indirizzi IP e gli indirizzi di collegamento non seguono lo stesso ordine. Per gli indirizzi IP, l'indirizzo sorgente viene prima di quello di destinazione; per gli indirizzi di collegamento invece l'indirizzo destinazione viene prima di quello sorgente. I datagrammi ed i frame sono stati progettati in questo modo e quindi rispettiamo questa convenzione.

7.3.1 Indirizzi MAC

In realtà, non sono i nodi (cioè, gli host e i router) ad avere indirizzi a livello di collegamento, ma sono i loro adattatori (schede di rete) a possederli. Un host o un router con più interfacce di rete avrà più indirizzi a livello di collegamento associati a esse, esattamente come accadeva per gli indirizzi IP.

Gli indirizzi a livello di collegamento sono indicati con varie terminologie che vanno da indirizzo fisico, indirizzo LAN o indirizzo MAC. Poiché quest'ultimo sembra essere il termine più utilizzato,

d'ora in avanti ci riferiremo a questi indirizzi come a indirizzi MAC. Per molte LAN (come Ethernet e 802.11), l'indirizzo MAC è lungo sei byte, il che consente di avere 2^{48} possibili indirizzi. Questi indirizzi sono generalmente espressi in notazione esadecimale, scrivendo due cifre esadecimali per ogni byte.

Un'interessante proprietà degli indirizzi MAC è che non esistono due schede di rete con lo stesso indirizzo. Ciò può sembrare sorprendente, dato che le schede di rete sono costruite in differenti paesi da diverse società. Come può un'azienda che costruisce schede a Taiwan essere sicura di usare indirizzi diversi da una che le costruisce in Belgio? La risposta è che la IEEE sovrintende alla gestione degli indirizzi MAC. In particolare, quando una società vuole costruire schede di rete, compra un blocco di spazio di indirizzi, costituito da 2^{24} indirizzi, a un prezzo simbolico. IEEE riserva il blocco di 2^{24} indirizzi, fissando i primi 24 bit dell'indirizzo e lasciando alla società il compito di assegnare a ciascuna scheda di rete una specifica combinazione dei 24 bit rimanenti.

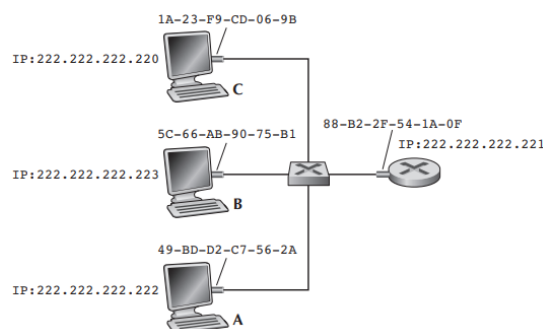
L'indirizzo MAC di una scheda di rete ha una struttura piatta (cioè non gerarchica) e non cambia mai. Ricordiamo che, al contrario, gli indirizzi IP hanno una struttura gerarchica (una parte per la rete e una parte per l'host), e che l'indirizzo IP di un nodo deve essere cambiato quando l'host si sposta, cioè quando cambia la rete al quale è collegato. L'indirizzo MAC di una scheda di rete è analogo al numero di codice fiscale di una persona, che ha anch'esso struttura orizzontale e non varia a seconda del luogo in cui la persona si trasferisce. Gli indirizzi IP invece sono analoghi all'indirizzo postale di una persona. Questi indirizzi hanno una struttura gerarchica e devono essere aggiornati quando la persona cambia residenza.

7.3.2 Protocollo per la risoluzione degli indirizzi (ARP)

Dato che esistono sia indirizzi del livello di rete (come gli indirizzi IP di Internet) sia indirizzi del livello di collegamento (gli indirizzi MAC), si presenta la necessità della loro conversione. Internet affida questo compito al protocollo di risoluzione degli indirizzi (ARP, address resolution protocol).

Esempio 7.3.2 – ARP

Per comprendere la necessità di un protocollo come ARP, considerate la rete illustrata in figura. In questo semplice esempio, ciascun nodo ha un indirizzo IP e la scheda di ciascun nodo ha un indirizzo MAC. Come al solito, gli indirizzi IP sono rappresentati in notazione decimale puntata e gli indirizzi MAC in notazione esadecimale.



Supponiamo ora che il nodo con l'indirizzo IP 222.222.222.220 voglia inviare un datagramma IP al nodo 222.222.222.222. In questo esempio i nodi sorgente e destinazione appartengono alla stessa sottorete, dal punto di vista dell'indirizzamento. Per trasmettere un datagramma, il nodo trasmittente deve fornire alla sua scheda di rete non solo il datagramma IP, ma anche l'indirizzo MAC del nodo destinatario 222.222.222.222. Quando gli vengono passati il datagramma IP e l'indirizzo MAC, la scheda del nodo trasmittente deve costruire un frame contenente l'indirizzo MAC del nodo di destinazione e immetterlo nella LAN.

Un'importante problematica affrontata in questo paragrafo è come il nodo trasmittente riesca a determinare l'indirizzo MAC del nodo destinazione con indirizzo IP 222.222.222.222. Come potete immaginare, bisogna utilizzare ARP. Un modulo ARP nel nodo sorgente riceve in input un indirizzo IP della stessa LAN e restituisce l'indirizzo MAC corrispondente. Nell'esempio trattato, il nodo trasmittente passa al suo modulo ARP l'indirizzo IP 222.222.222.222 e ARP gli restituisce il corrispondente indirizzo MAC, cioè 49-BD-D2-C7-56-2A.

Abbiamo quindi visto che un modulo ARP traduce un indirizzo IP in un indirizzo MAC. Ora che abbiamo visto che cosa fa ARP, cerchiamo di capire come funziona. Nella RAM dei nodi vi è una tabella ARP che contiene la corrispondenza tra indirizzi IP e MAC. La tabella contiene anche un valore relativo al TTL (time-to-live, tempo di vita), che indica quando bisognerà eliminare una data voce dalla tabella.

Indirizzo IP	Indirizzo MAC	TTL
222.222.222.221	88-B2-2F-54-1A-0F	13:45:00
222.222.222.223	5C-66-AB-90-75-B1	13:52:00

Figura 7.10: Una possibile tabella ARP per il nodo 222.222.222.220

Notiamo che la tabella non contiene necessariamente una voce per ciascun nodo della sottorete; alcuni nodi possono essere stati cancellati (perché scaduto il loro TTL), altri possono non essere mai stati inseriti. Il tipico TTL è di 20 minuti, da quando una voce viene inserita nella tabella ARP.

Il protocollo ARP è uno dei protocolli ausiliari definiti a livello di rete. Appartiene al livello di rete, ma abbiamo deciso di rimandare la sua analisi fino ad ora proprio perché il suo scopo è associare gli indirizzi IP agli indirizzi di collegamento. L'ARP accetta in input un indirizzo IP, individua l'indirizzo di collegamento corrispondente e lo passa al livello di collegamento.

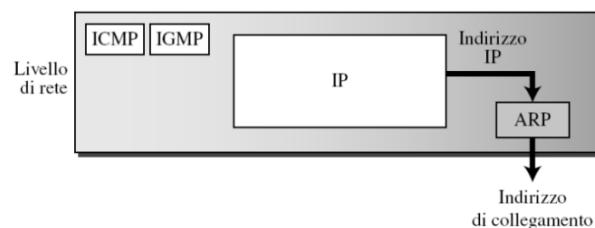


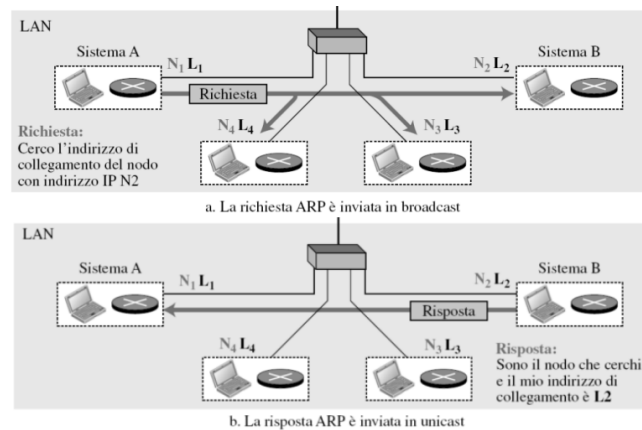
Figura 7.11: Posizione dell'ARP nella pila TCP/IP

Ogni volta che un host o un router deve trovare l'indirizzo di collegamento di un altro host o router che si trova all'interno della sua rete, invia un pacchetto di richiesta ARP che comprende l'indirizzo di collegamento, l'indirizzo IP del mittente e l'indirizzo IP del ricevente. Siccome il mittente non conosce ancora l'indirizzo di collegamento del ricevente, la richiesta viene trasmessa usando un broadcast a livello di collegamento. L'invio broadcast è ottenuto per mezzo del corrispondente indirizzo broadcast, in modo che la richiesta raggiunga tutti i nodi all'interno della rete locale.

Ogni host o router nella rete locale riceve ed elabora il pacchetto di richiesta ARP, ma solo il ricevente designato riconosce il suo indirizzo IP e restituisce un pacchetto di risposta ARP che contiene i suoi indirizzi IP e di collegamento. Il pacchetto di risposta viene inviato in modalità unicast direttamente al nodo che ha inviato la richiesta.

Esempio 7.3.3 – Protocollo ARP nella stessa sottorete

Nella figura, il sistema sulla sinistra (A) ha un pacchetto che deve essere consegnato ad un altro sistema (B) con indirizzo IP N2.



Il sistema A deve passare il pacchetto al suo livello di collegamento per la consegna vera e propria, ma non conosce l'indirizzo fisico (indirizzo di collegamento) del ricevente. Esso quindi utilizza i servizi di ARP, chiedendo al protocollo di inviare in broadcast un pacchetto di richiesta, per scoprire l'indirizzo fisico del un sistema che ha N2 come indirizzo IP. Il pacchetto di richiesta ARP viene ricevuto da ogni sistema sulla rete fisica, ma solo il sistema B risponde, come mostrato nella figura, inviando un pacchetto di risposta ARP che contiene il suo indirizzo fisico. Ora il sistema A può inviare tutti i pacchetti che ha per tale destinazione, usando l'indirizzo fisico ottenuto per mezzo di ARP.

Formato del pacchetto ARP

Viene mostrato il formato di un pacchetto ARP. I nomi dei campi spiegano da soli il loro significato. Il campo hardware type definisce il tipo del protocollo di livello collegamento che viene usato. Il campo protocol type definisce il protocollo del livello di rete.

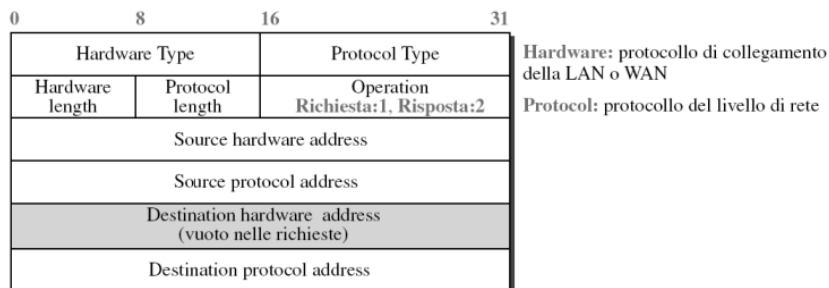
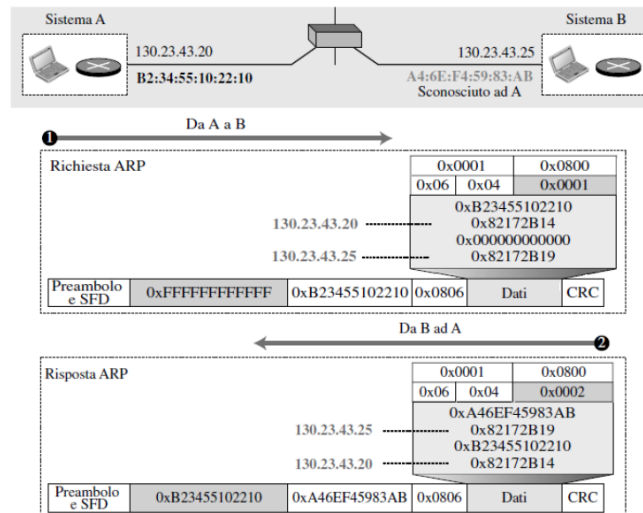


Figura 7.12: Formato del pacchetto ARP

I campi source hardware address e source protocol address definiscono, per il mittente, il suo indirizzo fisico e quello del protocollo di livello superiore (IP nel nostro esempio). Entrambi i campi, per i motivi discussi in precedenza, sono di lunghezza variabile (lunghezze definite per mezzo di hardware length e protocol length). I campi destination hardware address e destination protocol address definiscono l'indirizzo fisico e quello del protocollo di livello superiore, ma in questo caso per il ricevente. I pacchetti ARP vengono incapsulati direttamente all'interno dei frame di livello collegamento. Per questo motivo il frame dovrà indicare all'interno della sua intestazione che trasporta un pacchetto ARP invece di un datagramma del livello di rete.

Esempio 7.3.4 – Pacchetto ARP

Un host con indirizzo IP N1 e indirizzo MAC (indirizzo fisico) L1 ha un pacchetto da inviare a un altro host con indirizzo IP N2 e indirizzo fisico L2 (indirizzo fisico che al momento è sconosciuto al primo host). I due host si trovano all'interno della stessa rete. Mostrare il pacchetto di richiesta ARP e quello di risposta, racchiusi all'interno dei frame Ethernet.

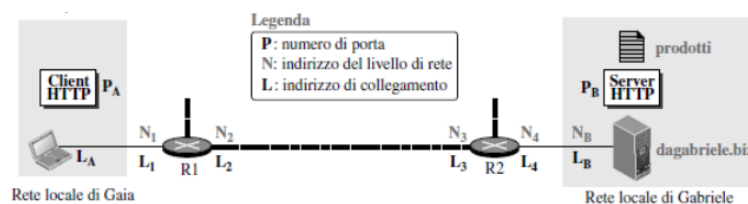


La figura mostra la richiesta ARP e il pacchetto di risposta. Da notare che il campo dati ARP è in questo caso di 28 byte e che i vari indirizzi contenuti al suo interno hanno lunghezze differenti: 32 bit per IPv4 e 48 bit per Ethernet. Inoltre, in questo caso anche gli indirizzi IP sono rappresentati in esadecimale.

Si è parlato di indirizzamento nei quattro livelli superiori della pila TCP/IP. A livello fisico invece non c'è alcun indirizzamento visto che questo livello si occupa di trasferire i singoli bit, che vengono spediti in broadcast nel mezzo trasmissivo e ricevuti da tutti i nodi che sono collegati a tale mezzo. Vediamo ora gli indirizzi dei vari livelli nell'ambito di un'unica transazione, così da poter fare attenzione alle varie interrelazioni che ci sono tra di loro. Vediamo anche come tutti questi indirizzi possono essere ricavati automaticamente dal sistema, purché sia noto il nome del sito che l'utente intende contattare.

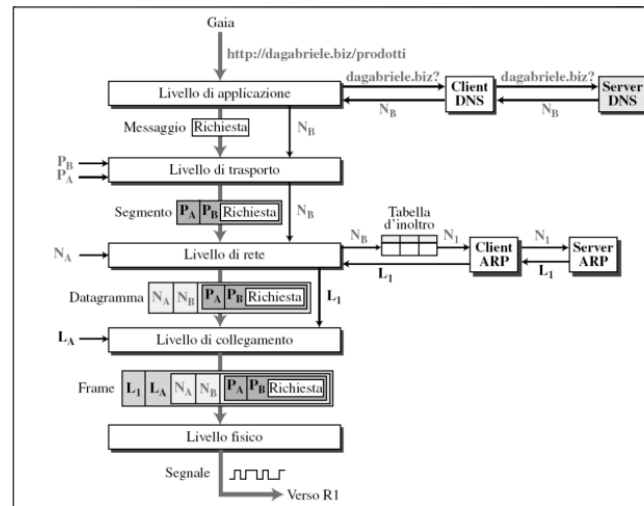
Esempio 7.3.5 – Indirizzamento

La figura mostra una internet semplificata dove ci sono Gaia e Gabriele. Gaia è un'utente che vuole accedere alla lista dei prodotti offerti da Gabriele. Gabriele ha una piccola attività chiamata *dagabriele.biz*, la lista dei prodotti e le relative specifiche sono memorizzate in un documento chiamato *prodotti*.



L'unico identificatore che Gaia deve conoscere è l'URL del sito che vuole visitare: <http://dagabriele.biz/prodotti>. Il resto degli indirizzi, sorgente e destinazione, vengono ricavati dal sistema, come mostriamo in questo esempio.

Supponiamo che Gaia sia collegata ad una LAN e che il suo computer conosca sia il suo indirizzo IP che quello di collegamento. Anche il server di Gabriele è collegato ad una LAN (una diversa rispetto a quella di Gaia) ed anche il suo computer conosce entrambi i suoi indirizzi. La stessa cosa vale per i router all'interno della internet semplificata: ciascun router conosce i propri indirizzi di rete e di collegamento. Nell'esempio verranno utilizzati degli indirizzi simbolici per rendere più leggibili le figure. Come spiegato in precedenza, Gaia non ha alcuna necessità di conoscere questi indirizzi: essi vengono ottenuti automaticamente nel corso della transazione.



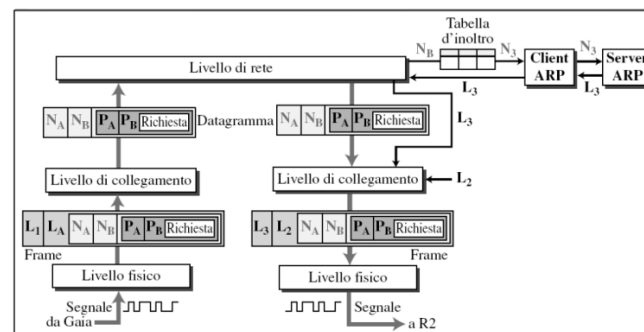
Flusso di pacchetti nel computer di Gaia

La figura mostra il flusso di pacchetti generati dal computer di Gaia. Gaia, usando un browser, digita l'URL del sito di Gabriele (`http://dagabriele.biz/prodotti`). Il computer di Gaia usa il client HTTP per soddisfare la sua richiesta. Il client HTTP, tuttavia, non conosce l'indirizzo IP di Gabriele, quindi chiama il client DNS per ottenere aiuto. Il client DNS invia una richiesta ad un server DNS e trova l'indirizzo IP del computer di Gabriele (N_B) che viene passato al livello trasporto.

A questo punto il livello trasporto crea un segmento utilizzando il numero di porta noto del server HTTP ($P_B = 80$) e un numero di porta temporaneo ottenuto dal sistema operativo (P_A) per il client HTTP. Ora il livello trasporto può predisporre un segmento e passarlo al livello di rete assieme all'indirizzo IP del computer di Gabriele (N_B), che è stato ottenuto nel passaggio precedente.

Il livello di rete riceve il segmento e l'indirizzo di rete N_B . Il livello di rete consulta la sua tabella d'instradamento e cerca il router successivo a cui instradare il datagramma che è destinato all'indirizzo N_B . La risposta in questo caso è il router di default della rete in cui si trova Gaia e quindi la tabella di routing fornisce l'indirizzo N_1 . Ma questo non è sufficiente, il livello di collegamento di Gaia ha la necessità di trovare l'indirizzo di collegamento del router R1. Utilizza quindi il suo client ARP per trovare l'indirizzo di collegamento L_1 . Il livello di rete può ora passare il datagramma con l'indirizzo di collegamento giusto al livello di collegamento. Come detto in precedenza, il livello di collegamento di Gaia conosce il suo indirizzo di collegamento, L_A . Crea quindi il frame e lo passa al livello fisico dove viene convertito in segnali e spedito attraverso il mezzo trasmissivo.

Ora vediamo cosa succede nel router R1. Come sappiamo, i router normalmente lavorano solamente ai tre livelli inferiori della pila TCP/IP. Il pacchetto ricevuto da una delle interfacce deve quindi salire tutti e tre questi livelli, ridiscenderli e poi essere spedito da un'altra interfaccia.



Flusso di pacchetti nel router R1

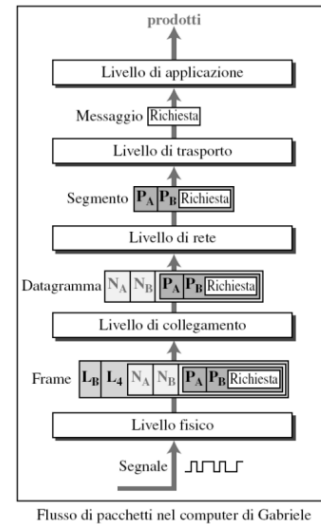
La figura mostra quanto avviene nel router. All'arrivo, il livello fisico del collegamento di sinistra (la rete a cui è collegata Gaia) riceve il frame e lo passa al livello di collegamento, che estrae il datagramma e lo passa al livello di rete. Esso esamina l'indirizzo del livello di rete del datagramma e scopre che deve essere consegnato al dispositivo con indirizzo IP N_B .

Il livello di rete consulta la sua tabella d'inoltro per scoprire qual'è il nodo successivo (router) nel percorso verso N_B . La tabella d'inoltro restituisce N_3 . L'indirizzo IP del router R2 indica che R2 si trova sullo stesso collegamento con R1. Il livello di rete di R1 ora usa ARP per trovare l'indirizzo di collegamento di R2, che risulta essere L_3 quindi passa il datagramma e l'indirizzo L_3 al livello di collegamento (lato destro del router R1). Il livello di collegamento incapsula il datagramma in un frame, aggiunge L_3 e L_2 (il suo indirizzo di collegamento sul lato destro) e passa il frame al livello fisico, che codifica i bit in segnali e li invia attraverso il mezzo trasmissivo a R2.

Le attività nel router R2 sono le stesse del router R1, ad eccezione del fatto che alcuni indirizzi sono diversi.

Ora vediamo cosa succede nella rete di Gabriele.

La figura mostra come i segnali del livello fisico nell'host di Gabriele vengono tradotti in un messaggio destinato al programma server HTTP. Nella rete locale di Gabriele non servono più indirizzi o risoluzioni di indirizzi. Il segnale ricevuto dal collegamento viene tradotto in un frame, che viene passato al livello di collegamento, il quale estrae il datagramma e lo passa al livello di rete. Quest'ultimo estrae il segmento e lo passa al livello trasporto, che apre il segmento e passa il messaggio contenuto alla socket in ascolto sulla porta $P_B(80)$. Il server HTTP riceve il messaggio, prepara una copia del file con i prodotti, lo trasforma in un documento HTML e lo restituisce a Gaia usando lo stesso flusso, ma in ordine inverso. Non è certo, ma il messaggio di ritorno probabilmente passerà attraverso i medesimi router, R2 e R1, per raggiungere Gaia.



7.4 LAN cablate: Ethernet

Nel 1985, la IEEE Computer Society (Institute of Electrical and Electronics Engineers) iniziò un progetto, chiamato Progetto 802, con l'obiettivo di definire uno standard per l'interconnessione tra dispositivi di produttori differenti. Il Progetto 802 non aveva lo scopo di sostituire una qualsiasi parte del modello OSI o la pila TCP/IP, ma piuttosto intendeva definire chiaramente le funzioni specifiche del livello fisico e di collegamento dei protocolli LAN. Questo lavoro ha portato alla definizione dell'IEEE Standard 802.

L'IEEE ha suddiviso il livello di collegamento in due sotto-livelli: Logical Link Control (LLC) e Media Access Control (MAC). Inoltre, l'IEEE ha creato anche vari standard relativi ai livelli fisici delle tecnologie LAN.

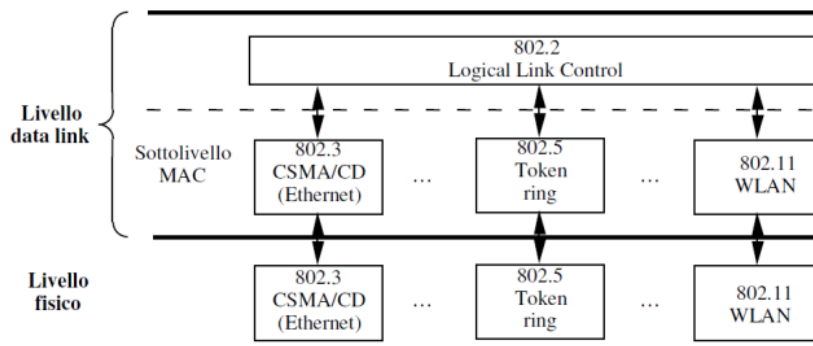


Figura 7.13: Gli standard IEEE per le LAN

In precedenza si è parlato del Data Link Control e in particolare si è detto che questo sotto-livello fornisce i servizi di framing, controllo di flusso e controllo degli errori. Nel caso di IEEE 802, il controllo di flusso, il controllo errori e parte delle funzioni di framing sono raccolte in un unico

sotto-livello, chiamato Logical Link Control (LLC). Più in dettaglio il framing viene gestito sia nel sotto-livello LLC che all'interno del sotto-livello MAC.

Uno dei vantaggi di LLC è il fatto che fornisce un singolo protocollo di controllo del livello di collegamento comune per tutte le LAN definite dall'IEEE. In altri termini, questo significa che grazie al protocollo LLC è possibile avere interoperabilità tra diverse tecnologie LAN. Questo è possibile dal momento che LLC rende il sotto-livello MAC trasparente (cioè invisibile) al livello superiore.

Si è visto in precedenza che esistono diversi approcci per l'accesso multiplo al canale condiviso: accesso casuale, accesso controllato e suddivisione in canali. L'IEEE 802 ha definito un sotto-livello chiamato Media Access Control (MAC, controllo dell'accesso al mezzo trasmissivo) che definisce il metodo d'accesso specifico per ciascuna tipologia di LAN. Ad esempio, il MAC per le LAN Ethernet è il CSMA/CD. Infine, come si è detto poco sopra, parte del framing è compito del livello MAC.

Ethernet detiene una posizione dominante nel mercato delle LAN cablate. E' stata la prima LAN ad alta velocità con vasta diffusione. Sempre al passo dei tempi con il tasso trasmissivo: 10 Mbps (Ethernet standard) - 10 Gbps (10 Gigabit Ethernet).

7.4.1 Ethernet standard

Il frame Ethernet contiene sette campi:

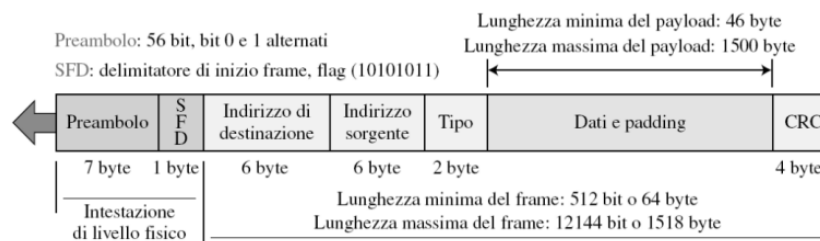


Figura 7.14: Struttura dei frame Ethernet

- *Preamble (preambolo).* Questo campo (7 byte quindi 56 bit) è composto da una serie di bit con valori 0 e 1 alternati. Il suo compito è quello di allertare il sistema ricevente circa l'arrivo del frame e permettere di sincronizzare il suo orologio con la trasmissione, nel caso non lo sia già. Il preambolo viene aggiunto a livello fisico e quindi non è propriamente parte del frame.
- *Start Frame Delimiter (SFD, delimitatore di inizio frame).* Questo campo (1 byte 10101011) segnala l'inizio del frame. L'SFD avverte la stazione o le stazioni riceventi che questa è la loro ultima possibilità di sincronizzazione. Gli ultimi 2 bit sono (11), e avvisano il ricevente che il campo seguente è l'indirizzo di destinazione. Quindi, in pratica, si tratta di un flag che definisce l'inizio del frame. Occorre ricordare che un frame Ethernet ha lunghezza variabile, quindi è necessario un flag per indicare l'inizio dei frame. Anche il campo SFD viene aggiunto dal livello fisico.
- *Destination Address (DA, Indirizzo di destinazione).* Il campo (6 byte quindi 48 bit) contiene l'indirizzo di collegamento della stazione o delle stazioni destinatarie del frame. Quando la stazione ricevente trova in questo campo il suo stesso indirizzo di livello collegamento (unicast), l'indirizzo multicast di un gruppo di cui fa parte, o l'indirizzo broadcast, estrae i dati dal frame e li passa al protocollo di livello superiore (definito dal valore del campo Type).
- *Source Address (SA, indirizzo della sorgente).* Anche questo campo è di 6 byte e contiene l'indirizzo di collegamento del mittente del frame.
- *Type (Tipo).* Questo campo definisce il protocollo di livello superiore del pacchetto incapsulato all'interno del frame. Il protocollo può essere IP, ARP, OSPF e così via. E' quindi necessario per il multiplexing e il de-multiplexing a livello di collegamento.
- *Data and padding (Dati).* Questo campo trasporta i dati incapsulati nel frame dai protocolli di livello superiore. Ha un minimo di 46 byte e un massimo di 1500 byte. Se i dati che provengono dal livello superiore superano i 1500 byte, essi devono essere frammentati e quindi racchiusi in più frame. Se sono meno di 46 byte, è necessario aggiungere tanti 0 quanti sono necessari per

arrivare alla dimensione minima (padding). Una volta decapsulati i dati contenuti nel frame vengono consegnati così come sono al protocollo di livello superiore, quindi senza rimuovere il padding (gli 0 aggiuntivi).

- **CRC.** L'ultimo campo contiene informazioni per il rilevamento errori. Il codice di ridondanza contenuto in questo campo viene calcolato sui campi indirizzo, tipo e dati. Se il ricevente lo calcola e constata che non è 0, allora il frame è stato alterato durante la trasmissione, e quindi il ricevente si limita a scartarlo.

Servizio senza connessione e inaffidabile

Ethernet fornisce un servizio senza connessione, questo significa che ogni frame inviato è indipendente dal frame precedente e dal successivo. In Ethernet non c'è alcuna fase di apertura o chiusura della connessione, il mittente si limita ad inviare un frame ogni volta che ne ha uno pronto, indipendentemente dal fatto che il destinatario sia pronto o meno a riceverlo. Può accadere che il mittente sommerga il ricevente di frame, con il risultato che alcuni di questi vengano scartati. Nel caso in cui un frame venga scartato, il mittente non viene informato dell'accaduto.

L'inaffidabilità di Ethernet è pari a quella di IP e UDP. Se un frame viene danneggiato durante la trasmissione e il ricevente riscontra il danneggiamento (grazie al codice di ridondanza contenuto nell'intestazione del frame), il ricevente si limiterà a scartare il frame, senza alcun'altra azione. E' dovere dei protocolli di livello superiore riscontrare la perdita dei dati e porvi rimedio.

Lunghezza del frame

Ethernet impone delle restrizioni sulla lunghezza minima dei frame e su quella massima. La restrizione sulla lunghezza minima è necessaria per il corretto funzionamento del meccanismo CSMA/CD. Un frame Ethernet deve avere una lunghezza minima di 512 bit (64 byte). Parte di questa lunghezza è relativa all'intestazione (header) e al trailer (la parte di frame che segue il campo dati). Se contiamo 18 byte di intestazione e trailer (6 byte di indirizzo della sorgente, 6 byte di indirizzo di destinazione, 2 byte per il Tipo e 4 byte di CRC), la lunghezza minima dei dati che il livello superiore deve fornire è $64 - 18 = 46$ byte. Se il pacchetto di livello superiore ha dimensione inferiore ai 46 byte, vengono aggiunti dei bit di riempimento per compensare la differenza (padding).

Lo standard Ethernet definisce la lunghezza massima del frame (senza contare il preambolo e il campo SFD) a 1518 byte. Se vengono sottratti i 18 byte dell'intestazione e del trailer, la lunghezza massima dei dati è 1500 byte. Questa restrizione deriva da due ragioni storiche: in primo luogo, la memoria, quando è stata inventata Ethernet, era molto costosa. Quindi questa restrizione permetteva di ridurre la dimensione dei buffer nei dispositivi. In secondo luogo essa evita che una stazione possa monopolizzare il mezzo trasmissivo condiviso, impedendo alle altre stazioni di inviare i loro frame.

Indirizzi

Tutte le stazioni che fanno parte di una rete Ethernet (ad esempio un computer, una postazione di lavoro o una stampante) sono dotate di una Network Interface Card (NIC), comunemente detta scheda di rete. La NIC viene installata nella stazione e fornisce un suo indirizzo di livello collegamento. Gli indirizzi Ethernet sono composti da 6 byte (48 bit) e vengono normalmente scritti in notazione esadecimale, con i due punti a dividere i vari byte.

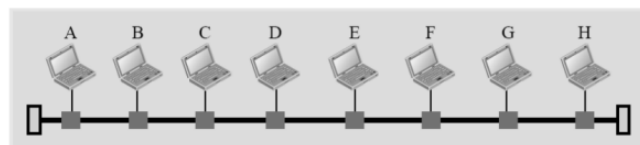
Quando gli indirizzi Ethernet vengono trasmessi, la loro codifica è diversa rispetto a come vengono rappresentati quando vengono scritti. Infatti, gli indirizzi vengono trasmessi da sinistra a destra, byte per byte, ma per ciascun byte il bit meno significativo viene inviato per primo e quello più significativo per ultimo. Grazie a questo semplice meccanismo il ricevente è in grado di capire già dal primo bit se il frame che sta ricevendo è unicast o multicast.

Metodo di accesso

Dato che il protocollo Ethernet Standard utilizza un approccio broadcast, era necessario implementare un metodo di accesso al mezzo trasmissivo condiviso. La modalità usata dall'Ethernet Standard era quella del CSMA/CD con il metodo dell'1 persistente. Per mezzo di un semplice esempio vediamo questi meccanismi applicati ad Ethernet.

Esempio 7.4.1 – Fasi operative del protocollo CSMA/CD

Supponiamo che la stazione A nella figura abbia un frame da inviare alla stazione D.



a. Rete LAN con topologia a bus che utilizza un cavo coassiale.

Per prima cosa la stazione A dovrebbe verificare se c'è un'altra stazione che sta già trasmettendo (carrier sense). Per far questo misura il livello di energia sul mezzo trasmissivo per un breve periodo di tempo (normalmente meno di $100\mu\text{s}$). L'assenza di segnale significa che nessuna stazione sta inviando (o il segnale non ha ancora raggiunto la stazione A). La stazione interpreta questa situazione come "mezzo inattivo" ed inizia a inviare il suo frame. Se invece rileva la presenza di segnale, questo significa che il mezzo trasmissivo è attualmente utilizzato da un'altra stazione. La stazione A controlla continuamente il mezzo finché non rimane inattivo per $100\mu\text{s}$, quindi inizia a inviare il suo frame. La stazione non può immediatamente gettare il frame, deve tenerne una copia nel buffer fino a quando non è sicura che non vi sia stata una collisione.

Il rilevamento del mezzo trasmissivo non può fermarsi dopo che la stazione A ha iniziato a inviare il frame, è necessario che continui a rimanere in ascolto anche mentre invia il frame. Possono verificarsi due casi:

1. La stazione A ha inviato 512 bit e non ha rilevato alcuna collisione; la stazione è quindi certa che il frame sia stato inviato senza collisioni ed interrompe la rilevazione sul mezzo trasmissivo. Da dove proviene la dimensione 512 bit? Considerando la velocità di trasmissione di Ethernet pari a 10 Mbps, questo significa che alla stazione occorrono $512 / (10 \text{ Mbps}) = 51,2 \mu\text{s}$ per inviare 512 bit. Con la velocità di propagazione in un cavo pari a 2×10 metri, il primo bit può aver percorso 10240 metri (solo andata) o 5120 metri (nel caso di andata e ritorno), aver avuto una collisione con un bit della stazione più lontana sul cavo ed essere tornato indietro. In altre parole, se si verifica una collisione, questa deve avvenire entro il momento in cui il mittente ha inviato 512 bit (caso peggiore) e il primo bit ha effettuato un'andata e ritorno di 5120 metri. Nel caso la collisione avvenga a metà del cavo, e non alla fine, la stazione rileverà la collisione ed interromperà la trasmissione.
2. La stazione A ha rilevato una collisione prima di inviare 512 bit. Ciò significa che uno dei precedenti bit è andato in collisione con un bit inviato da un'altra stazione. In questo caso entrambe dovrebbero astenersi dal trasmettere e mantenere il frame nei loro buffer per reinviarlo quando il mezzo trasmissivo sarà disponibile. Tuttavia, per informare le altre stazioni nella rete che vi è una collisione, la stazione invia un segnale di disturbo formato da 48 bit, che serve a creare sufficiente segnale (anche se la collisione avviene dopo pochi bit) per allertare le altre stazioni della collisione. Dopo l'invio del segnale di disturbo, la stazione deve incrementare il valore di K (numero di tentativi). Se, dopo l'incremento, K ha raggiunto il valore 15, allora si può assumere che la rete al momento sia troppo congestionata. La stazione quindi dovrebbe abbandonare i suoi sforzi e riprovare solamente più tardi. Se $K < 15$ la stazione può aspettare un tempo di back-off e riavviare il procedimento. La stazione genera un numero casuale compreso tra 0 e $2^K - 1$ che significa che ogni volta che avviene una collisione l'intervallo dei numeri casuali cresce in modo esponenziale. Dopo la prima collisione ($K = 1$) il numero casuale viene scelto nell'intervallo (0, 1). Dopo la seconda ($K = 2$) tra (0, 1, 2, 3), dopo la terza ($K = 3$) tra (0, 1, 2, 3, 4, 5, 6, 7). Quindi dopo ciascuna collisione, aumenta la probabilità che il tempo di back-off diventi maggiore. Ciò è dovuto al fatto che se la collisione avviene anche dopo il terzo o il quarto tentativo, significa che la rete è molto occupata, occorre quindi attendere per un tempo di back-off più lungo.

7.4.2 Fast Ethernet (100 Mbps)

Negli anni novanta sul mercato apparvero alcune tecnologie LAN con velocità di trasmissione superiore a 10 Mbps (come ad esempio FDDI e Fiber Channel). Ethernet quindi doveva competere con queste nuove tecnologie e la sua sopravvivenza era legata alla sua capacità di aumentare la velocità di trasferimento. L'evoluzione di Ethernet Standard, chiamata Fast Ethernet, ha fatto un notevole salto giungendo a ben 100 Mbps. Uno dei requisiti fondamentali in questa evoluzione era la necessità di mantenere Fast Ethernet compatibile con la versione precedente. Per questo motivo il sotto-livello MAC rimase del tutto immutato, questo significa che anche il formato del frame e le sue dimensioni (massima e minima) rimasero invariati. Con l'aumento della velocità di trasmissione, fu necessario riconsiderare alcune caratteristiche dell'Ethernet Standard che dipendono dalla velocità di trasmissione, ad esempio il metodo d'accesso.

Metodo di accesso

E' bene ricordare che il funzionamento corretto del CSMA/CD dipende dalla velocità di trasmissione, dalla dimensione minima del frame e dalla lunghezza massima della rete. Se si vuole mantenere inalterata la dimensione minima del frame è quindi necessario modificare la lunghezza massima della rete. In altre parole, se la dimensione minima del frame è ancora 512 bit e la trasmissione è 10 volte più veloce, allora le collisioni devono essere rilevate 10 volte più velocemente. Tutto questo porta al fatto che la rete dovrebbe essere 10 volte più corta (la velocità di propagazione ovviamente non cambia). Per questo motivo sono state previste due diverse soluzioni per il funzionamento di Fast Ethernet.

1. La prima soluzione è quella di abbandonare la topologia a bus, utilizzare un hub passivo con una topologia stella, ma fissare la dimensione massima della rete a 250 metri invece dei 2500 della versione Standard.
2. La seconda soluzione prevede l'uso di un commutatore di livello collegamento, detto anche switch dotato di buffer per memorizzare i frame e una connessione full-duplex (bi-direzionale) per ciascun host. In questo modo, in pratica, il mezzo trasmissivo è privato per ciascun host e non c'è alcuna ragione per usare il CSMA/CD, dal momento che gli host non sono più in competizione tra loro per accedere al mezzo trasmissivo. Lo switch riceve un frame da un host sorgente e lo memorizza nel buffer (coda) in attesa di elaborazione. Successivamente verifica l'indirizzo di destinazione e invia il frame attraverso l'interfaccia corrispondente. Dal momento che il collegamento al commutatore è full-duplex, il commutatore può persino inviare un frame a una destinazione mentre riceve un altro frame da un'altra stazione. In altre parole, il singolo mezzo trasmissivo condiviso è stato modificato in molti mezzi trasmissivi punto-punto e senza necessità di contesa.

7.4.3 Gigabit Ethernet

Il bisogno di velocità di trasferimento ancora maggiori ha portato alla progettazione di Gigabit Ethernet (1000 Mbps). Come è evidente anche dal nome, lo scopo di Gigabit Ethernet era quello di aggiornare la velocità di trasferimento dati a 1 Gbps mantenendo invariata la lunghezza dell'indirizzo, il formato e la lunghezza massima e minima dei frame.

La topologia delle reti Gigabit Ethernet prevede la presenza di uno switch al centro della stella, e tutti i computer (o altri switch) collegati ai vari rami della stella. Lo switch, per ogni porta, prevede la presenza di buffer dove i frame vengono memorizzati prima di essere trasmessi. Anche in questo caso, il mezzo trasmissivo tra le stazioni e lo switch centrale è privato, non ci sono collisioni e quindi non c'è alcuna necessità di usare CSMA/CD. L'assenza di collisioni implica che la massima lunghezza del cavo non è più determinata dal meccanismo di rilevazione delle collisioni bensì solo dall'attenuazione del segnale.

7.4.4 10-Gigabit Ethernet

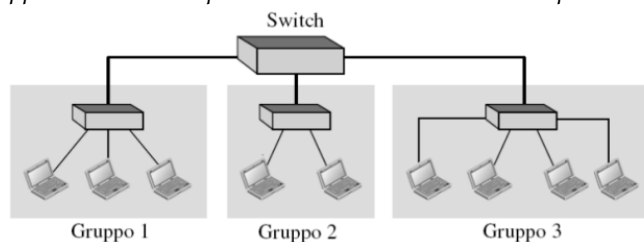
Negli ultimi anni è stata predisposta un'ulteriore revisione di Ethernet per l'uso nelle MAN (Metropolitan Area Network). L'idea alla base in questo caso è quella di estendere la tecnologia, la velocità di trasferimento dei dati e la distanza massima di copertura affinché Ethernet possa essere usata sia come LAN che come MAN, quindi su aree decisamente più vaste che in passato.

7.4.5 LAN virtuali

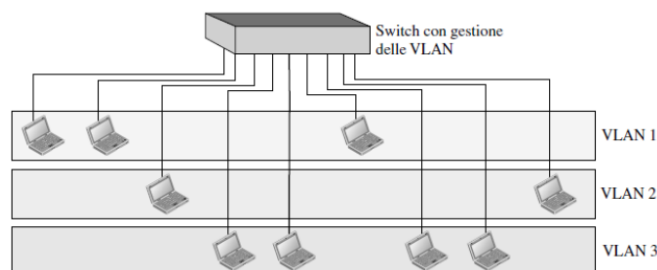
Una stazione è considerata parte di una LAN se ne fa fisicamente parte. Che cosa accade se occorre un "collegamento virtuale" tra due stazioni che appartengono a due LAN fisiche diverse? In maniera piuttosto generale possiamo definire una Virtual Local Area Network (VLAN) come una rete locale configurata per mezzo del software, anziché del cablaggio fisico.

Esempio 7.4.2 – Uno switch che supporta le VLAN

Nella figura viene mostrato un esempio di LAN commutata, ad esempio una piccola ditta, dove 10 stazioni sono raggruppate in 3 LAN e queste ultime sono interconnesse per mezzo di uno switch.



Il personale del primo gruppo lavora assieme, così come per il secondo ed il terzo gruppo. Ma cosa accade se c'è la necessità di spostare uno dei dipendenti del primo gruppo in un altro dei gruppi? E' necessario modificare la configurazione delle LAN, o più in dettaglio è necessario modificare i cavi di collegamento tra le varie stazioni, quindi sono necessari dei cambi fisici alla topologia di rete.



La figura mostra la stessa LAN suddivisa in VLAN. L'idea alla base della tecnologia VLAN è quella di dividere una LAN in segmenti logici, anziché fisici. Una LAN può essere quindi suddivisa in più LAN logiche chiamate VLAN. Nella pratica ciascuna di queste può rappresentare un gruppo di lavoro all'interno della ditta. Se un dipendente si sposta da un gruppo ad un altro, non c'è bisogno di cambiare la configurazione fisica. Nella VLAN il gruppo di appartenenza delle stazioni è definito dal software, non dall'hardware. Qualsiasi stazione può essere spostata facilmente da una VLAN ad un'altra. Tutti i membri appartenenti ad una VLAN possono ricevere i pacchetti broadcast inviati a questa particolare VLAN. Ciò significa che se una stazione si sposta dalla VLAN 1 alla VLAN 2, riceve i messaggi broadcast inviati alla VLAN 2, ma non riceve più quelli inviati alla VLAN 1.

7.5 Switch di livello collegamento

Uno switch di livello collegamento (commutatore di livello collegamento) opera sia a livello fisico che a livello di collegamento. Come dispositivo di livello fisico rigenera il segnale che riceve. Inoltre, come dispositivo di livello collegamento, è in grado di verificare gli indirizzi MAC (sorgente e destinazione) contenuti nel frame.

E' normale chiedersi qual è la differenza di funzionalità tra uno switch e un hub. Il primo ha capacità di filtraggio (filtering), è in grado di verificare l'indirizzo di destinazione del frame e decidere da quale porta in uscita deve essere inviato il frame. Uno switch ha una tabella che viene utilizzata per il filtraggio.

Uno switch trasparente è un commutatore di cui le stazioni presenti nella rete sono completamente ignare dell'esistenza del dispositivo di rete. Se uno switch viene aggiunto o eliminato dalla rete, non è necessaria alcuna operazione di riconfigurazione delle stazioni.

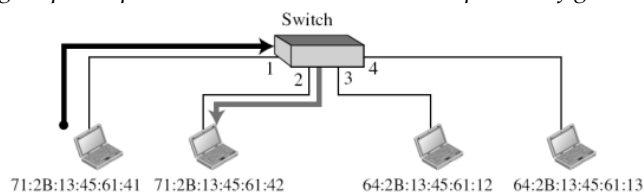
7.5.1 Apprendimento

Inizialmente gli switch operavano usando delle tabelle statiche di commutazione. Questo significa che l'amministratore di rete era costretto ad inserire manualmente ciascuna voce della tabella durante la configurazione dello switch. Nonostante il processo fosse abbastanza semplice, non era molto comodo. Infatti, se una stazione veniva aggiunta o cancellata, la tabella doveva essere modificata manualmente. La stessa cosa doveva essere fatta se l'indirizzo MAC di una stazione cambiava, un evento tutt'altro che raro (ad esempio per il cambio di una scheda di rete).

La soluzione, molto migliore della tabella statica, è una tabella dinamica che associa automaticamente gli indirizzi MAC alle porte (interfacce). Per creare una tabella dinamica, occorre uno switch che sia in grado di apprendere gradualmente dai frame che vengono inviati all'interno della rete. Per fare questo, lo switch analizza gli indirizzi sorgente e destinazione di ogni frame. L'indirizzo di destinazione viene utilizzato per decidere su quale interfaccia inoltrare il frame, quello sorgente invece serve per aggiungere nuove voci alla tabella e quindi, poco alla volta, configurarla automaticamente.

Esempio 7.5.1 – Switch con auto-apprendimento

Vediamo più in dettaglio questo procedimento utilizzando l'esempio della figura.



1. Quando la stazione A invia un frame alla stazione D, lo switch nella sua tabella non ha alcuna voce sia per D che per A. Il frame viene quindi inviato in broadcast su tutte le porte (ad esclusione di quella da cui è stato ricevuto). Analizzando l'indirizzo sorgente, lo switch apprende che la stazione A è collegata alla porta 1. Ciò indica che i frame destinati ad A, in futuro, dovranno essere inviati solo attraverso la porta 1. Lo switch aggiunge immediatamente questa informazione alla sua tabella, che ora contiene la sua prima voce.
2. Quando la stazione D invia un frame alla stazione B, lo switch non ha una voce relativa a B, così invia nuovamente in broadcast il frame. Tuttavia, questo frame permette di aggiungere un'ulteriore voce alla tabella: la porta a cui è collegata la stazione D.
3. Il processo di apprendimento continua fino a quando la tabella non contiene informazioni su tutte le stazioni e le relative porte. C'è da notare che il processo di apprendimento potrebbe richiedere molto tempo. Ad esempio, se una stazione non trasmette alcun frame essa non avrà mai una voce nella tabella, ma si tratta di una situazione estremamente improbabile.

Gli switch sono dispositivi plug-and-play che non richiedono intervento dell'amministratore di rete o dell'utente. Eliminano le collisioni ovvero bufferizzano i frame e non trasmettono più di un frame alla volta su ogni segmento di rete, inoltre interconnettono link eterogenei cioè collegamenti che operano a diverse velocità possono essere collegati a uno switch. Aumentano la sicurezza della rete e migliorano il network management. Infine forniscono informazioni su uso di banda, collisioni, tipi di traffico...

Capitolo 8

Reti wireless

Analizzeremo varie tecnologie wireless, quali le LAN wireless, le reti cellulari e le reti d'accesso wireless. La natura delle LAN wireless è diversa da quella delle LAN cablate, come vedremo in questo capitolo. In Internet sta aumentando l'uso di altre reti wireless come la telefonia cellulare. Quindi analizzeremo le reti basate su IEEE 802.11, che è lo standard dominante nelle LAN wireless. Poi parleremo delle LAN Bluetooth che vengono usate in molte applicazioni in maniera circoscritta all'utente senza connessione ad Internet.

8.1 LAN wireless

La comunicazione wireless è una tecnologia in forte espansione. La domanda di dispositivi di comunicazione senza l'uso di cavi sta aumentando ovunque. Le reti LAN wireless si possono trovare nei campus universitari, negli uffici e in molte aree pubbliche.

8.1.1 Introduzione

La Figura 8.1 mostra l'ambiente nel quale considereremo la comunicazione dati wireless e la mobilità. In una rete wireless possiamo identificare i seguenti elementi.

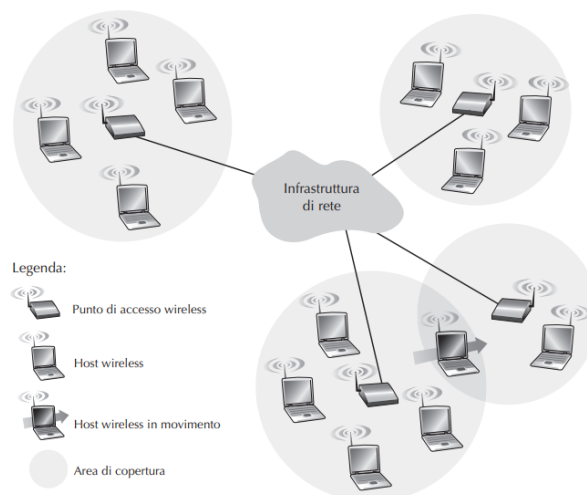


Figura 8.1: Componenti di una rete wireless

- *Host wireless.* Come nel caso delle reti cablate, gli host sono dispositivi periferici che eseguono applicazioni. L'host wireless può essere un portatile, un tablet, un telefono o un computer desktop. Gli host possono essere mobili o meno.
- *Collegamenti wireless.* L'host si connette alla stazione base o a un altro host attraverso un canale di comunicazione wireless. La Figura 8.2 mostra le due caratteristiche principali (area di copertura e frequenza del collegamento) dei più diffusi standard per collegamenti wireless.

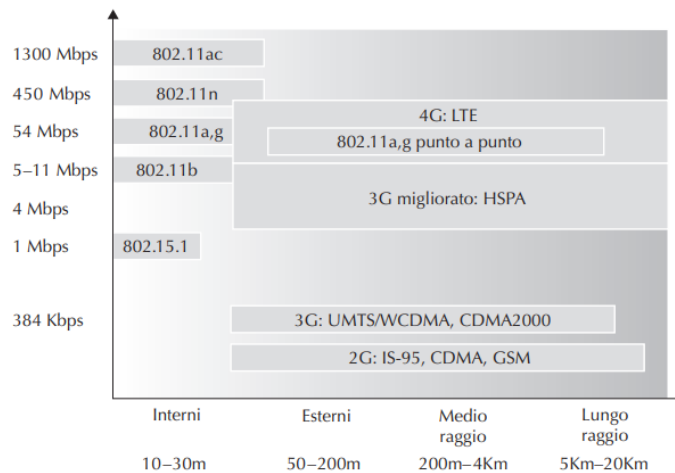


Figura 8.2: Caratteristiche dei collegamenti di alcuni standard per reti wireless

Nella Figura 8.1 sono illustrati i collegamenti wireless che connettono host collocati alle estremità di una rete. Vedremo che i collegamenti wireless sono utilizzati per connettere router, commutatori e anche altri dispositivi di rete.

- *Stazione base (base station)*. E' il componente chiave dell'infrastruttura delle reti wireless e, a differenza degli host e dei collegamenti wireless, non ha una controparte corrispondente nelle reti cablate. Una stazione base è responsabile dell'invio e della ricezione dei dati (pacchetti) tra gli host wireless e essa associati. Quando diciamo che un host wireless è "associato" a una stazione base, intendiamo che (1) l'host si trova nell'area di copertura della stazione base e che (2) la utilizza per trasmettere dati verso il resto della rete. Un esempio di queste stazioni base sono i ripetitori di cella (cell tower) nelle reti cellulari e gli access point (punti di accesso) nelle LAN 802.11.

Nella Figura 8.1 la stazione base è connessa al resto della rete (per esempio, Internet, rete aziendale, domestica o telefonica) e funziona quindi come ripetitore tra l'host e il resto del mondo con cui comunica. Gli host associati a una stazione base sono considerati come operanti in modalità infrastruttura (infrastructure mode), in quanto tutti i tradizionali servizi di rete (come l'assegnamento degli indirizzi e l'instradamento) sono forniti dalla rete attraverso la stazione base.

- *Infrastruttura di rete*. E' la rete più ampia alla quale l'host wireless potrebbe volersi connettere.

Confronto architetturale

Per prima cosa confrontiamo l'architettura delle LAN cablate e di quelle wireless per dare un'idea di quali fattori è necessario considerare quando studiamo le LAN wireless.

- *Mezzo trasmissivo*. La prima differenza che possiamo osservare tra una LAN cablata e una wireless è il mezzo trasmissivo. Nelle reti cablate usiamo dei cavi per connettere gli host. In una LAN wireless, il mezzo trasmissivo è l'aria e il segnale normalmente è broadcast. Quando gli host in una LAN wireless comunicano l'uno con l'altro, essi condividono lo stesso mezzo (ad accesso multiplo). In una situazione molto rara possiamo creare una comunicazione punto-punto tra due host wireless usando un'ampiezza di banda molto limitata e delle antenne bidirezionali.
- *Host wireless*. In una LAN cablata un host è sempre connesso alla sua rete in un punto con un indirizzo MAC fisso che si riferisce alla sua scheda di rete (Network Interface Card, NIC). Naturalmente un host in Internet può spostarsi da un punto all'altro. In questo caso il suo indirizzo MAC rimane lo stesso, ma l'indirizzo IP cambia. Tuttavia, prima che l'host possa usare i servizi di Internet, deve essere fisicamente connesso alla rete. In una LAN wireless, un host non è fisicamente connesso alla rete, ma può muoversi liberamente e usare i servizi da essa forniti.

- *LAN isolate.* Anche il concetto di LAN cablata isolata è diverso da quello di LAN wireless isolata. Una LAN cablata isolata è un insieme di host connessi tramite uno switch di collegamento (nella generazione recente dell'Ethernet). Una LAN wireless isolata, chiamata rete ad hoc nella terminologia LAN wireless, è un insieme di host che comunicano liberamente tra loro. Il concetto di switch di collegamento non esiste in questo tipo di rete.

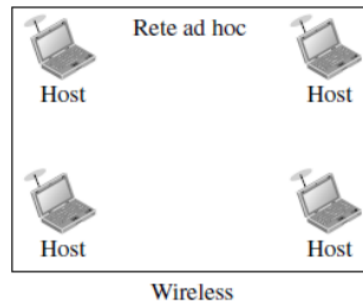


Figura 8.3: Reti ad hoc

- *Connessione ad altre reti.* Una LAN cablata si può connettere a un'altra rete o a Internet tramite un router. Una LAN wireless si può connettere a una rete con infrastruttura cablata, a una rete con infrastruttura wireless o a un'altra LAN wireless mediante una base station (stazione base), che nel caso delle reti LAN wireless 802.11 prende il nome di Access Point (AP) o punto di accesso.

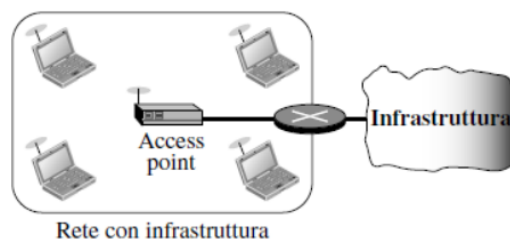


Figura 8.4: Connessione di una LAN wireless ad altre reti

In questo caso la LAN wireless viene definita rete con infrastruttura (o infrastrutturata) e la connessione all'infrastruttura cablata, come Internet, avviene tramite l'access point, il cui ruolo è completamente diverso da quello di uno switch di collegamento nell'ambiente cablato.

- *Migrazione dall'ambiente cablato a quello wireless.* Il funzionamento di una LAN cablata o una wireless dipende dai due livelli inferiori dello stack protocollare TCP/IP. Ciò significa che se abbiamo una LAN cablata in un edificio collegato a Internet tramite un router o un modem, per migrare dall'ambiente cablato a quello wireless è sufficiente cambiare le schede di rete progettate per ambienti cablati con quelle studiate per ambienti wireless e sostituire lo switch di collegamento con un AP. In questo cambiamento anche gli indirizzi MAC si modificheranno (a causa delle NIC diverse), mentre gli indirizzi IP resteranno gli stessi, ci stiamo spostando dai collegamenti cablati a quelli wireless.

Caratteristiche di rete

Le principali caratteristiche del link wireless sono:

- *Attenuazione del segnale.* Le radiazioni elettromagnetiche si attenuano quando attraversano determinati ostacoli (come le pareti delle case). Anche nello spazio libero l'intensità del segnale si attenua al crescere della distanza percorsa (path loss).
- *Interferenze da parte di altre sorgenti.* Sorgenti radio che trasmettono nella stessa banda di frequenza interferiscono tra loro. Per esempio, i telefoni a 2,4 GHz e le LAN 802.11b trasmettono

nella stessa banda di frequenze. Quindi, gli utenti della LAN 802.11b che parlano con un telefono wireless a 2,4 GHz non devono sorprendersi se rete e telefono presentano malfunzionamenti. Inoltre il rumore elettromagnetico ambientale (per esempio, di un motore o di un forno a microonde) può generare interferenze.

- *Propagazione su più cammini (multipath propagation)*. La propagazione su più cammini si verifica quando una parte delle onde elettromagnetiche si riflette su oggetti e sul terreno, percorrendo cammini di diversa distanza tra il trasmittente e il ricevente. Questo fenomeno disturba il segnale che giunge al destinatario. Oggetti in movimento tra il trasmittente e il ricevente possono causare il fenomeno di propagazione multipla, che varia in ogni momento.

Quanto osservato fin qui suggerisce che gli errori nei bit siano più frequenti nei collegamenti wireless che nelle reti cablate. Per questo motivo non dobbiamo sorprendersi se i protocolli di collegamento wireless utilizzano non solo potenti codici CRC di rilevazione degli errori, ma anche un protocollo di trasferimento affidabile a livello di collegamento, che ritrasmette i pacchetti danneggiati.

Dopo aver considerato i deterioramenti che possono avvenire in un canale wireless, rivolgiamo la nostra attenzione agli host che ricevono il segnale. Questi host ricevono un segnale elettromagnetico che è la combinazione di una forma degradata del segnale originale trasmesso dal mittente (degradato, per esempio, a causa degli effetti dell'attenuazione e della propagazione su cammini multipli) e un rumore di fondo dell'ambiente. Il rapporto segnale rumore (SNR, signal-to-noise ratio) è una misura relativa dell'intensità del segnale ricevuto, cioè dell'informazione che è stata trasmessa, e del rumore. Per i nostri scopi, ci basti sapere che un SNR più grande facilita il ricevente nel distinguere il segnale trasmesso dal rumore di fondo.

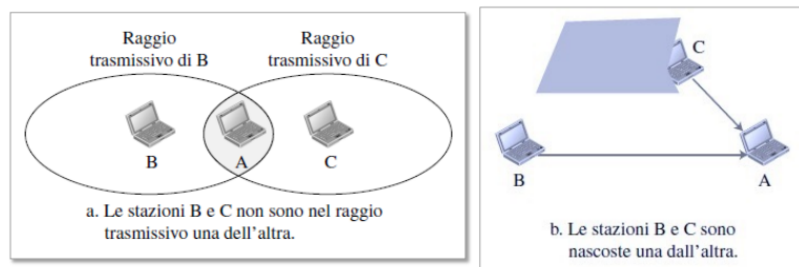
Controllo dell'accesso al mezzo

Forse il problema più importante di cui dobbiamo parlare in una LAN wireless è il controllo d'accesso, ovvero come un host wireless può avere accesso al mezzo di trasmissione condiviso (aria). Abbiamo detto che l'Ethernet Standard usava l'algoritmo CSMA/CD. Secondo tale metodo ogni host compete per accedere al mezzo e trasmette il suo frame se il mezzo è inattivo. Se avviene una collisione, essa viene rilevata e il frame inviato nuovamente. Il rilevamento delle collisioni nel CSMA/CD ha due scopi. Il fatto che venga rilevata una collisione significa che il frame non è stato ricevuto e deve essere inviato nuovamente. Se invece non viene rilevata alcuna collisione, allora vuol dire che il frame è stato ricevuto correttamente. L'algoritmo CSMA/CD non funziona nelle LAN wireless per tre ragioni.

1. Per rilevare una collisione un host deve poter inviare (il proprio frame) e ricevere (ascoltare il canale per capire se qualcun altro sta trasmettendo) contemporaneamente, il che significa che l'host deve operare in modalità duplex. Poiché la potenza del segnale ricevuto è molto inferiore a quella del segnale trasmesso, sarebbe troppo costoso impiegare un adattatore di rete in grado di rilevare collisioni (i dispositivi wireless hanno un'energia limitata, fornita dalla batteria, che non gli consente di utilizzare un tale dispositivo). Pertanto i dispositivi wireless non possono rilevare le collisioni (Collision Detection) come avviene per Ethernet.
2. A causa del problema del terminale nascosto (hidden terminal problem) secondo cui un host, nel seguito chiamato anche terminale o stazione, potrebbe non sapere che un altro host sta trasmettendo a causa di ostacoli o questioni dovute al raggio di trasmissione - una collisione potrebbe avvenire senza essere rilevata.

Esempio 8.1.1 – Problema del terminale nascosto

La figura mostra un esempio del problema del terminale nascosto.



Il raggio di trasmissione dell'host B è mostrato dall'ovale sinistro (sfera nello spazio); ogni host all'interno di tale raggio può sentire i segnali trasmessi da B. Analogamente, il raggio di trasmissione della stazione C è indicato con l'ovale destro (sfera nello spazio); ogni host situato entro tale raggio riceve i segnali trasmessi da C. L'host C è all'esterno del raggio di trasmissione di B; allo stesso modo B è all'esterno del raggio di C. L'host A, tuttavia, si trova nell'area coperta sia da B che da C quindi può sentire qualsiasi segnale trasmesso da B o da C. La figura mostra inoltre che il problema del terminale nascosto può presentarsi anche a causa di un ostacolo.

Supponiamo che B stia inviando dei dati ad A. Nel bel mezzo della trasmissione, anche C ha dei dati da inviare ad A. Tuttavia, C è fuori dal raggio di B e le trasmissioni di B non possono raggiungere C. Quindi C crede che il mezzo sia libero, e invia i suoi dati ad A, il che risulta in una collisione su A in quanto tale host si trova a ricevere dei dati sia da B che da C. In questo caso possiamo dire che i terminali B e C sono nascosti tra loro rispetto ad A. I terminali nascosti possono ridurre la capacità della rete a causa della possibilità di collisioni.

3. La distanza tra gli host può essere grande. L'attenuazione del segnale può impedire a un host di sentire una collisione che avviene ad un'altra estremità.

8.1.2 IEEE 802.11

L'IEEE ha definito le specifiche per le LAN wireless, chiamate IEEE 802.11, che copre i livelli fisico e di collegamento. In alcune nazioni, compresi gli Stati Uniti, si usa il termine Wi-Fi (abbreviazione di wireless fidelity) come sinonimo di LAN wireless. Wi-Fi è una LAN wireless certificata dalla Wi-Fi Alliance, un'associazione industriale globale no-profit, di cui fanno parte oltre 300 aziende, che si occupa di promuovere la crescita delle LAN wireless.

Architettura

Lo standard IEEE 802.11 definisce due tipi di architetture: il basic service set (BSS) e l'extended service set (ESS). L'architettura basic service set (BSS) è costituita da una o più stazioni wireless (dette anche host, dispositivi, nodi) e da un access point facoltativo. La Figura 8.5 mostra due BSS di questo standard.

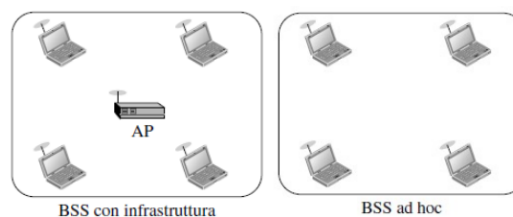


Figura 8.5: Basic service set (BSS)

Il secondo tipo di architettura, l'extended service set (ESS), è costituito da due o più BSS con infrastruttura. In questo caso i BSS sono collegati tramite un sistema di distribuzione, che è una rete cablata o wireless. Il sistema di distribuzione collega gli AP nei BSS. L'IEEE 802.11 non limita

il sistema di distribuzione, può essere una LAN IEEE come ad esempio una Ethernet. La Figura 8.6 illustra un ESS.

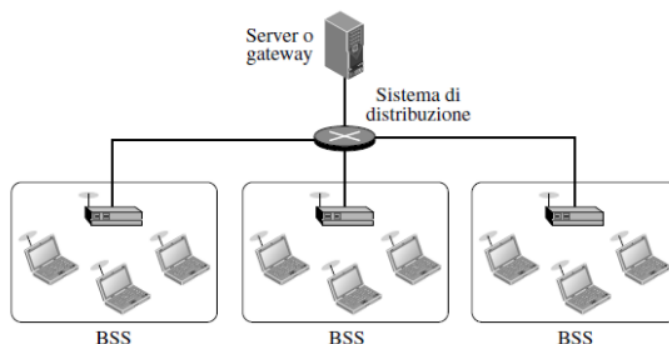


Figura 8.6: Extended service set (ESS)

Quando i BSS sono collegati, le stazioni che si possono raggiungere a vicenda (in visibilità) comunicano senza l'uso di un AP. Tuttavia, la comunicazione tra una stazione in un BSS e una al suo esterno avviene tramite l'AP.

Prima di inviare o ricevere pacchetti dati 802.11 le stazioni devono associarsi a un AP. Quando un amministratore di rete installa un AP, gli assegna uno o due identificativi, detti SSID (service set identifier). L'amministratore deve anche assegnare all'AP un numero di canale. Per comprendere questo concetto ricordiamo che 802.11 opera nella banda di frequenza da 2,4 GHz a 2,4835 GHz. In questi 85 MHz di banda, 802.11 definisce 11 canali parzialmente sovrapposti. Due canali non si sovrappongono solo se sono separati da quattro o più canali. In particolare, i canali 1, 6 e 11 costituiscono l'unica terna priva di sovrapposizione. Questo significa che un amministratore potrebbe creare una LAN wireless con una capacità trasmissiva globale di 33 Mbps, installando tre AP 802.11b nello stesso luogo, assegnando i canali 1, 6 e 11 agli AP e connettendoli con uno switch.

L'architettura IEEE 802.11 prevede che una stazione wireless si associ a un AP per accedere a Internet. Sorge dunque il problema di conoscere quali AP sono disponibili in un BSS e come avviene l'associazione. Lo standard 802.11 prevede che un AP invii dei segnali periodici, detti beacon, che includono l'identificatore dell'AP (Service Set Identifier - SSID) e il suo indirizzo MAC. La stazione wireless che vuole entrare in un BSS, scandisce gli 11 canali trasmissivi alla ricerca di frame beacon (passive scanning). Alla fine della scansione la stazione sceglie l'AP da cui ha ricevuto il beacon con la maggiore potenza di segnale e gli invia un frame con la richiesta di associazione. L'AP accetta l'associazione rispondendo con un frame di risposta di associazione, che permetterà all'host entrante di inviare una richiesta DHCP per ottenere un indirizzo IP nella rete.

La richiesta di associazione verso un particolare AP può avvenire mediante una fase di autenticazione: l'AP può richiedere alla stazione di autenticarsi mediante un nome utente e una password. Oppure l'autenticazione può avvenire in base all'indirizzo MAC della stazione. Una volta eseguita l'associazione, la stazione wireless può comunicare con l'AP per inviare e ricevere traffico anche verso/da Internet. Tuttavia, più stazioni possono voler trasmettere frame nello stesso momento, per cui è necessario regolare l'accesso al canale con un protocollo MAC che coordini le trasmissioni.

Protocollo MAC 802.11

L'IEEE 802.11 definisce due tecniche di accesso al mezzo: una distribuita (distributed coordination function - DCF) in cui i nodi si contendono l'accesso al canale, e un'altra centralizzata (point coordination function - PCF), in cui non c'è contesa e l'AP coordina l'accesso dei nodi al canale. In questo testo ci concentriamo sulla prima tecnica.

La tecnica Distributed Coordination Function (DCF) usa il protocollo CSMA/CA come metodo d'accesso. Poiché nelle reti wireless la rilevazione delle collisioni (collision detection) non può essere eseguita durante la trasmissione, è stato definito il protocollo di Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA) che ha lo scopo di evitare che si verifichino collisioni.

Al contrario del protocollo Ethernet 802.3, il protocollo MAC 802.11 non implementa la rilevazione delle collisioni. Esistono due fondamentali ragioni per questa scelta.

- La possibilità di rilevare collisioni richiede la capacità di inviare (il segnale proprio della stazione) e ricevere (per determinare se un'altra stazione sta anch'essa trasmettendo) contempora-

neamente. Essendo in genere la potenza del segnale ricevuto nettamente inferiore alla potenza del segnale trasmesso dall'adattatore 802.11, risulta molto costoso costruire hardware che possa rilevare una collisione.

- Estremamente più significativo è il fatto che l'adattatore, anche se fosse in grado di trasmettere e ricevere allo stesso istante (presumibilmente interrompendo la trasmissione quando trova il canale occupato), non potrebbe rilevare tutte le collisioni a causa del problema del terminale nascosto e dell'attenuazione del segnale.

Ricordiamo che un frame inviato da una stazione in una rete wireless potrebbe non raggiungere la stazione di destinazione per numerosi motivi. Per contrastare queste non trascurabili possibilità di fallimento, MAC 802.11 utilizza la conferma di avvenuta ricezione a livello di collegamento. Come mostrato nella Figura 8.7, quando la stazione di destinazione riceve un frame che passa il controllo CRC, attende per un breve periodo di tempo, noto come SIFS (short inter-frame space, spazio breve inter-frame), dopo il quale invia al mittente un frame di conferma di avvenuta ricezione. Se la stazione trasmittente non riceverà questo riscontro entro un arco di tempo stabilito, presupporrà un errore e ritrasmetterà il frame, utilizzando ancora il protocollo CSMA/CA per accedere al canale. Se il frame di conferma non sarà ricevuto dopo un numero prefissato di ritrasmissioni, la stazione trasmittente passerà oltre e scarnerà il frame.

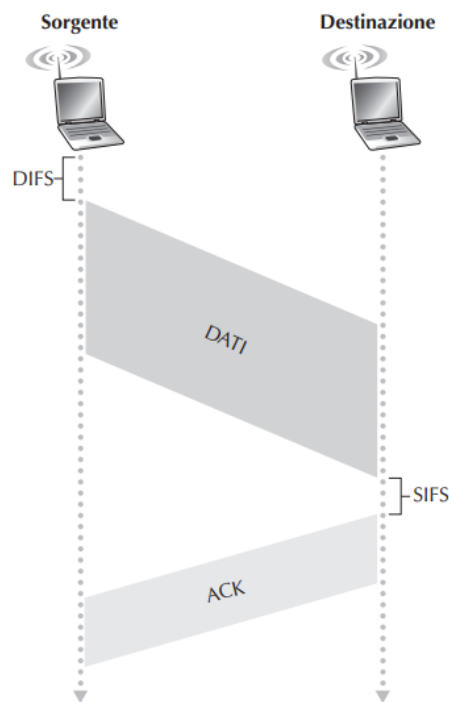


Figura 8.7: IEEE 802.11 usa acknowledgment a livello di collegamento

Prima di tutto vengono evitate le collisioni rimandando la trasmissione anche se il canale viene trovato libero: rilevata la portante, se il canale risulta libero, la stazione non trasmette immediatamente, ma attende un lasso di tempo chiamato interframe space o IFS. La motivazione di questa scelta sta nel fatto che il canale può apparire inattivo quando viene rilevato, ma una stazione distante può avere già iniziato a trasmettere. Il segnale della stazione distante non ha ancora raggiunto la stazione in oggetto. Il tempo IFS consente alla prima parte del segnale trasmesso dalla stazione distante di raggiungere la stazione che vuole trasmettere. Dopo aver atteso per un tempo IFS, se il canale è ancora inattivo, la stazione prima di trasmettere deve attendere un tempo di back-off, equivalente al tempo di contesa.

1. Se inizialmente la stazione percepisce il canale come inattivo, allora trasmette il suo frame dopo un breve periodo di tempo conosciuto come DIFS (distributed inter-frame space, spazio distribuito inter-frame).

2. Altrimenti, la stazione sceglie un valore casuale di ritardo usando una attesa binaria esponenziale e decrementa questo valore dopo DIFS solo quando il canale viene percepito come inattivo. Se il canale è percepito come occupato, il contatore rimane fermo.
3. Quando il contatore arriva a zero (notiamo che questo può verificarsi soltanto quando il canale è percepito come inattivo), la stazione trasmette l'intero frame e aspetta il frame di conferma.
4. Se riceve la conferma, la stazione sa che il frame è stato ricevuto correttamente e, qualora avesse un altro frame da inviare, riattiva il protocollo CSMA/CA dal passo 2. Se il frame di conferma non viene ricevuto, la stazione trasmittente ritorna al passo 2, ma con un valore di ritardo maggiore.

La finestra di contesa consiste di un lasso di tempo diviso in slot, detto back-off time. Una stazione che vuole trasmettere sceglie come tempo di attesa una quantità casuale di slot (back-off timer). Il numero di slot nella finestra cambia ad ogni trasmissione di un nuovo frame in base alla strategia di back-off binario esponenziale. Ciò significa che la prima volta il back-off viene impostato a uno slot e poi raddoppia ogni volta che la stazione non riesce a ottenere accesso al canale o a trasmettere con successo il frame. Un aspetto interessante della finestra di contesa è che la stazione deve ascoltare il canale dopo ogni slot di tempo: se trova il canale occupato, interrompe il timer e lo riavvia quando il canale viene rilevato libero.

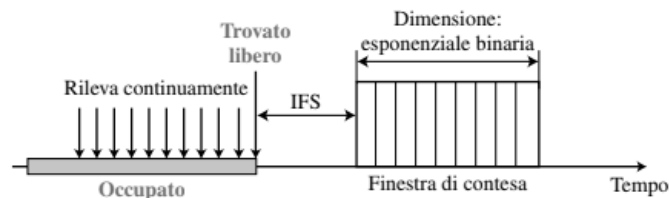
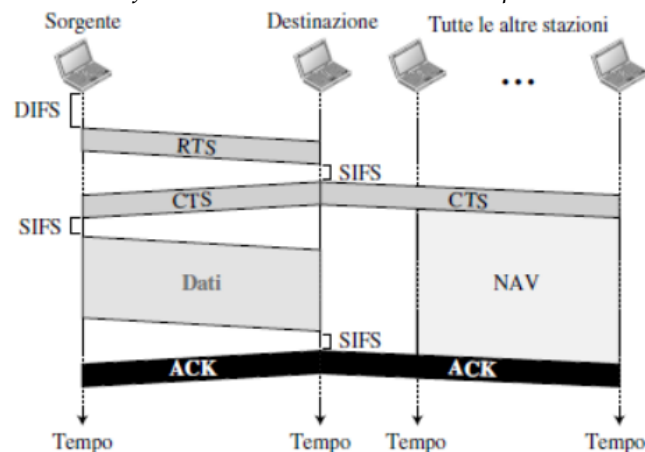


Figura 8.8: Finestra contesa

Poiché questi meccanismi non garantiscono che non avvenga una collisione, è necessario utilizzare riscontri positivi e timer per capire se la trasmissione del frame è andata a buon fine.

Esempio 8.1.2 – Modello temporale dello scambio di frame

La figura illustra lo scambio dei frame di controllo e di dati nel tempo.



1. Prima di inviare un frame la stazione sorgente ascolta il canale controllando il livello d'energia sulla frequenza portante. La stazione usa una strategia di persistenza con back-off finché il canale non è libero. Dopo che il canale viene rilevato inattivo, la stazione attende per un lasso di tempo chiamato DCF interframe space (DIFS), quindi invia un frame di controllo chiamato request to send (RTS), che indica la volontà di spedire un frame.
2. Dopo aver ricevuto l'RTS e aver atteso per un lasso di tempo chiamato short interframe space (SIFS), la stazione di destinazione invia un frame di controllo, chiamato clear to

send (CTS) alla stazione sorgente, che indica che la stazione di destinazione è pronta a ricevere i dati.

3. La stazione sorgente attende un lasso di tempo pari al SIFS, dopodiché invia i dati.
4. La stazione di destinazione, dopo aver atteso un lasso di tempo pari al SIFS, invia un riscontro (ack) per mostrare che il frame è stato ricevuto. Il riscontro è necessario in tale protocollo in quanto la stazione sorgente non ha alcun mezzo per verificare l'effettivo arrivo dei suoi dati a destinazione.

Se una stazione acquisisce l'accesso, come fanno le altre a posticipare l'invio dei loro dati? In altre parole, come si evita la collisione in questo protocollo? La chiave è una caratteristica chiamata NAV.

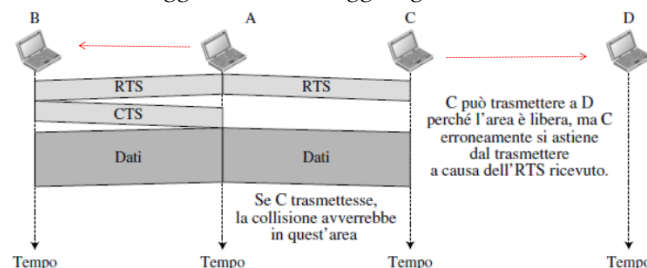
Quando una stazione invia un frame RTS include la durata di tempo in cui occuperà il canale per trasmettere il frame e ricevere l'ack. Le stazioni che sono influenzate da tale trasmissione avviano un timer chiamato network allocation vector (NAV) che indica quanto tempo deve passare prima che possano verificare l'inattività del canale. Ogni volta che una stazione accede al sistema ed invia un frame RTS le altre stazioni avviano i loro NAV. In altre parole ogni stazione, prima di ascoltare il canale per vedere se è inattivo, verifica che il suo NAV sia scaduto.

Che cosa accade se avviene una collisione durante il tempo in cui i frame di controllo RTS o CTS sono in transizione, spesso definito periodo di handshaking? Due o più stazioni possono cercare di inviare dei frame RTS contemporaneamente. Questi frame di controllo possono collidere. Tuttavia, siccome non c'è meccanismo per rilevare la collisione, il mittente suppone che ci sia stata una collisione se non ha ricevuto un frame CTS dal destinatario. Viene quindi applicata la strategia del back-off e il mittente prova di nuovo.

La soluzione al problema del terminale nascosto è l'uso dei frame RTS e CTS.

Esempio 8.1.3 – Problema del terminale nascosto

La figura mostra anche che il messaggio RTS da B raggiunge A, ma non C.



Tuttavia, siccome sia B che C si trovano entro il raggio di A, il messaggio CTS, che contiene la durata della trasmissione dei dati da B ad A, raggiunge C. La stazione C capisce quindi che c'è una trasmissione che interessa uno dei suoi vicini e si astiene dall'invio sino alla fine della trasmissione.

Il frame di livello MAC è costituito da nove campi, come mostrato nella Figura 8.9.

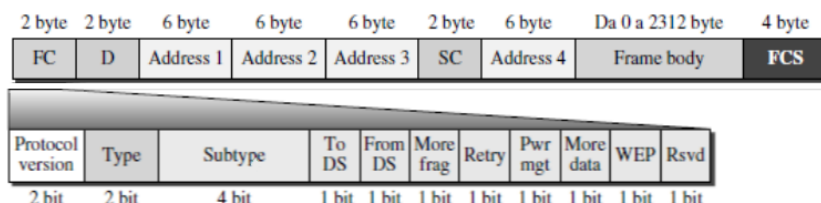


Figura 8.9: Formato del frame

- *Frame Control (FC)*. Il campo FC è lungo 2 byte e definisce il tipo di frame e alcune informazioni di controllo.
- *D*. Questo campo definisce la durata della trasmissione che viene usata per impostare il valore del NAV. In un frame di controllo esso definisce l'ID del frame.

- *Indirizzi*. Ci sono quattro campi indirizzo, ognuno lungo 6 byte. Il significato di ogni campo dipende dal valore dei sottocampi To DS e From DS che verranno analizzati in seguito.
- *SC (controllo sequenza)*. Questo campo definisce un valore da 16 bit. I primi quattro bit indicano il numero del frammento e gli ultimi 12 quello della sequenza, che è la stessa in tutti i frammenti.
- *Frame body (corpo del frame)*. Questo campo, che può essere da 0 a 2312 byte, contiene le informazioni sulla base del tipo e del sotto-tipo definito nel capo FC.
- *FCS*. Il campo FCS è lungo 4 byte e contiene una sequenza CRC-32 di rilevamento errori.

Una LAN wireless definita dall'IEEE 802.11 ha tre categorie di frame: gestione, controllo e dati.

- *Frame di gestione*. Vengono usati per la comunicazione iniziale tra stazioni e punti d'accesso. Per i frame di gestione il valore del campo tipo è 00.
- *Frame di controllo*. Si usano per accedere al canale e dare riscontro ai frame. Per i frame di controllo il valore del campo tipo è 01; i valori dei campi sotto-tipo per i frame di cui abbiamo parlato sono 1011:RTS, 1100:CTS, 1101:ACK.
- *Frame di dati*. Vengono usati per trasportare i dati e le informazioni di controllo.

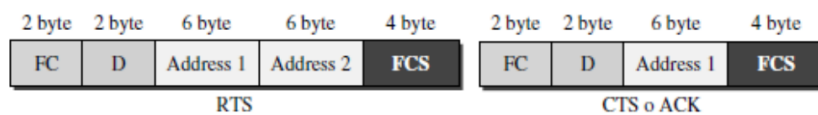


Figura 8.10: Frame di controllo

Indirizzamento

Il meccanismo d'indirizzamento dell'IEEE 802.11 specifica quattro casi, definiti dal valore dei due flag nel campo FC, To DS e From DS. Ogni flag può essere 0 o 1, di conseguenza possiamo avere quattro situazioni diverse. L'interpretazione dei quattro indirizzi (da 1 a 4) nel frame MAC dipende dal valore di tali flag.

To DS	From DS	Address 1	Address 2	Address 3	Address 4
0	0	Destinazione	Sorgente	DSS ID	N/A
0	1	Destinazione	AP mittente	Sorgente	N/A
1	0	AP ricevente	Sorgente	Destinazione	N/A
1	1	AP ricevente	AP mittente	Destinazione	Sorgente

Tabella 8.1: Indirizzi

Da notare che l'indirizzo 1 è sempre quello del dispositivo successivo che verrà visitato dal frame. L'indirizzo 2 è sempre quello del dispositivo precedente che il frame ha lasciato. L'indirizzo 3 è quello della stazione di destinazione finale se non viene definito dall'indirizzo 1 o quello della stazione sorgente originale se non è definito dall'indirizzo 2. L'indirizzo 4 è la sorgente originale quando anche il sistema di distribuzione è wireless.

- *Caso 00*. In questo caso $To DS = 0$ e $From DS = 0$. Ciò significa che il frame non sta andando verso un sistema di distribuzione ($To DS = 0$) e non sta arrivando da uno di essi ($From DS = 0$). Esso sta andando da una stazione in un BSS verso un'altra senza passare dal sistema di distribuzione.
- *Caso 01*. In questo caso $To DS = 0$ e $From DS = 1$. Ciò significa che il frame arriva da un sistema di distribuzione ($From DS = 1$). Esso giunge da un AP e si dirige verso una stazione. Da notare che l'indirizzo 3 contiene il mittente originale del frame (in un altro BSS).

- *Caso 10.* In questo caso $To DS = 1$ e $From DS = 0$. Ciò significa che il frame sta andando verso un sistema di distribuzione ($To DS = 1$). Esso va da una stazione a un AP. L'ACK viene inviato alla stazione originale. Da notare che l'indirizzo 3 contiene la destinazione finale del frame nel sistema di distribuzione.
- *Caso 11.* In questo caso $To DS = 1$ e $From DS = 1$. Questo è il caso in cui anche il sistema di distribuzione è wireless. Il frame sta andando da un AP a un altro AP in un sistema di distribuzione wireless. Qui servono quattro indirizzi per definire il mittente originale, la destinazione finale e due AP intermedi.

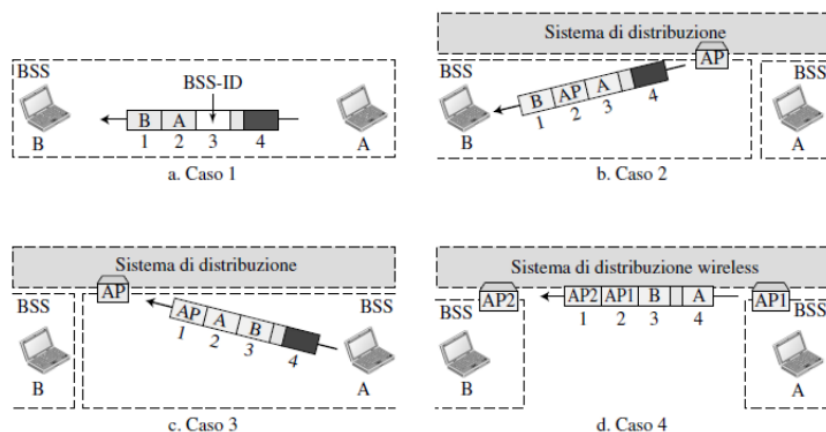


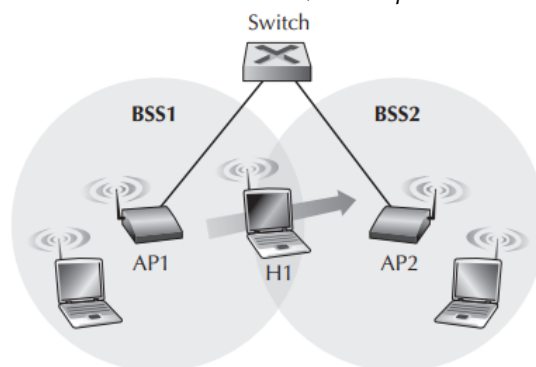
Figura 8.11: Meccanismo d'indirizzamento

Mobilità all'interno di una sottorete IP

Per aumentare la copertura di una LAN wireless, aziende e università dispongono spesso vari BSS all'interno di una stessa sottorete IP. Questo naturalmente fa nascere il problema della mobilità all'interno dei BSS: come può una stazione wireless muoversi da un BSS a un altro durante la stessa sessione TCP? Come vedremo ora, la mobilità può essere trattata in modo semplice quando i BSS fanno parte di una stessa sottorete.

Esempio 8.1.4 – Mobilità all'interno della stessa sottorete

La figura mostra due BSS interconnessi con l'host H1, che si sposta da BSS1 a BSS2.



Dato che in questo esempio il dispositivo che connette i due BSS non è un router, tutte le stazioni all'interno di queste, inclusi gli AP, apparterranno alla stessa sottorete IP. Quindi, quando H1 si sposta da BSS1 a BSS2, può mantenere il suo indirizzo IP e tutte le sue connessioni TCP aperte. Se il dispositivo d'interconnessione fosse stato un router, allora H1 avrebbe dovuto cambiare l'indirizzo IP e chiudere tutte le connessioni TCP, oppure usare un protocollo di mobilità a livello di rete come, per esempio, IP mobile.

Ma, in dettaglio, che cosa accade quando H1 si sposta da BSS1 a BSS2? Man mano che H1 si allontana da AP1, ne riceve sempre più debolmente il segnale e inizia a ricercarne uno più

intenso di un altro access point. H1 riceve i frame beacon da AP2 che, come spesso avviene, avrà lo stesso SSID di AP1. Quindi H1 si disconnette da AP1 e si associa ad AP2, mentre mantiene il suo indirizzo IP e le sue connessioni TCP aperte.

Questo risolve il problema dell'handoff dal punto di vista dell'host e dell'AP. Ma come fa uno switch a sapere che l'host si è spostato da un AP a un altro? Come ricorderete, gli switch auto-apprendono, costruendo automaticamente le proprie tabelle d'indirizzo. Grazie a questa qualità di auto-apprendimento si riescono ad affrontare positivamente spostamenti occasionali (per esempio, quando un impiegato si sposta da un ufficio a un altro); ma gli switch non sono stati progettati per supportare utenti che si spostano di frequente, pretendendo di mantenere le loro connessioni TCP mentre cambiano BSS. Per valutare questo problema, ricordiamo che prima dello spostamento, lo switch ha una voce nella sua tabella d'indirizzo che associa l'indirizzo MAC di H1 alla porta attraverso la quale H1 può essere raggiunto. Se H1 è inizialmente in BSS1, allora il datagramma destinato a H1 gli sarà inoltrato attraverso l'AP1. Una volta che H1 si sarà associato a BSS2, comunque, il suo frame dovrebbe essere diretto ad AP2. Una soluzione (un trucco, in realtà) potrebbe essere che, appena realizzata la nuova associazione, AP2 invii in broadcast allo switch un frame Ethernet, con l'indirizzo sorgente di H1. Ricevuto il frame, lo switch aggiornerà la sua tabella d'indirizzo, in modo che H1 sia raggiungibile attraverso AP2.

8.1.3 Bluetooth

Il Bluetooth è una tecnologia LAN wireless progettata per connettere dispositivi con diverse funzioni, come telefoni, notebook, computer (desktop e laptop), macchine fotografiche, stampanti e anche macchinette del caffè quando si trovano a una breve distanza l'una dall'altra. Una LAN Bluetooth è una rete ad hoc, il che significa che si forma spontaneamente, senza l'aiuto di alcuna stazione base (access point); i dispositivi si accordano l'un l'altro e creano una rete chiamata piconet, che può essere composta al massimo da 8 dispositivi. Una LAN Bluetooth può essere connessa persino a Internet se uno dei dispositivi ha tale capacità. Essa però, per natura, non può essere grande. Se ci sono molti dispositivi che cercano di connettersi si crea il caos.

Il Bluetooth originariamente è stato avviato come progetto dalla società Ericsson. Il suo nome deriva da Harald Blaatand, re di Danimarca (940-981) che unì Danimarca e Norvegia. Blaatand tradotto in inglese significa Bluetooth.

Oggigiorno la tecnologia Bluetooth è l'implementazione di un protocollo definito dallo standard IEEE 802.15. Lo standard definisce una rete wireless di area personale (PAN) funzionante in un'area delle dimensioni di una stanza. I dispositivi Bluetooth sono a bassa potenza, hanno una portata di 10 m, e usano una banda ISM da 2,4 GHz divisa in 79 canali da 1 MHz ciascuno.

Architettura

Il Bluetooth delinea due tipi di rete: la piconet e la scatternet.

Una rete Bluetooth viene chiamata piconet, o piccola rete. Essa può avere fino a otto stazioni, una delle quali è detta primaria (master), e le altre sono secondarie (slave). Tutte le stazioni secondarie sincronizzano i loro orologi e le sequenze di hopping con la stazione primaria. Da notare che una piconet può avere una sola stazione primaria. La comunicazione tra la stazione primaria e le secondarie può essere uno-a-uno o uno-a-molti.

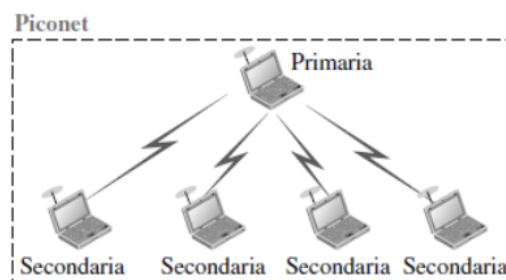


Figura 8.12: Piconet

Anche se una piconet può avere un massimo di sette stazioni secondarie, altre possono essere in stato parked (in sosta). Una stazione secondaria in stato parked è sincronizzata con la primaria, ma non può prendere parte alla comunicazione finché non viene spostata dallo stato parked allo stato attivo. Siccome solo otto stazioni possono essere attive in una piconet, attivare una stazione da uno stato parked significa che una stazione attiva deve andare in tale stato.

Le piconet si possono combinare per formare quella che viene definita una scatternet. Una stazione secondaria in una piconet può essere la primaria in un'altra piconet. Tale stazione può ricevere dei messaggi dalla stazione primaria nella prima piconet (come secondaria) e, agendo da primaria, consegnarli alle stazioni secondarie nella seconda piconet. Una stazione può appartenere a due piconet.

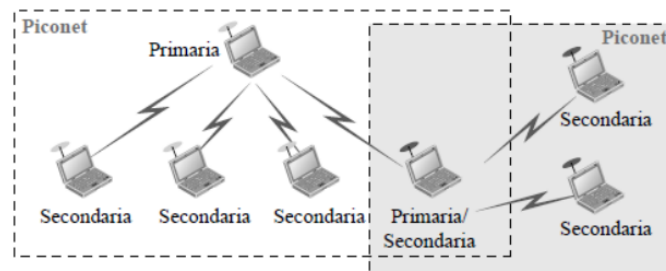


Figura 8.13: Scatternet

Un dispositivo Bluetooth ha un trasmettitore radio integrato a breve portata. La velocità di trasferimento dati corrente è 1 Mbps con ampiezza di banda da 2,4 GHz. Ciò significa che c'è una possibilità d'interferenza tra le LAN IEEE 802.11b wireless e le LAN Bluetooth.

Bluetooth usa una forma di TDMA come metodo d'accesso al mezzo. La stazione primaria e quelle secondarie comunicano tra loro usando degli slot temporali. La lunghezza di uno slot è $625 \mu s$. Ciò significa che durante il tempo in cui una frequenza viene usata, una stazione primaria invia un frame a una secondaria, o viceversa. Da notare che la comunicazione è solo tra la primaria e una secondaria; le secondarie non possono comunicare direttamente l'una con l'altra.

Il Bluetooth usa una variante di TDMA, detta TDD-TDMA (TDMA duplex a divisione del tempo). Il TDD-TDMA è un tipo di comunicazione half-duplex in cui il mittente e il destinatario inviano e ricevono dati, ma non contemporaneamente (half-duplex), tuttavia la comunicazione per ogni direzione usa diversi hop.

- *Comunicazione singola-secondaria.* Se la piconet ha una sola stazione secondaria, il funzionamento del TDMA è molto semplice. Il tempo viene diviso in slot da $625 \mu s$. La stazione primaria usa gli slot a numeri pari (0, 2, 4, ...); la secondaria a numeri dispari (1, 3, 5, ...). Il TDD-TDMA consente alla primaria e alla secondaria di comunicare in modalità half-duplex. Nello slot 0 la primaria invia e la secondaria riceve; nello slot 1 la secondaria invia e la primaria riceve. Il ciclo viene ripetuto.

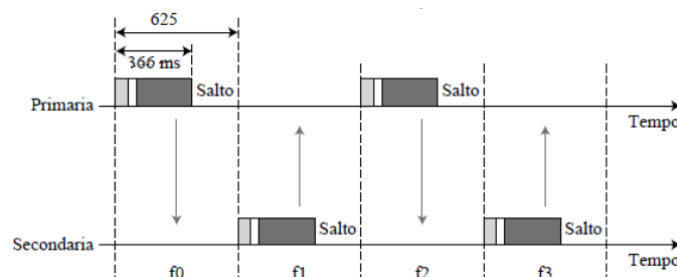


Figura 8.14: Comunicazione singola-secondaria

- *Comunicazione multipla-secondaria.* Il processo è un po' più complesso se ci sono più stazioni secondarie nella piconet. Anche in questo caso la primaria usa gli slot a numeri pari, mentre una secondaria trasmette nello slot a numero dispari successivo se il pacchetto della primaria nello slot precedente era indirizzato ad essa. Tutte le secondarie ascoltano gli slot a numeri pari, ma solo una secondaria trasmette in uno slot a numeri dispari.

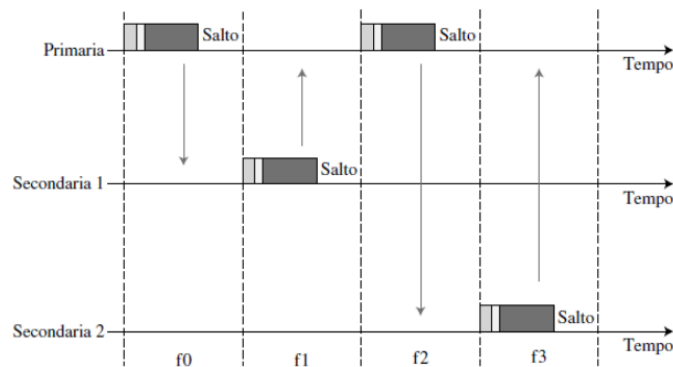


Figura 8.15: Comunicazione multipla-secondaria

1. Nello slot 0 la primaria invia un frame alla secondaria 1.
2. Nello slot 1 solo la secondaria 1 invia un frame alla primaria in quanto il frame precedente era indirizzato alla secondaria 1, le altre secondarie restano mute.
3. Nello slot 2 la primaria invia un frame alla secondaria 2.
4. Nello slot 3 solo la secondaria 2 invia un frame alla primaria in quanto il frame precedente era indirizzato alla secondaria 2, le altre secondarie sono mute.
5. Il ciclo continua.

Possiamo dire che questo metodo d'accesso è simile a un'operazione interrogazione/selezione con prenotazioni. Quando la primaria seleziona una secondaria, la interroga. Lo slot successivo è prenotato affinché la stazione interrogata possa inviare il suo frame. Se la secondaria interrogata non ha frame da inviare il canale resta inutilizzato.

8.2 Accesso al mezzo mediante suddivisione del canale

Nelle reti wireless, come le reti cellulari e quelle satellitari, vengono utilizzati dei meccanismi di accesso al mezzo trasmissivo basati sulla suddivisione del canale.

La channelization (o suddivisione del canale) è un metodo ad accesso multiplo nel quale l'ampiezza di banda disponibile di un link viene suddivisa in tempo, frequenza o codice, tra diverse stazioni. In questa sezione parleremo di tre protocolli di suddivisione del canale: FDMA, TDMA e CDMA.

8.2.1 Frequency-Division Multiple Access (FDMA)

Nel Frequency-Division Multiple Access (FDMA), o accesso multiplo a divisione di frequenza, l'ampiezza di banda disponibile viene divisa in bande di frequenza. Ad ogni stazione viene assegnata una banda per trasmettere i suoi dati. In altre parole ogni banda è riservata a una stazione specifica e appartiene sempre a tale stazione. Ogni stazione usa anche un filtro passa banda per limitare le frequenze del trasmettitore. Per evitare le interferenze della stazione, le bande assegnate vengono separate una dall'altra tramite piccole bande di guardia.

L'FDMA specifica una banda di frequenza predeterminata per l'intero periodo della comunicazione. Ciò significa che stream di dati (un flusso continuo di dati che non possono essere racchiusi in pacchetti) possono usare l'FDMA.

Dobbiamo sottolineare che l'FDMA è un protocollo del livello di collegamento, e non deve essere confuso con un processo di multiplex, quello a divisione di frequenza (FDM), che in realtà è l'implementazione dell'FDMA al livello fisico.

8.2.2 Time Division Multiple Access (TDMA)

Nel Time-Division Multiple Access (TDMA), o accesso multiplo a divisione del tempo, le stazioni condividono l'ampiezza di banda del canale nel tempo. A ogni stazione viene assegnato uno slot di

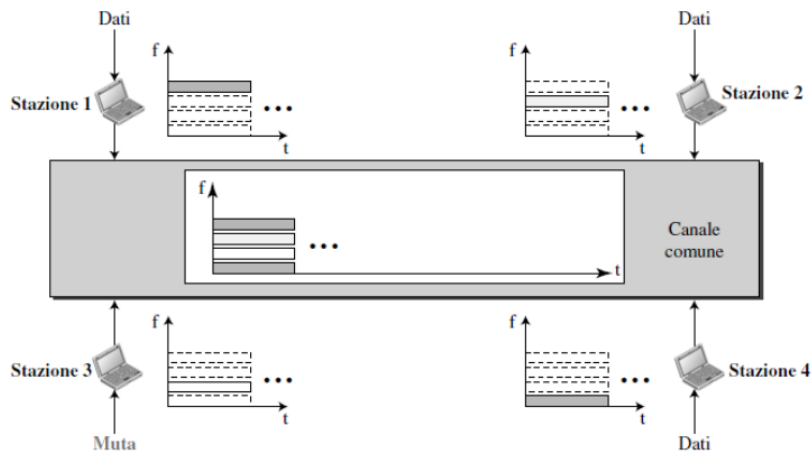


Figura 8.16: Accesso multiplo a divisione di frequenza (FDMA)

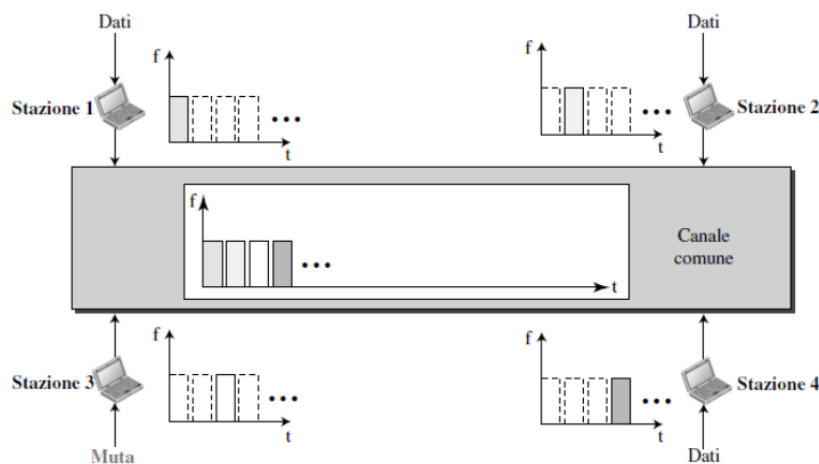


Figura 8.17: Accesso multiplo a divisione di tempo (TDMA)

tempo durante il quale può inviare i dati. Ogni stazione trasmette i suoi dati nello slot che le è stato assegnato.

Il problema principale con il TDMA sta nella sincronizzazione tra le varie stazioni. Ogni stazione deve conoscere l'inizio e la posizione del suo slot. Ciò può essere difficile a causa dei ritardi di propagazione introdotti nel sistema se le stazioni sono sparse in un'area ampia. Per compensare tali ritardi possiamo inserire dei tempi di guardia. La sincronizzazione normalmente viene realizzata tramite dei bit di sincronizzazione (normalmente chiamati bit preambolo) all'inizio di ogni slot.

Dobbiamo sottolineare che anche se il TDMA e il TDM (multiplex a divisione del tempo) sembrano concettualmente la stessa cosa, ci sono delle differenze tra di loro. Il TDM è una tecnica di livello fisico che unisce i dati dai canali più lenti e li trasmette usando un canale più veloce. Il processo usa un multiplexer fisico che inserisce delle unità dati da ogni canale. Il TDMA, d'altro canto, è un metodo d'accesso nel livello di collegamento. Questo livello in ogni stazione dice al suo livello fisico di usare lo slot che gli è stato assegnato. Non c'è multiplexer fisico al livello di collegamento.

8.2.3 Code-Divison Multiple Access (CDMA)

Il CDMA differisce dall'FDMA in quanto un solo canale occupa l'intera ampiezza di banda del link. Differisce dal TDMA in quanto tutte le stazioni possono inviare dati contemporaneamente, non essendoci suddivisione del tempo.

Per capire il CDMA facciamo un'analogia. CDMA significa semplicemente comunicazione con codici diversi. Per esempio in una stanza grande con molta gente, due persone possono parlare in inglese se nessun altro lo capisce. Altre due persone possono parlare cinese se sono gli unici due che

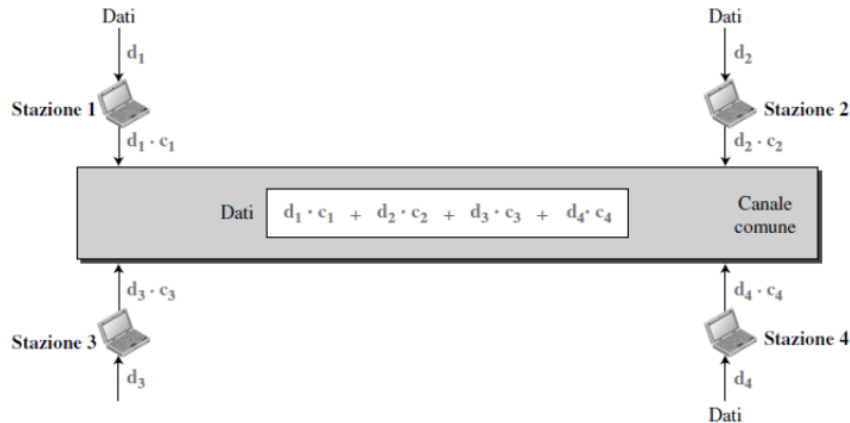
comprendono tale lingua e così via. In altre parole il canale comune, lo spazio della stanza in questo caso, può facilmente consentire la comunicazione tra varie coppie, ma in lingue diverse (codici).

Esempio 8.2.1 – Idea della comunicazione con codice

Supponiamo di avere quattro stazioni, 1, 2, 3 e 4, collegate allo stesso canale. I dati dalla stazione 1 sono d_1 , quelli dalla stazione 2 sono d_2 , e così via. Il codice assegnato alla prima stazione è c_1 , quello assegnato alla seconda è c_2 , e così di seguito. Supponiamo che i codici assegnati abbiano due proprietà.

1. Se moltiplichiamo ogni codice per un altro otteniamo 0.
2. Se moltiplichiamo ogni codice per sé stesso otteniamo 4 (il numero delle stazioni).

Tenendo a mente queste due proprietà vediamo come le quattro stazioni sopra indicate possono inviare dei dati usando lo stesso canale comune.



La stazione 1 moltiplica (un tipo speciale di moltiplicazione, come vedremo) i suoi dati per il suo codice per ottenere $d_1 \cdot c_1$. La stazione 2 moltiplica i suoi dati per il suo codice per ottenere $d_2 \cdot c_2$, e così via. I dati che vanno sul canale sono la somma di tutti questi termini, come mostrato nel riquadro. Qualsiasi stazione che voglia ricevere dei dati da una delle altre tre stazioni moltiplica i dati sul canale per il codice del mittente. Per esempio, supponiamo che le stazioni 1 e 2 stiano parlando tra loro. La stazione 2 vuole sentire ciò che sta dicendo la 1. Essa moltiplica i dati sul canale per c_1 , il codice della stazione 1.

Siccome $(c_1 \cdot c_1)$ fa 4, ma $(c_2 \cdot c_1)$, $(c_3 \cdot c_1)$ e $(c_4 \cdot c_1)$ sono tutti 0, la stazione 2 divide il risultato per 4 per ottenere i dati dalla stazione 1.

$$\begin{aligned} \text{Dati} &= [(d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4) \cdot c_1] / 4 \\ &= [d_1 \cdot c_1 \cdot c_1 + d_2 \cdot c_2 \cdot c_1 + d_3 \cdot c_3 \cdot c_1 + d_4 \cdot c_4 \cdot c_1] / 4 = (4 \times d_1) / 4 = d_1 \end{aligned}$$

Il CDMA si basa sulla teoria della codifica. Ad ogni stazione viene assegnato un codice, che è una sequenza di numeri chiamati chip. La figura mostra i chip per l'esempio precedente.

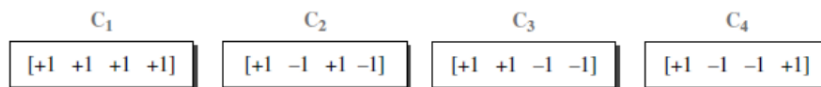


Figura 8.18: Sequenze di chip

Per ora dobbiamo sapere che le sequenze non vengono scelte in modo casuale, ma con molta attenzione. Vengono definite sequenze ortogonali e hanno le seguenti proprietà:

1. Ogni sequenza è composta da N elementi, dove N è il numero di stazioni e deve essere una potenza di 2.
2. Se moltiplichiamo una sequenza per un numero ogni elemento nella sequenza viene moltiplicato per tale numero. Ciò viene definito moltiplicazione di una sequenza per una scalare. Per esempio

$$2 \cdot [+1 +1 -1 -1] = [+2 +2 -2 -2]$$

Se moltiplichiamo due sequenze uguali, elemento per elemento, e sommiamo i risultati, otteniamo N , dove N è il numero di elementi di ogni sequenza. Tale dato viene definito il prodotto interno di due sequenze uguali. Per esempio

$$[+1 + 1 - 1 - 1] \cdot [+1 + 1 - 1 - 1] = 1 + 1 + 1 + 1 = 4$$

3. Se moltiplichiamo due sequenze diverse, elemento per elemento, e aggiungiamo i risultati, otteniamo 0. Ciò viene chiamato prodotto interno di due diverse sequenze. Per esempio

$$[+1 + 1 - 1 - 1] \cdot [+1 + 1 + 1 + 1] = 1 + 1 - 1 - 1 = 0$$

4. Sommare due sequenze significa aggiungere gli elementi corrispondenti. Il risultato è un'altra sequenza. Per esempio

$$[+1 + 1 - 1 - 1] + [+1 + 1 + 1 + 1] = [+2 + 2 + 0 + 0]$$

La codifica avviene in base ad alcune regole: se una stazione deve inviare il bit 0, essa lo codifica come -1, se deve inviare il bit 1, lo codifica come +1. Quando una stazione è inattiva, non invia alcun segnale, il che viene interpretato come uno 0.



Figura 8.19: Rappresentazione dei dati nel CDMA

Esempio 8.2.2 – Condivisione del canale nel CDMA

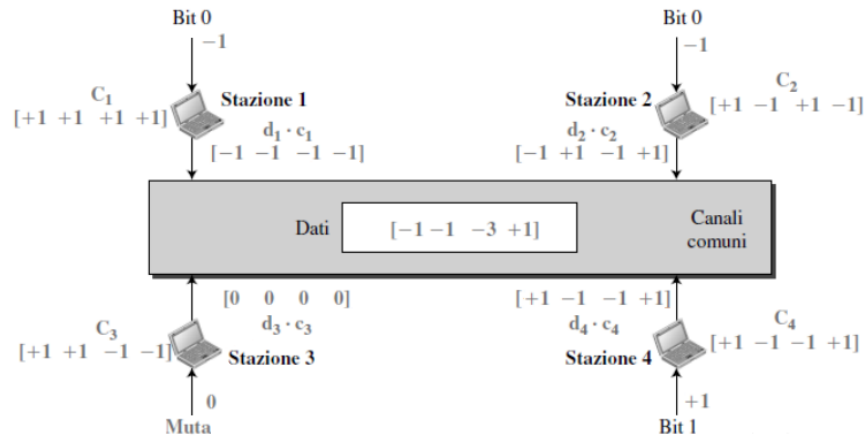
Come semplice esempio mostriamo come quattro stazioni condividono il link durante un intervallo da 1 bit. La procedura si può ripetere facilmente per altri intervalli. Supponiamo che le stazioni 1 e 2 stiano inviando un bit 0 e la stazione 4 stia trasmettendo un bit 1. La stazione 3 è muta. I dati sul sito mittente vengono tradotti in 1, -1, 0 e +1. Ogni stazione moltiplica il numero corrispondente per il suo chip (la sua sequenza ortogonale), che è unica per ogni stazione. Il risultato è una nuova sequenza che viene inviata al canale. Per semplicità supponiamo che tutte le stazioni inviino le sequenze risultanti contemporaneamente. La sequenza sul canale è la somma di tutte e quattro le sequenze sopra definite.

Ora immaginiamo che la stazione 3, che abbiamo detto essere muta, stia ascoltando la stazione 2. La stazione 3 moltiplica i dati totali sul canale per il codice per la stazione 2, che $c[1 - 1 + 1 - 1]$ per ottenere

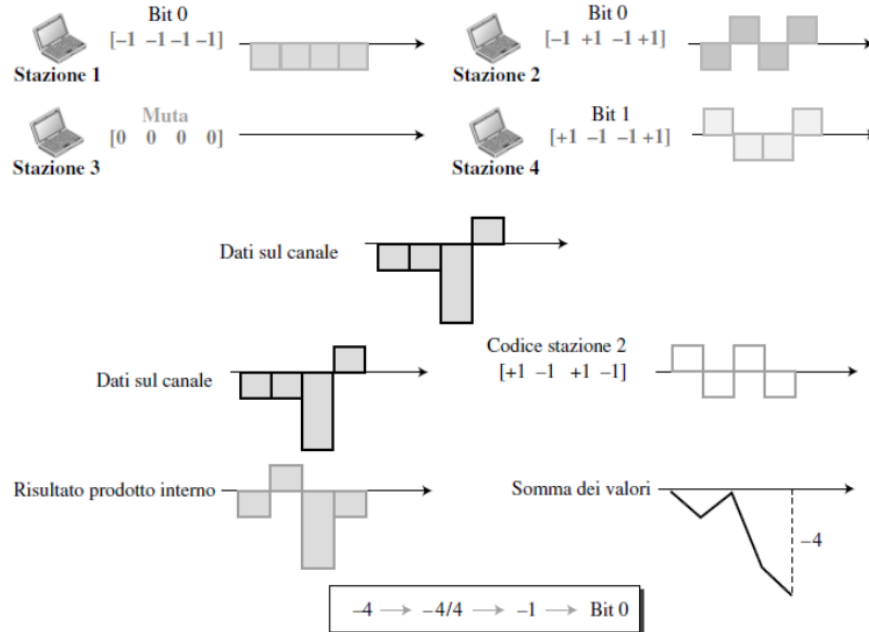
$$[-1 - 1 - 3 + 1] \cdot [+1 - 1 + 1 - 1] = -4$$

$$-4/4 = -1 \rightarrow \text{bit } 0$$

Il processo può essere meglio compreso se mostriamo il segnale digitale prodotto da ogni stazione e i dati recuperati a destinazione. La figura mostra i segnali corrispondenti per ogni stazione e il segnale che si trova sul canale comune.



La figura mostra come la stazione 3 possa rilevare i dati inviati dalla stazione 2 usando il codice per tale stazione. I dati totali sul canale vengono moltiplicati (operazione prodotto interno) per il segnale che rappresenta il codice chip della stazione 2 per ottenere un nuovo segnale. La stazione quindi integra e somma l'area sotto il segnale, per ottenere il valore -4, che viene diviso per 4 ed interpretato come bit 0.



Per generare delle sequenze di chip usiamo una tabella di Walsh, una tabella bidimensionale con un numero uguale di righe e colonne.

$$W_1 = \begin{bmatrix} +1 \end{bmatrix} \quad W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & \overline{W_N} \end{bmatrix}$$

a. Due regole base

$$W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} \quad W_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

b. Generazione di W_1, W_2 e W_4

Figura 8.20: Regola generale ed esempi di creazione di tabelle di Walsh

Nella tabella di Walsh, ogni riga è una sequenza di chip. W_1 indica una sequenza da un chip con una riga e una colonna. Possiamo scegliere -1 o +1 per il chip di questa tabella a titolo esemplificativo (scegliamo +1). Secondo tale tabella se conosciamo quella per N sequenze W_N possiamo crearne

un'altra per $2N$ sequenze W_{2N} . Il W_N indicato con una barra superiore indica il complemento di W_N in cui ogni +1 viene cambiato in -1 e viceversa. Viene mostrata anche come possiamo creare W_2 e W_4 da W_1 . Dopo aver selezionato W_1 , W_2 può essere composto da quattro W_1 con l'ultimo che fa da complemento a W_1 . Dopo che è stato generato W_2 , si può creare W_4 da quattro W_2 , con l'ultimo che fa da complemento a W_2 . Naturalmente W_8 è composto da quattro W_4 e così via. Da notare che dopo aver creato W_N ad ogni stazione viene assegnato un chip che corrisponde a una riga.

Il numero di sequenze N deve essere una potenza di 2. In altre parole dobbiamo avere $N = 2^m$.

Bibliografia

- [1] Behrouz A. Forouzan e Firouz Mosharraf. *Computer Networks: A Top-Down Approach*. McGraw-Hill, 2012.
- [2] James F. Kurose e Keith W. Ross. *Reti di calcolatori e Internet: Un approccio top-down*. 7^a ed. Pearson, 2017.